

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Kiến trúc Phần mềm (CO3017)

BÁO CÁO BÀI TẬP LỚN

Intelligent Restaurant Management System (IRMS)

Giảng viên: Trần Trương Tuấn Phát

Lớp học phần: L01

Nhóm: 8

Khoa: Khoa học và Kỹ thuật Máy tính

MSSV	Họ và tên	Email
2311896	Đỗ Quang Long	long.doquang1896@hcmut.edu.vn
2311982	Lê Hoàng Luân	luan.le2311982@hcmut.edu.vn
2310990	Lê Thúy Hiền	hien.lethuy@hcmut.edu.vn
2213698	Nguyễn Hải Trung	trung.nguyen2004@hcmut.edu.vn
2211384	Tô Thế Hưng	hung.totothehung@hcmut.edu.vn
2310737	Trần Nhật Đăng	dang.trannh388@hcmut.edu.vn

Mục lục

1	Tổng quan hệ thống	3
1.0.1	Bối cảnh nghiệp vụ	3
1.0.2	Mục tiêu của hệ thống	3
1.0.3	Phạm vi hệ thống	3
1.1	Nhóm chức năng trong phạm vi	4
1.2	Các bên liên quan	5
1.2.1	Cơ sở đo tải và ràng buộc dự án	6
1.3	Yêu cầu chức năng	6
1.4	Yêu cầu phi chức năng ở góc nhìn toàn dự án	6
2	Mô hình hóa hệ thống	8
2.1	Danh mục ca sử dụng	8
2.2	Góc nhìn sử dụng	12
2.2.1	Kịch bản 1: Từ đặt bàn đến tiếp nhận và xếp bàn	12
2.2.2	Kịch bản 2: Từ gọi món đến điều phối bếp và phục vụ	12
2.2.3	Kịch bản 3: Từ hóa đơn đến thanh toán và biên lai	13
2.2.4	Kịch bản 4: Từ cảnh báo mức tồn thấp đến quyết định của quản lý	13
2.3	Đặc tả ca sử dụng	21
3	Kiến trúc phần mềm	40
3.1	Tổng quan kiến trúc	40
3.2	Góc nhìn logic	42
3.2.1	Sơ đồ tổng quan	43
3.3	Tổng quan cho các góc nhìn chính	44
3.3.1	Tổng quan cho góc nhìn logic	45
3.3.2	Góc nhìn logic cho chuỗi phục vụ lỗi	46
3.3.3	Tổng quan cho góc nhìn mô-đun	47
3.3.4	Góc nhìn mô-đun cho tuyến order-to-cash	49
3.3.5	Tổng quan cho góc nhìn phân bổ	51
3.4	Đặc trưng kiến trúc	52
3.5	Góc nhìn phân rã	52
3.5.1	Các mô-đun phía người dùng	52
3.5.2	Các dịch vụ nghiệp vụ phía sau	53
3.5.3	Sơ đồ tổng quan	54
3.6	Góc nhìn thành phần và kết nối	59
3.6.1	Các thành phần chính	60
3.6.2	Các kiểu kết nối chính	60
3.6.3	Lý do chọn mô hình kết nối hỗn hợp	60
3.6.4	Sơ đồ thành phần cho tuyến lệnh vận hành	65
3.6.5	Sơ đồ thành phần cho xử lý nền, quản trị và mô hình đọc	68
3.7	Góc nhìn triển khai	69
3.7.1	Lý do thiết kế	69
3.7.2	Sơ đồ triển khai cho tính sẵn sàng cao và cô lập lỗi	75
3.8	Nguyên tắc thiết kế	77
3.9	Áp dụng nguyên lý SOLID vào thiết kế	78
3.9.1	Nguyên lý trách nhiệm đơn nhất	78
3.9.2	Nguyên lý đóng để ổn định, mở để mở rộng	78
3.9.3	Nguyên lý thay thế tương thích	79
3.9.4	Nguyên lý phân tách giao diện	79
3.9.5	Nguyên lý đảo ngược phụ thuộc	79

3.10	Khả năng mở rộng trong tương lai	79
4	Thiết kế chi tiết	80
4.1	Góc nhìn lớp	80
4.1.1	Phạm vi bao phủ của sơ đồ lớp	80
4.2	Giải thích theo từng cụm lớp	87
4.3	Thiết kế hành vi dựa trên ca sử dụng	99
4.3.1	Nhóm đặt bàn và bàn ăn	100
4.3.2	Nhóm gọi món và điều phối bếp	105
4.3.3	Nhóm hóa đơn, thanh toán và biên lai	112
4.3.4	Nhóm tồn kho, cảnh báo và phân tích	117
4.3.5	Nhóm quản trị cấu hình và vận hành	121
4.4	Phản tư về toàn bộ báo cáo và định hướng hiện thực	124
A	Phụ lục	125
A.1	Lý do lựa chọn kiến trúc và hồ sơ quyết định kiến trúc	125
A.1.1	So sánh các kiểu kiến trúc ứng viên	126
A.2	Liên kết giữa phụ lục và các phần chính	129
A.3	Lý do chọn các góc nhìn và loại sơ đồ	129
A.4	Lý do chọn các thành phần và loại hình kiến trúc nền	130
A.5	Phân công công việc	132

1 Tổng quan hệ thống

1.0.1 Bối cảnh nghiệp vụ

IRMS được thiết kế cho môi trường vận hành nhà hàng có phục vụ tại bàn, khu bếp, quầy thanh toán và hoạt động điều phối theo ca. Trong bối cảnh thực tế, các sai sót thường xuất hiện đúng vào giờ cao điểm: đơn gọi món ghi nhầm món, ghi chú dị ứng bị bỏ sót, trạng thái bàn thay đổi nhưng không được cập nhật kịp, hóa đơn bị lệch khi có tách hóa đơn hoặc áp dụng khuyến mãi, và tồn kho cuối ngày không phản ánh đúng mức tiêu hao thực tế.

Bài toán vì vậy không chỉ là “ghi đơn gọi món trên màn hình”. IRMS phải đóng vai trò như một nền tảng số trung tâm liên kết khu vực phục vụ phía trước, khu vực bếp và khu vực quản trị: đặt bàn, xếp bàn, gọi món, điều phối bếp, lập hóa đơn, thanh toán, tồn kho, báo cáo, kiểm soát truy cập, nhật ký kiểm toán và các tích hợp bên ngoài. Đây là một hệ thống nghiệp vụ hoàn chỉnh, có đường xử lý lỗi rõ ràng, có luồng hỗ trợ bất đồng bộ và có yêu cầu mở rộng về sau.

1.0.2 Mục tiêu của hệ thống

Các mục tiêu thiết kế chính của IRMS không chỉ dừng ở mức “có đủ chức năng”, mà còn phải chuyển thành kết quả vận hành có thể kiểm chứng trong phạm vi một dự án phần mềm cho nhà hàng quy mô trung bình. Vì vậy, báo cáo đặt ra các mục tiêu sau:

1. **Giảm sai sót nghiệp vụ trên tuyến gọi món:** mục tiêu là giữ tỷ lệ đơn gọi món bị sửa do nhập sai dưới 2% tổng số đơn đã xác nhận trong một ca, thông qua thực đơn số hóa, tùy chọn món có ràng buộc, cảnh báo ghi chú dị ứng và bước xác nhận trước khi gửi bếp.
2. **Rút ngắn độ trễ từ phục vụ xuống bếp:** phiếu món phải xuất hiện trên màn hình bếp trong vòng tối đa 2 giây ở điều kiện thường và không quá 3 giây ở giờ cao điểm, để tránh việc phục vụ phải gọi miệng hoặc ghi tay.
3. **Đồng bộ trạng thái vận hành theo thời gian gần thực:** trạng thái bàn, đơn gọi món, phiếu bếp, hóa đơn và thanh toán phải phản ánh trên các màn hình liên quan với độ trễ quan sát được không vượt quá 5 giây đối với các thay đổi quan trọng.
4. **Rút ngắn thời gian đóng phiên phục vụ:** với một bàn tiêu chuẩn không có tách hóa đơn phức tạp, thời gian từ lúc yêu cầu thanh toán đến lúc phát hành biên lai nên dưới 2 phút; với tách hóa đơn tiêu chuẩn cho 2–4 người, mục tiêu là dưới 4 phút.
5. **Tăng khả năng kiểm soát và truy vết:** 100% thao tác nhạy cảm như hoàn tiền, hủy đơn đã gửi bếp, ghi đè giá, mở lại hóa đơn hoặc đổi trạng thái thanh toán phải được ghi nhận đầy đủ người thực hiện, thời điểm và lý do.
6. **Tạo nền tảng mở rộng về sau:** khi cần thêm phương thức thanh toán, kênh gửi thông báo, thiết bị tự phục vụ hoặc mở rộng sang nhiều chi nhánh, kiến trúc phải cho phép mở rộng qua các giao diện và bộ kết nối phù hợp mà không buộc sửa chéo logic lõi của toàn bộ hệ thống.

1.0.3 Phạm vi hệ thống

Trong phạm vi

- **Đặt bàn và bàn ăn:** đặt bàn, danh sách chờ, tiếp nhận khách, gán bàn, giải phóng bàn, theo dõi phiên phục vụ tại bàn;
- **Gọi món số hóa:** tạo đơn gọi món, chỉnh sửa đơn gọi món, tùy chọn món, món kết hợp, ghi chú dị ứng, hướng dẫn đặc biệt;
- **Quản lý thực đơn:** quản lý món, nhóm món, nhóm tùy chọn, trạng thái còn bán, khuyến mãi cơ bản;
- **Điều phối bếp:** phát phiếu theo trạm, cập nhật trạng thái món, theo dõi hàng chờ của bếp;
- **Lập hóa đơn và thanh toán:** tạo hóa đơn, tách hóa đơn, thuế, phí phục vụ, tiền boa, tiền mặt, thẻ hoặc ví điện tử;
- **Theo dõi tồn kho:** trừ nguyên liệu, lưu giao dịch kho, cảnh báo mức tồn thấp;

- **Phân tích và báo cáo:** doanh thu, món bán chạy, giờ cao điểm, điểm nghẽn, thống kê hoàn tiền;
- **Quản trị:** người dùng, vai trò, quyền, nhật ký kiểm toán, cấu hình thực đơn, chính sách giá.

Ngoài phạm vi

- kế toán doanh nghiệp và đối soát tài chính ở mức tập đoàn;
- hệ quản trị chuỗi nhiều chi nhánh ở mức triển khai đầy đủ;
- học máy nâng cao cho dự báo nhu cầu hoặc định giá động;
- tích hợp nền tảng giao hàng bên thứ ba ở mức hoàn chỉnh;
- hệ quản trị quan hệ khách hàng và nền tảng khách hàng thân thiết mở rộng.

1.1 Nhóm chức năng trong phạm vi

IRMS bao phủ các nhóm chức năng sau:

- **Đặt bàn và xếp bàn:** đặt bàn, tiếp nhận khách, danh sách chờ, sơ đồ bàn, trạng thái bàn và vòng quay bàn.
- **Gọi món và thực đơn:** tra cứu thực đơn, tạo đơn gọi món, ghi chú dị ứng, tùy chọn món, món kết hợp và quản lý trạng thái món.
- **Điều phối bếp:** chuyển phiếu đến từng trạm bếp, theo dõi tiến độ chế biến, đồng bộ thông tin món sẵn sàng và món đã phục vụ.
- **Lập hóa đơn và thanh toán:** tạo hóa đơn, tách hóa đơn, tiền boa, thanh toán nhiều phương thức, biên lai và hoàn tiền.
- **Theo dõi tồn kho:** trừ tồn theo công thức món, theo dõi mức tồn thấp và hỗ trợ quyết định ẩn món.
- **Phân tích và quản trị:** báo cáo doanh thu, báo cáo vận hành, quản trị thực đơn, giá, khuyến mãi, vai trò và lịch làm việc.

1.2 Các bên liên quan

Bảng 1: Các bên liên quan chính của IRMS

Bên liên quan	Vai trò	Mối quan tâm chính
Khách hàng	Người dùng/đơn vị vận hành	- Đặt bàn trước - Nhận thông báo xác nhận và nhắc nhở - Thanh toán đa phương thức, nhận hóa đơn - Gọi món, yêu cầu tùy chỉnh, ghi chú dị ứng khi order
Lễ tân	Người dùng/đơn vị vận hành	- Quản lý sơ đồ bàn, theo dõi trạng thái trống/bận - Xử lý đặt bàn trước và danh sách chờ - Gửi thông báo xác nhận/nhắc nhở cho khách hàng - Ước tính thời gian chờ, phân bổ bàn hợp lý
Phục vụ	Người dùng/đơn vị vận hành	- Tạo đơn gọi món, nhận order từ khách - Theo dõi đơn, thời gian phục vụ và tình trạng thanh toán theo từng bàn
Nhân viên bếp	Người dùng/đơn vị vận hành	- Cập nhật trạng thái món: Đã bắt đầu, Đang nấu, Sẵn sàng phục vụ - Phản hồi cảnh báo khi món sắp trễ - Cập nhật nguyên liệu đã sử dụng sau khi chế biến
Thu ngân	Người dùng/đơn vị vận hành	- Lập hóa đơn tổng hợp (đồ ăn, đồ uống, dịch vụ) - Xử lý thanh toán: tiền mặt, ví điện tử - Tách bill và quản lý tip - Thực hiện hoàn tiền khi cần và ghi vào audit log
Quản lý	Người dùng/đơn vị vận hành	- Xem báo cáo: doanh thu, giờ cao điểm, món bán chạy - Giám sát tồn kho, nhận cảnh báo ngưỡng tối thiểu - Cấu hình menu, giá và chương trình khuyến mãi - Xuất báo cáo cho kế toán, đánh giá hiệu suất - Theo dõi hiệu quả nhân viên và điểm nghề bếp
Quản trị viên	Người dùng/đơn vị vận hành	- Phân quyền truy cập theo vai trò - Xem và quản lý audit log toàn hệ thống - Xử lý sự cố kỹ thuật, đảm bảo uptime - Cấu hình và bảo trì hệ thống POS
Cổng thanh toán	Hệ thống ngoài	- Xử lý giao dịch thẻ tín dụng/ghi nợ an toàn - Hỗ trợ ví điện tử (MoMo, ZaloPay, Apple Pay, ...) - Trả về kết quả giao dịch (thành công/thất bại) cho POS

1.2.1 Cơ sở đo tải và ràng buộc dự án

Đây là giả định thiết kế cho một nhà hàng tầm trung, được dùng thống nhất ở toàn bộ báo cáo khi nói về hiệu năng, độ tin cậy và khả năng mở rộng.

Bảng 2: Hai điều kiện vận hành dùng làm cơ sở thiết kế của IRMS

Điều kiện	Tải giả định	Ý nghĩa với kiến trúc
Điều kiện 1 — Ca thông thường	20–25 bàn hoạt động, 8–12 nhân viên đăng nhập đồng thời, tối đa 35 đơn gọi món mỗi giờ, khoảng 120 dòng món mỗi giờ.	Hệ thống phải phản hồi mượt cho các tác vụ tạo đơn gọi món, cập nhật màn hình bếp, đổi trạng thái bàn và chốt hóa đơn mà chưa cần mở rộng hay hy sinh trải nghiệm thao tác.
Điều kiện 2 — Giờ cao điểm	35–40 bàn hoạt động, 15–18 nhân viên đăng nhập đồng thời, tối đa 60 đơn gọi món mỗi giờ, khoảng 220 dòng món mỗi giờ và 20 hóa đơn mỗi giờ.	Kiến trúc phải giữ ổn định tuyến giao dịch nóng; các luồng nền như báo cáo, cảnh báo mức tồn thấp và đồng bộ thông báo không được làm nghẽn gọi món, điều phối bếp hay thanh toán.

Trong hai điều kiện trên, dự án chịu các ràng buộc thực tế sau:

- **Ràng buộc phạm vi:** phiên bản hiện tại phục vụ một nhà hàng đơn lẻ, chưa tối ưu cho vận hành nhiều chi nhánh ở quy mô đầy đủ, nhưng kiến trúc vẫn chưa điểm mở cho mở rộng về sau.
- **Ràng buộc thiết bị:** nhân viên thao tác qua máy tính bảng gọi món, màn hình bếp, thiết bị thu ngân hoặc giao diện web nội bộ; trải nghiệm phải ưu tiên thao tác nhanh, ít bước và dễ học theo ca.
- **Ràng buộc mạng:** mạng nội bộ có thể chập chờn ngắn hạn trong phạm vi dưới 60 giây; hệ thống được phép thử lại nhưng không được làm mất đơn gọi món đã xác nhận hoặc tạo phiếu trùng.
- **Ràng buộc thanh toán:** thanh toán điện tử luôn đi qua cổng đối tác ngoài; IRMS không lưu dữ liệu thẻ thô mà chỉ lưu kết quả giao dịch và mã tham chiếu.
- **Ràng buộc kiểm soát:** dữ liệu nghiệp vụ phải được lưu tập trung và có nhật ký kiểm toán cho hoàn tiền, hủy bỏ, ghi đề hoặc thay đổi nhạy cảm.
- **Ràng buộc học phần:** phạm vi hiện thực có thể tập trung vào một phần mô-đun lõi, nhưng kiến trúc vẫn phải phản ánh đầy đủ bức tranh hệ thống để chứng minh tư duy thiết kế tổng thể.

1.3 Yêu cầu chức năng

Các yêu cầu chức năng ở mức hạt thô được tổ chức thành năm cụm:

1. **Đặt bàn và xếp bàn:** tạo hoặc cập nhật đặt bàn, quản lý danh sách chờ, tiếp nhận khách, mở hoặc đóng phiên bàn.
2. **Gọi món và bếp:** tạo đơn gọi món, chỉnh sửa dòng món, chuyển phiếu sang màn hình bếp, cập nhật tiến độ và ra món.
3. **Thanh toán:** tạo hóa đơn, áp dụng khuyến mãi, tách hóa đơn, tiền boa, thanh toán, biên lai, hoàn tiền.
4. **Tồn kho và thực đơn:** quản lý thực đơn, giá, tùy chọn món, công thức, trừ tồn và cảnh báo mức tồn thấp.
5. **Quản trị và phân tích:** phân quyền theo vai trò, nhật ký kiểm toán, lịch ca, báo cáo doanh thu, hiệu suất nhân viên và điểm nghẽn của bếp.

1.4 Yêu cầu phi chức năng ở góc nhìn toàn dự án

Khác với ca sử dụng vốn mô tả hành vi theo từng kịch bản, yêu cầu phi chức năng của IRMS phải được nhìn ở mức **toàn dự án**. Chúng quyết định kiến trúc nào được chọn, mô-đun nào phải tách riêng, dữ liệu nào cần đồng bộ mạnh và luồng nào có thể xử lý bất đồng bộ. Vì vậy, mỗi yêu cầu dưới đây đều được gắn với **số liệu mục tiêu**, **điều kiện áp dụng** và **cách đáp ứng trong kiến trúc**.

Bảng 3: Yêu cầu phi chức năng của IRMS ở góc nhìn toàn dự án

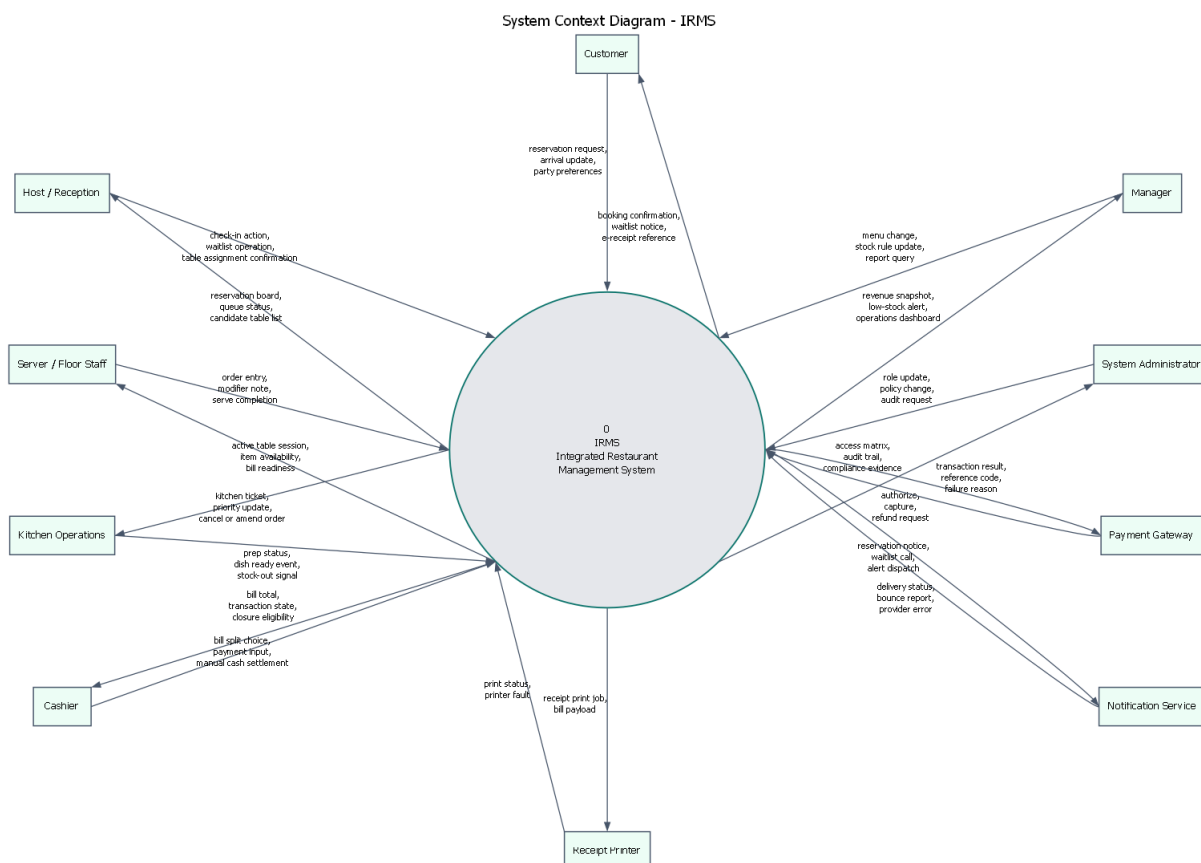
Thuộc tính	Mục tiêu đo được	Điều kiện áp dụng	Cách đáp ứng trong kiến trúc
Hiệu năng & độ phản hồi	Tra cứu thực đơn và trạng thái bàn có thời gian phản hồi bách phân vị thứ 95 dưới 1 giây; thao tác xác nhận đơn gọi món và gửi bếp có thời gian phản hồi bách phân vị thứ 95 dưới 2 giây ở Điều kiện 1 và dưới 3 giây ở Điều kiện 2; phiếu phải hiển thị trên màn hình bếp không quá 2 giây sau khi đơn được xác nhận.	Áp dụng cho toàn bộ giờ mở cửa, đặc biệt tại màn hình gọi món, màn hình bếp và quầy thu ngân là ba điểm chạm không được gây nghẽn thao tác người dùng.	Tách riêng các miền Ordering, Kitchen Coordination và Billing để giảm cạnh tranh tài nguyên; dùng mô hình đọc hoặc bộ nhớ đệm cho thực đơn và trạng thái thường đọc; giữ các tác vụ trên tuyến giao dịch nóng ở dạng gọi đồng bộ ngắn, còn báo cáo và cảnh báo chuyển sang xử lý nền.
Độ tin cậy & toàn vẹn giao dịch	Không làm mất đơn gọi món đã xác nhận; tỷ lệ tạo phiếu trùng hoặc ghi nhận thanh toán trùng phải thấp hơn 0.1%; dữ liệu gọi món và thanh toán có mục tiêu điểm khôi phục bằng 0 và khôi phục sau lỗi một nút ứng dụng trong vòng tối đa 10 phút.	Áp dụng ở cả Điều kiện 1 và Điều kiện 2; vẫn phải giữ đúng khi mạng nội bộ gián đoạn ngắn hạn dưới 60 giây hoặc khi đối tác thanh toán phản hồi chậm.	Dùng giao dịch cục bộ theo dịch vụ, khóa chống xử lý lặp cho gọi món, thanh toán và hoàn tiền, khóa lạc quan khi cập nhật trạng thái, cùng hộp thư ra và bộ truyền sự kiện cho sự kiện miền để tránh vừa mất dữ liệu vừa xử lý lặp.
Khả năng mở rộng	Hệ thống phải chịu được tối thiểu 1.5 lần tải của Điều kiện 2 trong các đợt cao điểm ngắn 10–15 phút mà không cần thiết kế lại; riêng các miền Ordering, Kitchen Coordination và Billing phải có thể tăng số phiên bản chạy độc lập khi tải tăng.	Áp dụng cho bữa trưa, bữa tối, ngày cuối tuần hoặc các đơn đặt nhóm lớn; báo cáo và phân tích không được kéo chậm tuyến giao dịch nóng.	Chọn kiến trúc tách miền nghiệp vụ rõ ràng để phân bổ tải theo nhu cầu thực tế; mở rộng theo chiều ngang các miền có nhu cầu cao; cô lập báo cáo bằng mô hình đọc và ảnh chụp dữ liệu để truy vấn phân tích không tranh chấp với cơ sở dữ liệu giao dịch.
Bảo mật & khả năng kiểm toán	100% thao tác hoàn tiền, hủy món đã gửi bếp, ghi đề giá, mở lại hóa đơn và thay đổi quyền phải có nhật ký người thực hiện, thời điểm, giá trị trước, giá trị sau và lý do; phiên thu ngân hoặc quản lý tự khóa sau 15 phút không hoạt động; hệ thống không lưu dữ liệu thẻ gốc.	Áp dụng liên tục trong toàn bộ vòng đời vận hành và đặc biệt quan trọng ở các ca đổi nhân sự, bàn giao ca hoặc xử lý khiếu nại sau thanh toán.	Dùng IAM tập trung, phân quyền theo vai trò, chính sách kiểm tra quyền trong từng dịch vụ, nhật ký kiểm toán bất biến cho hành động nhạy cảm, và tích hợp thanh toán qua đối tác ngoài để giới hạn phạm vi dữ liệu nhạy cảm nằm trong IRMS.
Khả năng quan sát & vận hành	100% yêu cầu ghi dữ liệu phải mang mã liên kết xuyên suốt; cảnh báo mức tồn thấp phải xuất hiện trong vòng 60 giây sau khi vượt ngưỡng; bảng điều khiển vận hành có độ trễ dữ liệu không quá 5 phút; lỗi ở cổng vào hoặc cổng thanh toán phải sinh cảnh báo vận hành trong vòng 1 phút.	Áp dụng cho cả thời gian phục vụ và giai đoạn chốt ca, khi quản lý cần đối chiếu chênh lệch đơn gọi món, hóa đơn, tồn kho và thao tác hoàn tiền.	Ghi nhật ký tập trung theo dịch vụ, truy vết xuyên suốt bằng mã liên kết, thu thập số đo cho độ trễ phiếu và thanh toán, tổng hợp bảng điều khiển từ mô hình đọc, cùng với dấu thời gian sự kiện để có thể truy vết một đơn từ lúc tạo đến lúc đóng hóa đơn.

Thuộc tính	Mục tiêu đo được	Điều kiện áp dụng	Cách đáp ứng trong kiến trúc
Khả năng bảo trì & mở rộng	Khi thêm một phương thức thanh toán hoặc một kênh thông báo mới, thay đổi chủ yếu chỉ nằm ở bộ kết nối, cấu hình và kiểm thử tích hợp; thay đổi thực đơn, giá, khuyến mãi hoặc ngưỡng mức tồn thấp không được buộc sửa chéo bộ điều khiển, thanh toán và tồn kho cùng lúc.	Áp dụng trong suốt vòng đời dự án, đặc biệt khi nhà hàng đổi chính sách giá, khuyến mãi theo mùa hoặc cần thí điểm thêm kênh thanh toán mới.	Phân rã theo miền nghiệp vụ, áp dụng nguyên lý SOLID và đảo ngược phụ thuộc cho công ngoài, tách lớp điều phối ứng dụng khỏi quy tắc miền, chuẩn hóa hợp đồng giữa các dịch vụ để mỗi thay đổi được khu trú trong biên chức năng tương ứng.

2 Mô hình hóa hệ thống

2.1 Danh mục ca sử dụng

Ba hình dưới đây không chỉ có vai trò minh họa mà còn là phần chuyển tiếp quan trọng trước khi đi vào đặc tả ca sử dụng. Chúng lần lượt trả lời ba câu hỏi nền tảng: **IRMS giao tiếp với ai, IRMS phải hỗ trợ những nhóm nghiệp vụ nào và một vòng vận hành đầu-cuối trong nhà hàng diễn ra ra sao**. Việc diễn giải ba hình này giúp phần ca sử dụng phía sau bám đúng bối cảnh toàn hệ thống thay vì bị nhìn như những kịch bản rời rạc; phần định vị và biện minh tương ứng được nối trực tiếp về Bảng 20 và Bảng 21 trong Phụ lục A.2 và Phụ lục A.3.



Hình 1: Sơ đồ ngữ cảnh của IRMS

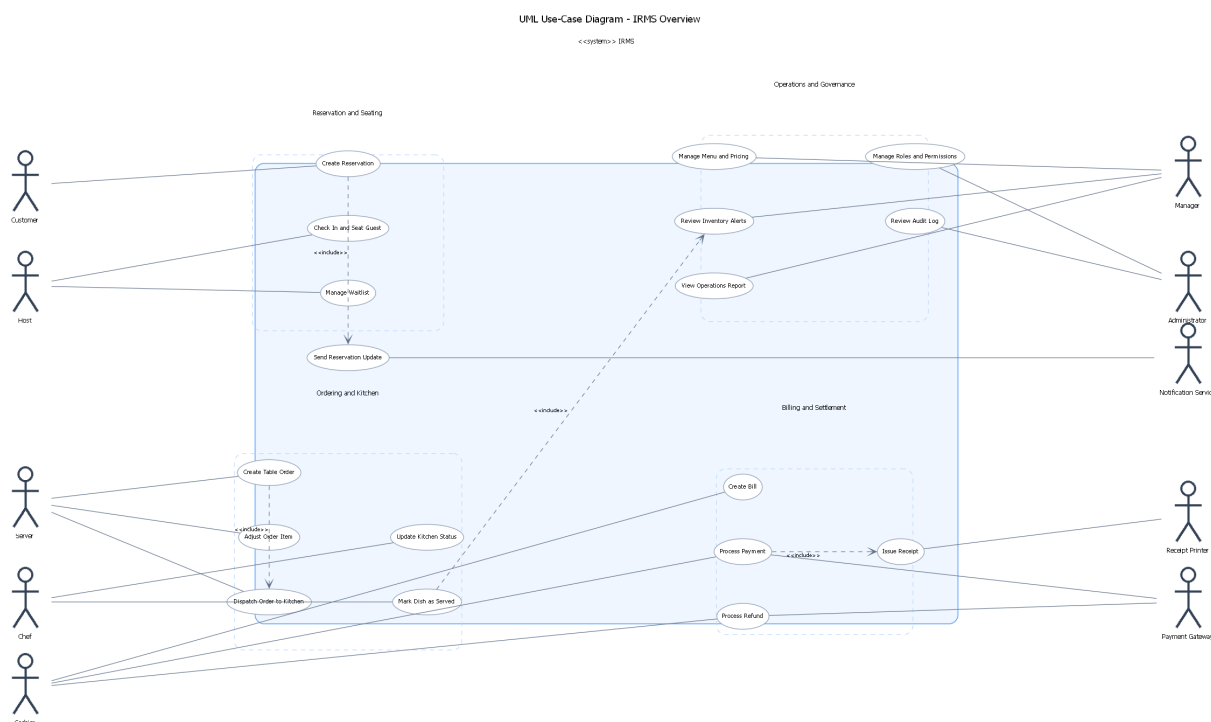
Hình 1 chốt rõ ranh giới hệ thống theo đúng kiểu *context/process diagram*: IRMS được đặt ở trung tâm như một tiến trình điều phối duy nhất, còn xung quanh là các thực thể bên ngoài và hệ thống đối tác trao đổi dữ liệu với nó. Cách thể hiện này không chỉ trả lời câu hỏi “ai dùng IRMS”, mà còn làm rõ “dữ liệu gì đi vào, kết quả

gì đi ra” ở từng điểm chạm như đặt bàn, xếp bàn, gọi món, điều phối bếp, quyết toán, kiểm soát truy cập và tích hợp dịch vụ ngoài.

Các điểm cần đọc từ sơ đồ ngữ cảnh.

- **Tuyến phục vụ phía trước:** Customer, Host / Reception, Server / Floor Staff và Cashier trao đổi trực tiếp các luồng dữ liệu thời gian thực như yêu cầu đặt bàn, trạng thái chờ, đơn gọi món, tách hóa đơn và nhập thông tin thanh toán.
- **Tuyến thực thi phía sau:** Kitchen Operations trả về trạng thái chế biến, tín hiệu món sẵn sàng hoặc hết nguyên liệu; Manager và System Administrator gửi thay đổi cấu hình, truy vấn báo cáo và yêu cầu kiểm toán ở lớp điều hành.
- **Các hợp đồng tích hợp ngoài:** Payment Gateway, Notification Service và Receipt Printer được đặt ngoài biên hệ thống để nhấn mạnh rằng thanh toán, gửi thông báo và in biên lai là các phụ thuộc cần được cách ly bằng giao tiếp có tên gọi rõ ràng.
- **Các luồng dữ liệu đều được đặt tên:** hình không dừng ở mức liệt kê vai trò, mà chỉ ra cụ thể các trao đổi như “reservation request”, “kitchen ticket”, “transaction result”, “audit trail” hoặc “print status”; nhờ đó ranh giới trách nhiệm của IRMS được khóa chặt hơn trước khi đi vào use case và kiến trúc.

Chính vì vậy, hình ngữ cảnh là cầu nối trực tiếp từ bối cảnh nghiệp vụ ở Chương 1 sang sơ đồ tổng quan kiến trúc ở Mục 3.1: nó khóa ranh giới hệ thống trước khi tải liệu đi sâu vào cách phân rã các năng lực bên trong.



Hình 2: Sơ đồ tổng quan ca sử dụng của IRMS

Hình 2 trình bày tổng quan ca sử dụng theo đúng ký pháp UML: các actor đứng ngoài biên hệ thống, các ca sử dụng được biểu diễn bằng ellipse và được gom thành năm gói nghiệp vụ lớn gồm đặt bàn và vòng đời bàn, gọi món và điều phối bếp, hóa đơn và thanh toán, tồn kho và dự báo nhập hàng, cùng quản trị, kiểm toán và phân tích. Cách tổ chức này cho thấy phạm vi của IRMS không dừng ở các thao tác trực tiếp trước khách hàng, mà bao trùm cả phân kết thúc vòng đời bàn, giao tiếp chủ động với khách hàng, điều phối ưu tiên trong bếp, hỗ trợ quyết định nhập hàng từ dữ liệu tiêu hao lịch sử và truy vết thao tác nhạy cảm ở tầng quản trị.

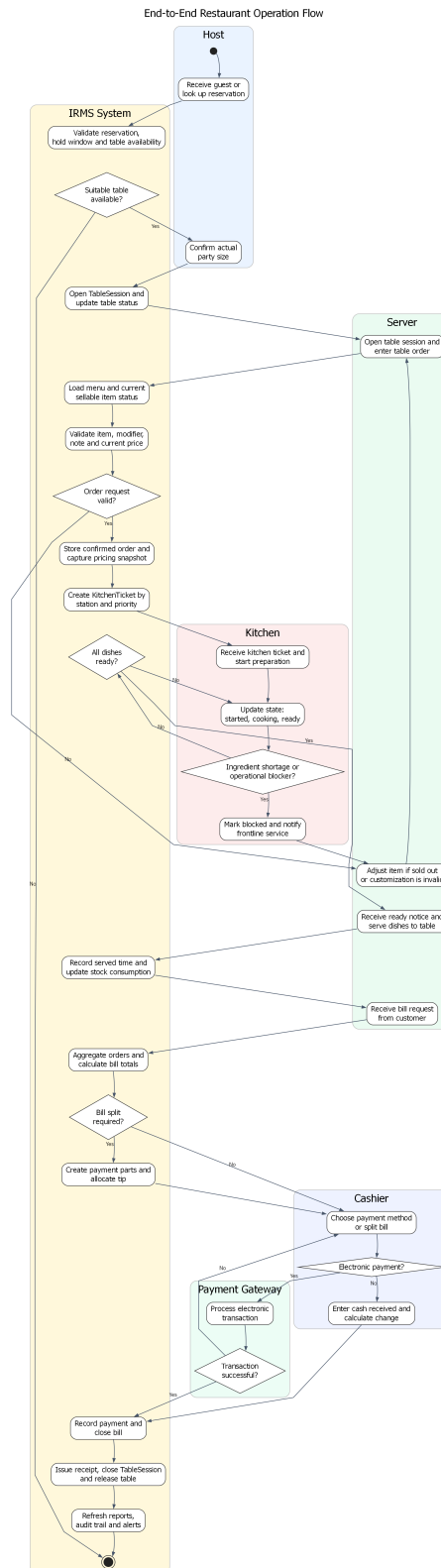
Nhìn theo góc độ nghiệp vụ, năm cụm này cũng tương ứng với năm lớp trách nhiệm khác nhau trong hệ thống. Cụm đặt bàn và bàn ăn quản lý năng lực tiếp nhận khách theo thời gian thực; cụm gọi món và bếp điều khiển luồng thực thi nóng nhất của nhà hàng; cụm hóa đơn và thanh toán chốt quyết định tài chính; cụm tồn kho và dự

bảo hỗ trợ điều hành vật tư; còn cụm quản trị, kiểm toán và phân tích bảo đảm hệ thống có thể được kiểm soát, truy vết và cải tiến sau vận hành. Vì vậy, sơ đồ tổng quan ca sử dụng không chỉ là danh mục chức năng, mà còn là bản đồ chỉ ra nơi nào cần phản hồi tức thời, nơi nào cần độ đúng đắn cao, và nơi nào có thể chấp nhận xử lý sau trên tuyến nền.

Năm gói ca sử dụng chính thể hiện trong hình.

- **Reservation and Seating:** gom toàn bộ vòng đời đặt bàn, check-in, danh sách chờ, giải phóng bàn và thông báo cho khách.
- **Ordering and Kitchen:** gom tuyến gọi món, tùy biến món, điều phối bếp, cập nhật trạng thái chế biến và xử lý ưu tiên trong giờ cao điểm.
- **Billing and Payment:** gom các thao tác tạo hóa đơn, tách hóa đơn, thanh toán, phát hành biên lai và hoàn tiền.
- **Inventory and Reorder:** gom tiêu hao tồn kho, cảnh báo mức tồn thấp và hỗ trợ quyết định nhập hàng.
- **Governance and Analytics:** gom các ca sử dụng quản trị menu, phân quyền, nhật ký kiểm toán, báo cáo và lịch ca.

Việc gom theo gói ở mức tổng quan giúp hình giữ được khả năng đọc, còn phần chi tiết của từng mã ca sử dụng vẫn được triển khai ngay sau đó trong danh mục và các đặc tả cụ thể. Cách trình bày này cũng tạo nhịp chuyển hợp lý sang các góc nhìn logic và mô-đun ở chương kiến trúc, nơi năm gói nghiệp vụ này được ánh xạ thành các miền trách nhiệm ổn định hơn; phần định vị và giải thích mở rộng của sơ đồ tổng quan ca sử dụng được dẫn ngay tới Bảng 20 và Bảng 21.



Hình 3: Sơ đồ hoạt động của luồng vận hành đầu-cuối nhà hàng

Hình 3 mô tả luồng vận hành đầu-cuối tiêu biểu của nhà hàng, bắt đầu từ khâu tiếp nhận khách hoặc kiểm tra đặt bàn, chuyển sang mở phiên bàn, tạo đơn gọi món, gửi phiếu xuống bếp, cập nhật tiến độ chế biến, giao món, lập hóa đơn, xử lý thanh toán và kết thúc bằng giải phóng bàn. Điểm quan trọng của sơ đồ hoạt động này là nó cho thấy những trạng thái nào phải được đồng bộ mạnh giữa các bộ phận: một bàn chỉ được mở một phiên phục vụ, một đơn đã xác nhận phải sinh phiếu bếp, và một hóa đơn chỉ được đóng khi thanh toán hoàn tất hoặc được ghi nhận là thất bại có kiểm soát.

Nhìn dưới góc độ kiến trúc, luồng đầu-cuối này cũng chỉ ra ranh giới giữa **đường giao dịch nóng** và **đường xử lý nền**. Các bước như xác nhận đơn gọi món, cập nhật trạng thái phiếu Ready, ghi nhận thanh toán thành công và đóng TableSession thuộc về tuyến đồng bộ bắt buộc vì chúng tác động trực tiếp đến trải nghiệm khách hàng. Ngược lại, các bước như cập nhật báo cáo, phát cảnh báo mức tồn thấp, gửi thông báo xác nhận hoặc đồng bộ số liệu quản trị có thể được đẩy sang luồng bất đồng bộ để giảm tải cho các điểm chạm thời gian thực.

Những mắt xích kiến trúc được khóa bởi sơ đồ đầu-cuối.

- **Điểm bàn giao giữa các khu vực:** lễ tân mở phiên bản, phục vụ xác nhận đơn, bếp cập nhật trạng thái, thu ngân quyết toán; mỗi điểm bàn giao đều gắn với một thay đổi trạng thái có hậu quả nghiệp vụ.
- **Điểm đồng bộ bắt buộc:** mở TableSession, xác nhận đơn, đánh dấu món sẵn sàng, ghi nhận thanh toán và giải phóng bàn không được nhập nhằng về trạng thái.
- **Điểm bất đồng bộ chấp nhận được:** thông báo, báo cáo và một phần cảnh báo tồn kho được đẩy ra sau mà không làm sai tuyến phục vụ trước mặt khách hàng.
- **Điểm nối sang chương kiến trúc:** hình này là cơ sở để kiểm tra xem các sơ đồ logic, mô-đun, thành phần và triển khai ở chương sau có thật sự bảo vệ được luồng vận hành đầu-cuối hay không.

Bảng 4: Danh mục 25 ca sử dụng chính được đặc tả chi tiết

Nhóm	Mã ca sử dụng
Nhóm đặt bàn và bàn ăn	UC-RES-01, UC-RES-02, UC-RES-03, UC-RES-04, UC-RES-05
Nhóm gọi món và điều phối bếp	UC-ORD-01, UC-ORD-02, UC-ORD-03, UC-KIT-01, UC-KIT-02, UC-KIT-03, UC-ORD-04
Nhóm thanh toán	UC-BIL-01, UC-BIL-02, UC-PAY-01, UC-PAY-02, UC-PAY-03
Nhóm tồn kho, cảnh báo và dự báo	UC-INV-01, UC-INV-02, UC-INV-03
Nhóm quản trị, kiểm toán và phân tích	UC-REP-01, UC-ADM-01, UC-ADM-02, UC-ADM-03, UC-OPS-01

Năm ca sử dụng giữ vai trò khóa kín phạm vi nghiệp vụ ở phần này là UC-RES-04, UC-RES-05, UC-KIT-03, UC-INV-03 và UC-ADM-03. Chúng làm rõ các mắt xích dễ bị xem nhẹ khi mô tả hệ thống nhà hàng ở mức khái quát: kết thúc vòng đời bàn sau thanh toán, giao tiếp chủ động với khách hàng, điều phối ưu tiên trong bếp, hỗ trợ quyết định nhập hàng từ dữ liệu lịch sử và truy vết thao tác nhạy cảm ở tầng quản trị.

2.2 Góc nhìn sử dụng

Góc nhìn sử dụng cho thấy cách các tác nhân tương tác với các dịch vụ qua các kịch bản nghiệp vụ tiêu biểu. Về mặt lý thuyết, góc nhìn sử dụng là công cụ rất tốt để **kiểm chứng kiến trúc bằng kịch bản**. Một phương án phân rã đẹp trên góc nhìn mô-đun vẫn có thể thất bại khi đi qua tình huống thực, nếu chuỗi tương tác quá dài, lỗi khó giải thích hoặc ranh giới giao dịch không rõ. Do đó, bốn kịch bản dưới đây được chọn vì chúng chạm vào bốn luồng cốt lõi nhất của IRMS.

2.2.1 Kịch bản 1: Từ đặt bàn đến tiếp nhận và xếp bàn

1. Khách hàng hoặc lễ tân tạo đặt bàn.
2. Reservation Service kiểm tra khả dụng và gán bàn hoặc giữ chỗ.
3. Khi khách đến, Host UI xác nhận tiếp nhận và mở TableSession.
4. TableSession được chuyển tới Server POS UI để phục vụ gọi món.

2.2.2 Kịch bản 2: Từ gọi món đến điều phối bếp và phục vụ

1. Phục vụ tạo đơn gọi món và cấu hình tùy chọn món hoặc gợi ý dự ứng trên Server POS UI.

2. `Ordering Service` xác nhận đơn gọi món và phát sự kiện tạo phiếu bếp.
3. `Kitchen Coordination Service` tách phiếu theo trạm bếp và hiển thị lên `Kitchen Display UI`.
4. Bếp cập nhật trạng thái từng dòng phiếu; màn hình gọi món nhận lại trạng thái sẵn sàng.
5. Phục vụ giao món và đánh dấu đã phục vụ; `Inventory Service` nhận sự kiện để trừ tồn.

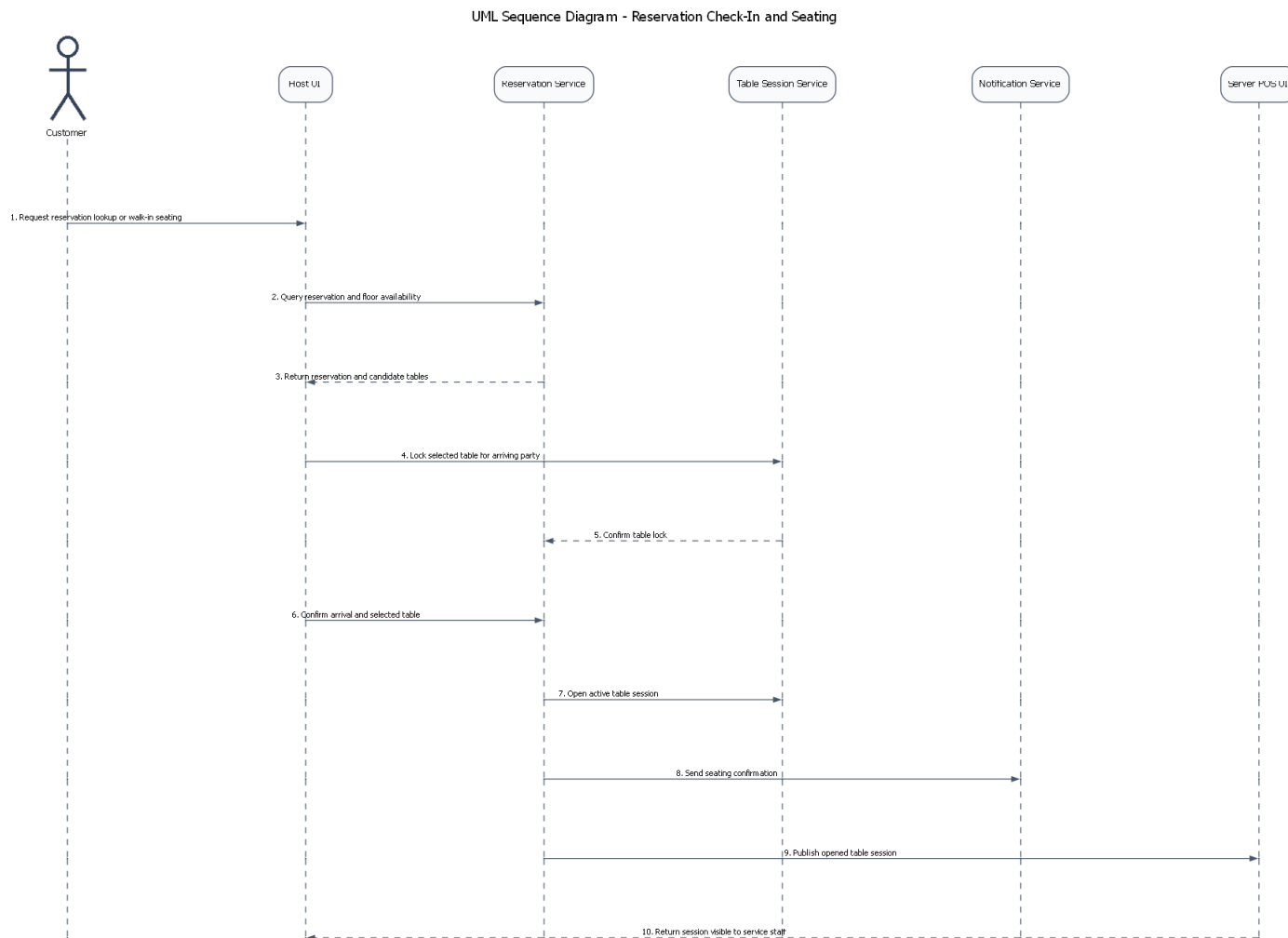
2.2.3 Kịch bản 3: Từ hóa đơn đến thanh toán và biên lai

1. Thu ngân hoặc phục vụ gom các đơn gọi món của bàn thành hóa đơn.
2. `Billing Service` áp dụng khuyến mãi, tiền boa và tách hóa đơn nếu cần.
3. Thanh toán được thực hiện qua tiền mặt hoặc cổng thanh toán điện tử.
4. Khi thanh toán đủ, biên lai được phát hành và `TableSession` chuyển sang chuẩn bị đóng.

2.2.4 Kịch bản 4: Từ cảnh báo mức tồn thấp đến quyết định của quản lý

1. `Inventory Service` cập nhật lượng tồn thực tế sau mỗi món đã phục vụ hoặc sau mỗi lần điều chỉnh thủ công.
2. Nếu xuống dưới ngưỡng, hệ thống tạo sự kiện cảnh báo mức tồn thấp.
3. `Manager Portal` nhận cảnh báo, hiển thị nguyên liệu bị ảnh hưởng và món có nguy cơ hết hàng.
4. Quản lý quyết định nhập thêm hàng hoặc tạm ẩn món trong menu.

Giải thích chi tiết cho góc nhìn sử dụng. Góc nhìn sử dụng đặc biệt quan trọng với IRMS vì hệ thống không phải một khối xử lý tĩnh; nó là một chuỗi phối hợp giữa người và máy theo thời gian thực. Mỗi kịch bản ở trên đều cho thấy ranh giới nào cần đồng bộ mạnh, chẳng hạn như thanh toán, và ranh giới nào chấp nhận xử lý bất đồng bộ, chẳng hạn như báo cáo hoặc cảnh báo mức tồn thấp.



Hình 4: Sơ đồ trình tự cho kịch bản từ đặt bàn đến tiếp nhận và xếp bàn

Hình 4 cho thấy `Reservation` và `TableSession` không phải là cùng một thực thể nghiệp vụ dù chúng liên quan trực tiếp. Điểm mạnh của cách tách này là hệ thống xử lý được nhiều trường hợp thực tế: khách có đặt bàn nhưng không đến, khách vào trực tiếp không có đặt bàn nhưng vẫn mở phiên phục vụ, hoặc lễ tân phải đổi bàn khi năng lực phục vụ thay đổi. Nếu trộn hai khái niệm này thành một, kiến trúc sẽ khó biểu diễn các trạng thái chuyển tiếp quan trọng của vận hành nhà hàng.

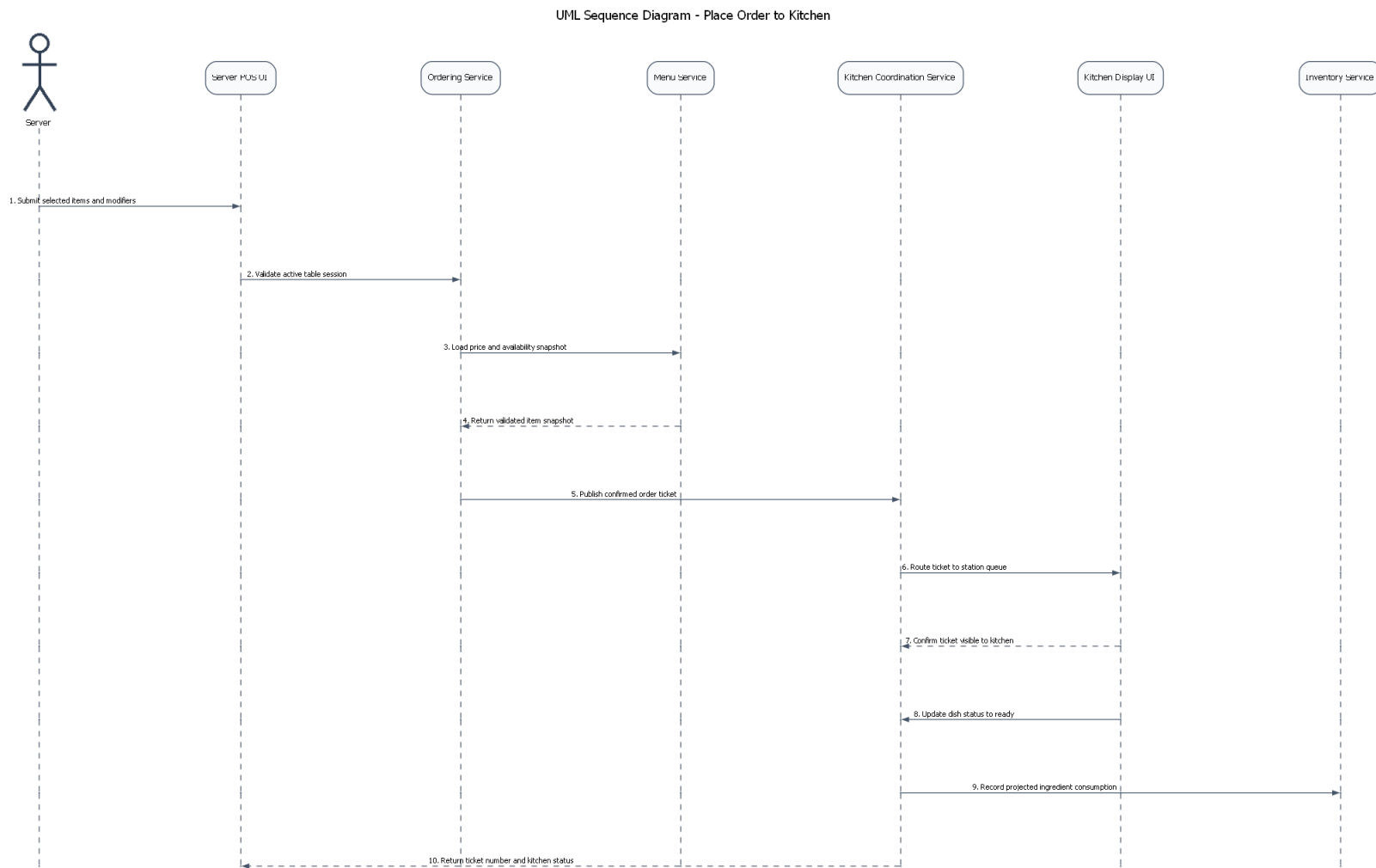
Đánh đổi là luồng xếp bàn phải cẩn thận hơn về đồng bộ trạng thái: đặt bàn có thể còn hiệu lực nhưng bàn ăn chưa sẵn sàng; bàn có thể vừa được giải phóng nhưng vẫn đang dọn dẹp; thao tác tiếp nhận có thể đến muộn so với thời điểm giữ chỗ ban đầu. Do đó, góc nhìn sử dụng này nhắc rằng đặt bàn và xếp bàn tuy nhìn đơn giản, nhưng thực ra là nơi chứa nhiều quy tắc nghiệp vụ về thời gian và năng lực bàn.

Các thành phần chính thể hiện trong hình.

- **Khách hàng và Host UI:** khởi tạo yêu cầu đặt bàn hoặc check-in.
- **Reservation Service:** chịu trách nhiệm xác thực trạng thái đặt bàn và gợi ý bàn.
- **TableSession và Server POS UI:** nhận bàn đã mở để chuyển sang luồng phục vụ thực tế.

Lý do thiết kế của sơ đồ này.

- Sơ đồ làm rõ ranh giới giữa dữ liệu đặt bàn và phiên bàn thực tế.
- Đây là kịch bản giúp chứng minh tại sao bài toán xếp bàn cần được mô hình hóa riêng thay vì gộp vào một trạng thái bàn đơn giản.



Hình 5: Sơ đồ trình tự cho kịch bản từ gọi món đến điều phối bếp và phục vụ

Hình 5 là kịch bản quan trọng nhất để kiểm chứng **khả năng đáp ứng**. Nguyên tắc thiết kế nên là: *lưu đơn gọi món thành công trước, rồi mới lan truyền sang bếp*. Điều này bảo vệ tuyến đầu theo hai cách. Thứ nhất, màn hình gọi món nhận phản hồi nhanh hơn vì không phải đợi toàn bộ chuỗi bếp, tồn kho và báo cáo. Thứ hai, khi việc chuyển sang bếp gặp trục trặc, hệ thống vẫn có thể thử lại từ trạng thái đơn đã xác nhận thay vì mất dữ liệu đơn gọi món.

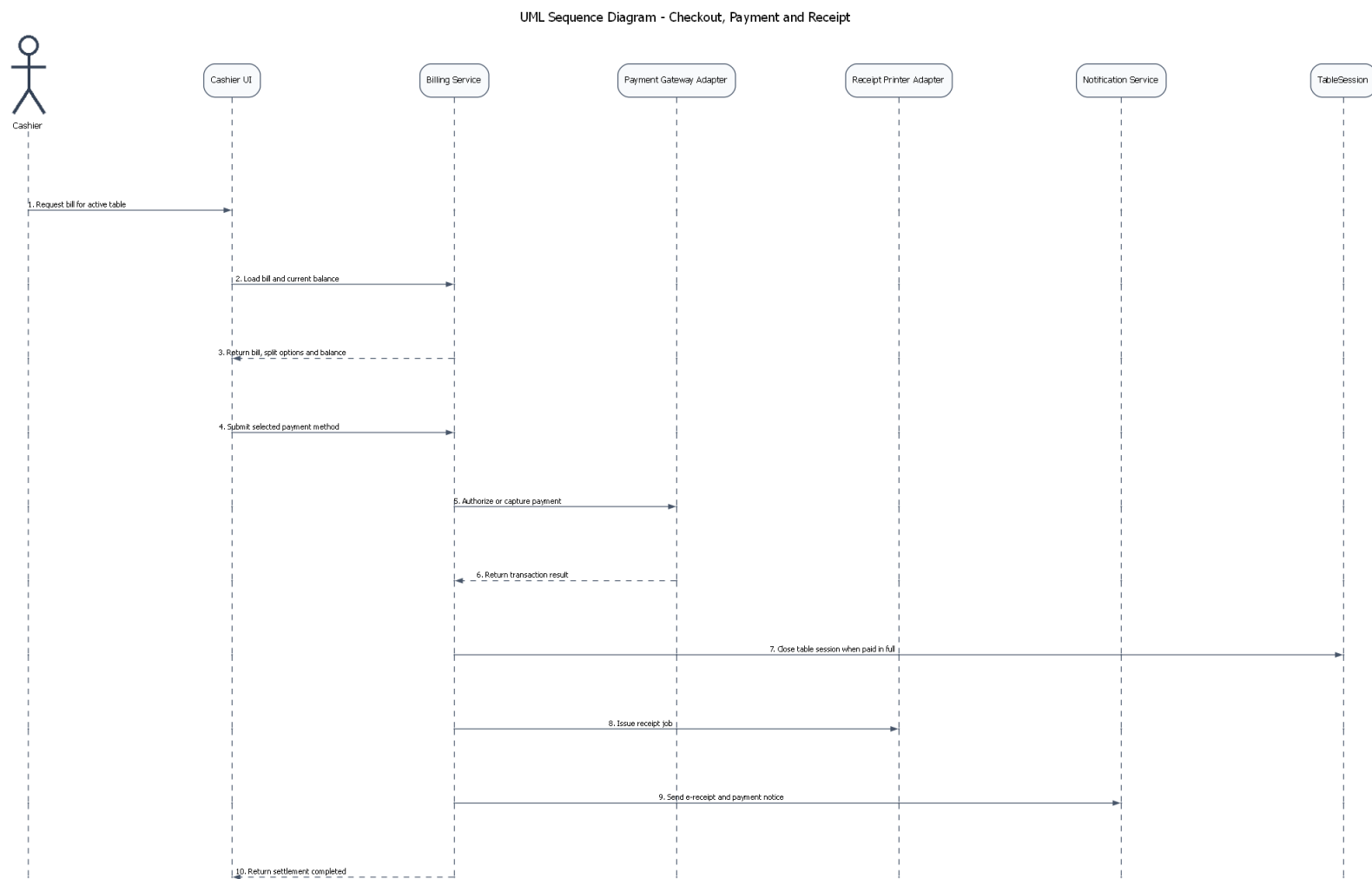
Ưu điểm của mô hình này là tách được **bước chốt giao dịch** khỏi **bước lan truyền vận hành**. Tuy nhiên, đánh đổi là màn hình bếp và màn hình gọi món phải xử lý được trạng thái trung gian: đơn đã xác nhận nhưng phiếu chưa hiện lên màn hình bếp ngay lập tức; một dòng món có thể đã sẵn sàng ở bếp nhưng màn hình gọi món chưa nhận được cập nhật cuối cùng; việc trừ tồn cũng có thể đi sau sự kiện phục vụ xong. Vì thế, góc nhìn sử dụng này là bằng chứng rằng cơ chế nhất quán trễ ở IRMS phải được dùng có kiểm soát, không dùng tùy tiện.

Các thành phần chính thể hiện trong hình.

- **Server POS UI**: là điểm bắt đầu của thao tác gọi món.
- **Ordering Service**: là nơi chốt giao dịch gọi món.
- **Kitchen Coordination Service và Kitchen Display UI**: nhận luồng lan truyền sang bếp sau khi đơn đã được xác nhận.
- **Inventory Service**: chỉ nhận cập nhật sau khi món đã đi qua luồng phục vụ phù hợp.

Lý do thiết kế của sơ đồ này.

- Sơ đồ này nhấn mạnh thứ tự “xác nhận trước, lan truyền sau” của tuyến gọi món.
- Đây là kịch bản quan trọng nhất để giải thích vì sao kiến trúc phải cân bằng giữa phản hồi nhanh và nhất quán có kiểm soát.



Hình 6: Sơ đồ trình tự cho kịch bản từ hóa đơn đến thanh toán và biên lai

Hình 6 là nơi kiến trúc phải nghiêng mạnh về **độ tin cậy**. Ở kịch bản này, khối lập hóa đơn không chỉ làm phép cộng cuối cùng, mà còn là nơi áp dụng khuyến mãi, tiền boa, tách hóa đơn, chọn phương thức thanh toán, giao tiếp với cổng thanh toán, phát hành biên lai và đối trạng thái phiên phục vụ. Đây là luồng có mật độ chuyển trạng thái cao nhất và hệ quả nghiệp vụ lớn nhất nếu xử lý sai.

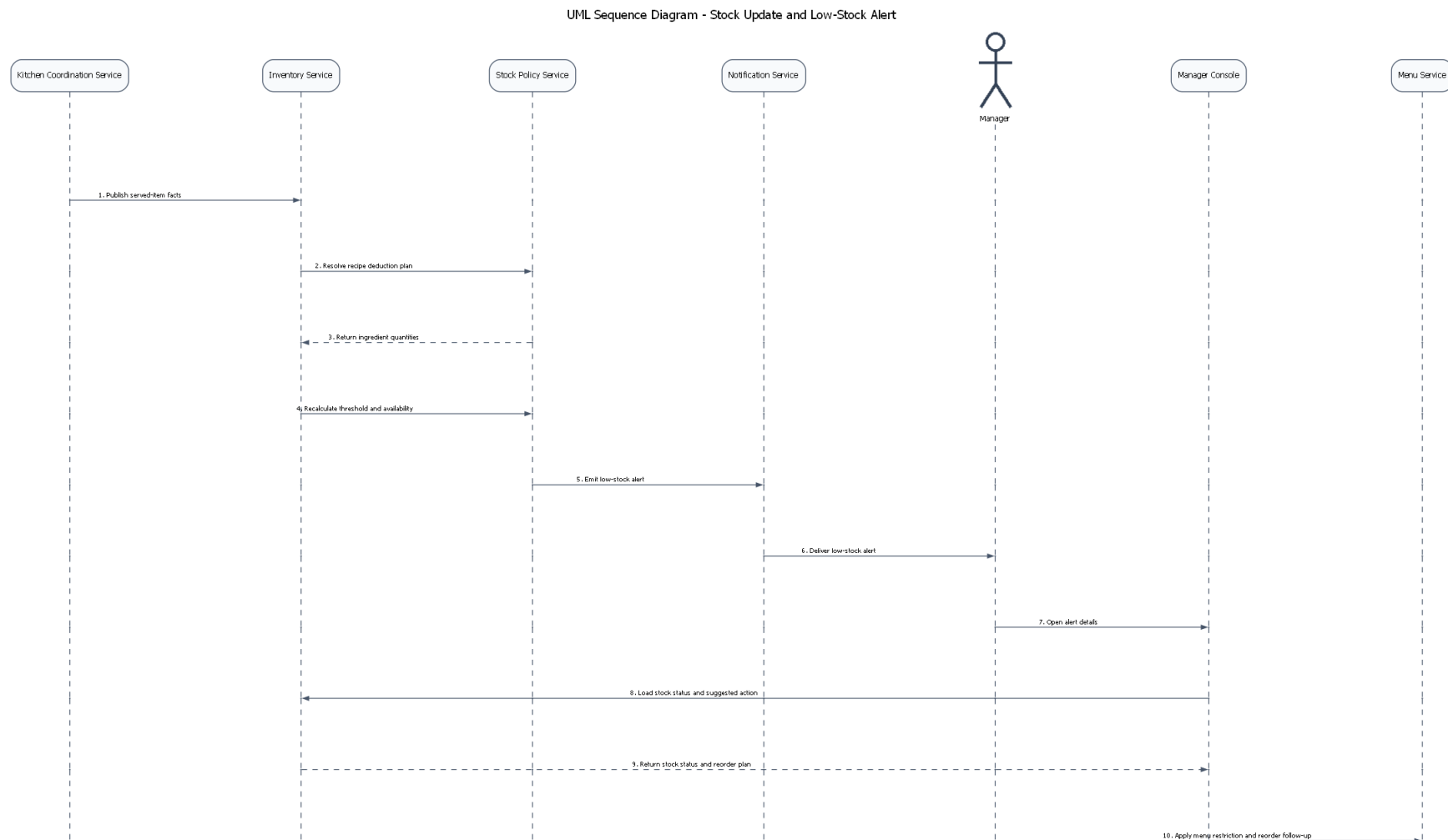
Ưu điểm của việc giữ thanh toán trong luồng đồng bộ là hệ thống có thể trả lời rất rõ: giao dịch thành công hay thất bại, hóa đơn còn mở hay đã đóng, phiên phục vụ còn tiếp diễn hay đã kết thúc. Ngược lại, nếu thanh toán bị đẩy sang bất đồng bộ quá mức, thu ngân và khách hàng sẽ rất khó hiểu trạng thái thật sự của giao dịch. Nhược điểm của cách làm đồng bộ là đường xử lý thanh toán trở thành điểm nhạy cảm với hệ ngoài, đặc biệt khi cổng thanh toán chậm hoặc lỗi. Vì thế, đây là nơi cần thời gian chờ rõ ràng, khóa chống xử lý lặp, nhật ký kiểm toán và cơ chế xử lý lỗi minh bạch nhất.

Các thành phần chính thể hiện trong hình.

- **Cashier UI**: là điểm điều khiển nghiệp vụ của thu ngân.
- **Billing Service**: giữ trung tâm của tuyến quyết toán.
- **Payment Gateway Adapter**: đại diện cho tích hợp thanh toán ngoài hệ thống.
- **Receipt Printer Adapter và TableSession**: kết thúc luồng bằng phát hành biên lai và đóng phiên bàn.

Lý do thiết kế của sơ đồ này.

- Đây là luồng nhạy cảm nhất về độ đúng đắn nghiệp vụ, nên cần được tách riêng để làm rõ ranh giới trách nhiệm và ranh giới giao dịch.
- Việc tách rõ cổng thanh toán, phát hành biên lai và đóng phiên bàn giúp người đọc thấy ranh giới trách nhiệm của từng bước sau khi thanh toán hoàn tất.



Hình 7: Sơ đồ trình tự cho kịch bản từ cảnh báo mức tồn thấp đến quyết định của quản lý

Hình 7 cho thấy rõ vì sao quản lý tồn kho và gửi thông báo không nên chen trực tiếp vào tuyến giao dịch nóng. Việc theo dõi tồn kho cần chính xác ở mức vận hành, nhưng không nhất thiết phải khóa trải nghiệm gọi món của phục vụ chỉ vì một cảnh báo quản lý chưa được gửi xong. Cách đặt việc trừ tồn và cảnh báo mức tồn thấp trên luồng sự kiện giúp hệ thống giữ được **khả năng sửa đổi**: khi chính sách trừ kho, ánh xạ công thức hay ngưỡng cảnh báo thay đổi, phần ảnh hưởng chủ yếu nằm trong khối tồn kho và khối gửi thông báo.

Nhược điểm là lượng tồn hiển thị cho quản lý có thể hơi chậm so với giao dịch mới nhất nếu bộ xử lý nền bị trễ hoặc sự kiện đang được thử lại. Điều này chấp nhận được trong phạm vi IRMS, miễn là không nhầm lẫn cảnh báo tồn kho với xác nhận thanh toán. Nói cách khác, góc nhìn sử dụng này giúp chứng minh nơi nào được phép chấp nhận độ trễ đồng bộ, và nơi nào thì tuyệt đối không nên chấp nhận điều đó.

Các thành phần chính thể hiện trong hình.

- **Ordering Service**: là nguồn phát sinh biến động vận hành cần suy diễn sang kho.
- **Inventory Service và LowStockRule**: chịu trách nhiệm cập nhật tồn và đánh giá ngưỡng cảnh báo.
- **Notification Adapter và Manager UI**: là đầu ra quản trị của luồng cảnh báo.

Lý do thiết kế của sơ đồ này.

- Sơ đồ cho thấy rõ tồn kho và cảnh báo nên đi trên một tuyến hỗ trợ thay vì chen vào đường giao dịch nóng.
- Đây là ví dụ rõ nhất để giải thích sự khác nhau giữa **đúng về vận hành** và **phải phản hồi tức thời** trong kiến trúc.

2.3 Đặc tả ca sử dụng

longtable enumitem

Use Case ID	UC-RES-01
Use Case Name	Tạo đặt bàn
Primary Actors	Khách hàng, Lễ tân
Description	Tạo một đặt bàn hợp lệ cho ngày, giờ và số lượng khách cụ thể.
Trigger	Khách hàng gửi yêu cầu đặt bàn qua quầy lễ tân, điện thoại hoặc web/app.
Preconditions	• Nhà hàng có mở nhận đặt bàn trong khung giờ yêu cầu. • Có tối thiểu tên và số điện thoại khách hàng. • Sơ đồ bàn và năng lực phục vụ đã được cấu hình.
Postconditions	• Tạo đặt bàn với trạng thái <i>Pending</i> hoặc <i>Confirmed</i>. • Hệ thống giữ chỗ hoặc gợi ý bàn phù hợp. • Khách nhận mã đặt bàn và thông báo xác nhận.
Normal Flow	1. Lễ tân nhập thời gian, số khách, tên và số điện thoại. 2. Hệ thống kiểm tra và validate dữ liệu đầu vào. 3. Hệ thống kiểm tra sức chứa, bàn trống và xung đột đặt bàn. 4. Hệ thống gợi ý bàn hoặc nhóm bàn phù hợp. 5. Lễ tân chọn bàn và nhập ghi chú nếu có. 6. Hệ thống tạo đặt bàn và sinh mã đặt bàn. 7. Hệ thống gửi thông báo xác nhận cho khách.

Alternative Flows	A1. Không đủ sức chứa (từ bước 3) 3.1 Hệ thống đề xuất khung giờ khác hoặc đưa khách vào danh sách chờ. A2. Yêu cầu đặc biệt (từ bước 4) 4.1 Lễ tân lọc danh sách bàn theo khu vực hoặc thuộc tính yêu cầu. A3. Không gán được bàn (từ bước 5) 5.1 Hệ thống tạo đặt bàn trạng thái Pending và giữ chỗ theo số lượng khách.
Exception Flows	E1. Dữ liệu không hợp lệ (từ bước 2) 2.1 Hệ thống yêu cầu nhập lại thông tin khách. E2. Lỗi đồng bộ sơ đồ bàn (từ bước 3) 3.2 Hệ thống khóa thao tác và yêu cầu tải lại dữ liệu.
Business Rules	- Không vượt quá sức chứa tối đa của bàn hoặc nhóm bàn. - Nhóm khách lớn phải được xác nhận bởi lễ tân hoặc quản lý.

Use Case ID	UC-RES-02
Use Case Name	Nhận khách và sắp bàn
Primary Actors	Lễ tân, Phục vụ
Description	Tiếp nhận khách đến nhà hàng và gán khách vào bàn hoặc nhóm bàn phù hợp.
Trigger	Khách đến quầy lễ tân hoặc được gọi từ danh sách chờ.
Preconditions	- Khách có đặt bàn hoặc có thể được xếp chỗ tại chỗ. - Sơ đồ bàn và trạng thái bàn được đồng bộ theo thời gian thực.
Postconditions	- Một phiên phục vụ bàn (TableSession) được mở. - Bàn được chuyển sang trạng thái Occupied hoặc Reserved-Arrived. - Phục vụ nhận được thông báo tiếp nhận bàn mới.
Normal Flow	- Lễ tân tìm đặt bàn theo mã, số điện thoại hoặc tên khách. - Hệ thống hiển thị thông tin đặt bàn, trạng thái, thời điểm đến và bàn gợi ý. - Lễ tân xác nhận khách đến, số khách thực tế và thời gian check-in. - Hệ thống kiểm tra hiệu lực đặt bàn và trạng thái bàn. - Lễ tân điều chỉnh hoặc ghép bàn nếu số lượng khách thay đổi. - Hệ thống mở phiên phục vụ (TableSession). - Hệ thống cập nhật trạng thái bàn thành Occupied hoặc Reserved-Arrived. - Hệ thống phân công bàn cho phục vụ theo khu vực. - Phục vụ nhận thông báo và chuẩn bị phục vụ.

Alternative Flows	A1. Khách vắng lại (từ bước 1) 1.1 Lễ tân chọn chức năng xếp bàn trực tiếp. 1.2 Hệ thống gợi ý bàn trống phù hợp. 1.3 Lễ tân chọn bàn và tiếp tục từ bước 5. A2. Thay đổi số lượng khách (từ bước 3) 3.1 Lễ tân cập nhật số khách thực tế. 3.2 Hệ thống gợi ý lại bàn phù hợp hoặc yêu cầu ghép bàn.
Exception Flows	E1. Bàn chưa sẵn sàng (từ bước 4) 4.1 Hệ thống đề xuất bàn thay thế hoặc đưa khách vào danh sách chờ. E2. Đặt bàn hết hiệu lực (từ bước 4) 4.2 Hệ thống yêu cầu xác nhận phục hồi từ lễ tân hoặc quản lý. E3. Lỗi đồng bộ sơ đồ bàn (từ bước 4) 4.3 Hệ thống tạm dừng thao tác và yêu cầu tải lại dữ liệu.
Business Rules	- Mỗi bàn chỉ có một <code>TableSession</code> hoạt động tại một thời điểm. - Không được gán bàn nếu bàn đang được sử dụng. - Thời gian giữ chỗ tối đa được cấu hình theo ca phục vụ.

Use Case ID	UC-RES-03
Use Case Name	Quản lý danh sách chờ
Primary Actors	Lễ tân
Description	Ghi nhận khách chờ bàn và điều phối khách khi có bàn phù hợp.
Trigger	Nhà hàng hết bàn phù hợp hoặc khách bị trì hoãn.
Preconditions	- Lễ tân có quyền truy cập danh sách chờ. - Hệ thống có khả năng ước tính thời gian chờ.
Postconditions	- Khách được thêm vào danh sách chờ với trạng thái <code>Waiting</code>. - Khách được gọi theo thứ tự ưu tiên khi có bàn phù hợp.
Normal Flow	- Lễ tân nhập thông tin khách, số lượng khách và kênh liên hệ. - Hệ thống ước tính thời gian chờ dựa trên tình trạng bàn. - Lễ tân xác nhận thêm khách vào danh sách chờ. - Khi có bàn trống, hệ thống tìm khách phù hợp theo sức chứa và ưu tiên. - Lễ tân chọn khách để gọi. - Hệ thống gửi thông báo và cập nhật trạng thái <code>Called</code>. - Lễ tân xác nhận khách đến và chuyển sang use case Nhận khách và sắp bàn.
Alternative Flows	A1. Ưu tiên khách (từ bước 4) 4.1 Lễ tân chọn khách có ưu tiên cao hơn (VIP, đặt trước) thay vì theo FIFO. A2. Cập nhật thông tin (từ bước 3) 3.1 Lễ tân cập nhật số điện thoại, số lượng khách hoặc hủy yêu cầu.

Exception Flows	E1. Khách không phản hồi (từ bước 6) 6.1 Nếu khách không phản hồi trong thời gian quy định, hệ thống chuyển trạng thái sang Skipped. E2. Thời gian chờ thay đổi (từ bước 2) 2.1 Nếu thời gian chờ tăng đột biến, hệ thống yêu cầu lễ tân xác nhận lại trước khi hiển thị.
Business Rules	- Thứ tự mặc định là FIFO trong cùng nhóm sức chứa. - Có thể override thứ tự theo ưu tiên. - Chỉ gọi khách khi có bàn phù hợp với số lượng và yêu cầu. - Thời gian giữ lượt gọi được cấu hình (timeout).

Use Case ID	UC-RES-04
Use Case Name	Cập nhật trạng thái bàn và giải phóng bàn
Primary Actors	Lễ tân, Phục vụ, Thu ngân
Description	Cập nhật trạng thái bàn, đóng phiên phục vụ và giải phóng bàn cho lượt khách tiếp theo.
Trigger	Khách rời bàn, hoàn tất phục vụ hoặc cần thay đổi trạng thái bàn.
Preconditions	- Bàn tồn tại trong hệ thống. - Người thao tác có quyền cập nhật trạng thái. - Nếu có <code>TableSession</code>, hệ thống truy xuất được trạng thái hóa đơn.
Postconditions	- Trạng thái bàn được cập nhật nhất quán. <code>TableSession</code> được đóng nếu đủ điều kiện. - Lịch sử thay đổi được ghi nhận.
Normal Flow	- Người dùng chọn bàn trên sơ đồ bàn. - Hệ thống hiển thị trạng thái bàn, phiên phục vụ và hóa đơn liên quan. - Người dùng chọn trạng thái đích (Cleaning, Available, OutOfService). - Nếu chuyển sang trạng thái giải phóng, hệ thống kiểm tra hóa đơn hoặc split còn mở. - Nếu tất cả hóa đơn đã hoàn tất, hệ thống đóng <code>TableSession</code> và chuyển bàn sang Cleaning. - Phục vụ xác nhận đã dọn bàn xong. - Hệ thống chuyển bàn sang trạng thái Available. - Hệ thống cập nhật danh sách chờ và gợi ý khách tiếp theo nếu có.
Alternative Flows	A1. Chuyển sang OutOfService (từ bước 3) 3.1 Người dùng chuyển bàn sang trạng thái OutOfService để bảo trì hoặc khóa khu vực. A2. Chuyển bàn trong khi phục vụ (từ bước 3) 3.2 Nếu khách đổi bàn, hệ thống cập nhật bàn mới nhưng giữ nguyên <code>TableSession</code>.

Exception Flows	E1. Chưa thanh toán (từ bước 4) 4.1 Hệ thống từ chối giải phóng bàn và yêu cầu hoàn tất thanh toán. E2. Xung đột cập nhật (từ bước 2) 2.1 Hệ thống phát hiện cập nhật đồng thời và yêu cầu tải lại dữ liệu. E3. Không đủ quyền (từ bước 3) 3.3 Hệ thống chặn thao tác nếu người dùng không có quyền.
Business Rules	- Chỉ chuyển sang Available khi không còn TableSession và hóa đơn mở. - Mọi thay đổi trạng thái phải được ghi log (thời gian, người thực hiện). - Không được giải phóng bàn khi còn công nợ.

Use Case ID	UC-RES-05
Use Case Name	Gửi thông báo cho khách hàng
Primary Actors	Hệ thống, Lễ tân
Description	Gửi thông báo cho khách hàng khi có sự kiện như đặt bàn, thay đổi hoặc đến lượt chờ.
Trigger	Sự kiện nghiệp vụ phát sinh hoặc lễ tân kích hoạt gửi thủ công.
Preconditions	- Có thông tin liên hệ hợp lệ. - Mẫu thông báo và kênh gửi đã được cấu hình. - Hệ thống kết nối được dịch vụ gửi thông báo.
Postconditions	- Tạo bản ghi NotificationMessage. - Thông báo được gửi hoặc đưa vào hàng chờ. - Lịch sử gửi được lưu lại.
Normal Flow	- Hệ thống nhận sự kiện cần gửi thông báo. - Hệ thống xác định loại thông báo và chọn mẫu nội dung phù hợp. - Hệ thống kiểm tra thông tin liên hệ và kênh gửi ưu tiên. - Hệ thống cá nhân hóa nội dung thông báo. - Hệ thống tạo NotificationMessage và gửi qua kênh đã chọn. - Hệ thống ghi nhận kết quả và cập nhật trạng thái gửi.
Alternative Flows	A1. Gửi thủ công (từ bước 1) 1.1 Lễ tân kích hoạt gửi lại thông báo cho khách. A2. Chuyển kênh gửi (từ bước 3) 3.1 Nếu kênh chính không khả dụng, hệ thống chuyển sang kênh dự phòng.
Exception Flows	E1. Thiếu thông tin liên hệ (từ bước 3) 3.2 Hệ thống đánh dấu cần xử lý thủ công. E2. Lỗi dịch vụ gửi (từ bước 5) 5.1 Hệ thống đưa thông báo vào hàng chờ retry. E3. Bị chặn spam (từ bước 5) 5.2 Hệ thống bỏ qua gửi trùng và ghi nhận lý do.

Business Rules	- Mỗi thông báo phải liên kết với Reservation hoặc WaitlistEntry. - Thông báo có thời hạn phải kèm thời gian phản hồi rõ ràng. - Không gửi trùng thông báo trong khoảng thời gian cấu hình.
-----------------------	---

Use Case ID	UC-ORD-01
Use Case Name	Tạo đơn gọi món tại bàn
Primary Actors	Phục vụ
Description	Tạo đơn gọi món cho một bàn đang phục vụ, bao gồm món, số lượng và ghi chú.
Trigger	Phục vụ chọn chức năng tạo đơn gọi món trên bàn đang hoạt động.
Preconditions	- TableSession đang hoạt động. - Phục vụ có quyền thao tác trên khu vực.
Postconditions	- Đơn gọi món được lưu với trạng thái Draft hoặc Confirmed. - Các dòng món được liên kết với phiên phục vụ.
Normal Flow	- Phục vụ mở bàn và chọn chức năng tạo đơn gọi món. - Hệ thống kiểm tra trạng thái TableSession. - Hệ thống hiển thị thực đơn và danh sách món. - Phục vụ chọn món, số lượng và ghi chú cho từng dòng món. - Hệ thống tính tạm tính theo giá hiện hành. - Phục vụ lưu đơn ở trạng thái Draft hoặc xác nhận gửi bếp. - Hệ thống lưu đơn gọi món và cập nhật trạng thái.
Alternative Flows	A1. Tạo nhiều đơn (từ bước 1) 1.1 Phục vụ tạo nhiều đơn gọi món liên tiếp trong cùng một phiên phục vụ. A2. Dừng mẫu đơn (từ bước 3) 3.1 Phục vụ chọn mẫu đơn có sẵn cho bàn đoàn hoặc combo.
Exception Flows	E1. Món hết hàng (từ bước 4) 4.1 Hệ thống từ chối thêm món và yêu cầu chọn món thay thế. E2. Phiên bàn không hợp lệ (từ bước 2) 2.1 Hệ thống từ chối tạo đơn nếu TableSession đã đóng.
Business Rules	- Mỗi đơn phải thuộc một TableSession. - Giá món được cố định tại thời điểm xác nhận đơn. - Một bàn có thể có nhiều đơn trong cùng phiên phục vụ.

Use Case ID	UC-ORD-02
Use Case Name	Tùy biến món và ghi chú dị ứng
Primary Actors	Phục vụ, Bếp
Description	Ghi nhận tùy chọn món, mức chế biến và ghi chú dị ứng cho từng dòng món.

Trigger	Phục vụ chọn chỉnh sửa chi tiết một dòng món trong đơn gọi món.
Preconditions	- Món hỗ trợ tùy chọn hoặc ghi chú. - Đơn gọi món ở trạng thái <code>Draft</code> hoặc chưa gửi bếp. - Cấu hình modifier đã được thiết lập.
Postconditions	- Tùy chọn được lưu dưới dạng <code>OrderItemModifier</code>. - Ghi chú dị ứng được gắn vào dòng món.
Normal Flow	- Phục vụ mở chi tiết dòng món. - Hệ thống hiển thị các nhóm tùy chọn và giới hạn chọn. - Phục vụ chọn các tùy biến cho món. - Phục vụ nhập ghi chú dị ứng hoặc yêu cầu đặc biệt. - Hệ thống kiểm tra quy tắc chọn (min/max). - Hệ thống lưu cấu hình tùy chọn vào dòng món. - Khi đơn được gửi bếp, thông tin tùy biến và ghi chú được hiển thị.
Alternative Flows	A1. Dừng cấu hình mẫu (từ bước 2) 2.1 Phục vụ chọn nhanh cấu hình có sẵn cho món. A2. Gắn cờ dị ứng (từ bước 4) 4.1 Hệ thống tự động đánh dấu nổi bật các ghi chú dị ứng quan trọng.
Exception Flows	E1. Vi phạm quy tắc chọn (từ bước 5) 5.1 Hệ thống yêu cầu điều chỉnh số lượng lựa chọn. E2. Ghi chú quá dài (từ bước 4) 4.2 Hệ thống yêu cầu rút gọn hoặc chuyển sang ghi chú nội bộ. E3. Đơn đã gửi bếp (từ bước 1) 1.1 Hệ thống từ chối chỉnh sửa nếu đơn đã được xác nhận.
Business Rules	- Ghi chú dị ứng phải hiển thị rõ ràng trên phiếu bếp. - Modifier có thể thay đổi giá món. - Không cho phép chỉnh sửa sau khi gửi bếp.

Use Case ID	UC-ORD-03
Use Case Name	Gửi đơn gọi món sang bếp
Primary Actors	Phục vụ, Bếp
Description	Chuyển đơn gọi món đã xác nhận đến các trạm bếp để chế biến.
Trigger	Phục vụ nhấn gửi bếp hoặc hệ thống tự động gửi.
Preconditions	- Đơn có ít nhất một dòng món hợp lệ. - Đơn ở trạng thái <code>Draft</code>. - Kết nối tới hệ thống bếp khả dụng.
Postconditions	- Tạo một hoặc nhiều <code>KitchenTicket</code>. - Đơn chuyển sang trạng thái <code>Confirmed/Queued</code>.

Normal Flow	1. Phục vụ xác nhận các dòng món cần gửi. 2. Hệ thống kiểm tra trạng thái món, modifier và tồn kho logic. 3. Hệ thống xác định trạm bếp cho từng dòng món. 4. Hệ thống tách đơn thành các <code>KitchenTicket</code> theo trạm bếp. 5. Hệ thống xếp hàng phiếu bếp theo ưu tiên và thời gian. 6. Hệ thống cập nhật trạng thái đơn thành <code>Confirmed/Queued</code>. 7. Màn hình bếp hiển thị phiếu và phát tín hiệu thông báo.
Alternative Flows	A1. Gửi món sau (từ bước 1) 1.1 Phục vụ đánh dấu một số món để gửi sau. A2. Gộp phiếu bếp (từ bước 4) 4.1 Hệ thống gộp các phiếu bếp gần nhau theo cấu hình thời gian.
Exception Flows	E1. Không có trạm bếp (từ bước 3) 3.1 Hệ thống chặn gửi và cảnh báo lỗi cấu hình menu. E2. Mất kết nối bếp (từ bước 5) 5.1 Hệ thống đưa phiếu vào hàng đợi và retry sau.
Business Rules	• Mỗi dòng món phải được định tuyến đến ít nhất một trạm bếp. • Chỉ dòng món hợp lệ mới được gửi. • Không cho phép gửi lại các dòng đã được gửi trước đó.

Use Case ID	UC-KIT-01
Use Case Name	Cập nhật tiến độ chế biến
Primary Actors	Bếp
Description	Theo dõi và cập nhật trạng thái món từ khi nhận phiếu bếp đến khi sẵn sàng phục vụ.
Trigger	Nhân viên bếp chọn một phiếu bếp trên màn hình KDS.
Preconditions	• <code>KitchenTicket</code> đã được tạo và gán trạm bếp. • Nhân viên đã đăng nhập vào hệ thống bếp.
Postconditions	• Trạng thái món và phiếu bếp được cập nhật. • POS nhận được tiến độ mới nhất.
Normal Flow	1. Nhân viên bếp mở phiếu bếp ở trạng thái <code>Queued</code>. 2. Nhân viên cập nhật trạng thái từng món (<code>Started</code>, <code>Cooking</code>, <code>Ready</code>). 3. Hệ thống ghi nhận thời gian cho từng trạng thái. 4. Hệ thống so sánh thời gian chế biến với SLA. 5. Khi tất cả món hoàn tất, hệ thống chuyển phiếu sang <code>Ready</code>. 6. Hệ thống gửi thông báo đến POS để phục vụ ra món.
Alternative Flows	A1. Ưu tiên phiếu (từ bước 1) 1.1 Nhân viên bếp ưu tiên xử lý phiếu VIP hoặc món cần ra trước. A2. Giữ món (từ bước 2) 2.1 Nhân viên đánh dấu <code>Hold for service</code> để chờ đồng bộ với món khác.

Exception Flows	E1. Thiếu nguyên liệu (từ bước 2) 2.2 Nhân viên đánh dấu Blocked và gửi yêu cầu thay món. E2. Mất kết nối KDS (từ bước 3) 3.1 Hệ thống chuyển sang hàng đợi cục bộ và đồng bộ lại sau.
Business Rules	- Chỉ trạm bếp được phân công mới được cập nhật trạng thái. - Mọi thay đổi trạng thái phải được ghi log. - Trạng thái phiếu phụ thuộc vào trạng thái tất cả món.

Use Case ID	UC-KIT-02
Use Case Name	Ra món cho bàn
Primary Actors	Phục vụ
Description	Nhận món từ bếp và phục vụ đúng bàn, đúng thứ tự.
Trigger	Hệ thống thông báo món hoặc phiếu bếp ở trạng thái Ready.
Preconditions	- Món ở trạng thái Ready. - TableSession còn hoạt động.
Postconditions	- Dòng món được cập nhật trạng thái Served. - Thời gian phục vụ được ghi nhận.
Normal Flow	- Phục vụ mở danh sách món sẵn sàng. - Phục vụ xác nhận nhận món từ bếp. - Hệ thống hiển thị thông tin bàn, vị trí ngồi và ghi chú. - Phục vụ giao món cho khách. - Phục vụ xác nhận “Đã phục vụ”. - Hệ thống cập nhật trạng thái từng dòng món thành Served. - Nếu tất cả món đã phục vụ, hệ thống cập nhật trạng thái phục vụ của bàn.
Alternative Flows	A1. Xác nhận theo phiếu (từ bước 1) 1.1 Phục vụ xác nhận toàn bộ phiếu thay vì từng món. A2. Giữ món theo course (từ bước 3) 3.1 Phục vụ trì hoãn ra món để đồng bộ theo course.
Exception Flows	E1. Sai món hoặc sai tùy biến (từ bước 3) 3.2 Phục vụ trả món về bếp và ghi lý do. E2. Bàn tạm vắng (từ bước 4) 4.1 Phục vụ đánh dấu trì hoãn phục vụ. E3. Trạng thái không hợp lệ (từ bước 5) 5.1 Hệ thống từ chối nếu món chưa ở trạng thái Ready.
Business Rules	- Chỉ món ở trạng thái Ready mới được phục vụ. - Mọi thao tác trả món phải có lý do. - Trạng thái phục vụ phải được ghi log để phân tích SLA.

Use Case ID	UC-KIT-03
--------------------	-----------

Use Case Name	Ưu tiên hàng chờ bếp và xử lý món gấp
Primary Actors	Bếp, Quản lý
Description	Điều chỉnh mức ưu tiên của ticket hoặc món để xử lý các trường hợp gấp và giảm trễ hạn.
Trigger	Ticket gần vượt SLA, khách VIP hoặc yêu cầu can thiệp từ quản lý.
Preconditions	• KitchenTicket hoặc item tồn tại trong hàng chờ. • Có dữ liệu SLA và thứ tự hàng chờ. • Người thao tác có quyền thay đổi ưu tiên.
Postconditions	• Mức ưu tiên được cập nhật. • Hàng chờ được sắp xếp lại. • Lý do thay đổi được ghi log.
Normal Flow	1. Nhân viên mở hàng chờ của trạm bếp. 2. Hệ thống hiển thị độ trễ và các ticket có nguy cơ trễ. 3. Người thao tác chọn ticket hoặc món cần ưu tiên. 4. Hệ thống áp dụng quy tắc ưu tiên (SLA, VIP, course). 5. Người thao tác xác nhận hoặc nhập lý do ưu tiên. 6. Hệ thống cập nhật mức ưu tiên. 7. Hệ thống sắp xếp lại hàng chờ. 8. Hệ thống làm nổi bật ticket ưu tiên trên KDS.
Alternative Flows	A1. Tự động ưu tiên (từ bước 2) 2.1 Hệ thống tự động tăng ưu tiên nếu ticket gần vượt SLA. A2. Ưu tiên theo món (từ bước 3) 3.1 Người thao tác chỉ ưu tiên một phần của ticket (ví dụ món khai vị).
Exception Flows	E1. Không đủ quyền (từ bước 3) 3.2 Hệ thống yêu cầu xác nhận từ quản lý. E2. Quá tải trạm bếp (từ bước 2) 2.2 Hệ thống cảnh báo và gợi ý điều phối sang trạm khác. E3. Thiếu dữ liệu SLA (từ bước 4) 4.1 Hệ thống chỉ cho phép ưu tiên thủ công.
Business Rules	• Mọi hành động ưu tiên phải có lý do. • Không được gây trì hoãn vô hạn cho ticket khác (anti-starvation). • Ưu tiên có thể áp dụng ở mức ticket hoặc item.

Use Case ID	UC-ORD-04
Use Case Name	Sửa hoặc hủy dòng món
Primary Actors	Phục vụ, Quản lý
Description	Điều chỉnh hoặc hủy dòng món trong đơn gọi món theo trạng thái và quyền hạn.
Trigger	Phục vụ chọn dòng món để sửa hoặc hủy.

Preconditions	- Đơn chưa hoàn tất thanh toán. - Người dùng có quyền chỉnh sửa theo trạng thái món.
Postconditions	- Dòng món được cập nhật hoặc chuyển trạng thái Voided. - Lịch sử thay đổi được ghi nhận. - Nếu đã gửi bếp, hệ thống phát thông báo đến KDS.
Normal Flow	- Phục vụ chọn dòng món cần chỉnh sửa. - Hệ thống hiển thị trạng thái món và quyền chỉnh sửa. - Phục vụ chọn sửa thông tin hoặc hủy dòng món. - Nếu món đã gửi bếp, hệ thống yêu cầu nhập lý do. - Nếu cần, hệ thống yêu cầu phê duyệt từ quản lý. - Hệ thống cập nhật dữ liệu dòng món. - Hệ thống phát sự kiện đến KDS và cập nhật tạm tính.
Alternative Flows	A1. Chỉnh sửa trước khi gửi bếp (từ bước 3) 3.1 Phục vụ chỉnh sửa tự do khi món ở trạng thái Draft. A2. Hủy sau khi chế biến (từ bước 4) 4.1 Hệ thống yêu cầu phê duyệt bắt buộc khi món đã ở trạng thái Cooking hoặc Ready.
Exception Flows	E1. Không đủ quyền (từ bước 5) 5.1 Hệ thống từ chối thao tác nếu không có quyền. E2. Xung đột cập nhật (từ bước 2) 2.1 Hệ thống phát hiện chỉnh sửa đồng thời và yêu cầu tải lại.
Business Rules	- Hủy món sau khi gửi bếp phải có lý do. - Có thể yêu cầu phê duyệt nếu vượt ngưỡng cấu hình. - Không được mất lịch sử trạng thái và giá ban đầu.

UC-BIL-01 — Tạo hóa đơn cho bàn

Tác nhân chính: Phục vụ, Thu ngân

Mục tiêu: Tổng hợp các đơn gọi món của bàn thành hóa đơn có thể thanh toán.

Kích hoạt: Khách yêu cầu tính tiền hoặc phục vụ chọn “Tạo hóa đơn”.

Tiền điều kiện

- Phiên bản đang mở và có ít nhất một đơn gọi món hợp lệ.
- Các món cần tính tiền đã ở trạng thái cho phép xuất hóa đơn.

Hậu điều kiện

- Một hóa đơn được tạo với tổng tiền, phí, giảm giá và trạng thái thanh toán ban đầu.
- Bàn có thể được chuyển sang trạng thái chờ thanh toán.

Luồng chính

- Phục vụ chọn bàn cần tính tiền.
- Hệ thống gom toàn bộ đơn gọi món chưa được quyết toán của bàn.
- Hệ thống tính subtotal, phụ phí, thuế, khuyến mãi và tổng thanh toán.
- Phục vụ/thu ngân xem trước hóa đơn và xác nhận phát hành.
- Hệ thống tạo bản ghi Bill và danh sách phân bổ thanh toán mặc định.

Luồng thay thế

- Nhà hàng có thể tạo hóa đơn tạm thời để khách kiểm tra trước khi thanh toán.
- Có thể tạo nhiều hóa đơn con nếu bàn yêu cầu tách theo ghế hoặc theo nhóm món.

Ngoại lệ

- Một đơn gọi món vừa bị hủy hoặc điều chỉnh trong lúc tạo hóa đơn: hệ thống yêu cầu tính lại.
- Chưa có cấu hình thuế hoặc phí cho chi nhánh: hệ thống chặn phát hành hóa đơn.

Quy tắc nghiệp vụ

- Một đơn gọi món không được gắn vào hơn một hóa đơn đang mở cùng lúc.
- Công thức tính phí và thuế phải được chụp lại tại thời điểm tạo hóa đơn.

UC-BIL-02 — Tách hóa đơn và thêm tiền boa

Tác nhân chính: Thu ngân, Phục vụ

Mục tiêu: Chia hóa đơn thành nhiều phần thanh toán và ghi nhận tiền boa.

Kích hoạt: Khách yêu cầu thanh toán riêng theo đầu người, theo món hoặc theo số tiền.

Tiền điều kiện

- Hóa đơn đã được tạo.
- Hóa đơn chưa đóng hoàn toàn.

Hậu điều kiện

- Các phần thanh toán (BillSplit) được tạo và sẵn sàng nhận giao dịch thanh toán.
- Tiền boa được phân bổ theo chính sách nhà hàng.

Luồng chính

- Thu ngân mở hóa đơn cần tách.
- Hệ thống hiển thị các món, số ghế, subtotal và các quy tắc chia hóa đơn.
- Thu ngân chọn kiểu chia: đều, theo món, theo ghế, theo số tiền hoặc kết hợp.
- Thu ngân nhập tiền boa hoặc chọn mức tiền boa gợi ý.
- Hệ thống tính lại số tiền của từng phần thanh toán và hiển thị chênh lệch nếu có.
- Thu ngân xác nhận cấu hình tách hóa đơn.

Luồng thay thế

- Khách có thể chỉ tách một phần món ra thanh toán trước, phần còn lại giữ nguyên hóa đơn gốc.
- Tiền boa có thể áp dụng cho toàn bộ hóa đơn hoặc gắn riêng từng split.

Ngoại lệ

- Tổng các split không khớp tổng hóa đơn: hệ thống từ chối lưu và yêu cầu cân bằng lại.
- Một split đã thanh toán không được phép thay đổi phạm vi món trừ khi hoàn tiền/phê duyệt.

Quy tắc nghiệp vụ

- Tổng các split cộng tiền boa phải khớp với công thức tiền của hóa đơn.
- Tiền boa có thể bị giới hạn tối đa theo chính sách địa phương hoặc hệ thống POS.

UC-PAY-01 — Xử lý thanh toán

Tác nhân chính: Thu ngân

Mục tiêu: Thu tiền cho một hóa đơn hoặc một split bằng tiền mặt, thẻ hoặc ví điện tử.

Kích hoạt: Thu ngân chọn “Thanh toán” trên hóa đơn hoặc split cụ thể.

Tiền điều kiện

- Hóa đơn/split ở trạng thái chờ thanh toán.
- Cổng thanh toán và cấu hình máy POS khả dụng đối với phương thức không tiền mặt.

Hậu điều kiện

- Một bản ghi Payment được tạo với trạng thái thành công, thất bại hoặc chờ xác nhận.
- Nếu đã thanh toán đủ, hóa đơn được đóng.

Luồng chính

1. Thu ngân chọn split cần thanh toán và phương thức thanh toán.
2. Hệ thống hiển thị số tiền phải thu và các bước xác nhận.
3. Nếu thanh toán điện tử, hệ thống gọi PaymentGateway để ủy quyền và ghi nhận giao dịch.
4. Nếu thanh toán tiền mặt, thu ngân nhập số tiền khách đưa và hệ thống tính tiền thối.
5. Hệ thống lưu kết quả thanh toán, cập nhật số dư còn lại và trạng thái hóa đơn.
6. Nếu hóa đơn đã được thanh toán đủ, hệ thống đóng hóa đơn và mở bước phát hành biên lai.

Luồng thay thế

- Một hóa đơn có thể được thanh toán bằng nhiều phương thức trong nhiều lần khác nhau.
- Nhà hàng có thể cho phép đặt cọc trước và trừ vào tổng hóa đơn khi check-out.

Ngoại lệ

- Cổng thanh toán trả về thất bại hoặc quá thời gian chờ: hệ thống ghi nhận trạng thái Failed/Pending Review.
- Thiết bị thanh toán ngoại vi không phản hồi: hệ thống cho phép chuyển phương thức khác.
- Số tiền thanh toán vượt mức còn lại: hệ thống yêu cầu xác nhận hoặc tự giới hạn theo cấu hình.

Quy tắc nghiệp vụ

- Mọi thanh toán điện tử phải lưu mã tham chiếu giao dịch.
- Hóa đơn chỉ đóng khi tổng thanh toán hợp lệ lớn hơn hoặc bằng tổng phải thu.

UC-PAY-02 — Phát hành biên lai

Tác nhân chính: Thu ngân, Khách hàng

Mục tiêu: Tạo và gửi biên lai sau khi thanh toán thành công.

Kích hoạt: Một hóa đơn hoặc split được thanh toán thành công.

Tiền điều kiện

- Thanh toán đã được ghi nhận hợp lệ.
- Kênh phát hành biên lai (in giấy, email, SMS) đã được cấu hình.

Hậu điều kiện

- Một Receipt được tạo và liên kết với hóa đơn/thanh toán.
- Khách hàng nhận biên lai theo kênh đã chọn.

Luồng chính

1. Hệ thống tạo nội dung biên lai gồm món, giảm giá, phí, phương thức thanh toán và thời gian.
2. Thu ngân chọn in giấy hoặc gửi điện tử.
3. Hệ thống phát hành biên lai và cập nhật trạng thái thành công.
4. Nếu cần, hệ thống gửi bản sao tới email khách hàng hoặc số điện thoại đã lưu.

Luồng thay thế

- Khách có thể yêu cầu xuất biên lai rút gọn hoặc chi tiết theo chính sách nhà hàng.
- Trong trường hợp split bill, mỗi split có thể có biên lai riêng.

Ngoại lệ

- Máy in lỗi hoặc hết giấy: hệ thống gợi ý chuyển sang gửi điện tử hoặc in lại sau.
- Email/SMS không gửi được: hệ thống vẫn lưu biên lai và cho phép gửi lại thủ công.

Quy tắc nghiệp vụ

- Biên lai phải tham chiếu tới ít nhất một thanh toán thành công.
- Việc phát hành lại biên lai không được tạo thêm thanh toán mới.

UC-PAY-03 — Hoàn tiền

Tác nhân chính: Thu ngân, Quản lý

Mục tiêu: Thực hiện hoàn tiền một phần hoặc toàn phần cho giao dịch đã thanh toán.

Kích hoạt: Khách khiếu nại, hủy món sau thanh toán hoặc cần điều chỉnh hóa đơn.

Tiền điều kiện

- Đã tồn tại thanh toán thành công cần hoàn.
- Người thao tác có quyền hoàn tiền hoặc được quản lý phê duyệt.

Hậu điều kiện

- Một bản ghi Refund được tạo và liên kết với giao dịch thanh toán gốc.
- Sổ audit ghi nhận người hoàn, lý do và số tiền hoàn.

Luồng chính

1. Thu ngân chọn giao dịch thanh toán cần hoàn.
2. Hệ thống hiển thị số tiền đã thanh toán, số tiền đã hoàn trước đó và mức hoàn còn khả dụng.
3. Thu ngân nhập số tiền hoàn và lý do.
4. Nếu vượt ngưỡng cho phép, hệ thống yêu cầu quản lý xác nhận.
5. Hệ thống gửi yêu cầu hoàn tiền tới gateway hoặc ghi nhận hoàn tiền mặt.
6. Hệ thống cập nhật trạng thái hoàn tiền, số dư hóa đơn và nhật ký kiểm toán.

Luồng thay thế

- Có thể hoàn tiền một phần cho riêng món bị lỗi mà không đóng lại toàn bộ hóa đơn.
- Quản lý có thể chuyển hoàn tiền thành tín dụng khách hàng nếu nhà hàng áp dụng chính sách ví nội bộ.

Ngoại lệ

- Gateway từ chối hoàn tiền: hệ thống giữ trạng thái Pending Review và cảnh báo kế toán.
- Số tiền hoàn vượt mức thanh toán rỗng còn lại: hệ thống chặn thao tác.
- Giao dịch thanh toán gốc không cho phép hoàn tiền sau thời hạn quy định: hệ thống yêu cầu xử lý ngoại lệ.

Quy tắc nghiệp vụ

- Mỗi khoản hoàn tiền phải tham chiếu tới giao dịch thanh toán gốc và lý do hoàn tiền chuẩn hóa.
- Refund vượt ngưỡng cấu hình phải có phê duyệt của quản lý.

UC-INV-01 — Cập nhật tiêu hao tồn kho sau phục vụ

Tác nhân chính: Hệ thống, Quản lý kho

Mục tiêu: Khấu trừ tồn kho theo món đã bán hoặc điều chỉnh thủ công khi cần.

Kích hoạt: Dòng món được xác nhận đã phục vụ hoặc quản lý kho tạo giao dịch điều chỉnh.

Tiền điều kiện

- Món ăn đã được ánh xạ công thức nguyên liệu qua RecipeLine.
- Tồn kho của chi nhánh đang hoạt động.

Hậu điều kiện

- Các giao dịch kho (StockTransaction) được ghi nhận.
- Số lượng tồn (onHand) của nguyên liệu được cập nhật.

Luồng chính

1. Khi món được phục vụ, hệ thống tra công thức nguyên liệu tương ứng.
2. Hệ thống tính tổng định mức theo số lượng món đã ra.
3. Hệ thống tạo các giao dịch kho trừ tồn cho từng nguyên liệu.
4. Nếu có điều chỉnh thủ công, quản lý kho nhập số lượng và lý do điều chỉnh.
5. Hệ thống cập nhật tồn kho hiện tại và lịch sử giao dịch.

Luồng thay thế

- Nhà hàng có thể cấu hình khấu trừ khi gửi bếp thay vì khi phục vụ để phản ánh thực tế chế biến.
- Các nguyên liệu hao hụt do prep đầu ca được ghi nhận bằng giao dịch riêng.

Ngoại lệ

- Thiếu ánh xạ công thức: hệ thống không tự trừ tồn và tạo cảnh báo cho quản lý menu/kho.
- Cập nhật đồng thời dẫn đến âm kho bất thường: hệ thống chặn hoặc đưa vào hàng chờ rà soát.

Quy tắc nghiệp vụ

- Mọi điều chỉnh thủ công phải có lý do chuẩn hóa như waste, spoilage, recount, prep.
- Tồn kho và lịch sử giao dịch phải được tách theo chi nhánh.

UC-INV-02 — Nhận cảnh báo sắp hết hàng

Tác nhân chính: Quản lý, Bếp, Hệ thống

Mục tiêu: Phát hiện nguyên liệu xuống dưới ngưỡng và gửi cảnh báo vận hành kịp thời.

Kích hoạt: Tồn kho sau cập nhật nhỏ hơn ngưỡng trong LowStockRule.

Tiền điều kiện

- Ngưỡng cảnh báo tồn kho đã được cấu hình cho từng nguyên liệu và chi nhánh.
- Kênh gửi cảnh báo nội bộ hoạt động.

Hậu điều kiện

- Cảnh báo sắp hết hàng được gửi đến đúng vai trò chịu trách nhiệm.
- Người dùng có thể xem danh sách nguyên liệu cần hoặc khóa bán món liên quan.

Luồng chính

1. Sau mỗi lần cập nhật tồn, hệ thống kiểm tra điều kiện sắp hết hàng.
2. Khi vượt ngưỡng cảnh báo, hệ thống tạo sự kiện và thông báo nội bộ.
3. Quản lý mở danh sách cảnh báo, xem nguyên liệu ảnh hưởng và các món liên quan.
4. Quản lý quyết định đặt hàng, điều chỉnh ngưỡng hoặc tạm ẩn món phụ thuộc.

Luồng thay thế

- Nếu nhà hàng dùng tích hợp nhà cung cấp, hệ thống có thể gợi ý lượng đặt hàng theo lịch sử tiêu thụ.
- Cảnh báo có thể được gom theo ca hoặc gửi ngay lập tức với nguyên liệu trọng yếu.

Ngoại lệ

- Ngưỡng cấu hình thiếu hoặc sai đơn vị: hệ thống ghi cảnh báo cấu hình thay vì gửi cảnh báo vận hành.
- Kênh thông báo lỗi: hệ thống lưu thông báo chờ gửi lại để thực hiện lại sau.

Quy tắc nghiệp vụ

- Một cảnh báo đang mở không được gửi lặp lại quá thường xuyên trong cùng khoảng thời gian chống spam.
- Nguyên liệu trọng yếu có thể kích hoạt tự động ẩn món phụ thuộc theo chính sách chi nhánh.

UC-INV-03 — Phân tích tiêu hao và đề xuất nhập hàng

Tác nhân chính: Quản lý, Quản lý kho

Mục tiêu: Phân tích lịch sử tiêu hao, hao hụt và cảnh báo tồn để đề xuất lượng nhập hàng phù hợp cho kỳ tiếp theo.

Kích hoạt: Quản lý mở màn hình phân tích nhập hàng theo ngày, tuần hoặc kỳ vận hành.

Tiền điều kiện

- Hệ thống đã lưu lịch sử `StockTransaction`, cảnh báo sắp hết hàng và số liệu tiêu hao theo chi nhánh.
- Quy tắc đặt lại hàng như lead time, mức an toàn hoặc min-max đã được cấu hình ở mức tối thiểu.
- Người dùng có quyền xem dữ liệu tồn kho và báo cáo vận hành.

Hậu điều kiện

- Một tập đề xuất nhập hàng được sinh ra với lượng khuyến nghị và mức độ tin cậy.
- Quản lý có thể lưu snapshot, xuất báo cáo hoặc dùng kết quả để lập kế hoạch đặt hàng.
- Các nguyên liệu rủi ro cao, tiêu hao bất thường hoặc hao hụt lớn được làm nổi bật.

Luồng chính

1. Quản lý chọn kỳ phân tích, chi nhánh và nhóm nguyên liệu cần xem.
2. Hệ thống tải lịch sử tiêu hao, cảnh báo sắp hết hàng, dữ liệu hao hụt và thời gian cung ứng.
3. Hệ thống tính mức tiêu hao bình quân, độ biến động theo ca và xác suất thiếu hàng trong kỳ kế tiếp.
4. Hệ thống sinh lượng tồn mục tiêu, điểm đặt lại hàng và lượng nhập khuyến nghị.
5. Quản lý rà soát các đề xuất, điều chỉnh nếu cần và lưu hoặc xuất báo cáo.

Luồng thay thế

- Nếu dữ liệu lịch sử chưa đủ dày, hệ thống chuyển sang quy tắc min-max và mức an toàn mặc định.
- Quản lý có thể so sánh nhiều kỳ như cuối tuần, ngày lễ hoặc giờ cao điểm để chọn phương án nhập hàng.

Ngoại lệ

- Dữ liệu đơn vị tính không nhất quán hoặc thiếu ánh xạ nguyên liệu: hệ thống đánh dấu cần rà soát thủ công.
- Lịch sử giao dịch quá ít: hệ thống vẫn cho phép tạo đề xuất nhưng phải gắn cảnh báo mức tin cậy thấp.

Quy tắc nghiệp vụ

- Đề xuất nhập hàng là khuyến nghị ra quyết định, không tự động sinh giao dịch mua hàng.
- Việc tính lượng nhập phải xét theo từng chi nhánh vì nhu cầu, lead time và mức hao hụt có thể khác nhau.

UC-REP-01 — Xem báo cáo doanh thu và vận hành

Tác nhân chính: Quản lý

Mục tiêu: Theo dõi doanh thu, giờ cao điểm, món bán chạy, độ trễ bếp và hiệu suất phục vụ.

Kích hoạt: Quản lý mở bảng điều khiển phân tích hoặc yêu cầu xuất báo cáo.

Tiền điều kiện

- Dữ liệu giao dịch, gọi món, bếp và tồn kho đã được tổng hợp vào mô hình đọc báo cáo.
- Người dùng có quyền xem báo cáo.

Hậu điều kiện

- Quản lý có được ảnh chụp báo cáo hiện tại hoặc tệp xuất PDF/CSV.
- Các chỉ số dùng cho hợp vận hành được chuẩn hóa.

Luồng chính

1. Quản lý chọn khoảng thời gian, chi nhánh và bộ lọc báo cáo.
2. Hệ thống truy xuất ReportSnapshot hoặc tính toán số liệu mới nhất.
3. Hệ thống hiển thị doanh thu, số bàn quay vòng, món bán chạy, phiếu bếp chậm, tỷ lệ hủy món và hiệu suất nhân viên.
4. Quản lý phân tích biểu đồ, đi sâu theo ca, trạm bếp hoặc khu vực.
5. Nếu cần, quản lý chọn xuất báo cáo sang PDF/CSV.

Luồng thay thế

- Bảng điều khiển có thể hiển thị cảnh báo bất thường như thời gian ra món tăng mạnh hoặc hoàn tiền tăng đột biến.
- Quản lý có thể lưu bộ lọc ưa thích để dùng lại.

Ngoại lệ

- Read model báo cáo chưa được làm mới: hệ thống hiển thị thời điểm đồng bộ cuối cùng.
- Khối lượng dữ liệu quá lớn: hệ thống chuyển truy vấn sang chế độ bất đồng bộ và thông báo khi hoàn tất.

Quy tắc nghiệp vụ

- Báo cáo phân tích dùng dữ liệu tổng hợp, không truy cập trực tiếp vào luồng ghi giao dịch thời gian thực.
- Mỗi lần xuất báo cáo phải lưu dấu vết ai xuất và thời điểm xuất.

UC-ADM-01 — Quản lý menu và giá bán

Tác nhân chính: Quản lý

Mục tiêu: Tạo, cập nhật, tạm ẩn món, thay đổi giá và chương trình khuyến mãi.

Kích hoạt: Quản lý mở màn hình cấu hình menu.

Tiền điều kiện

- Người dùng có quyền quản trị menu.
- Chi nhánh mục tiêu đang được cấu hình và có danh mục món cơ sở.

Hậu điều kiện

- Thực đơn mới hoặc thay đổi giá được lưu và có thể công bố theo lịch hiệu lực.
- Những món liên quan đến đơn gọi món đang mở vẫn giữ giá đã được chụp lại trước đó.

Luồng chính

1. Quản lý chọn danh mục món hoặc món cụ thể.

2. Hệ thống hiển thị thông tin giá, tình trạng bán, modifier và món liên quan.
3. Quản lý chỉnh sửa tên, giá, tình trạng bán, hình thức khuyến mãi hoặc thuộc tính món.
4. Quản lý lưu thay đổi và chọn công bố ngay hoặc theo lịch.
5. Hệ thống kiểm tra tác động tới POS/KDS và phát hành bản menu mới.

Luồng thay thế

- Có thể công bố theo ca sáng, ca tối hoặc theo ngày lễ.
- Khuyến mãi có thể áp dụng theo món, danh mục hoặc mức hóa đơn.

Ngoại lệ

- Dữ liệu cấu hình thiếu bắt buộc, ví dụ chưa gán trạm bếp hoặc thiếu thuế suất: hệ thống không cho công bố.
- Xung đột lịch hiệu lực giữa nhiều chương trình giá: hệ thống yêu cầu người dùng giải quyết.

Quy tắc nghiệp vụ

- Không được thay đổi trực tiếp giá của đơn gọi món đã xác nhận.
- Mọi lần công bố thực đơn phải được ghi vào nhật ký kiểm toán và có phiên bản.

UC-ADM-02 — Quản lý vai trò và phân quyền

Tác nhân chính: Quản trị viên, Quản lý

Mục tiêu: Gán vai trò và quyền thao tác cho lễ tân, phục vụ, bếp, thu ngân và quản lý.

Kích hoạt: Quản trị viên mở màn hình quản lý người dùng.

Tiền điều kiện

- Tài khoản người dùng đã tồn tại hoặc đang được tạo mới.
- Chính sách RBAC và danh mục quyền đã được định nghĩa.

Hậu điều kiện

- Tài khoản người dùng được gán đúng vai trò/quyền theo chi nhánh.
- Các thao tác nhạy cảm bị kiểm soát bởi `AuthorizationPolicy`.

Luồng chính

1. Quản trị viên tìm người dùng cần cập nhật.
2. Hệ thống hiển thị vai trò hiện tại, chi nhánh và phạm vi quyền.
3. Quản trị viên thêm/bỏ vai trò hoặc gán quyền theo chính sách.
4. Hệ thống lưu thay đổi và yêu cầu người dùng đăng nhập lại nếu cần.
5. Audit log ghi nhận toàn bộ thay đổi quyền.

Luồng thay thế

- Có thể gán vai trò tạm thời cho ca làm việc hoặc theo khoảng thời gian hiệu lực.
- Nhà hàng nhiều chi nhánh có thể áp dụng vai trò khác nhau theo từng địa điểm.

Ngoại lệ

- Người dùng đang tự gỡ quyền quản trị cuối cùng của hệ thống: thao tác bị chặn.
- Vai trò xung đột với chính sách hiện hành: hệ thống yêu cầu xác nhận hoặc điều chỉnh.

Quy tắc nghiệp vụ

- Nguyên tắc tối thiểu quyền được áp dụng mặc định.
- Hoàn tiền, ghi đề giá và điều chỉnh tồn kho là các quyền nhạy cảm phải được kiểm soát riêng.

UC-ADM-03 — Xem nhật ký kiểm toán và truy vết thao tác nhạy cảm

Tác nhân chính: Quản lý, Quản trị viên

Mục tiêu: Tra cứu, đối chiếu và truy vết các hành động nhạy cảm như hoàn tiền, hủy món, ghi đề giá, đổi quyền hoặc mở lại hóa đơn.

Kích hoạt: Người có thẩm quyền mở màn hình kiểm toán để điều tra sự cố, rà soát cuối ca hoặc phục vụ kiểm tra nội bộ.

Tiền điều kiện

- Hệ thống đang ghi nhận AuditLog cho các hành động nhạy cảm.
- Người dùng có quyền xem nhật ký kiểm toán ở phạm vi chi nhánh hoặc toàn hệ thống theo vai trò.
- Dữ liệu correlation id, thời gian và đối tượng bị tác động có thể truy xuất được.

Hậu điều kiện

- Người dùng xem được chuỗi hành động liên quan đến một giao dịch hoặc một người thao tác.
- Hệ thống lưu lại dấu vết việc xem hoặc xuất audit log nếu chính sách yêu cầu.
- Có thể tạo báo cáo hoặc gắn cờ theo dõi cho bản ghi cần điều tra thêm.

Luồng chính

1. Người dùng mở màn hình nhật ký kiểm toán.
2. Hệ thống kiểm tra quyền truy cập vào dữ liệu kiểm toán nhạy cảm.
3. Người dùng nhập bộ lọc như khoảng thời gian, người thao tác, thực thể, loại hành động hoặc mã liên kết.
4. Hệ thống truy vấn các bản ghi audit bất biến và hiển thị danh sách kết quả phù hợp.
5. Người dùng mở chi tiết một bản ghi để xem giá trị trước, giá trị sau, lý do, correlation id và ngữ cảnh liên quan.
6. Nếu cần, người dùng xuất báo cáo hoặc đánh dấu follow-up để tiếp tục điều tra.

Luồng thay thế

- Người dùng có thể đi từ một hóa đơn, payment hoặc order cụ thể sang chuỗi audit liên quan.
- Các trường nhạy cảm như thông tin liên hệ hoặc dữ liệu thanh toán tham chiếu có thể được làm mờ theo quyền.

Ngoại lệ

- Người dùng không đủ quyền xem chi tiết audit log: hệ thống từ chối truy cập hoặc chỉ cho xem bản rút gọn.
- Không có bản ghi phù hợp với bộ lọc: hệ thống thông báo không có kết quả và cho phép điều chỉnh bộ lọc.
- Kho audit tạm thời chậm hoặc không sẵn sàng: hệ thống chuyển sang chế độ chỉ đọc hạn chế và cảnh báo quản trị.

Quy tắc nghiệp vụ

- Bản ghi kiểm toán đã phát sinh không được phép sửa nội dung theo cách làm mất dấu vết gốc.
- Việc xem hoặc xuất audit log mức nhạy cảm cao cũng phải được ghi nhận vào audit log.

UC-OPS-01 — Quản lý ca làm nhân viên

Tác nhân chính: Quản lý

Mục tiêu: Phân công nhân sự cho từng chi nhánh, khu vực và ca làm việc.

Kích hoạt: Quản lý mở lịch làm việc và tạo/cập nhật ca.

Tiền điều kiện

- Tài khoản nhân viên đã tồn tại.
- Mẫu ca làm và cấu hình khu vực phục vụ đã được tạo.

Hậu điều kiện

- Một hoặc nhiều ShiftAssignment được tạo hoặc cập nhật.
- Nhân viên nhìn thấy ca được phân trong lịch làm việc.

Luồng chính

1. Quản lý chọn ngày, chi nhánh và loại ca.
2. Hệ thống hiển thị danh sách nhân viên khả dụng cùng kỹ năng/vai trò.
3. Quản lý gán nhân viên cho ca và khu vực phụ trách.
4. Hệ thống kiểm tra trùng ca, thiếu vị trí bắt buộc và giới hạn giờ làm.
5. Quản lý lưu lịch; hệ thống gửi thông báo cho nhân viên.

Luồng thay thế

- Quản lý có thể sao chép lịch từ tuần trước rồi tinh chỉnh.
- Ca thiếu người có thể được đánh dấu chờ xác nhận hoặc mở đăng ký tự nguyện.

Ngoại lệ

- Nhân viên bị phân ca trùng thời gian hoặc vượt giới hạn giờ làm: hệ thống chặn lưu.
- Người dùng không thuộc chi nhánh được chọn: hệ thống yêu cầu thay đổi phạm vi gán.

Quy tắc nghiệp vụ

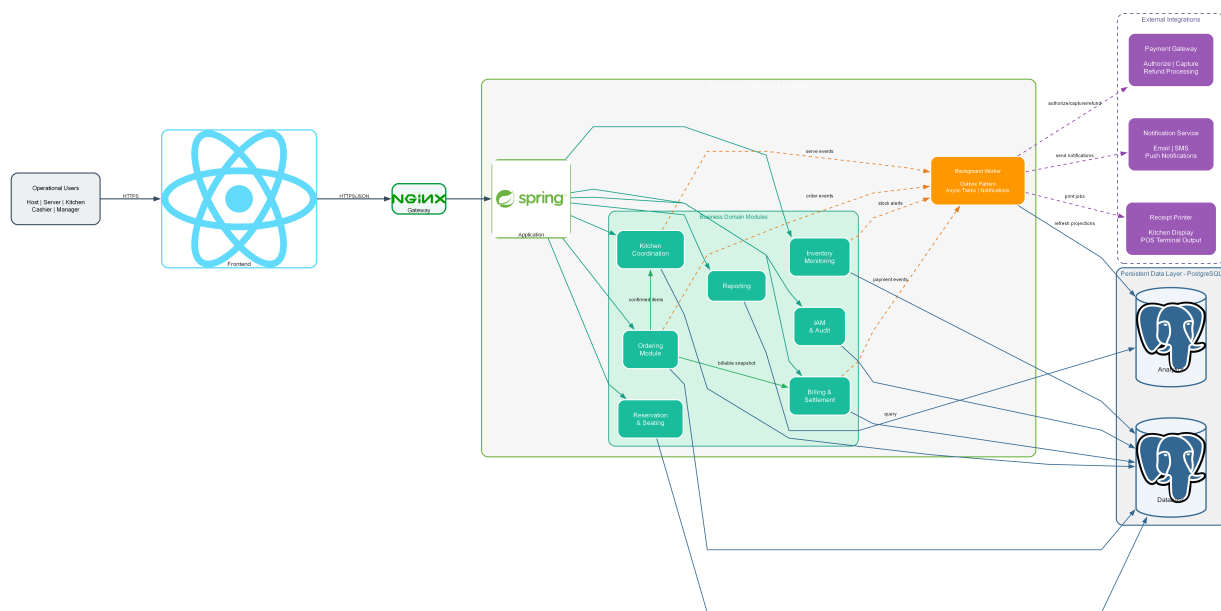
- Một nhân viên không được có hai ca chồng lấn ở cùng thời điểm.
- Ca bếp, ca thu ngân và ca quản lý phải đáp ứng số lượng tối thiểu theo chính sách vận hành.

3 Kiến trúc phần mềm

3.1 Tổng quan kiến trúc

IRMS được tổ chức theo **kiến trúc lai** gồm một lớp biên web thống nhất, một **backend Spring Boot** được chia **mô-đun theo miền nghiệp vụ** và các **luồng hỗ trợ nền** cho làm mới mô hình đọc, gửi thông báo, in biên lai và phát tán sự kiện vận hành. Cách mô tả này giữ đúng với những lát cắt ở các mục sau: hệ thống được phân rã rõ theo năng lực nghiệp vụ, nhưng ở giai đoạn hiện tại các mô-đun lõi vẫn cùng chạy trong một runtime tập trung để giữ giao dịch nóng đơn giản, nhất quán và dễ vận hành hơn.

Phân biện minh cho lựa chọn kiến trúc, thành phần và kiểu kết nối của sơ đồ tổng quan này được dẫn trực tiếp tại Bảng 20 và Bảng 22, đồng thời được giải thích sâu hơn trong Phụ lục A.1 và Phụ lục A.4.



Hình 8: Sơ đồ tổng quan kiến trúc của IRMS

Hình 8 là sơ đồ nền tảng để đọc toàn bộ chương kiến trúc. Hình này không chỉ trả lời câu hỏi “IRMS có những khối nào”, mà còn trả lời ba câu hỏi quan trọng hơn: yêu cầu đi vào hệ thống từ đâu, quyết định nghiệp vụ được giữ ở tầng nào, và đâu là ranh giới giữa phần lõi có thể kiểm soát với phần tích hợp ngoài cần cách ly rủi ro.

Ở lớp ngoài cùng, ReactJS đóng vai trò mặt trước thống nhất cho lễ tân, phục vụ, bếp, thu ngân và quản lý. Việc dùng một nền giao diện thống nhất không có nghĩa mọi vai trò dùng cùng một màn hình, mà có nghĩa mọi vai trò đi qua cùng một cơ chế xác thực, cùng quy tắc điều hướng và cùng cách đồng bộ trạng thái quan trọng như bàn, đơn gọi món, phiếu bếp và hóa đơn. Đây là quyết định quan trọng vì IRMS là hệ thống có nhiều điểm chạm nghiệp vụ nhưng cần một cảm giác vận hành liền mạch. Nếu mỗi vai trò bị tách thành những ứng dụng rời rạc ngay từ đầu, chi phí đồng bộ trạng thái, quản lý phiên làm việc và kiểm soát truy cập sẽ tăng rất nhanh.

Ở lớp cổng vào, Nginx được đặt làm điểm tiếp nhận thống nhất. Vai trò của nó không chỉ là chuyển tiếp yêu cầu. Về kiến trúc, đây là nơi phù hợp để đặt các kiểm tra chung như xác thực sơ bộ, giới hạn lưu lượng, ghi nhật ký ở biên, chuẩn hóa tiêu đề yêu cầu và tách ứng dụng khách khỏi cấu trúc bên trong của hệ thống. Nhờ có lớp này, giao diện người dùng không cần biết trực tiếp từng mô-đun phía sau đang chạy ra sao, ở đâu, hay được tách thành bao nhiêu phần. Điều đó giúp cấu trúc bên trong được thay đổi dần theo thời gian mà không làm gãy các điểm chạm ở tuyến đầu.

Phần quan trọng nhất của hình là lớp xử lý phía sau. Ở đây, IRMS được tổ chức theo **các mô-đun nghiệp vụ trong một backend runtime chung** thay vì theo một tập dịch vụ mạng độc lập ngay từ đầu. Reservation & Seating giữ bối cảnh đặt bàn, danh sách chờ và phiên bàn; Ordering giữ dữ liệu gọi món và ảnh chụp giá trị tại thời điểm xác nhận; Kitchen Coordination quản lý việc chuyển từ đơn gọi món sang thực thi tại bếp; Billing & Settlement chịu trách nhiệm cho toàn bộ chuỗi lập hóa đơn, tách hóa đơn, thanh toán và hoàn tiền; Inventory Monitoring xử lý mức tồn, quy tắc cảnh báo và lịch sử biến động; Reporting phục vụ phía đọc quản trị; còn IAM / Audit bảo vệ biên truy cập và truy vết các quyết định nhạy cảm. Cách chia này làm rõ một nguyên tắc quan trọng: phần mềm phải phản ánh cách nhà hàng vận hành ngoài đời, không phản ánh cách tổ chức kỹ thuật thuần túy.

Lớp dữ liệu trong hình cũng cần được đọc kỹ. PostgreSQL không chỉ là nơi lưu bảng dữ liệu, mà là nơi neo giữ tính nhất quán của những quyết định có hậu quả nghiệp vụ cao như xác nhận đơn gọi món, đổi trạng thái bàn, đóng hóa đơn hay ghi nhận hoàn tiền. Đồng thời, sơ đồ cũng chú ý cho thấy **mô hình đọc phục vụ báo cáo** được làm mới qua tuyến nền, thay vì bắt tuyến quản trị truy vấn trực tiếp lên dữ liệu giao dịch nóng. Đây là một quyết định cân bằng rất thực tế: giao diện phục vụ tại bàn, màn hình bếp và giao diện thu ngân phải phản hồi trước; báo cáo chi tiết có thể đi sau vài giây hoặc vài phút miễn là đúng và truy vết được.

Cuối cùng, tuyến Outbox / Background Worker cùng các tích hợp ngoài như cổng thanh toán,

kênh gửi thông báo và máy in biên lai được đặt ở rìa hệ thống. Đây là chi tiết kiến trúc rất quan trọng. Những thành phần này có thể chậm, lỗi, thay nhà cung cấp hoặc thay giao thức tích hợp. Nếu chúng chen trực tiếp vào miền nghiệp vụ, mọi thay đổi ngoài hệ thống sẽ lan ngược vào lõi. Khi bị đẩy ra rìa và đi qua tuyến nền hoặc các bộ kết nối trừu tượng, IRMS có thể giữ ổn định phần trung tâm ngay cả khi phải thay đổi cách gửi thông báo, đổi thiết bị in hoặc đổi đối tác thanh toán.

1. **Lớp giao diện người dùng (ReactJS).** Đây là nơi tối ưu cho thao tác nhanh, ít bước và nhất quán giữa nhiều vai trò vận hành. Kiến trúc lựa chọn một nền giao diện chung để giảm chi phí học hệ thống, giữ một mô hình điều hướng thống nhất và đơn giản hóa quản lý phiên làm việc theo ca.
2. **Cổng vào giao diện lập trình ứng dụng (Nginx).** Thành phần này tạo một biên truy cập rõ ràng, là nơi phù hợp để gom các kiểm tra chung trước khi yêu cầu đi sâu vào lõi nghiệp vụ. Nó cũng giúp che giấu cấu trúc bên trong khỏi ứng dụng khách.
3. **Backend mô-đun (Spring Boot).** Các năng lực nghiệp vụ được chia thành những mô-đun rõ ràng trong cùng một runtime, nhờ đó thay đổi ở vùng gọi món không tự động làm vỡ vùng thanh toán, còn thay đổi ở vùng tồn kho không ép vùng đặt bàn phải sửa theo.
4. **Tuyến nền (Outbox / Background Worker).** Các hệ quả phụ như làm mới mô hình đọc, gửi thông báo, in biên lai hoặc phát cảnh báo được đẩy sang xử lý nền để không làm nghẽn tuyến giao dịch nóng.
5. **Kho dữ liệu và mô hình đọc (PostgreSQL).** Dữ liệu nghiệp vụ cốt lõi và dữ liệu đọc phục vụ báo cáo được tách vai trò rõ ràng. Việc này giúp truy vấn quản trị không làm nặng tuyến giao dịch nóng như gọi món, điều phối bếp và thanh toán.
6. **Các tích hợp ngoài.** Những điểm phụ thuộc đối tác được đặt ở rìa kiến trúc để dễ thay thế, dễ cô lập lỗi và không làm ô nhiễm mô hình miền nghiệp vụ.

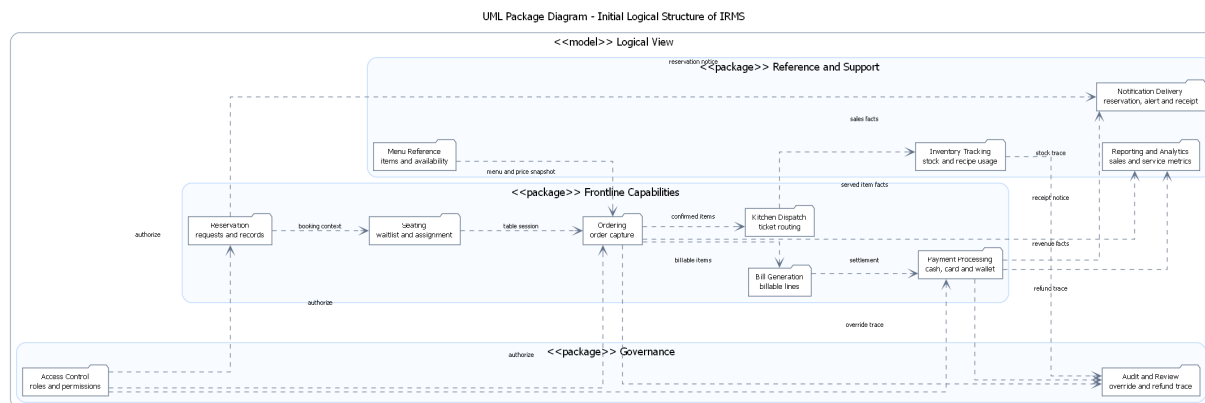
Nhìn tổng thể, sơ đồ này cho thấy IRMS đi theo một tư duy kiến trúc rất thực dụng: giữ tuyến đầu gọn, giữ lõi nghiệp vụ có ranh giới rõ, và đẩy phần dễ biến động ra ngoài biên. Chính vì vậy, các góc nhìn logic, mô-đun, thành phần và triển khai ở các mục sau không phải là những bức tranh rời rạc, mà là những lát cắt khác nhau của cùng một quyết định kiến trúc.

3.2 Góc nhìn logic

Ở góc nhìn logic, trọng tâm không còn là đường đi từ người dùng vào hệ thống, mà là cách IRMS tách các năng lực nghiệp vụ thành những miền trách nhiệm rõ ràng. Trong báo cáo này, góc nhìn logic được dùng theo hướng **capability and responsibility mapping**: hệ thống được đọc như một tập các năng lực nghiệp vụ, mỗi năng lực giữ một vùng quyết định riêng, có mối liên hệ rõ với các năng lực còn lại nhưng không bị trộn vào cùng một khối xử lý mơ hồ. Cách nhìn đó phù hợp với mục tiêu của báo cáo kiến trúc vì nó giúp kiểm tra xem kiến trúc có thật sự phản ánh vận hành nhà hàng hay chỉ mới phản ánh danh sách chức năng.

Hai sơ đồ dưới đây lần lượt thể hiện bước phân rã logic ban đầu và bước gom các năng lực thành ranh giới kiến trúc ổn định hơn để dùng nhất quán ở các phần sau.

3.2.1 Sơ đồ tổng quan



Hình 9: Sơ đồ phân rã năng lực logic ban đầu của IRMS

Hình 9 phản ánh bước phân rã năng lực nghiệp vụ ban đầu. Ở đây, hệ thống được nhìn như một tập hợp các năng lực chức năng còn khá gần với thao tác người dùng: đặt bàn và xếp bàn, duyệt thực đơn, tiếp nhận đơn gọi món, điều phối bếp, lập hóa đơn, xử lý thanh toán, theo dõi tồn kho, báo cáo, kiểm soát truy cập, rà soát kiểm toán và gửi thông báo. Cách nhìn này có ích ở giai đoạn đầu vì nó bám sát đầu bài và tác nhân, giúp việc kiểm tra phạm vi chức năng bắt buộc trở nên trực quan hơn.

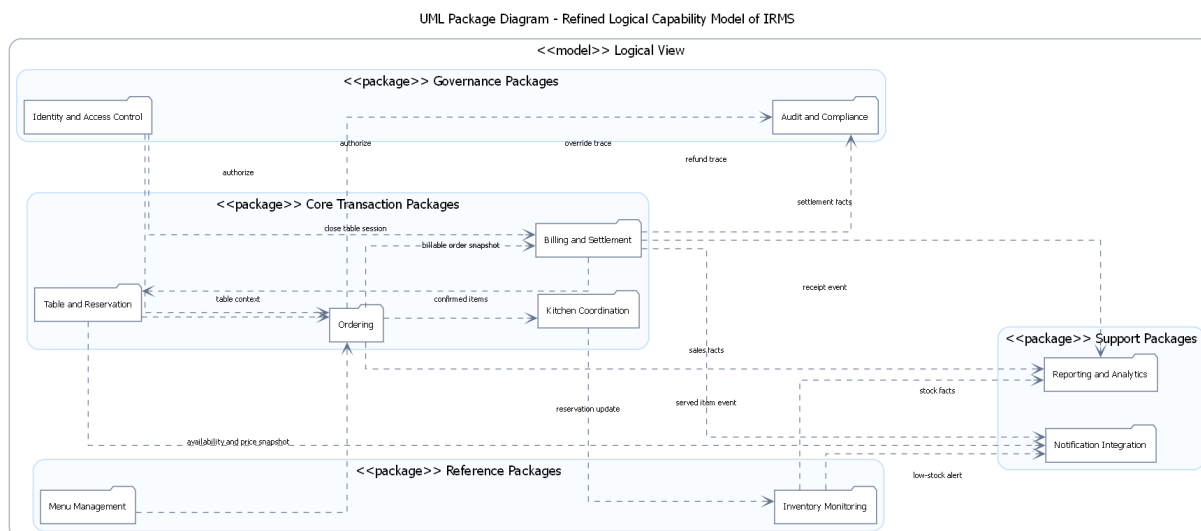
Tuy nhiên, nếu dừng ở mức này thì kiến trúc sẽ còn khá *thiên về chức năng bề mặt*. Ví dụ, Bill Generation và Payment Processing thực ra nằm trong cùng một vùng quyết toán nghiệp vụ; Access Control và Audit Review cũng chưa được phân biệt rõ giữa chính sách kiểm soát và dữ liệu kiểm toán. Vì thế, sơ đồ này nên được hiểu là bước khám phá năng lực nghiệp vụ, không phải phương án phân rã cuối cùng.

Các thành phần chính thể hiện trong hình.

- **Nhóm tuyến đầu:** gồm các năng lực gần trực tiếp với thao tác người dùng như đặt bàn, gọi món, bếp và thanh toán.
- **Nhóm hỗ trợ vận hành:** gồm tồn kho, báo cáo và gửi thông báo.
- **Nhóm kiểm soát hệ thống:** gồm kiểm soát truy cập và rà soát kiểm toán.

Lý do thiết kế của sơ đồ này.

- Sơ đồ ban đầu giúp quan sát xuất phát điểm theo chức năng bề mặt trước khi hệ thống được gom lại theo các ranh giới kiến trúc chặt chẽ hơn.
- Việc đặt các năng lực gần nhau trong cùng một hình giúp người đọc thấy xuất phát điểm thiết kế trước khi báo cáo khóa lại thành các miền nghiệp vụ rõ ràng hơn.



Hình 10: Sơ đồ phân rã năng lực logic của IRMS

Hình 10 khóa lại phương án phân rã logic được sử dụng xuyên suốt báo cáo. Ở mức này, các năng lực nghiệp vụ được gom thành những ranh giới phù hợp hơn cho kiến trúc: Table and Reservation, Menu Management, Ordering, Kitchen Coordination, Billing and Settlement, Inventory Monitoring, Reporting and Analytics, Identity and Access Control, Audit and Compliance, và Notification Integration. Đây là điểm rất quan trọng vì nó cho thấy kiến trúc không còn chỉ phản ánh thao tác giao diện, mà đã chuyển sang phản ánh **các ranh giới năng lực nghiệp vụ**.

Về mặt kiến trúc, cách phân rã này giúp ba việc. Thứ nhất, nó tăng **tính kết tụ**: mỗi năng lực nghiệp vụ gom những quy tắc gần nhau vào cùng một ranh giới. Thứ hai, nó giảm **mức độ phụ thuộc**: tồn kho, báo cáo và gửi thông báo không bị kéo vào cùng tuyến giao dịch nóng của gọi món. Thứ ba, nó tạo nền cho việc chọn kiểu kết nối phù hợp: Ordering và Billing and Settlement cần đồng bộ mạnh hơn, trong khi Inventory Monitoring, Reporting and Analytics và Notification Integration có thể đi qua sự kiện hoặc bộ xử lý nền.

Các thành phần chính thể hiện trong hình.

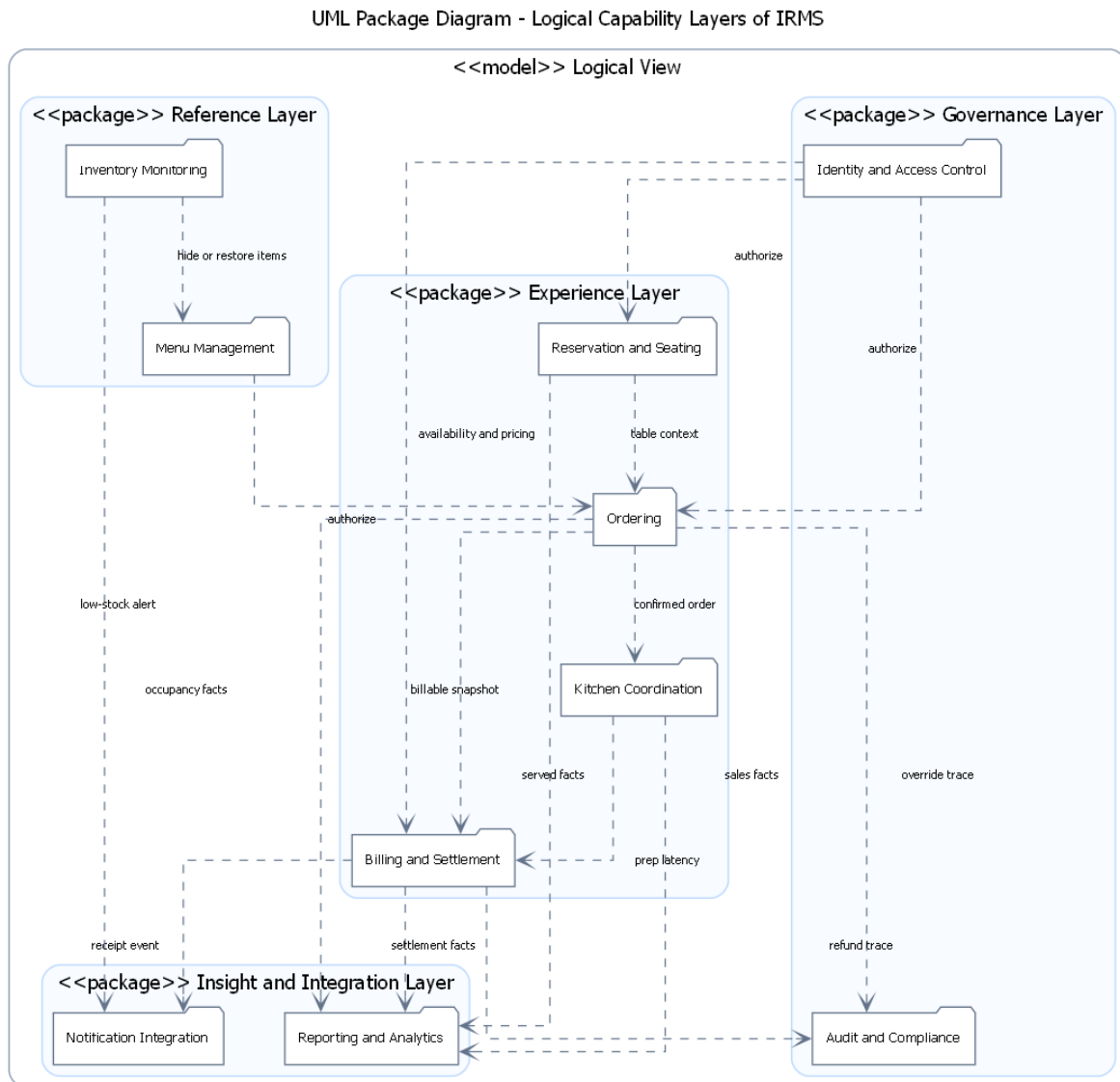
- **Các năng lực nghiệp vụ lõi**: Table and Reservation, Menu Management, Ordering, Kitchen Coordination và Billing and Settlement.
- **Các năng lực hỗ trợ**: Inventory Monitoring, Reporting and Analytics và Notification Integration.
- **Các năng lực kiểm soát**: Identity and Access Control cùng Audit and Compliance.

Lý do thiết kế của sơ đồ này.

- Đây là sơ đồ tổng quan quan trọng nhất của góc nhìn logic vì nó khóa ranh giới miền nghiệp vụ trước khi đi sang mô-đun và thành phần thực thi.
- Việc làm rõ các nhóm lõi, hỗ trợ và kiểm soát giúp những phần sau của báo cáo giải thích được vì sao có luồng đồng bộ và luồng bất đồng bộ trong kiến trúc.

3.3 Tổng quan cho các góc nhìn chính

3.3.1 Tổng quan cho góc nhìn logic



Hình 11: Sơ đồ tổng quan cho góc nhìn logic của IRMS

Hình 11 làm rõ tầng khái niệm của góc nhìn logic, tức là trả lời câu hỏi “trong bức tranh nghiệp vụ tổng thể, các năng lực của IRMS được nhóm thành những lớp nào và quan hệ chính giữa các lớp đó là gì”. Khác với hình logic ở mức liệt kê năng lực riêng lẻ, hình này nhấn mạnh **bốn tầng năng lực**: tuyến phục vụ trực tiếp với khách, nhóm tham chiếu và hoạch định, nhóm kiểm soát và nhóm hỗ trợ phân tích. Cách nhìn đó giúp phân logic trở thành một bản đồ năng lực có cấu trúc, thay vì một danh sách chức năng rời rạc; phân định vị và lý do chọn kiểu nhìn này được nối thẳng tới Bảng 20, Bảng 21 và Bảng 22.

Các nhóm năng lực chính thể hiện trong hình.

- **Nhóm năng lực tuyến đầu:** gồm Table & Reservation, Ordering, Kitchen Coordination và Billing & Settlement; đây là tuyến giao dịch nóng, nơi mọi thao tác phải phản hồi nhanh và nhất quán.
- **Nhóm năng lực tham chiếu và hoạch định:** gồm Menu Management và Inventory Monitoring;

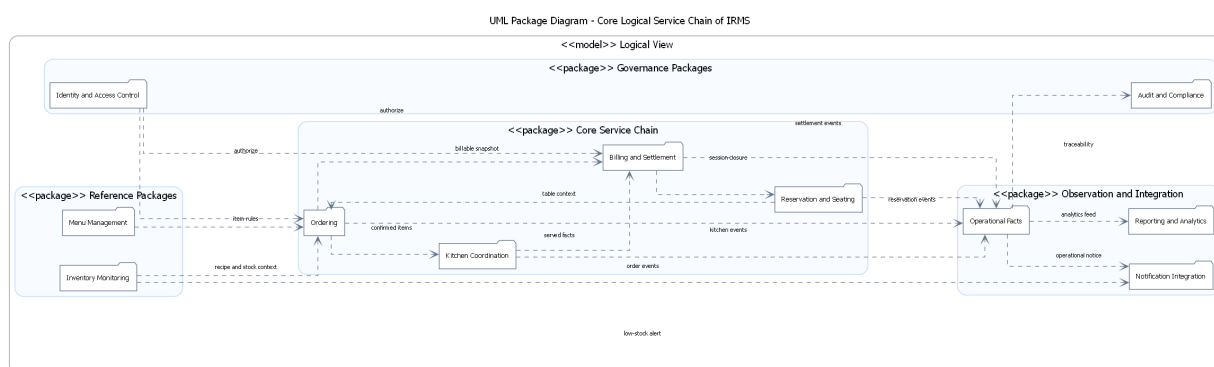
hai năng lực này cung cấp dữ liệu điều kiện và quy tắc cho tuyến vận hành, nhưng không nên làm nghìn tuyến phục vụ.

- **Nhóm năng lực kiểm soát và quản trị:** gồm Identity & Access Control và Audit & Compliance; đây là lớp bảo vệ và truy vết cho những hành động nhạy cảm.
- **Nhóm năng lực hỗ trợ và phân tích:** gồm Notification Integration và Reporting & Analytics; đây là lớp mở rộng giá trị vận hành, thiên về xử lý bất đồng bộ hoặc tổng hợp dữ liệu.

Ý nghĩa thiết kế của sơ đồ tổng quan này.

- Nó khóa lại ranh giới logic ở mức **nhóm năng lực**, giúp những phần sau của báo cáo giải thích được vì sao có luồng đồng bộ mạnh ở tuyến đầu nhưng lại có luồng dựa trên sự kiện cho cảnh báo, báo cáo và gửi thông báo.
- Nó cho thấy rõ **chiều phụ thuộc đúng**: thực đơn và tồn kho phục vụ gọi món; gọi món phát sinh dữ liệu cho bếp và quyết toán; món đã phục vụ phát sinh dữ liệu tiêu hao cho tồn kho, còn thanh toán hoàn tất kéo theo thông báo, báo cáo và khép phiên bản.
- Nó tạo nền cho góc nhìn phân bổ vì mỗi năng lực sau đó sẽ được ánh xạ sang vùng thực thi, nơi lưu trữ và bộ kết nối phù hợp thay vì ánh xạ ngẫu nhiên theo giao diện người dùng.

3.3.2 Góc nhìn logic cho chuỗi phục vụ lõi



Hình 12: Góc nhìn logic cho chuỗi phục vụ lõi của IRMS

Hình 12 đi từ bản đồ năng lực sang chuỗi năng lực lõi, tức phần quyết định trực tiếp trải nghiệm phục vụ của khách hàng từ lúc khách được nhận bàn cho tới khi bàn được giải phóng sau thanh toán. Trong góc nhìn logic, chuỗi này gồm bốn miền chính: Reservation & Seating mở bối cảnh phục vụ, Ordering giữ dữ liệu giao dịch của món, Kitchen Coordination biến dữ liệu bán hàng thành công việc chế biến, và Billing & Settlement chốt nghĩa vụ tài chính cũng như đóng vòng đời phiên bản. Sơ đồ giúp người đọc thấy rõ hệ thống không vận hành như một chuỗi màn hình, mà như một chuỗi năng lực nghiệp vụ có đầu vào, đầu ra và trách nhiệm riêng.

Điểm quan trọng nhất của hình là **mỗi miền logic chỉ công bố loại thông tin cần thiết cho miền kế tiếp**. Reservation & Seating không truyền toàn bộ chi tiết đặt bàn sang gọi món; nó chỉ truyền bối cảnh phiên bản hợp lệ. Ordering không giao thẳng các cấu trúc giao diện sang bếp; nó truyền ảnh chụp đơn gọi món đã xác nhận và có khả năng định tuyến. Kitchen Coordination không quyết toán thay cho thu ngân; nó chỉ phát hành trạng thái thực thi như Ready, Blocked hoặc Served. Billing & Settlement sau đó sử dụng ảnh chụp giao dịch đủ tin cậy để tính tiền, thu tiền và khép lại phiên bản. Nhờ giới hạn như vậy, chuỗi nghiệp vụ giữ được độ kết tụ cao mà không làm các miền trở nên phụ thuộc quá mức.

Một chi tiết quan trọng khác là nút Published Operational Facts ở phần dưới của hình. Nút này không phải một thành phần triển khai, mà là cách biểu diễn ở mức logic cho nhóm dữ kiện được phát ra sau khi các quyết định lõi đã được chốt, ví dụ sự kiện đặt bàn, ảnh chụp đơn gọi món, trạng thái phục vụ món hay kết

quả quyết toán. Cách biểu diễn này giúp tách rõ hai loại quan hệ: quan hệ trên **trực phục vụ chính** và quan hệ trên **trực quan sát, tích hợp, truy vết**. Nhờ đó, sơ đồ vừa giữ được chuỗi lỗi dễ đọc, vừa vẫn thể hiện được vì sao báo cáo, thông báo và kiểm toán có thể bám theo giao dịch mà không chen vào quyết định lỗi.

Sơ đồ cũng cho thấy vị trí của các miền tham chiếu và kiểm soát xung quanh chuỗi lỗi. Menu Management và Inventory Monitoring không đứng trên trực phục vụ chính, nhưng chúng cung cấp điều kiện vận hành như khả dụng món, giá, ảnh xạ công thức và tín hiệu ăn món. Identity & Access Control cùng Audit & Compliance lại bao quanh chuỗi lỗi như một lớp kiểm soát: không tác động trực tiếp vào giá trị món ăn hay số tiền phải thu, nhưng quyết định ai được phép ghi đề, hoàn tiền, hủy món hay xem dữ liệu nhạy cảm. Đây là cách nhìn logic phù hợp với chuẩn mô tả kiến trúc: phân biệt rõ **năng lực nghiệp vụ chính, năng lực tham chiếu và năng lực điều tiết**.

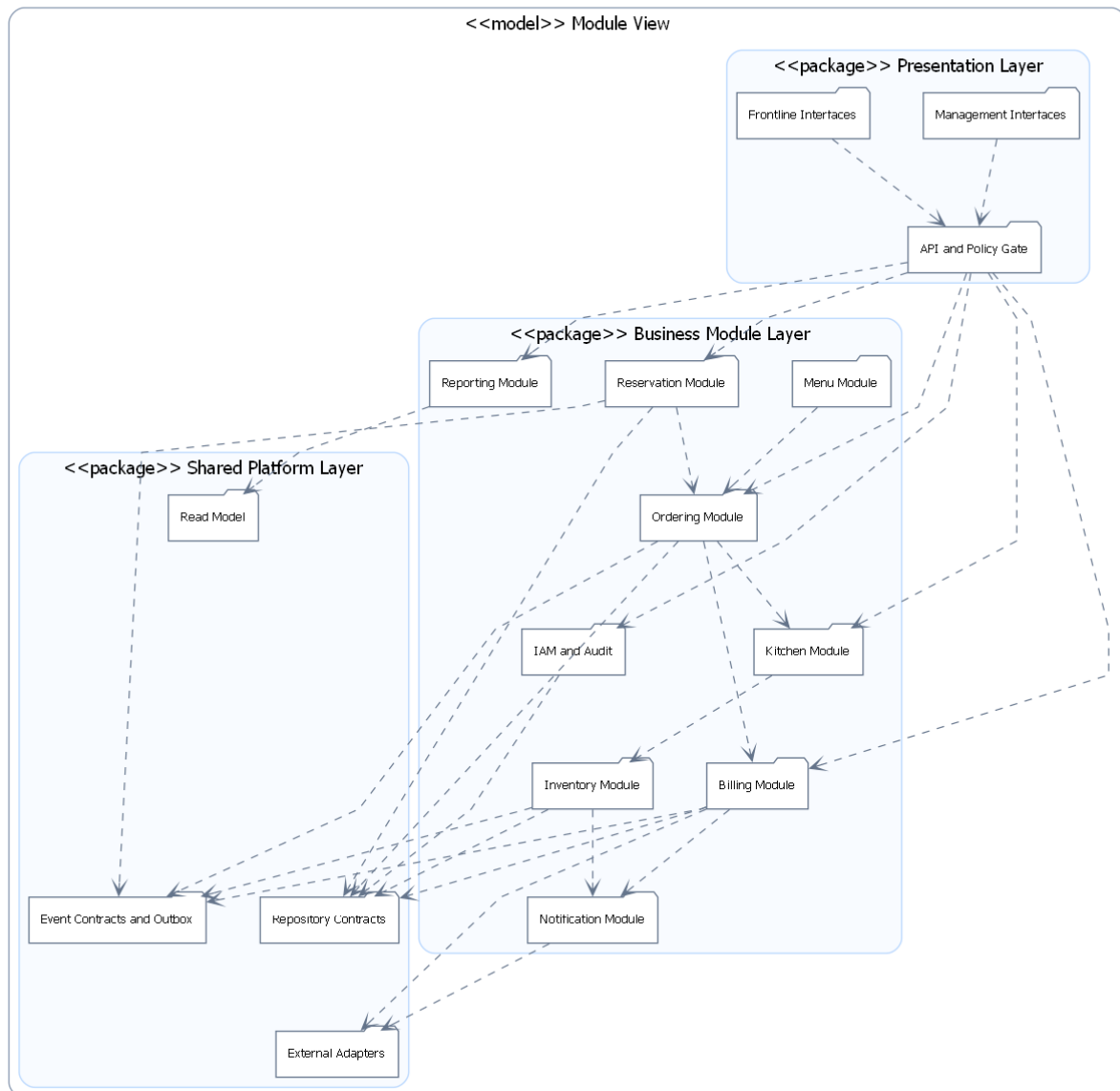
Những điểm chính cần đọc từ hình.

- **Chuỗi lỗi** đi theo thứ tự Reservation & Seating → Ordering → Kitchen Coordination → Billing & Settlement, phản ánh đúng vòng đời phục vụ trong nhà hàng.
- **Menu Management** và **Inventory Monitoring** đóng vai trò năng lực điều kiện: chúng không thay mặt tuyến đầu xử lý giao dịch nhưng ảnh hưởng trực tiếp tới tính hợp lệ của món và quyết định ăn hoặc mở bán.
- **Published Operational Facts** biểu diễn lớp dữ kiện được công bố ra ngoài chuỗi lỗi sau khi các quyết định chính đã được chốt, làm cầu nối tới báo cáo, thông báo và truy vết.
- **Identity & Access Control** cùng **Audit & Compliance** là lớp bảo vệ xuyên suốt, giúp những hành động nhạy cảm có cơ sở quyền hạn và dấu vết truy vết rõ ràng.
- **Notification Integration** và **Reporting & Analytics** nằm ở vòng ngoài vì chúng tiêu thụ dữ kiện vận hành hơn là điều khiển giao dịch lỗi theo thời gian thực.

3.3.3 Tổng quan cho góc nhìn mô-đun

Trong báo cáo này, góc nhìn mô-đun được trình bày theo tinh thần **decomposition view** kết hợp với **uses/layered constraints**. Nói cách khác, phần mô-đun không chỉ liệt kê hệ thống có những khối nào, mà còn chỉ ra khối nào được phép phụ thuộc vào khối nào, thông tin nào là phụ thuộc tĩnh, và đâu là những liên hệ chỉ nên xuất hiện dưới dạng hệ quả sự kiện sau giao dịch.

UML Package Diagram - Layered Module Dependencies of IRMS



Hình 13: Sơ đồ tổng quan cho góc nhìn mô-đun của IRMS

Hình 13 làm rõ quy tắc phụ thuộc theo lớp trong góc nhìn mô-đun. Góc nhìn mô-đun không chỉ trả lời hệ thống có những mô-đun nào, mà còn phải trả lời mô-đun nào được phép gọi trực tiếp mô-đun nào, mô-đun nào chỉ nên giao tiếp qua sự kiện, và mô-đun nào là nền tảng dùng chung chứ không phải mô-đun nghiệp vụ. Vì vậy hình này tách rõ Client Modules, Business Modules và Shared Platform Modules.

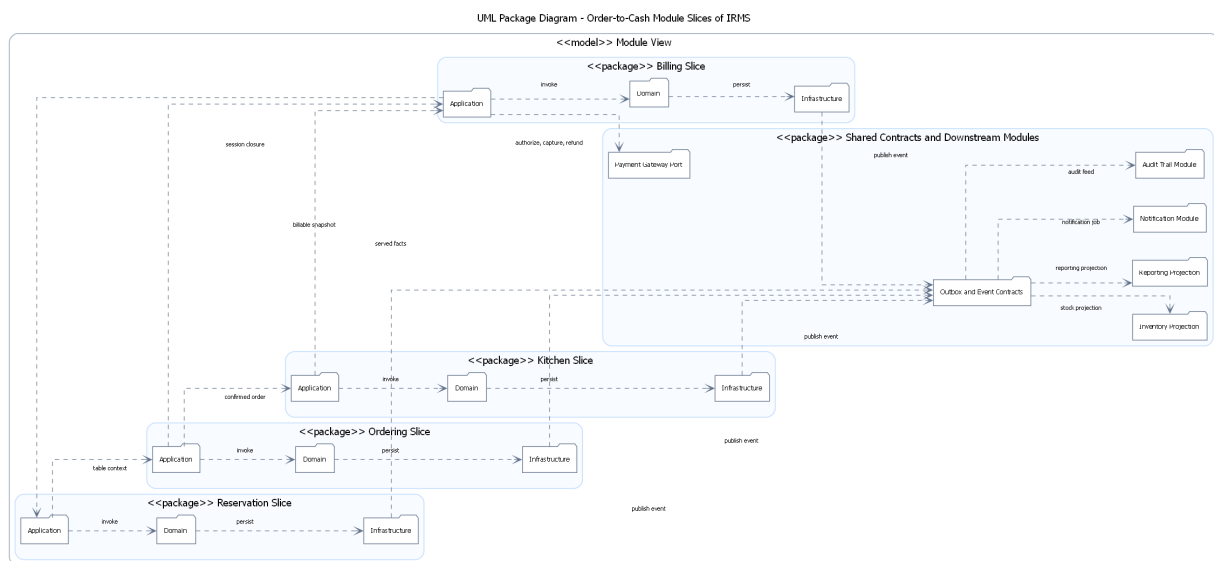
Các ý chính cần đọc từ hình.

- Các giao diện người dùng chỉ đi vào API / Web Layer; chúng không được gọi chéo vào lớp kho lưu trữ hay bộ kết nối.
- Các mô-đun nghiệp vụ giữ trách nhiệm riêng, ví dụ Ordering không “ôm” logic thanh toán, còn Billing không trực tiếp điều phối trạng thái bếp.
- Những năng lực như Repositories / Unit of Work, Event Bus / Outbox, Reporting Views và External Adapters là hạ tầng dùng chung, được phụ thuộc theo hướng có kiểm soát.

Vì sao sơ đồ tổng quan này quan trọng.

- Nó giúp phần góc nhìn mô-đun không bị dừng ở mức “danh sách mô-đun”, mà nâng lên thành **quy tắc tổ chức mã nguồn và phụ thuộc**.
- Nó tạo cầu nối trực tiếp sang góc nhìn thành phần: từ phụ thuộc ở mức mô-đun, người đọc có thể suy ra vì sao ở sơ đồ thành phần tồn tại bộ điều khiển, dịch vụ ứng dụng, kho lưu trữ, hộp thư ra và bộ kết nối như những loại thành phần khác nhau.
- Nó cũng hỗ trợ phần SOLID vì từ sơ đồ có thể thấy rõ đường phụ thuộc được bẻ vào lớp trừu tượng hoặc hạ tầng dùng chung thay vì gọi chéo trực tiếp giữa các mô-đun.

3.3.4 Góc nhìn mô-đun cho tuyến order-to-cash



Hình 14: Góc nhìn mô-đun cho tuyến order-to-cash của IRMS

Hình 14 làm rõ ở mức chi tiết hơn tuyến từ nhận bàn, gọi món, điều phối bếp, quyết toán cho tới phát sinh dữ kiện báo cáo và thông báo. Ở mức này, hệ thống được nhìn như một chuỗi lát cắt mô-đun nối tiếp nhau thay vì một tập tên mô-đun đứng cạnh nhau. Đây là sơ đồ cần thiết vì trong nhiều báo cáo kiến trúc, phần mô-đun thường chỉ dừng ở danh sách mô-đun mà không chỉ ra đường phụ thuộc tinh của một tuyến nghiệp vụ quan trọng. Với IRMS, tuyến order-to-cash là nơi mọi quyết định về ranh giới mô-đun lộ ra rõ nhất.

Điểm cốt lõi của sơ đồ là **mỗi lát cắt mô-đun sở hữu dữ liệu, luật nghiệp vụ và nhịp thực thi tương ứng**. Trong hình, mỗi lát cắt được biểu diễn như một khối dọc gồm ba tầng Application, Domain và Repository. Đây là cách biểu diễn súc tích nhưng vẫn đủ mạnh về mặt kiến trúc: Reservation sở hữu bối cảnh phiên bàn và trạng thái bàn; Ordering sở hữu Order, OrderItem và ảnh chụp thực đơn; Kitchen sở hữu KitchenTicket và hàng chờ chế biến; Billing sở hữu Bill, BillSplit, Payment và Refund. Các phụ thuộc giữa chúng đi qua các giao diện công khai hoặc ảnh chụp nghiệp vụ, chứ không qua việc truy cập thẳng vào cấu trúc bên trong nhau. Nhờ đó, thay đổi ở một mô-đun không kéo theo nghĩa vụ sửa mã nội bộ của mô-đun khác.

Hình này còn nhấn mạnh sự khác nhau giữa **đường thực thi đồng bộ** và **đường hệ quả sau giao dịch**. Đường thực thi đồng bộ xuất hiện ở các mũi tên đi từ Reservation Slice sang Ordering Slice, từ Ordering Slice sang Kitchen Slice, từ Ordering Slice sang Billing Slice và từ Billing Slice quay lại Reservation Slice để khép phiên bàn. Đường hệ quả sau giao dịch lại đi qua Outbox and Event Contracts rồi mới lan sang Inventory Projection, Reporting Projection, Notification Module và Audit Trail Module. Việc tách hai đường này là một nguyên tắc quan trọng của báo cáo: tác vụ tuyến đầu phải ngắn và giải thích được trạng thái, còn tác vụ hỗ trợ

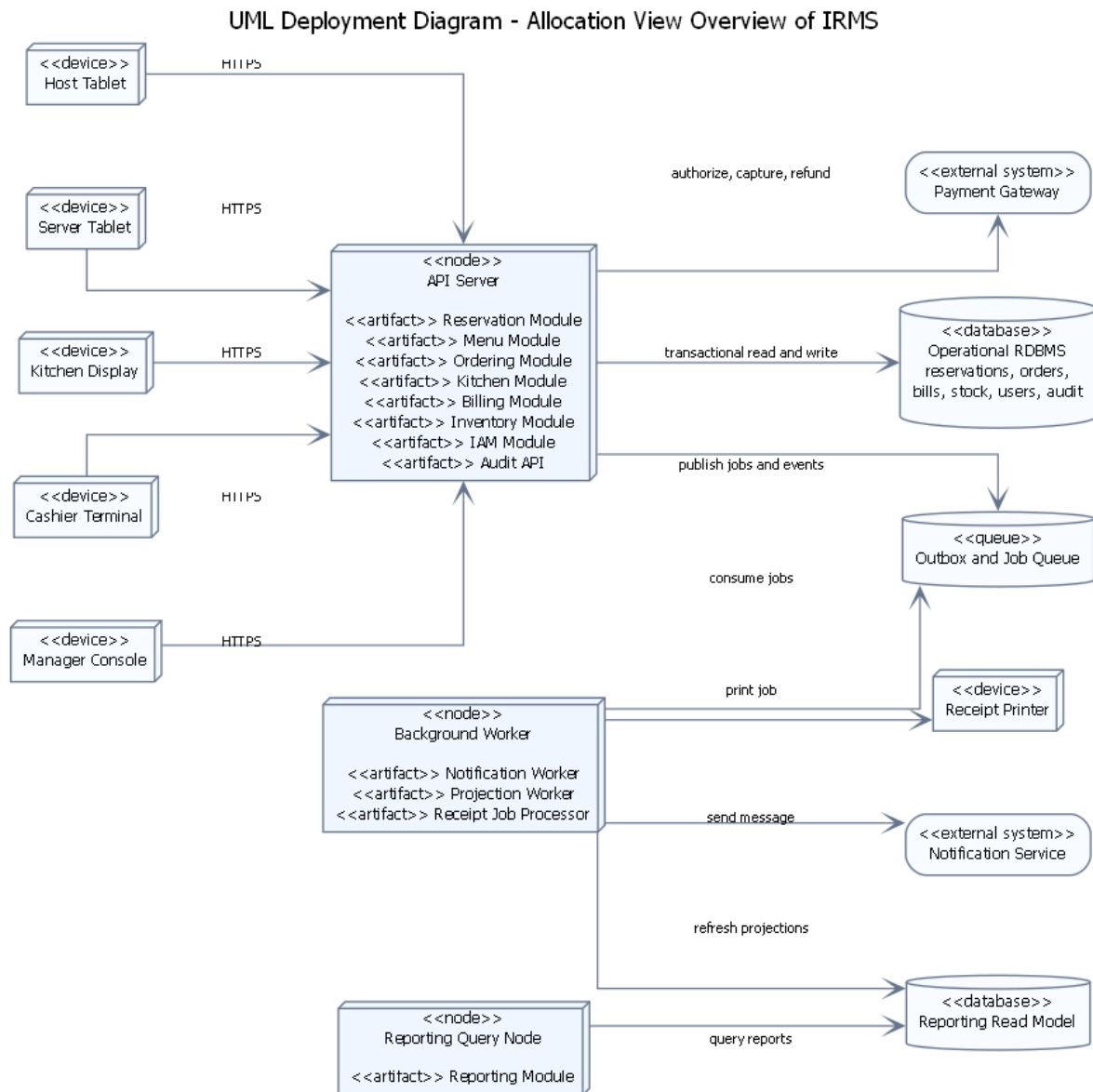
phải được tách nhíp để không làm nghẽn giao dịch.

Ngoài ra, sơ đồ còn cho thấy vai trò của **API and Policy Gate** như một mô-đun biên nằm trước các lát cắt nghiệp vụ. Trong thực tế hiện thực, đây là nơi hợp lý để gom các kiểm tra đầu vào, xác thực, kiểm tra quyền ở mức yêu cầu và chuẩn hóa lời gọi vào các mô-đun lõi. Nhờ có tầng biên này, các lát cắt phía sau không phải lặp lại quá nhiều logic kỹ thuật, trong khi phần điều phối ứng dụng vẫn giữ được biên giao dịch riêng của từng mô-đun.

Những điểm chính cần đọc từ hình.

- **Lát cắt mô-đun** được tổ chức theo chiều dọc: mỗi mô-đun có lớp ứng dụng, lớp miền và kho lưu trữ riêng, thay vì dùng chung một lớp kỹ thuật cho toàn bộ hệ thống.
- **Tuyến phụ thuộc tính** chỉ đi qua giao diện công khai và ảnh chụp nghiệp vụ cần thiết, nhờ đó tránh phụ thuộc chéo vào chi tiết nội bộ.
- **Tuyến hệ quả sự kiện** được tách sang **Outbox, Reporting, Notification** và một phần **Inventory**, bảo vệ đường nóng khỏi tác vụ phụ.
- **API and Policy Gate** là biên chung trước các lát cắt nghiệp vụ, giúp gom các kiểm tra kỹ thuật và kiểm tra quyền trước khi lời gọi đi vào từng mô-đun.
- **Quy tắc mô-đun** ở hình này là cơ sở để kiểm chứng phản hiện thực mã nguồn về sau: nếu mô-đun gọi xuyên qua nội tạng của nhau, kiến trúc sẽ bị phá vỡ ngay ở mức code.

3.3.5 Tổng quan cho góc nhìn phân bố



Hình 15: Sơ đồ tổng quan cho góc nhìn phân bố của IRMS

Hình 15 đưa ra bức tranh tổng quan cho góc nhìn phân bố. Nếu góc nhìn logic trả lời “hệ thống gồm những năng lực nào” và góc nhìn mô-đun trả lời “hệ thống phân rã thành những mô-đun nào”, thì góc nhìn phân bố phải trả lời “những mô-đun và năng lực đó cuối cùng được đặt ở đâu trong vùng thực thi, dữ liệu và hệ thống ngoài”. Hình này vì vậy ánh xạ ba chiều: **logic/mô-đun** → **phân bố thực thi** → **phân bố dữ liệu**.

Các lớp phân bố chính thể hiện trong hình.

- **Lớp biên và thiết bị:** là nơi phát sinh thao tác nghiệp vụ thực tế như máy của lễ tân, máy của phục vụ, màn hình bếp, máy thu ngân và màn hình quản lý.
- **Lớp phân bố thực thi:** gồm nút giao diện lập trình ứng dụng, bộ xử lý nền và dịch vụ truy vấn báo cáo; đây là nơi các năng lực được hiện thực thành tiến trình hoặc vùng thực thi cụ thể.

- **Lớp phân bổ dữ liệu:** gồm Operational RDBMS, Reporting Views / Read Model và Outbox / Job Queue; chúng tách dữ liệu giao dịch, dữ liệu đọc tổng hợp và hàng đợi xử lý nền.
- **Lớp hệ thống ngoài:** gồm Payment Gateway, Notification Service và Receipt Printer; chúng là các điểm tích hợp nằm ngoài lõi IRMS.

Lý do thiết kế của sơ đồ này.

- Sơ đồ giúp người đọc nhìn ra ngay **mô-đun nào chạy trên nút giao diện lập trình ứng dụng, mô-đun nào đi qua bộ xử lý nền, và mô-đun nào đọc từ mô hình báo cáo thay vì đánh trực tiếp vào cơ sở dữ liệu giao dịch.**
- Nó cho thấy rõ vì sao Notification và một phần Reporting không nên bị buộc vào tuyến giao dịch nóng.
- Nó là phần nối đúng giữa góc nhìn thành phần và góc nhìn triển khai: sơ đồ thành phần cho thấy thành phần hợp tác ra sao; góc nhìn phân bổ cho thấy các thành phần đó được đặt trên vùng thực thi và nơi lưu trữ nào.

3.4 Đặc trưng kiến trúc

Dựa trên các yêu cầu phi chức năng đã được lượng hóa ở phần tổng quan, những *đặc trưng kiến trúc* quan trọng nhất của IRMS là:

1. **Khả năng đáp ứng:** màn hình gọi món, màn hình bếp và màn hình thu ngân phải phản hồi nhanh để không làm chậm tuyến phục vụ trực tiếp với khách.
2. **Độ tin cậy:** đơn gọi món, thanh toán, hoàn tiền và cập nhật trạng thái bàn là các luồng không được mất dữ liệu hoặc xử lý lặp sai lệch.
3. **Khả năng mở rộng:** điểm nóng thường dồn vào giờ cao điểm và tập trung ở các miền Ordering, Kitchen Coordination và Billing thay vì toàn hệ thống.
4. **Bảo mật và khả năng kiểm toán:** mọi thao tác nhạy cảm phải được kiểm soát bằng quyền và truy vết lại được về sau.
5. **Khả năng quan sát:** hệ thống phải đủ nhật ký, vết thực thi và số đo để thấy độ trễ của phiếu bếp, thanh toán và các điểm nghẽn vận hành.
6. **Khả năng sửa đổi:** thực đơn, giá, khuyến mãi, ngưỡng cảnh báo tồn kho, kênh gửi thông báo và bộ kết nối thanh toán thay đổi thường xuyên nên cần biến thay đổi hợp.

Sáu đặc trưng trên không độc lập tuyệt đối mà thường kéo nhau theo hướng căng thẳng. Ví dụ, để tăng **khả năng đáp ứng**, hệ thống có xu hướng rút ngắn chuỗi gọi đồng bộ; nhưng để giữ **độ tin cậy**, một số bước như thanh toán vẫn phải nằm trong giao dịch hoặc giao thức chặt hơn. Tương tự, **khả năng sửa đổi** khuyến khích tách ranh giới nghiệp vụ, nhưng nếu tách quá sớm hoặc quá mạnh thì có thể làm tăng độ phức tạp tích hợp, ảnh hưởng ngược lại **khả năng đáp ứng** và **khả năng bảo trì**. Vì vậy, các quyết định kiến trúc ở phần sau đều được đọc dưới góc nhìn cân bằng giữa các đặc trưng này, không phải tối ưu một đặc tính duy nhất.

3.5 Góc nhìn phân rã

Ở góc nhìn phân rã, hệ thống được chia thành hai lớp lớn: **các ứng dụng phía người dùng** và **các dịch vụ nghiệp vụ phía sau**. Việc phân rã không dựa trên tầng kỹ thuật thuần túy, mà bám theo miền nghiệp vụ của nhà hàng.

3.5.1 Các mô-đun phía người dùng

- **Host/Reservation UI:** phục vụ đặt bàn, danh sách chờ, sơ đồ bàn và check-in.
- **Server POS UI:** phục vụ gọi món tại bàn, chỉnh món, theo dõi phiếu bếp và hỗ trợ tính tiền.
- **Kitchen Display UI:** hiển thị phiếu bếp theo trạm, mức ưu tiên, hạn hoàn thành và tiến độ chế biến.
- **Cashier UI:** tạo hóa đơn, tách hóa đơn, thanh toán, biên lai và hoàn tiền.

- **Manager Portal:** quản trị thực đơn, chương trình khuyến mãi, cảnh báo tồn kho, lịch làm việc và bảng điều khiển báo cáo.

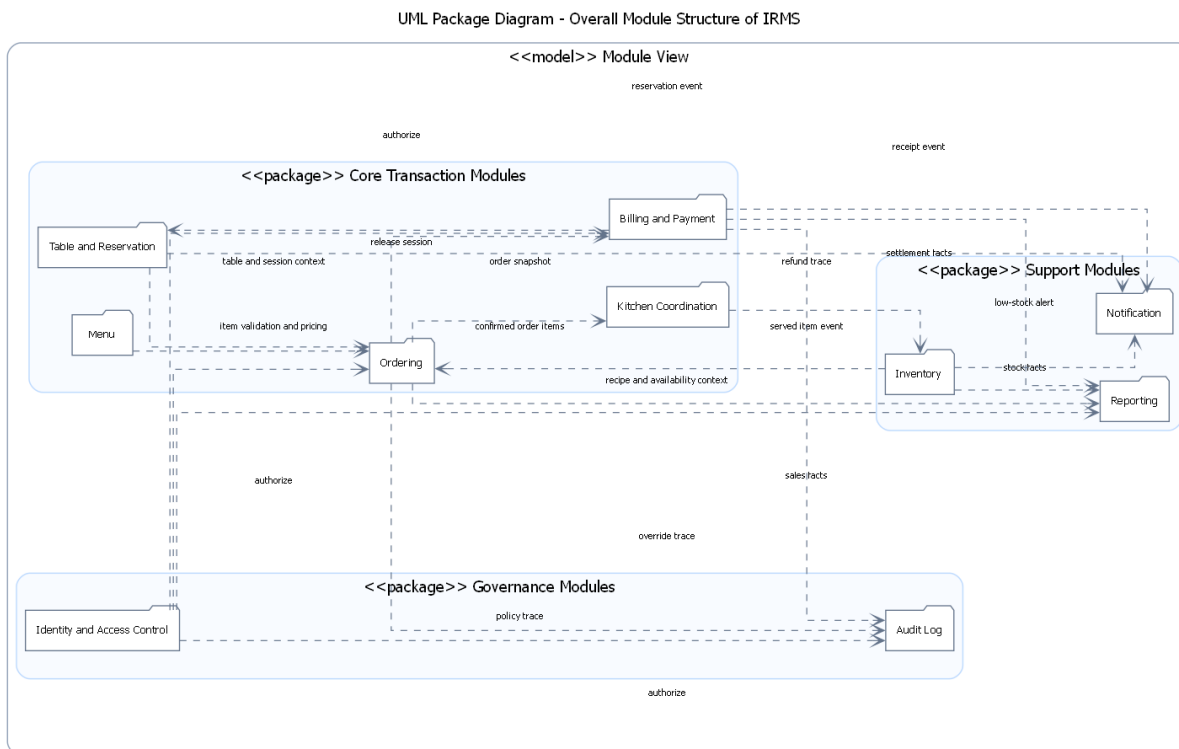
3.5.2 Các dịch vụ nghiệp vụ phía sau

- **Reservation & Seating Service:** quản lý đặt bàn, danh sách chờ, bàn ăn và phiên phục vụ tại bàn.
- **Ordering Service:** quản lý thực đơn, tùy chọn món, đơn gọi món, dòng món và trạng thái của đơn từ đầu đến cuối.
- **Kitchen Coordination Service:** điều phối phiếu bếp, trạm bếp và cam kết thời gian ra món.
- **Billing Service:** quản lý hóa đơn, tách hóa đơn, thanh toán, biên lai, hoàn tiền và áp dụng khuyến mãi.
- **Inventory Service:** quản lý công thức, mặt hàng tồn kho, giao dịch kho và quy tắc cảnh báo mức tồn thấp.
- **Reporting Service:** quản lý mô hình đọc, ảnh chụp dữ liệu báo cáo và xuất báo cáo.
- **IAM/Access Service:** định danh, vai trò, quyền và thực thi chính sách truy cập.

Lưu ý về thuật ngữ “dịch vụ”. Trong góc nhìn phân rã này, từ “dịch vụ” được dùng theo nghĩa **ranh giới năng lực nghiệp vụ ở mức logic**, không mặc nhiên có nghĩa rằng mỗi dịch vụ phải được triển khai thành một tiến trình mạng độc lập. Điểm này rất quan trọng để tránh nhầm lẫn giữa *ranh giới mô-đun* và *cách bố trí triển khai*. Một hệ thống có thể phân rã theo dịch vụ rất rõ về mặt logic, nhưng khi triển khai vẫn chạy tập trung trong một khối xử lý phía sau nếu điều đó phù hợp hơn với quy mô và ranh giới giao dịch.

Giải thích chi tiết cho góc nhìn phân rã. Phân rã như trên giúp ranh giới trách nhiệm rõ ràng: thay đổi ở bếp không kéo trực tiếp vào thanh toán; thay đổi ở thực đơn không làm nứt mô hình đặt bàn; báo cáo không đung vào luồng ghi thời gian thực. Đây là cách phân rã phù hợp với IRMS vì ranh giới miền nghiệp vụ của nhà hàng đã rất rõ theo thực tế vận hành.

3.5.3 Sơ đồ tổng quan



Hình 16: Góc nhìn mô-đun tổng quan của IRMS

Hình 16 thể hiện góc nhìn mô-đun ở mức đủ rộng để nhìn ra toàn bộ cấu trúc trách nhiệm của hệ thống mà chưa sa vào chi tiết cách hiện thực. Đây là một trong những sơ đồ quan trọng nhất của báo cáo vì nó trả lời câu hỏi mà bất kỳ người phân biện nào cũng sẽ quan tâm: hệ thống được chia thành các khối nào, mỗi khối thật sự chịu trách nhiệm cho điều gì, và ranh giới nào là ranh giới kiến trúc đáng bảo vệ.

Điều cần đọc đầu tiên ở hình này không phải là số lượng mô-đun, mà là **cách chia mô-đun theo nghĩa nghiệp vụ**. Reservation không chỉ là nơi lưu đặt bàn; nó giữ toàn bộ quy tắc về tiếp nhận khách, danh sách chờ, sức chứa và mở phiên bàn. Ordering không chỉ là nơi ghi dòng món; nó giữ ảnh chụp thực đơn tại thời điểm gọi, các tùy chọn món, ghi chú đặc biệt và trạng thái đơn gọi món. Kitchen không phải là phần mở rộng nhỏ của gọi món; nó là miền chịu trách nhiệm cho hàng chờ chế biến, ưu tiên, đồng bộ trạng thái và bàn giao món. Billing là miền quyết toán có vòng đời riêng. Khi được tách như vậy, mô-đun trở thành nơi trú ngụ của quyết định nghiệp vụ, không chỉ là vỏ bọc kỹ thuật.

Một điểm quan trọng khác của sơ đồ là **các phụ thuộc giữa mô-đun**. Ordering cần biết ngữ cảnh bàn đang phục vụ và cần lấy thông tin món từ menu, nhưng không vì vậy mà được ôm luôn toàn bộ quy tắc của bàn hay toàn bộ vòng đời khuyến mãi. Billing cần ảnh chụp đáng tin cậy của những gì đã được gọi và phục vụ, nhưng không nên quay ngược trở lại để điều khiển trực tiếp Kitchen. Inventory, Reporting, Notification, Audit và IAM được bố trí thành các mô-đun hỗ trợ hoặc xuyên suốt nhằm nhấn mạnh rằng chúng rất quan trọng, nhưng không nên chen vào mọi quyết định ở tuyến giao dịch nóng. Cách đặt đó giúp người đọc thấy ngay nơi nào phải ưu tiên tính đúng đắn tức thời, nơi nào có thể chấp nhận độ trễ nhỏ và nơi nào phải ưu tiên khả năng truy vết.

Sơ đồ còn quan trọng ở chỗ nó làm rõ **điểm dễ hỏng nếu thiết kế kém kỹ luật**. Trong một hệ thống nhà hàng, rất nhiều khái niệm có liên hệ chặt với nhau: Order liên quan đến Bill; KitchenTicket liên quan đến OrderItem; tồn kho lại suy diễn từ các món đã phục vụ hoặc các quy tắc nghiệp vụ khác. Nếu không có ranh giới mô-đun đủ chặt, hệ thống sẽ rất dễ rơi vào tình trạng bất kỳ thay đổi nhỏ nào ở khuyến mãi, thực đơn hay thanh toán cũng kéo theo một chuỗi sửa chéo trên nhiều vùng mã. Hình này vì vậy không chỉ mô tả cấu trúc hiện

tại, mà còn là công cụ để tự kiểm tra chất lượng kiến trúc trong suốt vòng đời phát triển.

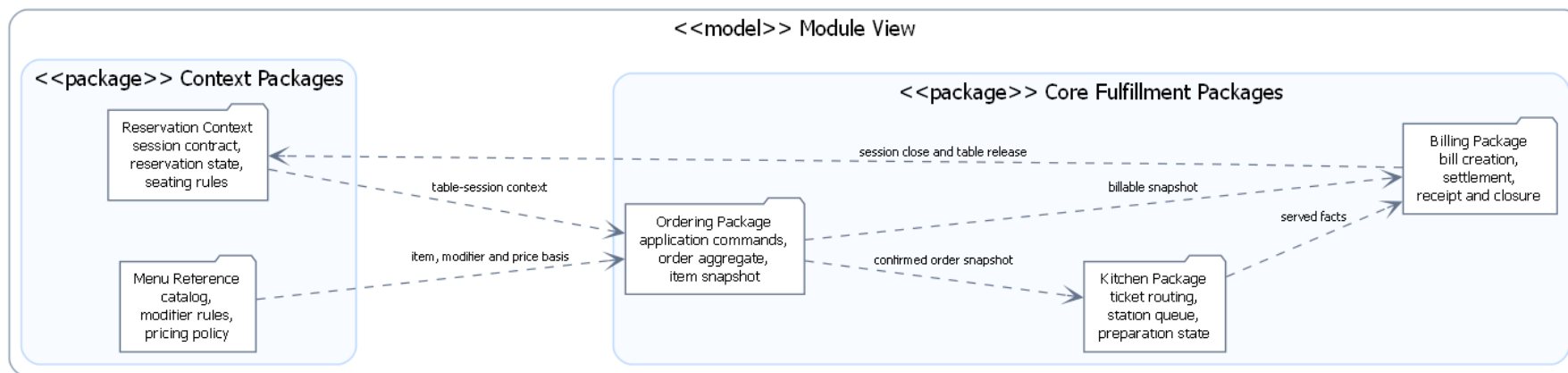
Các thành phần chính thể hiện trong hình.

- **Các mô-đun nghiệp vụ lõi:** Reservation, Ordering, Kitchen, Billing.
- **Các mô-đun hỗ trợ và xuyên suốt:** Inventory, Reporting, Notification, Audit, IAM.
- **Các phụ thuộc mô-đun:** cho thấy dữ liệu hoặc quyết định nào đi qua biên mô-đun nào.

Lý do thiết kế của sơ đồ này.

- Đây là sơ đồ tổng quan của góc nhìn mô-đun vì nó mô tả toàn bộ cấu trúc phân rã trước khi tách riêng tuyến lõi và tuyến hỗ trợ.
- Việc nhìn dependencies ở mức mô-đun giúp người đọc đánh giá nhanh mức kết tụ và mức phụ thuộc chéo của kiến trúc.

UML Package Diagram - Core Fulfillment Path of IRMS



Hình 17: Góc nhìn mô-đun của luồng thực thi nghiệp vụ lõi

Hình 17 đi sâu vào tuyến thực thi nghiệp vụ lõi của IRMS, tức tuyến mà mỗi giây chậm đi hoặc mỗi quyết định sai lệch đều tác động trực tiếp tới khách hàng đang ngồi tại bàn. Đây là vùng kiến trúc phải được đọc cẩn thận nhất vì nó quyết định hệ thống có thực sự dùng được trong giờ cao điểm hay không.

Trung tâm của hình là *Ordering*. Cách đặt *Ordering* ở tâm không nhằm làm mô-đun này trở nên “quyền lực”, mà để phản ánh đúng bản chất dữ liệu của bài toán. Tại thời điểm gọi món, hệ thống phải biết khách đang ở bàn nào, phiên bản có còn hiệu lực hay không, món còn bán hay không, cấu hình tùy chọn có hợp lệ hay không, giá nào cần được chụp lại để về sau hóa đơn không bị sai lệch, và sau khi xác nhận thì thông tin nào phải được lan truyền sang bếp. Tất cả các quyết định đó gặp nhau ở *Ordering*. Vì vậy, nếu đặt trung tâm vào nơi khác, mô hình sẽ méo theo kỹ thuật chứ không còn bám đúng nghiệp vụ.

Tuy nhiên, chính vì ở trung tâm nên *Ordering* cũng là mô-đun dễ bị phình to nhất. Nếu cả khuyến mãi phức tạp, tính toán kho, gửi thông báo, phân tích và kiểm toán chi tiết đều bị dồn vào cùng một chỗ, mô-đun này sẽ nhanh chóng trở thành “điểm dồn mọi thứ”. Hình này vì vậy không chỉ cho thấy vai trò trung tâm của *Ordering*, mà còn nhắc rất rõ biên của nó: nó phải tập trung vào dữ liệu gọi món và chuyển tiếp nghiệp vụ trực tiếp; còn các hệ quả hỗ trợ cần được chuyển sang mô-đun khác hoặc luồng xử lý khác.

Mối quan hệ giữa *Ordering*, *Kitchen* và *Billing* trong hình cũng rất quan trọng. Sau khi đơn gọi món được xác nhận, *Kitchen* phải nhận đủ thông tin để tổ chức chế biến nhưng không nên quyết định ngược trở lại những bất biến gốc của đơn gọi món. Tương tự, *Billing* cần một ảnh chụp đáng tin cậy của món đã gọi và món đã phục vụ; nhưng miền quyết toán không nên trở thành nơi quay lại chỉnh sửa cấu trúc gọi món gốc. Cách sắp xếp này giúp chuỗi nghiệp vụ có hướng đi rõ ràng: từ ngữ cảnh bàn, sang xác nhận đơn gọi món, sang điều phối bếp, rồi tới quyết toán. Khi hướng đi rõ, ranh giới giao dịch, khóa đồng thời và xử lý ngoại lệ cũng dễ được xác định hơn.

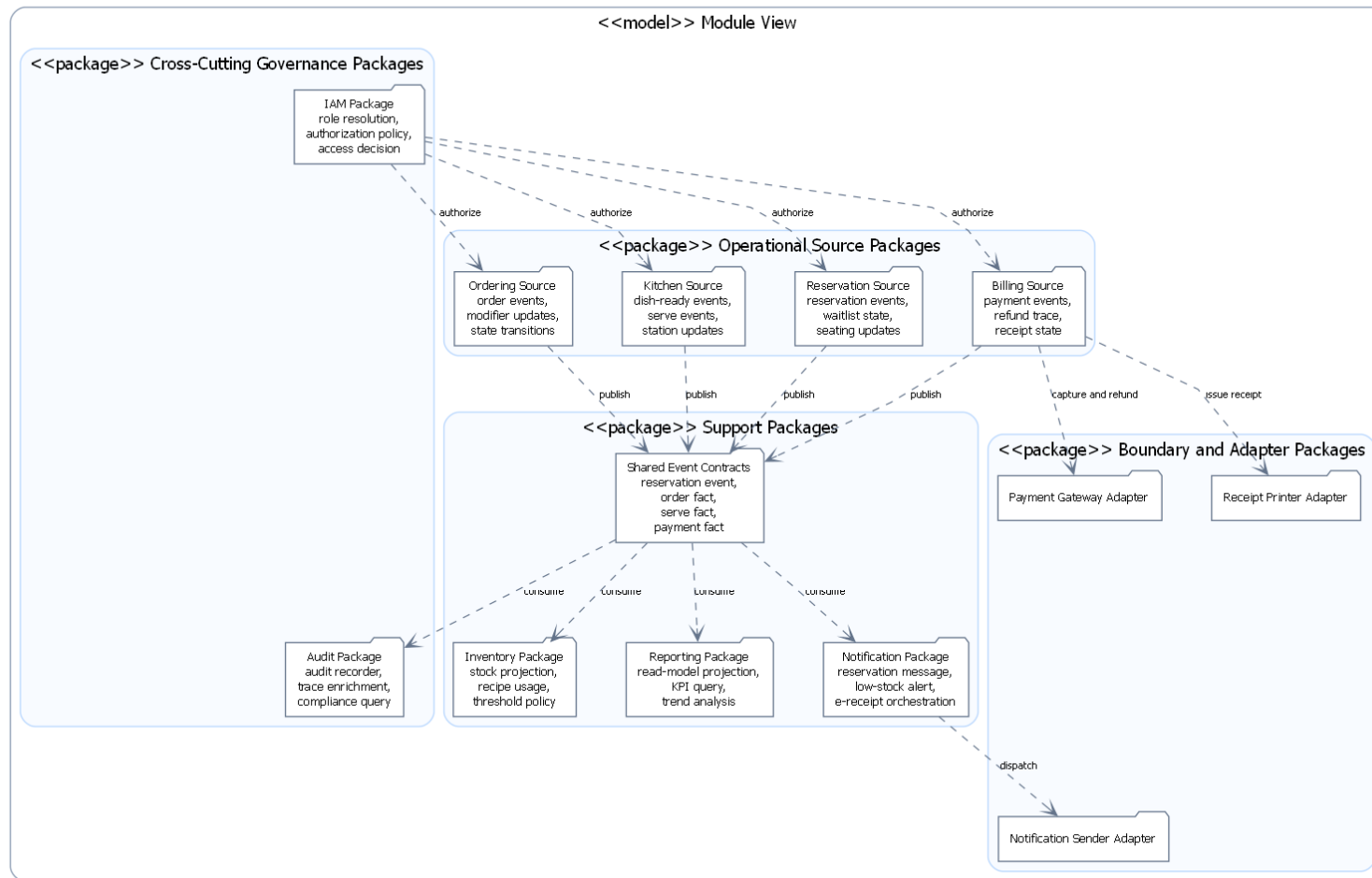
Các thành phần chính thể hiện trong hình.

- **Reservation và TableSession:** cung cấp ngữ cảnh bàn đang phục vụ.
- **Ordering:** giữ vai trò trung tâm của tuyến nghiệp vụ nóng.
- **Kitchen và Billing:** là hai hướng lan truyền quan trọng sau khi đơn được xác nhận.

Lý do thiết kế của sơ đồ này.

- Sơ đồ được tách riêng để người đọc nhìn rõ đường nghiệp vụ nóng thay vì bị chìm trong toàn bộ hệ thống.
- Việc nhấn mạnh tâm của *Ordering* giúp giải thích vì sao mô-đun này cần được bảo vệ khỏi việc ôm thêm quá nhiều trách nhiệm.

UML Package Diagram - Supporting and Cross-Cutting Modules of IRMS



Hình 18: Góc nhìn mô-đun của các mô-đun hỗ trợ và xuyên suốt

Hình 18 giải thích phần còn lại của kiến trúc, tức những mô-đun không trực tiếp đứng trên đường giao dịch nóng nhưng vẫn quyết định chất lượng vận hành của hệ thống trong dài hạn. Đây là phần thường bị trình bày quá mỏng trong nhiều tài liệu, khiến các năng lực hỗ trợ bị nhìn như phần phụ. Với IRMS, điều đó không đúng: Inventory, Reporting, Notification, Identity & Access Control và Audit & Compliance đều là những mô-đun hạng nhất, chỉ khác ở chỗ chúng phục vụ mục tiêu khác với mô-đun gọi món và thanh toán.

Inventory là mô-đun suy diễn từ kết quả vận hành sang biến động kho; Reporting chuyển dữ liệu nghiệp vụ thành góc nhìn quản trị; Notification chịu trách nhiệm lan truyền thông tin ra ngoài; còn Identity & Access Control và Audit Log bảo vệ tính kiểm soát của toàn hệ thống. Sơ đồ hiện tại làm rõ thêm một điểm rất quan trọng: các mô-đun hỗ trợ không tồn tại lơ lửng, mà được nuôi bởi **các mô-đun nguồn vận hành** như Reservation, Ordering, Kitchen và Billing thông qua một lớp Shared Event Contracts. Cách tổ chức này làm rõ sự khác nhau giữa mô-đun sinh dữ kiện vận hành, hợp đồng trung gian mang nghĩa nghiệp vụ, mô-đun tiêu thụ dữ kiện và bộ kết nối đặt ở rìa hệ thống.

Tuy vậy, tách ra riêng không có nghĩa là xem nhẹ. Chính các mô-đun hỗ trợ này mới giúp IRMS trở thành một hệ thống vận hành hoàn chỉnh thay vì chỉ là công cụ nhập món. Nhà quản lý cần số liệu; bộ phận vận hành cần cảnh báo tồn kho; tổ chức cần khả năng truy vết thao tác; hệ thống cần phân quyền đủ chặt để tránh lạm dụng quyền hạn. Có thể hiểu ngắn gọn rằng tuyên lối phản ánh trạng thái phục vụ đang diễn ra, còn nhóm mô-đun hỗ trợ phản ánh cách toàn bộ nhà hàng được điều hành, quan sát và kiểm soát.

Đánh đổi đi cùng cách tổ chức này là hệ thống phải chấp nhận một mức **độ trễ quan sát có kiểm soát**. Báo cáo có thể chậm hơn vài giây hoặc vài phút; cảnh báo mức tồn thấp có thể đến sau giao dịch gốc một khoảng nhỏ; dữ liệu kiểm toán có thể được làm giàu thêm chi tiết sau khi hành động chính đã hoàn tất. Nhưng đó là đánh đổi đúng trong bối cảnh nhà hàng, vì cái cần phản hồi ngay là đơn gọi món, bếp và thanh toán; còn cái cần đúng, đủ và truy vết được ở tầng quản trị có thể đi sau một nhịp nhỏ mà vẫn hoàn toàn chấp nhận được.

Một điểm cần nhấn mạnh thêm là Identity & Access Control không đi cùng Shared Event Contracts. Đây là chủ ý thiết kế. Phân quyền là ràng buộc được áp trước hoặc ngay trong lúc xử lý yêu cầu, không phải hệ quả đi sau giao dịch. Vì vậy, mô-đun này được vẽ như một lớp điều tiết trực tiếp lên các mô-đun nguồn. Ngược lại, Audit & Compliance vừa có phần kiểm soát chính sách, vừa có phần tiêu thụ dấu vết sau hành động; bởi vậy trong sơ đồ nó đứng giữa vai trò governance và vai trò hậu kiểm. Cách đặt đó giúp hình không chỉ liệt kê mô-đun, mà còn làm rõ nhịp can thiệp của từng mô-đun vào đời sống vận hành của hệ thống.

Các thành phần chính thể hiện trong hình.

- **Reservation, Ordering, Kitchen và Billing:** là các mô-đun nguồn phát sinh dữ kiện vận hành để nhóm hỗ trợ tiêu thụ.
- **Shared Event Contracts:** là lớp hợp đồng trung gian chuẩn hóa các dữ kiện được phát ra từ mô-đun nguồn trước khi sang nhóm hỗ trợ.
- **Inventory, Reporting và Notification:** đại diện cho nhóm mô-đun suy diễn, tổng hợp và tích hợp có thể chạy lệch nhịp với đường nóng.
- **Identity & Access Control và Audit & Compliance:** đại diện cho nhóm kiểm soát xuyên suốt.
- **Các bộ kết nối ngoài:** cho thấy ranh giới cuối cùng giữa mô-đun hỗ trợ của IRMS và hạ tầng hoặc đối tác bên ngoài.

Lý do thiết kế của sơ đồ này.

- Tách sơ đồ này ra khỏi sơ đồ tuyên lối giúp người đọc thấy rõ phần nào không nên chen trực tiếp vào tuyên giao dịch.
- Hình này cũng làm rõ tại sao một số phần có thể chấp nhận độ trễ nhỏ nhưng vẫn phải giữ được tính truy vết và khả năng kiểm soát.

3.6 Góc nhìn thành phần và kết nối

Ở góc nhìn thành phần và kết nối, hệ thống được mô tả bằng các thành phần thực thi chính và các kiểu liên kết giữa chúng. Góc nhìn này đặc biệt quan trọng vì nó cho biết một yêu cầu đi qua những khối nào, khối nào giữ

quyết định nghiệp vụ và khối nào chỉ đóng vai trò tích hợp hoặc hỗ trợ kỹ thuật.

3.6.1 Các thành phần chính

- **Các ứng dụng giao diện:** gồm các giao diện cho lễ tân, phục vụ, bếp, thu ngân và quản lý. Nhóm này chịu trách nhiệm nhận thao tác của người dùng và hiển thị kết quả theo đúng ngữ cảnh công việc.
- **Cổng vào giao diện lập trình ứng dụng:** là điểm vào thống nhất của hệ thống, chịu trách nhiệm xác thực, kiểm tra yêu cầu, định tuyến và ghi nhật ký ở biên.
- **Các dịch vụ ứng dụng:** điều phối các ca sử dụng thuộc các miền đặt bàn, gọi món, điều phối bếp, thanh toán, tồn kho, báo cáo và quản trị.
- **Các mô-đun miền nghiệp vụ:** chứa thực thể gốc, dịch vụ miền, chính sách và các bất biến nghiệp vụ cốt lõi.
- **Lớp lưu trữ dữ liệu:** thực hiện lưu trữ và truy xuất dữ liệu giao dịch thông qua kho lưu trữ, bộ quản lý giao dịch và bộ kết nối cơ sở dữ liệu.
- **Bộ phát sự kiện và bộ xử lý nền:** phát sự kiện, xử lý tác vụ nền, thử lại khi lỗi và làm mới mô hình đọc.
- **Các bộ kết nối ngoài:** kết nối tới công thanh toán, kênh gửi thông báo và thiết bị in biên lai mà không để logic nghiệp vụ phụ thuộc trực tiếp vào đối tác ngoài.

3.6.2 Các kiểu kết nối chính

Bảng 17: Các kiểu kết nối chính trong góc nhìn thành phần và kết nối

Kiểu kết nối	Vai trò	Ví dụ trong IRMS
Giao thức HTTP và lớp giao tiếp ứng dụng	giao tiếp giữa thiết bị người dùng và khối xử lý phía sau, ưu tiên phản hồi nhanh và thông điệp lỗi rõ ràng	các giao diện lễ tân, phục vụ và thu ngân gửi yêu cầu nghiệp vụ
Lời gọi nội bộ trong tiến trình	điều phối ca sử dụng bên trong khối xử lý phía sau	bộ điều khiển gọi Application Service
Kết nối kho lưu trữ và giao dịch	đọc ghi dữ liệu giao dịch và bảo vệ ranh giới giao dịch	ghi Order, Bill, Payment, TableSession
Công bố và tiếp nhận sự kiện	lan truyền trạng thái hoặc kích hoạt xử lý phụ	OrderConfirmed, DishServed, PaymentCompleted
Bộ kết nối thích nghi	giao tiếp với hệ thống ngoài mà không để miền nghiệp vụ phụ thuộc trực tiếp	thanh toán, gửi thông báo, in biên lai
Kết nối truy vấn	đọc mô hình đọc hoặc khung nhìn báo cáo	bảng điều khiển quản lý, xuất báo cáo doanh thu

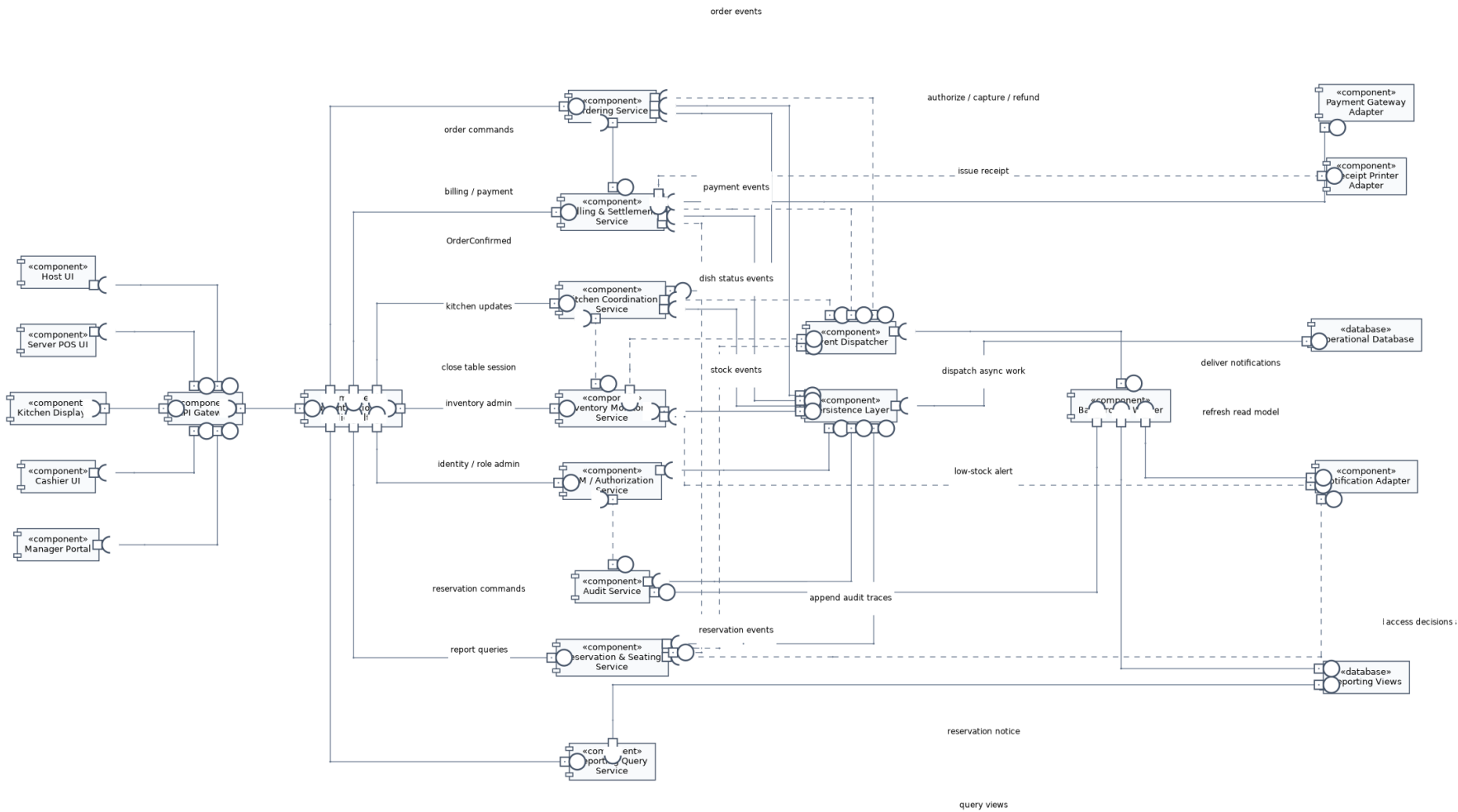
3.6.3 Lý do chọn mô hình kết nối hỗn hợp

IRMS không thể dùng một kiểu kết nối duy nhất cho mọi luồng nghiệp vụ. Nếu tất cả đều gọi đồng bộ nối chuỗi, hệ thống sẽ dễ nghẽn vào giờ cao điểm, đặc biệt trên các màn hình gọi món, màn hình bếp và màn hình thu ngân. Ngược lại, nếu tất cả đều chuyển sang xử lý bất đồng bộ, người dùng sẽ khó biết thao tác nào đã thành công, thao tác nào đang chờ và thao tác nào thất bại.

Vì vậy, kiến trúc dùng mô hình kết nối hỗn hợp với hai nguyên tắc. Thứ nhất, các bước nằm trên tuyến phục vụ trực tiếp cho khách như xác nhận đơn gọi món, chuyển món sang bếp, cập nhật trạng thái sẵn sàng và ghi nhận

thanh toán phải ưu tiên phản hồi ngay. Thứ hai, các luồng hỗ trợ như cập nhật báo cáo, gửi thông báo, làm giàu nhật ký kiểm toán hoặc cảnh báo tồn kho nên đi qua sự kiện và bộ xử lý nền để không chặn tuyến giao dịch chính.

Giải thích chi tiết cho góc nhìn thành phần và kết nối. Điểm then chốt của IRMS là phân biệt *kết nối phục vụ trải nghiệm người dùng* với *kết nối phục vụ phối hợp nội bộ*. Chọn đúng kiểu kết nối cho từng nhóm thành phần sẽ quyết định trực tiếp đến độ trễ phản hồi, khả năng giải thích lỗi và khả năng mở rộng của toàn hệ thống.



Hình 19: Sơ đồ thành phần và kết nối tổng quan của IRMS

Hình 19 cho thấy cấu trúc thực thi tổng quan của hệ thống theo cách nhìn thành phần. Thiết bị người dùng chỉ giao tiếp với lớp công vào; từ đó yêu cầu đi qua lớp điều phối ca sử dụng, xuống miền nghiệp vụ, lớp lưu trữ và các bộ kết nối ra ngoài. Ý nghĩa học thuật của hình này là chứng minh hệ thống không được tổ chức như một khối gọi chéo tùy ý, mà có hướng phụ thuộc rõ ràng từ ngoài vào trong.

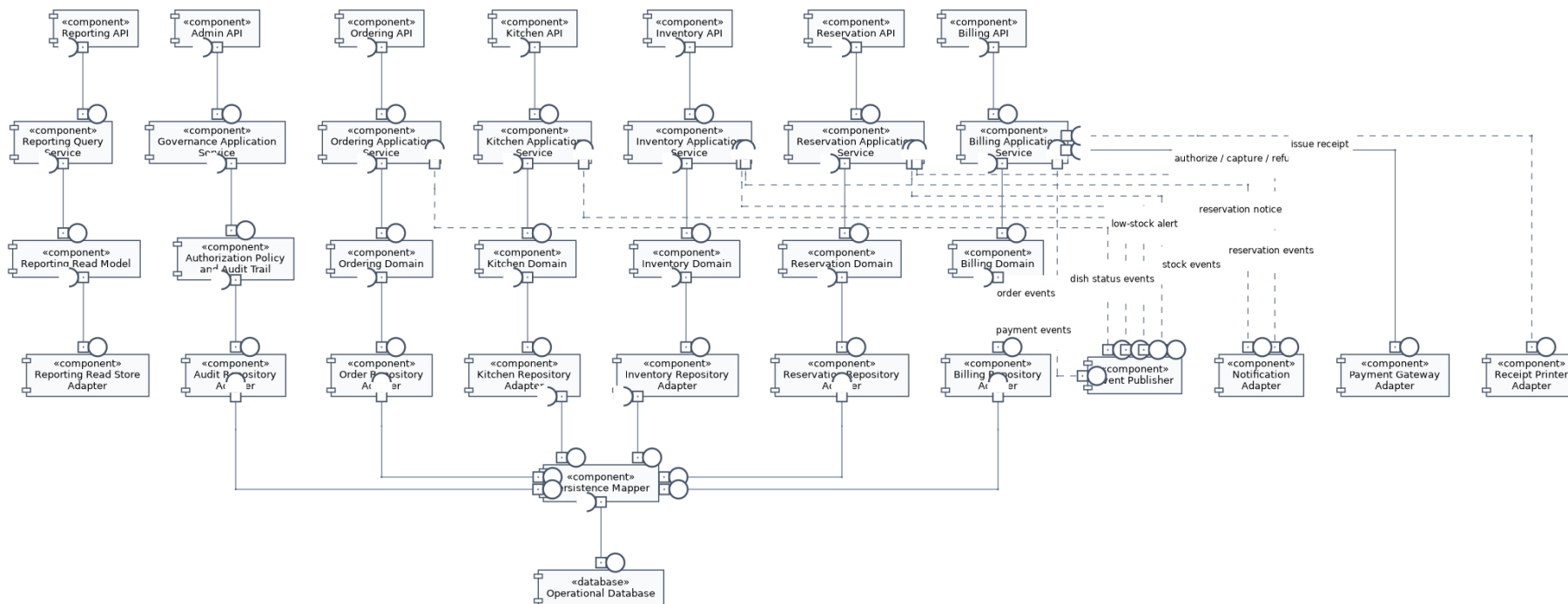
Các thành phần chính thể hiện trong hình.

- **Các ứng dụng giao diện:** là nơi phát sinh yêu cầu từ lễ tân, phục vụ, bếp, thu ngân và quản lý.
- **Cổng vào giao diện lập trình ứng dụng:** tiếp nhận yêu cầu, xác thực, kiểm tra đầu vào và chuyển tiếp tới đúng ca sử dụng.
- **Các dịch vụ ứng dụng:** điều phối từng ca sử dụng cụ thể như tạo đặt bàn, xác nhận đơn gọi món, cập nhật phiếu bếp hoặc chốt hóa đơn.
- **Các mô-đun miền nghiệp vụ:** giữ luật nghiệp vụ cốt lõi, bất biến dữ liệu và quyết định nghiệp vụ của nhà hàng.
- **Lớp lưu trữ dữ liệu:** chịu trách nhiệm lưu trữ trạng thái giao dịch và bảo vệ ranh giới giao dịch.
- **Bộ phát sự kiện và bộ xử lý nền:** xử lý sự kiện và tác vụ nền không nên nằm trên tuyến giao dịch nóng.
- **Các bộ kết nối ngoài:** bao bọc các tích hợp ra ngoài như thanh toán, gửi thông báo và in biên lai.

Lý do thiết kế của sơ đồ này.

- Việc đặt **Cổng vào giao diện lập trình ứng dụng** ở phía trước giúp thống nhất xác thực, kiểm tra yêu cầu và ghi nhật ký ở biên thay vì lặp lại ở mọi điểm vào.
- Việc tách **Các dịch vụ ứng dụng** khỏi **Các mô-đun miền nghiệp vụ** giúp phần điều phối ca sử dụng không lẫn với luật nghiệp vụ cốt lõi.
- Việc đẩy **Các bộ kết nối ngoài** ra rìa giúp miền nghiệp vụ không phụ thuộc trực tiếp vào cổng thanh toán, dịch vụ gửi thông báo hay thiết bị in.
- Việc tách **Bộ phát sự kiện và bộ xử lý nền** thành thành phần riêng giúp các tác vụ phụ không làm chậm các thao tác đang diễn ra trực tiếp trước khách hàng.

Ưu điểm của sơ đồ này là chiều phụ thuộc được kiểm soát rõ: giao diện không phụ thuộc trực tiếp vào miền nghiệp vụ, miền nghiệp vụ không gọi ngược bộ điều khiển, còn bộ kết nối ngoài chỉ xuất hiện ở rìa hệ thống. Nhược điểm là việc hiện thực phải giữ kỷ luật rõ ràng; nếu đưa luật nghiệp vụ vào bộ điều khiển hoặc để miền nghiệp vụ gọi thẳng bộ kết nối ngoài thì giá trị của kiến trúc sẽ bị phá vỡ.



Hình 20: Sơ đồ thành phần chi tiết của khối xử lý phía sau trong IRMS

Hình 20 là sơ đồ quan trọng nhất của góc nhìn thành phần vì nó mở lớp vỏ của khối xử lý phía sau và cho thấy hệ thống được tổ chức theo trật tự trách nhiệm như thế nào. Rất nhiều tài liệu chỉ dừng ở việc nói rằng hệ thống có giao diện, dịch vụ và cơ sở dữ liệu. Hình này đi xa hơn: nó chỉ ra thành phần nào được quyền giữ luật nghiệp vụ, thành phần nào chỉ nên điều phối tiến trình, và thành phần nào tuyệt đối không được chen ngược vào lỗi.

Lớp tiếp nhận chỉ làm việc của lớp tiếp nhận: nhận yêu cầu, kiểm tra hình dạng dữ liệu, chuyển đổi dữ liệu vào ra và trả kết quả. Điều quan trọng ở đây là nó không giữ luật nghiệp vụ. Khi lớp này phình to, mọi quy tắc quan trọng sẽ bị rải khắp các bộ điều khiển và hệ thống rất khó kiểm thử. **Lớp điều phối ứng dụng** là nơi ghép các bước của từng ca sử dụng: mở giao dịch, gọi đúng đối tượng miền nghiệp vụ, điều phối kho lưu trữ và quyết định điểm nào cần phát sự kiện. Lớp này không nên chứa bản thân các bất biến nghiệp vụ, nhưng phải đủ gần nghiệp vụ để điều phối đúng trình tự. **Lớp miền nghiệp vụ** mới là nơi cư trú của các đối tượng gốc, chính sách, quy tắc và bất biến cốt lõi. Đây là trái tim của thiết kế. Cuối cùng, **lớp hạ tầng kỹ thuật** giữ các chi tiết thay đổi nhanh như cơ sở dữ liệu, cổng thanh toán, kênh gửi thông báo và cơ chế phát sự kiện.

Sự phân lớp này đặc biệt quan trọng với IRMS vì hệ thống có nhiều loại thay đổi với tốc độ khác nhau. Quy trình thu ngân có thể thay đổi nhẹ theo chính sách nhà hàng; cách tính phí và khuyến mãi có thể đổi theo mùa; đối tác thanh toán hoặc máy in biên lai có thể thay. Nếu không có ranh giới giữa miền nghiệp vụ và chi tiết công nghệ, mọi thay đổi dạng đó sẽ lan thẳng vào lỗi. Hình này cho thấy kiến trúc đã chủ động chặn điều đó: bộ kết nối ngoài bị giữ ở rìa, kho lưu trữ bị ẩn sau giao diện trừu tượng, và phần quyết định nghiệp vụ được bảo vệ ở trung tâm.

Một giá trị khác của hình là nó chứng minh rằng các nguyên lý thiết kế được nói ở phần sau không chỉ mang tính khẩu hiệu. Khi quan sát sơ đồ này, người đọc có thể thấy trực tiếp nơi áp dụng trách nhiệm đơn nhất, nơi áp dụng phân tách giao diện và nơi áp dụng đảo ngược phụ thuộc. Nói cách khác, đây không chỉ là sơ đồ đẹp; đây là sơ đồ dùng để kiểm tra xem những lời biện minh về thiết kế có thực sự được phản ánh vào cấu trúc hay không.

Các thành phần chính thể hiện trong hình.

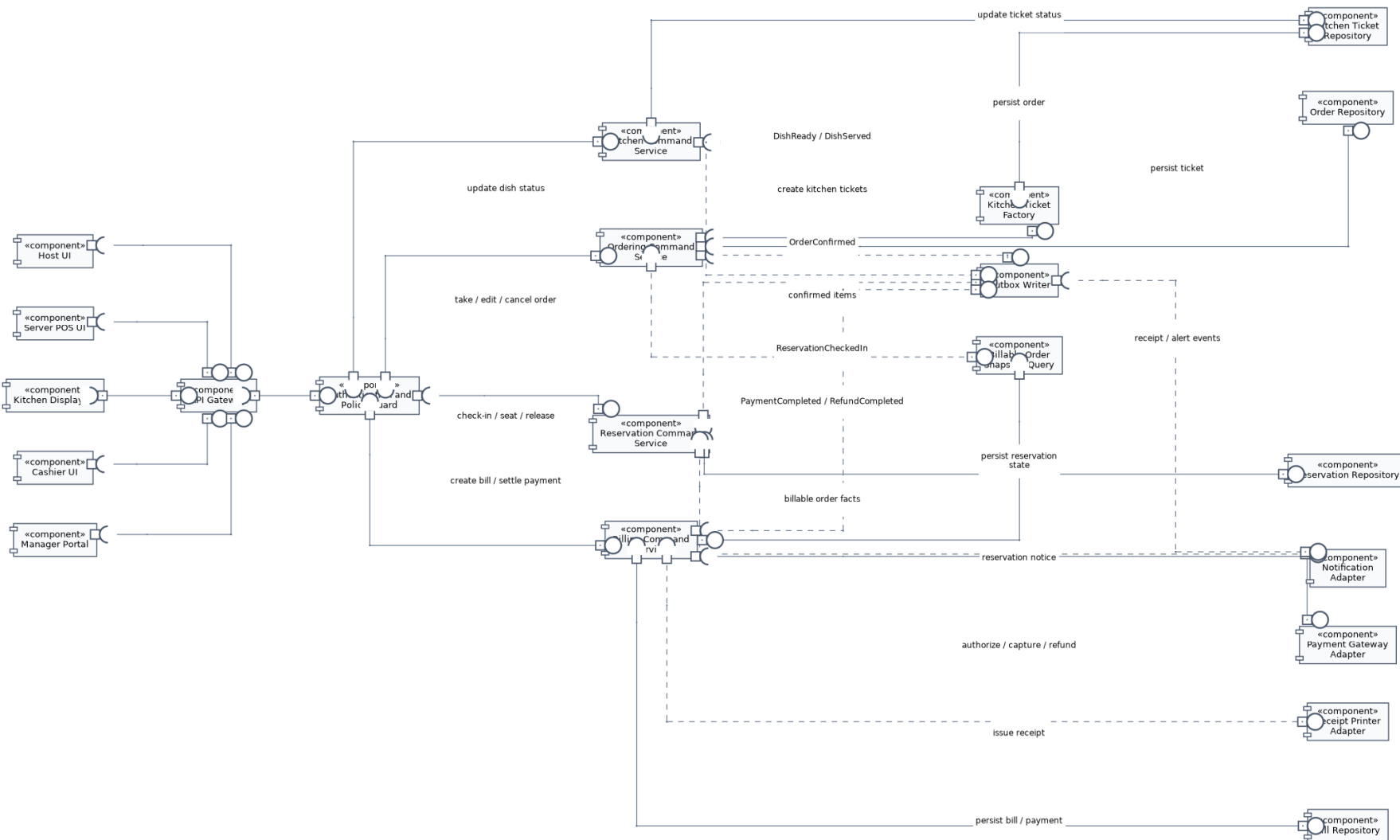
- **Lớp tiếp nhận:** tiếp nhận yêu cầu, chuyển đổi dữ liệu vào và dữ liệu ra, nhưng không giữ luật nghiệp vụ.
- **Lớp điều phối ứng dụng:** điều phối từng ca sử dụng, kiểm soát ranh giới giao dịch và gọi đúng đối tượng miền nghiệp vụ.
- **Lớp miền nghiệp vụ:** chứa thực thể gốc, chính sách nghiệp vụ, quy tắc kiểm tra và các bất biến cốt lõi.
- **Lớp hạ tầng kỹ thuật:** cung cấp lưu trữ, công bố sự kiện, kết nối cơ sở dữ liệu và tích hợp với hệ thống ngoài.
- **Kho lưu trữ, bộ kết nối, trục sự kiện nội bộ:** là các thành phần kỹ thuật cụ thể, được đặt ở lớp hạ tầng thay vì chen vào lỗi.

Lý do thiết kế của sơ đồ này.

- Sơ đồ chứng minh khối xử lý phía sau không phải một khối mờ, mà có cấu trúc nội bộ chặt chẽ và có chiều phụ thuộc rõ ràng.
- Việc tách lớp tiếp nhận, lớp điều phối ứng dụng, lớp miền nghiệp vụ và lớp hạ tầng giúp truy ra chính xác nơi nên đặt từng loại thay đổi.
- Đây là sơ đồ then chốt để nối góc nhìn thành phần với phần thiết kế chi tiết và phần cài đặt.

Điểm mạnh lớn nhất của cấu trúc này là khả năng giữ cho hệ thống phát triển lâu dài mà không bị rối. Đổi lại, nó đòi hỏi kỷ luật tổ chức mã nguồn ngay từ đầu: phải chấp nhận viết nhiều giao diện hơn, nhiều lớp điều phối hơn và nhiều lớp chuyển đổi dữ liệu hơn. Trong bối cảnh của IRMS, đó là chi phí hợp lý để đổi lấy một lỗi nghiệp vụ không bị bào mòn bởi thay đổi công nghệ.

3.6.4 Sơ đồ thành phần cho tuyến lệnh vận hành



Hình 21: Sơ đồ thành phần cho tuyến lệnh vận hành của IRMS

Hình 21 được đưa vào để giải thích thật rõ đường đi của một lệnh vận hành điển hình, từ lúc người dùng thao tác trên giao diện đến lúc dữ liệu lỗi được ghi bền vững và các hệ quả phụ được chuẩn bị để xử lý tiếp. Đây là sơ đồ rất quan trọng vì IRMS không chỉ cần “xử lý được”, mà còn cần xử lý theo một trình tự có thể giải thích và kiểm toán được.

Điểm đáng chú ý đầu tiên là yêu cầu đi qua lớp kiểm soát ở biên trước khi chạm vào lõi. Bộ lọc xác thực và kiểm tra chính sách không phải là chi tiết phụ. Chúng quyết định ai được phép gọi thao tác nào, trong bối cảnh nào, và với dữ liệu nào. Sau khi qua biên này, yêu cầu đi vào dịch vụ ứng dụng của từng vùng như Reservation, Ordering, Kitchen hoặc Billing. Dịch vụ ứng dụng ở đây chỉ nên điều phối: nó gọi đúng đối tượng miền nghiệp vụ, mở và đóng giao dịch, và quyết định thời điểm thích hợp để ghi hộp thư ra hoặc phát tín hiệu cho các phần hỗ trợ.

Điểm quan trọng thứ hai là sơ đồ tách rất rõ giữa **hoàn thành giao dịch lõi** và **xử lý hệ quả phụ**. Một thao tác như xác nhận đơn gọi món hay ghi nhận thanh toán phải đạt trạng thái bền vững trước. Chỉ sau khi phần đó an toàn, hệ thống mới nên phát thông tin sang bếp, làm mới báo cáo, gửi thông báo hoặc cập nhật phần đọc. Cách tổ chức này là nền tảng của luận điểm “lỗi đồng bộ, hỗ trợ bất đồng bộ”. Nó cho phép IRMS phản hồi nhanh ở tuyến đầu mà không đánh đổi tính đúng đắn của dữ liệu.

Điểm đáng đọc thứ ba là vị trí của các bộ kết nối như Payment Gateway Adapter, Receipt Printer Adapter và Notification Adapter. Chúng xuất hiện ở rìa thay vì ở giữa. Điều đó nói lên rằng công thanh toán, máy in hay kênh gửi thông báo không được phép định hình cấu trúc miền nghiệp vụ. Miền nghiệp vụ chỉ biết các năng lực mà nó cần; cách kết nối cụ thể được để cho lớp ngoài đảm nhiệm. Đây chính là điều làm cho thiết kế giữ được khả năng thay thế nhà cung cấp và cô lập lỗi khi tích hợp ngoài gặp trục trặc.

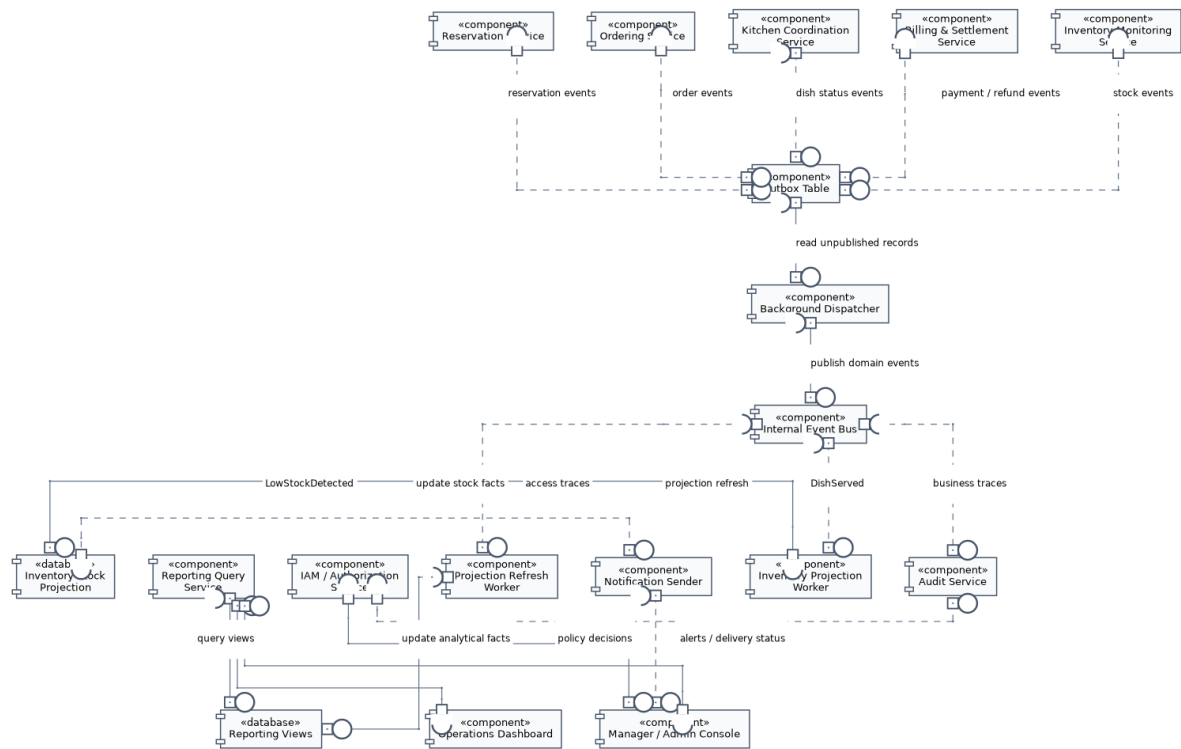
Các thành phần chính thể hiện trong hình.

- **Cổng vào giao diện lập trình ứng dụng và lớp bộ điều khiển** cùng **bộ lọc xác thực / kiểm tra chính sách**: tiếp nhận yêu cầu, chuẩn hóa dữ liệu vào, xác thực và chặn hành vi vượt quyền trước khi đi vào lõi.
- **Các dịch vụ ứng dụng** của từng vùng Reservation, Ordering, Kitchen và Billing: điều phối ca sử dụng, gọi đúng thành phần miền nghiệp vụ và kiểm soát biên giao dịch.
- **Các thành phần miền nghiệp vụ và kho lưu trữ**: hiện thực quy tắc miền và nơi lưu trạng thái bền vững.
- **Hộp thư ra / bộ phát sự kiện** cùng các bộ kết nối như Payment Gateway Adapter, Receipt Printer Adapter và Notification Adapter: tách bước chốt giao dịch khỏi bước tích hợp hoặc xử lý nền.

Lý do cần thêm sơ đồ này.

- Nó mô tả đường đi của một lệnh nghiệp vụ cụ thể, thay vì chỉ liệt kê danh mục thành phần.
- Nó bảo vệ luận điểm kiến trúc rằng những gì cần xác nhận ngay phải được chốt ở lõi trước, còn các hệ quả phụ đi sau qua cơ chế có kiểm soát.
- Nó làm rõ nơi đặt kiểm tra chính sách, giúp bảo mật và kiểm toán trở thành một phần hữu cơ của kiến trúc, không phải đoạn xử lý thêm về sau.

3.6.5 Sơ đồ thành phần cho xử lý nền, quản trị và mô hình đọc



Hình 22: Sơ đồ thành phần cho xử lý nền, quản trị và mô hình đọc

Hình 22 mô tả nửa còn lại của bức tranh thành phần: phần xử lý nền, phần quản trị và phần mô hình đọc. Nếu sơ đồ trước giải thích đường lệnh, sơ đồ này giải thích chuyện gì xảy ra sau khi giao dịch lỗi đã được chốt an toàn. Đây là nơi tư duy kiến trúc của IRMS bộc lộ rất rõ: hệ thống không đẩy mọi thứ vào đồng bộ, nhưng cũng không tùy tiện đưa mọi thao tác sang nền.

Điều cốt lõi của hình là nguyên tắc **sự kiện là hậu quả của giao dịch, không phải giao dịch tự nó**. Khi một thao tác lỗi hoàn tất, dữ liệu nghiệp vụ được ghi trước; sau đó Outbox Table, Background Dispatcher và Internal Event Bus mới tiếp nhận phân việc tiếp theo. Cách tổ chức này giúp giảm đáng kể nguy cơ mất đồng bộ kiểu “thông báo đã gửi nhưng giao dịch gốc chưa được ghi” hoặc “báo cáo đã cập nhật nhưng trạng thái gốc chưa chắc chắn”. Đối với một hệ thống có cả giao dịch tiền bạc lẫn điều phối vận hành như IRMS, đây là nguyên tắc sống còn.

Phần thứ hai cần chú ý là cách Projection Refresh Worker, Reporting Query Service và Reporting Views được đặt thành một chuỗi đọc riêng. Điều đó thể hiện một quyết định rõ ràng: báo cáo và bảng điều khiển không được tranh chấp trực tiếp với mô hình ghi nóng. Nhà quản lý có thể cần nhiều bộ lọc, nhiều thống kê, nhiều lát cắt dữ liệu theo giờ, theo ca, theo nhân viên hoặc theo trạm bếp; nhưng những nhu cầu đó không nên làm chậm giao diện phục vụ tại bàn, màn hình bếp hay giao diện thu ngân. Khi mô hình đọc được tách riêng, hệ thống có thể tối ưu phân đọc cho phân tích mà không làm ô nhiễm phân ghi giao dịch.

Phần thứ ba là vai trò của các thành phần quản trị như Audit Service, IAM / Authorization Service và Notification Sender. Hình này nhấn mạnh rằng kiểm toán, phân quyền và gửi thông báo không phải là những đoạn phụ dán thêm vào cuối chương trình. Chúng là các thành phần có vị trí kiến trúc rõ ràng, nhận dữ liệu từ tuyến giao dịch qua cơ chế có kiểm soát và thực hiện trách nhiệm theo nhịp phù hợp. Chính nhờ cách tổ chức này mà IRMS vừa có thể vận hành mượt ở tuyến đầu, vừa giữ được năng lực kiểm soát, truy vết và quản trị ở tuyến sau.

Các thành phần chính thể hiện trong hình.

- **Các thành phần sinh giao dịch:** là nơi phát sinh sự kiện nghiệp vụ từ các mô-đun ghi như đặt bàn, gọi món, bếp, thanh toán và tồn kho.

- **Outbox Table, Background Dispatcher và Internal Event Bus:** tạo đường chuyển tiếp có kiểm soát từ giao dịch sang xử lý nền.
- **Projection Refresh Worker, Reporting Query Service và Reporting Views:** xây dựng phía đọc cho bảng điều khiển và báo cáo quản trị.
- **Audit Service, IAM / Authorization Service và Notification Sender:** hiện thực các trách nhiệm quản trị và tích hợp ngoài mà không ép chúng phải đồng giao dịch với gọi món hoặc thanh toán.

Điểm kiến trúc được nhấn mạnh bởi sơ đồ.

- Giao dịch lỗi phải được hoàn tất trước; phần còn lại đi sau theo một đường có thể thử lại, giám sát và truy vết.
- Báo cáo đọc từ mô hình đọc riêng, không tranh chấp trực tiếp với mô hình ghi trong giờ cao điểm.
- Kiểm toán và phân quyền được đặt đúng chỗ trong kiến trúc, cho thấy chúng là thành phần hạng nhất chứ không phải tiện ích phụ.

3.7 Góc nhìn triển khai

Ở góc nhìn triển khai, hệ thống được ánh xạ lên hạ tầng vật lý và hạ tầng chạy chương trình như sau:

1. Tầng thiết bị đầu cuối

- Máy tính bảng dành cho nhân viên phục vụ.
- Thiết bị lễ tân hoặc ki-ốt tiếp nhận khách.
- Màn hình bếp tại từng trạm chế biến.
- Máy thu ngân.
- Trình duyệt của quản lý.

2. Tầng cổng vào và phân phối

- Bộ đảo ngược lưu lượng hoặc **API Gateway**.
- Máy chủ phân phối giao diện tĩnh hoặc giao diện phía máy chủ.

3. Tầng ứng dụng

- Khối xử lý phía sau chứa các mô-đun đặt bàn, gọi món, điều phối bếp, thanh toán, tồn kho, báo cáo và quản lý truy cập.
- Bộ xử lý nền thực hiện tác vụ sự kiện, hàng đợi thử lại và làm mới ảnh chụp dữ liệu báo cáo.

4. Tầng dữ liệu

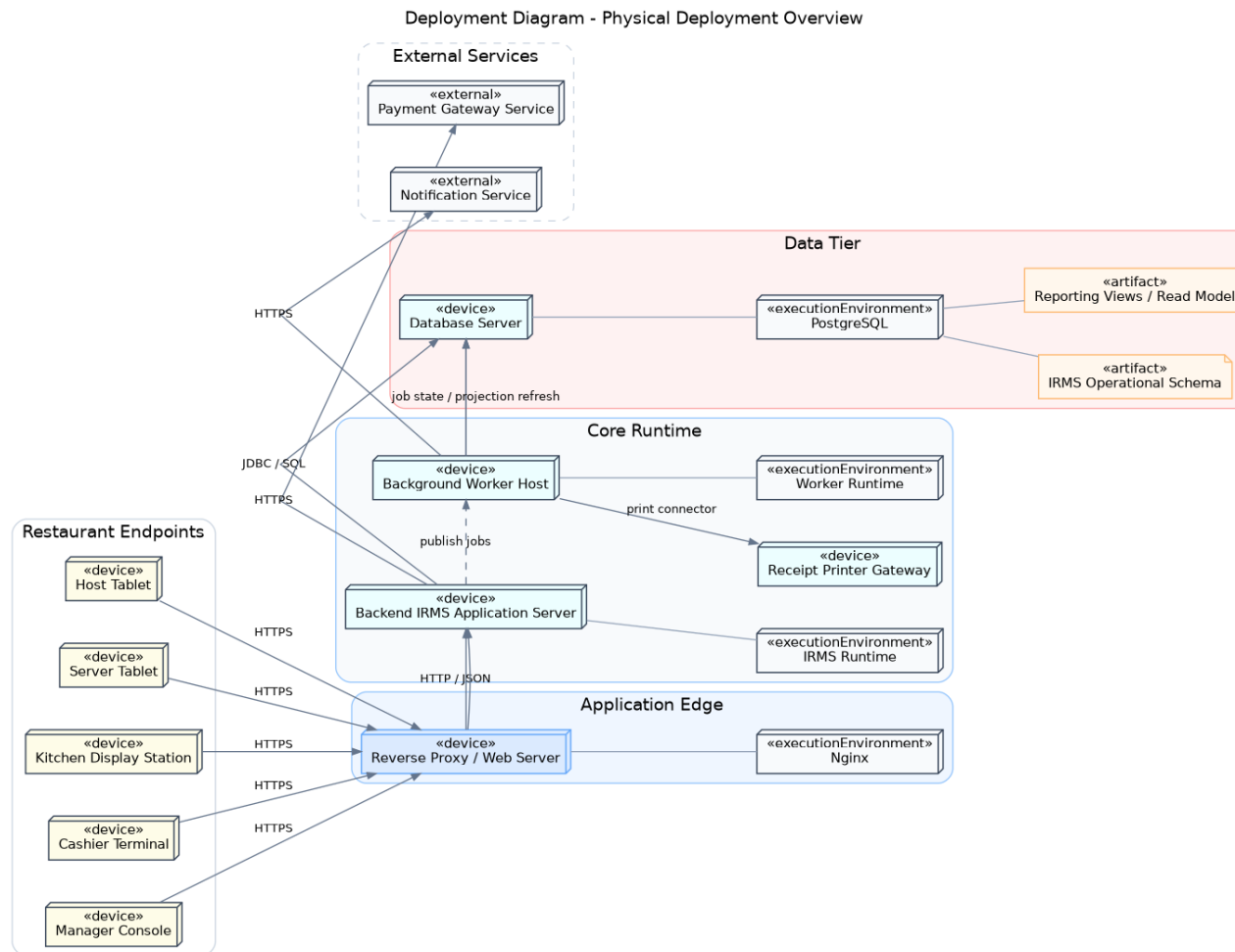
- Cơ sở dữ liệu giao dịch hoặc lược đồ giao dịch theo từng miền.
- Mô hình đọc và khung nhìn báo cáo.
- Kho lưu trữ đối tượng cho biên lai xuất ra hoặc tài nguyên của thực đơn.
- Hạ tầng nhật ký, số đo và vết thực thi.

3.7.1 Lý do thiết kế

- Thiết bị gọi món và thiết bị bếp phải đặt gần môi trường mạng nội bộ của nhà hàng và có cơ chế kết nối lại tốt vì đây là hai điểm bị ảnh hưởng trực tiếp nhất khi mạng dao động.
- Khối thanh toán và khối gọi món có yêu cầu sẵn sàng cao hơn các khối còn lại, nên cần được ưu tiên về giám sát, sao lưu và khả năng mở rộng.
- Khối báo cáo cần tách khỏi tuyến giao dịch để truy vấn phân tích không cản trở đường xử lý gọi món và thanh toán.

- Các nút dữ liệu phải có sao lưu và chính sách phân quyền riêng; dữ liệu thanh toán, hoàn tiền và nhật ký kiểm toán là nhóm cần ưu tiên bảo vệ cao.
- Việc tách bộ xử lý nền khỏi khối xử lý yêu cầu trực tuyến giúp những tác vụ như gửi thông báo, in biên lai điện tử hoặc cập nhật báo cáo không tranh chấp tài nguyên với thao tác tại quầy.

Giải thích chi tiết cho góc nhìn triển khai. Góc nhìn triển khai giúp chứng minh rằng kiến trúc được chọn không chỉ hợp lý trên sơ đồ logic mà còn có thể triển khai thực tế. Với IRMS, bản chất *nhiều điểm chạm vật lý* — quầy lễ tân, bàn phục vụ, trạm bếp, quầy thu ngân — làm cho góc nhìn triển khai quan trọng hơn nhiều so với các hệ thống chỉ có một giao diện web thông thường.



Hình 23: Sơ đồ triển khai tổng quan về bố trí vật lý của IRMS

Hình 23 là sơ đồ triển khai ở mức vật lý, tức mức trả lời câu hỏi hệ thống sẽ hiện diện như thế nào trong nhà hàng thật. Giá trị lớn nhất của hình này là nó kéo kiến trúc ra khỏi trạng thái trừu tượng thuần túy. Khi nhìn vào sơ đồ, người đọc không còn thấy các mô-đun dưới dạng khái niệm, mà thấy chúng gắn vào máy tính bảng gọi món, thiết bị lễ tân, màn hình bếp, quầy thu ngân, khối xử lý trung tâm, bộ xử lý nền, cơ sở dữ liệu và các hệ thống ngoài.

Điều quan trọng nhất cần rút ra là IRMS có **nhiều điểm chạm vật lý nhưng một lõi điều phối tập trung**. Đây là lựa chọn rất phù hợp với phạm vi của bài toán. Nhà hàng cần nhiều thiết bị tại hiện trường để thao tác gần nơi công việc diễn ra nhất, nhưng lại cần một lõi chung để giữ trạng thái thống nhất của bàn, đơn gọi món, bếp và hóa đơn. Nếu đẩy quá nhiều quyết định xuống từng thiết bị cục bộ, hệ thống sẽ rất dễ mất nhất quán khi mạng dao động hoặc khi hai vai trò thao tác chồng thời điểm. Vì vậy, hình này làm rõ rằng thiết bị đầu cuối nên gần người dùng, còn quyết định nghiệp vụ bền vững phải được neo ở khối xử lý trung tâm và hạ tầng dữ liệu.

Sơ đồ cũng cho thấy vì sao bộ xử lý nền cần đứng riêng. Nhiều tác vụ như gửi thông báo, phát hành biên lai điện tử, làm mới báo cáo hoặc xử lý một số hệ quả phụ không cần tranh tài nguyên trực tiếp với thao tác ở bàn, ở bếp hay ở quầy thu ngân. Khi chúng được tách ra, hệ thống giữ được nhịp phản hồi ổn định hơn ở tuyến trước khách hàng. Đây là lựa chọn triển khai có ý nghĩa thực tế rất lớn, vì giờ cao điểm của nhà hàng thường không cho phép những tác vụ phụ chen vào tuyến chính.

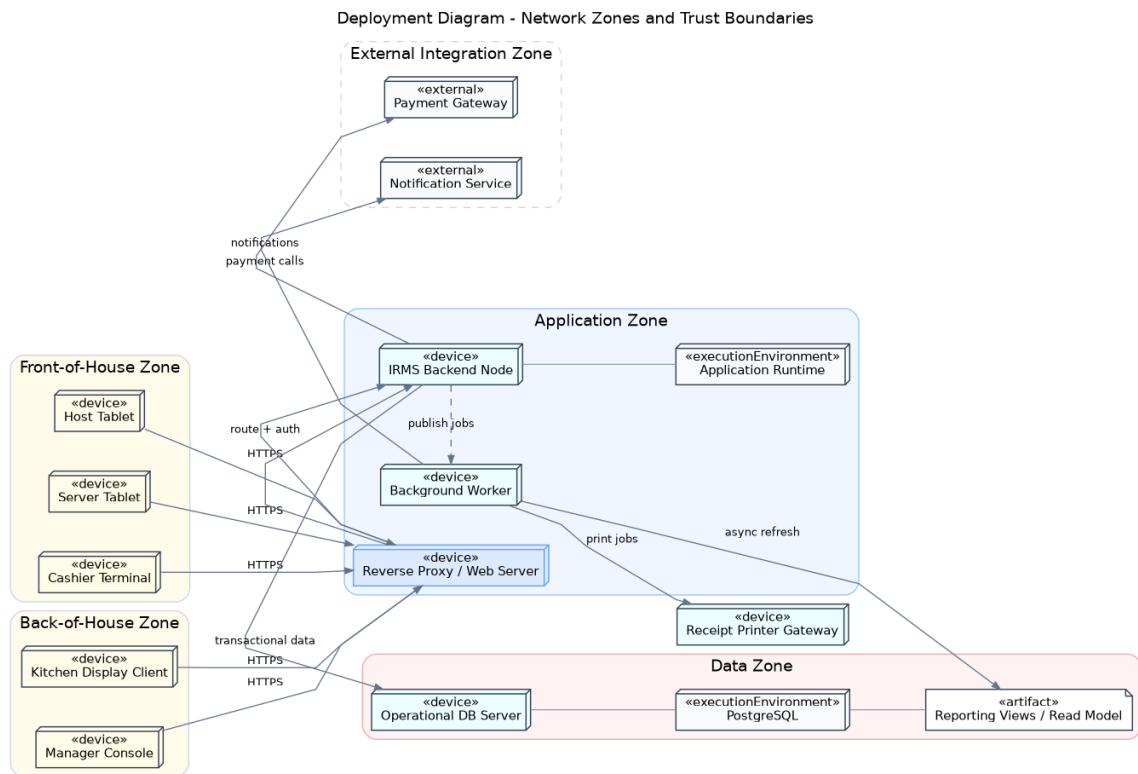
Các thành phần chính thể hiện trong hình.

- **Thiết bị đầu cuối:** gồm máy tính bảng gọi món, thiết bị lễ tân, màn hình bếp, máy thu ngân và trình duyệt quản lý.
- **Khối cổng vào:** tiếp nhận yêu cầu từ các thiết bị và chuyển tiếp vào khối xử lý.
- **Khối xử lý phía sau:** thực hiện các chức năng nghiệp vụ chính của hệ thống.
- **Bộ xử lý nền:** nhận tác vụ không cần phản hồi ngay như gửi thông báo hoặc cập nhật báo cáo.
- **Hạ tầng dữ liệu:** lưu trữ dữ liệu giao dịch, dữ liệu báo cáo và tệp đính kèm.
- **Các hệ thống ngoài:** gồm cổng thanh toán, kênh gửi thông báo và thiết bị hoặc dịch vụ in biên lai.

Lý do thiết kế của sơ đồ này.

- Bố trí một khối xử lý trung tâm giúp đơn giản hóa triển khai, dễ kiểm thử đầu-cuối và dễ giữ ranh giới giao dịch ngắn gọn.
- Tách bộ xử lý nền khỏi khối tiếp nhận yêu cầu trực tuyến giúp tác vụ phụ không tranh chấp quá mạnh với yêu cầu tại quầy.
- Đặt các hệ thống ngoài ở rìa giúp phân biệt rõ phần lõi của IRMS với phần tích hợp phụ thuộc đối tác.

Phương án này mạnh ở chỗ đơn giản và dễ vận hành, rất phù hợp với bài toán một nhà hàng hoặc một nhóm nhỏ chi nhánh. Đổi lại, khối xử lý trung tâm trở thành vùng cần được giám sát và bảo vệ cẩn thận hơn, vì nó là nơi hội tụ của nhiều quyết định nghiệp vụ.



Hình 24: Sơ đồ triển khai theo vùng mạng và ranh giới tin cậy

Hình 24 làm rõ góc nhìn mà sơ đồ vật lý tổng quan mới chỉ chạm tới ở mức khái quát: **ranh giới tin cậy**. Với một hệ thống như IRMS, bảo mật không chỉ là câu chuyện tài khoản và mật khẩu. Nó còn là câu chuyện thành phần nào được đặt ở vùng mạng nào, ai được phép nói chuyện trực tiếp với ai, và dữ liệu nhạy cảm nằm ở đâu trong toàn bộ mặt bằng triển khai.

Việc tách thành vùng phục vụ phía trước, vùng bếp, vùng ứng dụng, vùng dữ liệu và vùng tích hợp ngoài giúp kiến trúc thể hiện rõ những lớp bảo vệ khác nhau. Thiết bị ở hiện trường phải đủ gần nghiệp vụ để thao tác nhanh, nhưng không nên có quyền đi thẳng tới cơ sở dữ liệu. Cơ sở dữ liệu lại phải ở một vùng được bảo vệ hơn, tránh truy cập trực tiếp từ thiết bị đầu cuối hoặc từ các kết nối ngoài. Các hệ thống đối tác như cổng thanh toán hay dịch vụ gửi thông báo nằm ngoài biên tin cậy nội bộ, nên mọi lưu lượng đi qua chúng đều phải được kiểm soát, ghi nhật ký và hạn chế quyền truy cập. Nhờ vậy, sơ đồ này làm nổi bật một nguyên tắc quan trọng: không có “một vùng an toàn duy nhất”, mà có nhiều lớp biên, mỗi lớp có mức tin cậy khác nhau.

Sơ đồ còn có giá trị vận hành thực tế. Khi xảy ra sự cố, cách chia vùng giúp khoanh nhanh câu hỏi: lỗi nằm ở lớp thiết bị hiện trường, ở vùng ứng dụng hay ở kết nối với hệ ngoài. Tương tự, khi thiết kế tường lửa, mã hóa đường truyền, nhật ký truy cập hay chính sách cấp quyền mạng, chính sơ đồ vùng mạng là cơ sở để quyết định cái gì phải bị chặn tuyệt đối, cái gì được phép đi qua một cổng duy nhất và cái gì chỉ được mở từ một vùng đã kiểm soát.

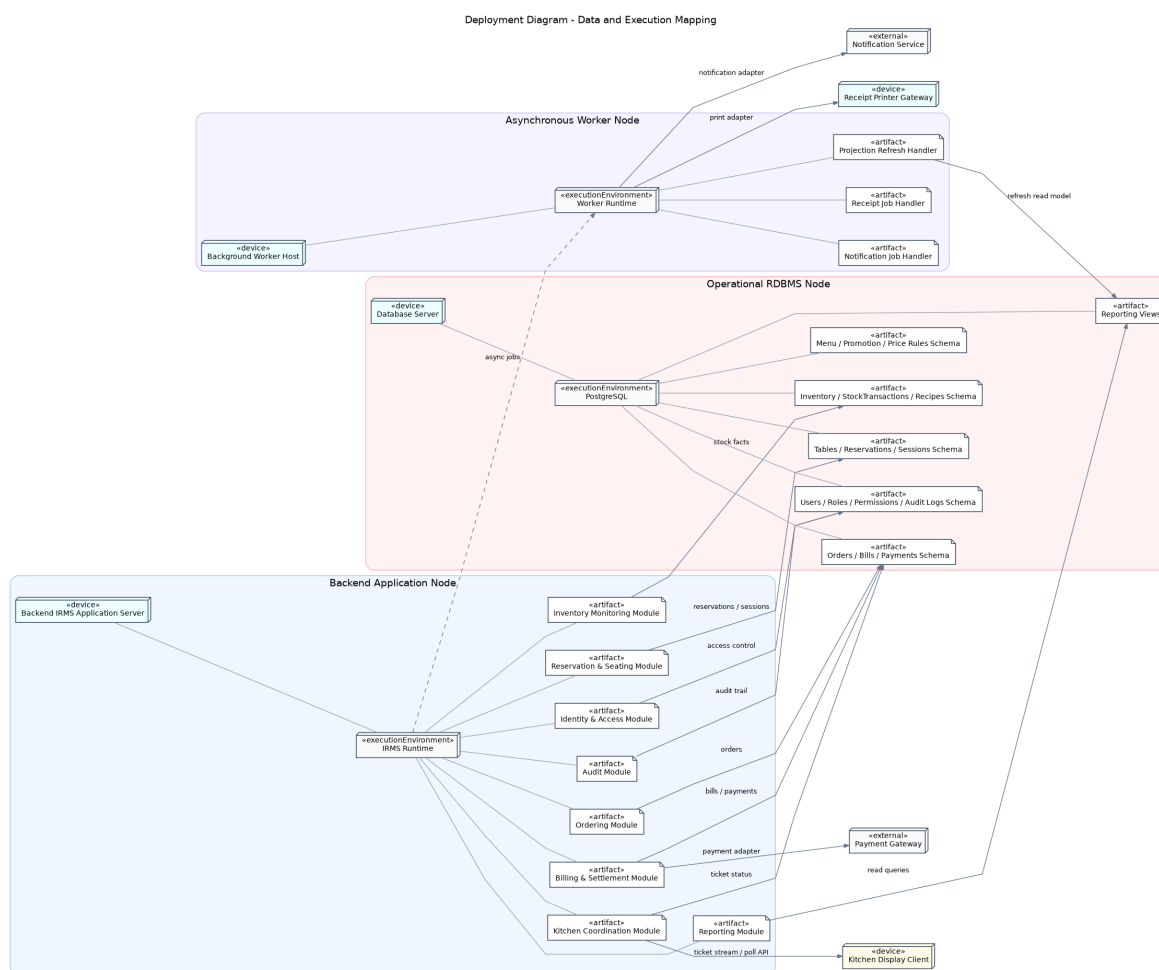
Các vùng chính thể hiện trong hình.

- **Vùng phục vụ phía trước:** chứa thiết bị của lễ tân, phục vụ và thu ngân.
- **Vùng bếp:** chứa màn hình bếp và các thiết bị phục vụ chế biến.
- **Vùng ứng dụng:** chứa cổng vào, khối xử lý phía sau và bộ xử lý nền.
- **Vùng dữ liệu:** chứa cơ sở dữ liệu giao dịch, mô hình đọc và kho lưu trữ đối tượng.
- **Vùng tích hợp ngoài:** chứa cổng thanh toán, dịch vụ gửi thông báo và các hệ thống ngoài khác.

Lý do thiết kế của sơ đồ này.

- Sơ đồ làm rõ nơi nào là mạng nội bộ hiện trường, nơi nào là vùng ứng dụng, nơi nào là vùng dữ liệu nhạy cảm và nơi nào là thế giới bên ngoài.
- Việc tách vùng dữ liệu thành một lớp riêng giúp giảm nguy cơ truy cập trực tiếp từ thiết bị đầu cuối hoặc từ Internet.
- Mọi tích hợp ngoài đi qua khối ứng dụng giúp tăng khả năng kiểm soát, ghi nhật ký và áp dụng chính sách bảo vệ dữ liệu.

Ưu điểm của cách biểu diễn theo vùng là giúp người đọc hiểu được nơi nào là nội bộ nhà hàng, nơi nào là biên kết nối ra ngoài và nơi nào là dữ liệu nhạy cảm. Nó cũng hỗ trợ suy nghĩ về tường lửa, cắt lớp mạng, mã hóa đường truyền, ghi nhật ký và ứng phó sự cố. Nhược điểm là sơ đồ này không diễn tả chi tiết luồng dữ liệu nghiệp vụ như sơ đồ trình tự, nên cần được đọc cùng góc nhìn sử dụng và góc nhìn thành phần.



Hình 25: Sơ đồ triển khai về ánh xạ dữ liệu và thực thi

Hình 25 là sơ đồ có ý nghĩa quyết định khi cần biện minh kiểu kiến trúc được chọn ở mức thực thi. Nó cho thấy các miền nghiệp vụ được đóng gói như các mô-đun trong cùng một khối xử lý trực tuyến, trong khi bộ xử lý nền đảm nhiệm các công việc chấp nhận độ trễ nhỏ hơn như gửi thông báo, phát hành biên lai điện tử và làm mới mô hình đọc. Nói cách khác, đây là sơ đồ chứng minh IRMS đi theo **đơn khối theo mô-đun** ở mức thực thi, chứ không phải một hệ phân tán nặng ngay từ đầu.

Điểm mạnh nhất của cách bố trí này là giữ được cả hai lợi ích tương tự như khó đi cùng nhau. Một mặt, ranh giới mô-đun vẫn rõ nên hệ thống không trượt thành một khối mã hỗn loạn. Mặt khác, các giao dịch lõi như tạo đơn gọi món, cập nhật trạng thái bếp, lập hóa đơn và thanh toán vẫn diễn ra trong một không gian thực thi tập trung hơn, từ đó tránh được rất nhiều phức tạp của giao dịch phân tán, đồng bộ lược đồ và phối hợp lỗi qua nhiều

tiến trình mạng. Đây là lựa chọn rất hợp lý cho bối cảnh bài toán và quy mô dự án.

Sơ đồ cũng làm rõ vai trò của mô hình đọc và kho lưu trữ đối tượng. Mô hình đọc không phải là bản sao tùy hứng của dữ liệu giao dịch, mà là một mặt cắt phục vụ truy vấn quản trị. Kho lưu trữ đối tượng lại là nơi phù hợp cho biên lai, tệp đính kèm hoặc tài nguyên thực đơn khi cần. Nhờ tách như vậy, khối xử lý trực tuyến không phải ôm mọi nhu cầu lưu trữ vào cùng một hình thức.

Các thành phần chính thể hiện trong hình.

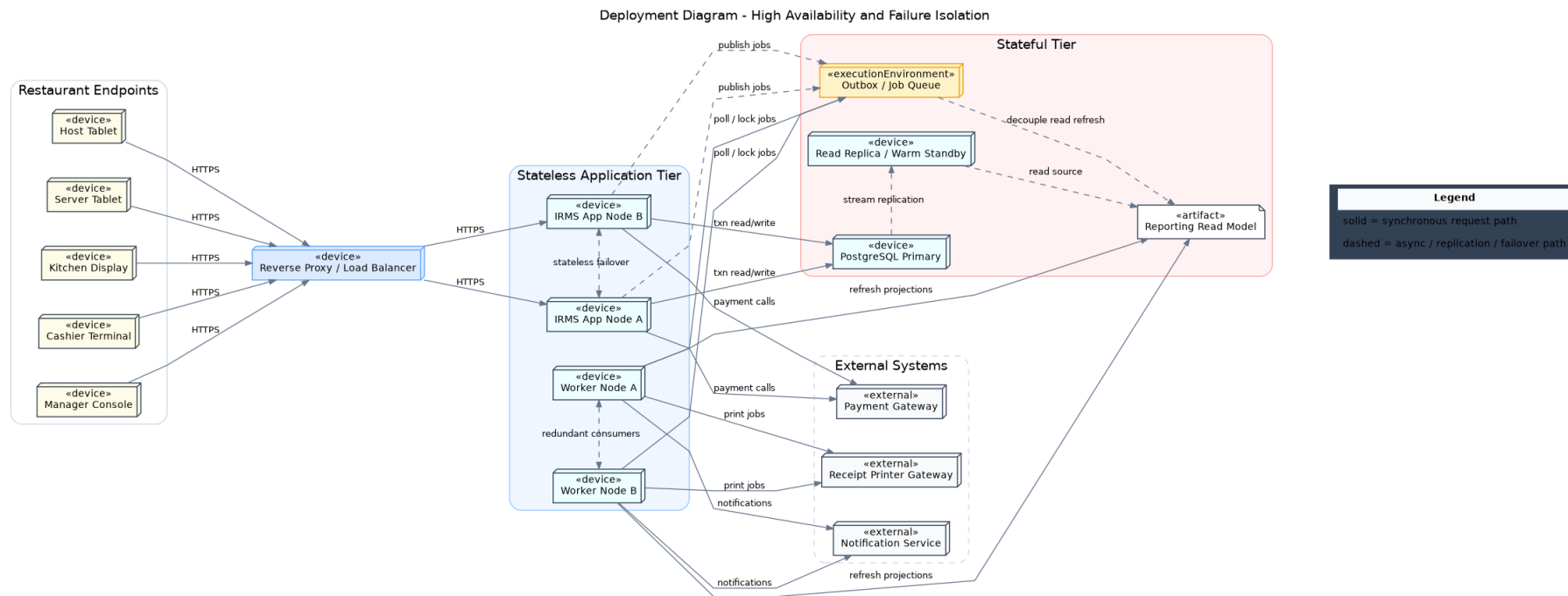
- **Khối xử lý trực tuyến:** chứa các mô-đun nghiệp vụ phục vụ yêu cầu thời gian thực.
- **Bộ xử lý nền:** xử lý các tác vụ chấp nhận độ trễ nhỏ.
- **Lược đồ giao dịch:** lưu dữ liệu giao dịch cốt lõi của hệ thống.
- **Mô hình đọc và dữ liệu báo cáo:** tối ưu cho truy vấn quản trị và thống kê.
- **Kho lưu trữ đối tượng:** giữ biên lai, tệp đính kèm hoặc tài nguyên thực đơn nếu có.

Lý do thiết kế của sơ đồ này.

- Đóng gói nhiều miền nghiệp vụ trong cùng một khối xử lý giúp giữ giao dịch lõi đơn giản hơn so với việc tách thành nhiều dịch vụ mạng độc lập ngay từ đầu.
- Tách mô hình đọc khỏi dữ liệu giao dịch giúp báo cáo phong phú hơn mà không làm nặng tuyến xử lý trực tuyến.
- Giữ bộ xử lý nền ở ngoài khối trực tuyến giúp hệ thống vừa rõ ràng về thực thi, vừa linh hoạt khi sau này cần tách riêng một số năng lực lớn.

Ưu điểm của cách tổ chức này là thực dụng, nhất quán và phù hợp với phạm vi đề tài. Đổi lại, kiến trúc phải được bảo vệ bằng kỷ luật mô-đun bên trong; nếu không, việc cùng chạy trong một khối xử lý có thể dẫn tới tình trạng phụ thuộc chéo quá mức.

3.7.2 Sơ đồ triển khai cho tính sẵn sàng cao và cô lập lỗi



Hình 26: Sơ đồ triển khai cho tính sẵn sàng cao và cô lập lỗi

Hình 26 đi sâu vào một câu hỏi thường được đặt ra khi đánh giá kiến trúc vận hành: nếu một nút ứng dụng, một bộ xử lý nền hoặc một thành phần tích hợp gặp sự cố, IRMS còn giữ được điều gì và suy giảm dịch vụ ra sao. Đây là sơ đồ rất đáng giá vì nó buộc thiết kế phải trả lời về hành vi khi lỗi xảy ra, thay vì chỉ mô tả trạng thái lý tưởng khi mọi thứ đều hoạt động.

Điểm đầu tiên cần rút ra là **tuyến yêu cầu trực tuyến** và **tuyến xử lý nền** đều có phương án dự phòng. Bộ chuyển tiếp ngược hoặc bộ cân bằng tải đặt trước các nút ứng dụng giúp ứng dụng khách không gán cứng vào một nút cụ thể. Nếu một nút ứng dụng lỗi, lưu lượng có thể được đưa sang nút còn lại mà không làm người dùng phải thay đổi điểm truy cập. Tương tự, việc có nhiều nút bộ xử lý nền giúp các tác vụ không bị tắc nghẽn toàn bộ khi một bộ tiêu thụ gặp lỗi.

Điểm thứ hai là sơ đồ làm rõ cách dữ liệu được bảo vệ. PostgreSQL Primary kết hợp với Read Replica / Warm Standby cho thấy dữ liệu giao dịch lỗi không nên chỉ có một nơi duy nhất. Đồng thời, phân đọc báo cáo và phân xử lý nền được đi qua hàng đợi hoặc hộp thư ra, nhờ đó sự cố ở bảng điều khiển hoặc một nhiệm vụ nền không làm sập ngay tuyến giao dịch. Đây là một dạng cô lập lỗi rất quan trọng trong nhà hàng: nếu kênh thông báo bị chậm, khách vẫn phải thanh toán được; nếu báo cáo chưa kịp làm mới, bàn vẫn phải mở và món vẫn phải ra đúng.

Điểm thứ ba là sơ đồ thể hiện rõ **đường suy giảm dịch vụ chấp nhận được**. Không phải mọi lỗi đều phải dẫn tới dừng toàn hệ thống. Một số lỗi chỉ nên làm chậm phân đọc, làm chậm thông báo hoặc chuyển một số tác vụ sang hàng chờ. Khả năng chấp nhận suy giảm có kiểm soát như vậy mới là dấu hiệu của một hệ thống vận hành trưởng thành.

Các ý chính cần rút ra từ sơ đồ.

- **Bộ chuyển tiếp ngược / bộ cân bằng tải** tách lớp ứng dụng khách khỏi từng nút ứng dụng riêng lẻ, nên lỗi một nút không đồng nghĩa lỗi toàn bộ điểm vào.
- **IRMS App Node A/B** cùng **Worker Node A/B** cho phép dự phòng ở cả tuyến xử lý yêu cầu và tuyến xử lý nền.
- **PostgreSQL Primary** cùng **bản sao đọc / nút dự phòng ấm** hỗ trợ tách truy vấn đọc hoặc phục hồi nhanh hơn sau lỗi nút chính.
- **Outbox / Job Queue** ngăn các tác vụ nền và việc làm mới mô hình chiếu làm nghẽn trực tiếp tuyến giao dịch.

Vì sao sơ đồ này cần có trong góc nhìn triển khai.

- Góc nhìn triển khai không chỉ nên cho thấy hệ thống đặt ở đâu, mà còn phải cho thấy nó phản ứng ra sao khi có lỗi.
- Với IRMS, nếu cổng thanh toán hoặc dịch vụ gửi thông báo chậm, hệ thống vẫn phải giữ đúng giao dịch đã hoàn tất trong lỗi; sơ đồ này minh họa chính xác cơ chế tách biệt đó.
- Nó cũng làm rõ hướng mở rộng sau này: có thể tăng số nút ứng dụng, nút bộ xử lý nền hoặc năng lực đọc mà không phải thiết kế lại mô hình miền nghiệp vụ.

3.8 Nguyên tắc thiết kế

Kiến trúc của IRMS bám theo các nguyên tắc thiết kế sau:

- **Kết tụ cao, phụ thuộc thấp**: mỗi mô-đun chỉ công khai phân giao diện thật sự cần thiết và tránh làm lộ chi tiết cài đặt.
- **Phân tách mối quan tâm**: bộ điều khiển, dịch vụ ứng dụng, miền nghiệp vụ và hạ tầng kỹ thuật không bị trộn vai trò.
- **Phụ thuộc thông qua lớp trừu tượng**: các tích hợp ngoài đi qua giao diện trừu tượng thay vì để lỗi nghiệp vụ phụ thuộc trực tiếp vào nhà cung cấp hoặc bộ thư viện cụ thể.

- **Lỗi đồng bộ, hỗ trợ bất đồng bộ:** các giao dịch lỗi như tạo đơn gọi món, cập nhật bếp và thanh toán ưu tiên xử lý đồng bộ; các hệ quả phụ như cảnh báo tồn kho, làm mới báo cáo và gửi thông báo đi qua sự kiện hoặc bộ xử lý nền.
- **Mô hình hóa bắt đầu từ miền nghiệp vụ:** sơ đồ lớp phải phản ánh hành vi thật của nhà hàng, không bị giản lược thành sơ đồ quan hệ dữ liệu thuần túy.

Các nguyên tắc trên không chỉ là danh sách khái niệm, mà là bộ tiêu chí nhất quán xuyên suốt toàn bộ tài liệu. Trong góc nhìn phân rã, chúng dẫn tới ranh giới năng lực nghiệp vụ rõ ràng; trong góc nhìn thành phần, chúng dẫn tới sự tách bạch giữa bộ điều khiển, dịch vụ ứng dụng, miền nghiệp vụ và bộ kết nối; trong góc nhìn triển khai, chúng dẫn tới quyết định tách bộ xử lý nền và đường báo cáo khỏi đường xử lý yêu cầu; còn trong sơ đồ lớp, chúng dẫn tới việc ưu tiên thực thể gốc, chính sách và giao diện trừu tượng thay vì chỉ vẽ các bảng dữ liệu rời rạc.

3.9 Áp dụng nguyên lý SOLID vào thiết kế

Bảng 18: Ánh xạ nguyên lý SOLID vào thiết kế của IRMS

Nguyên lý	Quyết định thiết kế chính	Biểu hiện trong IRMS
Trách nhiệm đơn nhất	tách trách nhiệm theo ca sử dụng, thực thể gốc và mối quan tâm tích hợp	bộ điều khiển chỉ tiếp nhận yêu cầu; dịch vụ ứng dụng chỉ điều phối; thực thể gốc giữ bất biến; bộ xử lý nền xử lý tác vụ phụ
Đóng để ổn định, mở để mở rộng	mở các điểm mở rộng qua chiến lược, chính sách và bộ kết nối	thêm phương thức thanh toán, kênh thông báo hoặc quy tắc khuyến mãi mà không phải sửa lỗi giao dịch
Thay thế tương thích	thiết kế hợp đồng rõ cho các lớp trừu tượng tích hợp	mọi cách hiện thực của <code>PaymentGateway</code> , <code>NotificationSender</code> , <code>ReceiptPrinter</code> phải thay thế được nhau mà không làm đổi hành vi mong đợi của lớp dùng chúng
Phân tách giao diện	tránh giao diện lớn kiểu “làm mọi thứ”	<code>Billing</code> chỉ phụ thuộc công thanh toán và biên lai; <code>Inventory</code> chỉ phụ thuộc công cảnh báo; <code>Reporting</code> chỉ phụ thuộc giao diện truy vấn phù hợp
Đảo ngược phụ thuộc	giữ chiều phụ thuộc hướng vào lớp trừu tượng	dịch vụ nghiệp vụ phụ thuộc kho lưu trữ, chính sách và công kết nối trừu tượng thay vì phụ thuộc trực tiếp vào cách hiện thực cụ thể

3.9.1 Nguyên lý trách nhiệm đơn nhất

Nguyên lý này được áp dụng theo nhiều lớp khác nhau. Ở biên hệ thống, bộ điều khiển chỉ làm nhiệm vụ nhận yêu cầu, kiểm tra sơ bộ và chuyển cho dịch vụ ứng dụng; nó không chứa quy tắc nghiệp vụ cốt lõi. Ở lớp điều phối, mỗi dịch vụ ứng dụng chỉ chịu trách nhiệm cho một nhóm hành vi gần nhau: `OrderingApplicationService` không xử lý hoàn tiền, còn `BillingApplicationService` không làm nhiệm vụ điều phối bếp. Ở miền nghiệp vụ, các thực thể gốc như `Order`, `Bill` hay `InventoryItem` giữ bất biến tương ứng thay vì đẩy toàn bộ quy tắc vào một lớp trung tâm khổng lồ. Nhờ vậy, khi một chính sách thay đổi, phạm vi sửa đổi được khoanh gọn đúng nơi có trách nhiệm.

3.9.2 Nguyên lý đóng để ổn định, mở để mở rộng

IRMS có nhiều điểm biến động cao: đối tác thanh toán, kênh gửi thông báo, cách phát hành biên lai, quy tắc khuyến mãi, quy tắc cảnh báo mức tồn thấp. Nếu lỗi nghiệp vụ gọi thẳng từng nhà cung cấp hoặc mã hóa cứng từng biến thể, mỗi thay đổi sẽ buộc phải sửa phần trung tâm. Thiết kế vì thế ưu tiên các lớp trừu tượng như

PaymentGateway, NotificationSender, ReceiptPrinter, cùng các chính sách riêng cho định giá, khả dụng và phân quyền. Nhờ đó, hệ thống mở cho mở rộng nhưng vẫn giữ được phần lõi ổn định.

3.9.3 Nguyên lý thay thế tương thích

Nguyên lý này đặc biệt quan trọng ở các bộ kết nối tích hợp. Một cách hiện thực mới của PaymentGateway phải tuân thủ cùng một hợp đồng hành vi về giao dịch thành công, thất bại, hoàn tiền, tính chống xử lý lặp và mã tham chiếu. Tương tự, một NotificationSender mới có thể gửi thư điện tử, tin nhắn hay thông báo đẩy, nhưng không được khiến dịch vụ ứng dụng phải viết nhánh xử lý đặc biệt chỉ dành riêng cho nó. Khi một lớp trừu tượng được thiết kế đúng, lớp dùng nó không cần biết bộ kết nối phía sau là nhà cung cấp nào.

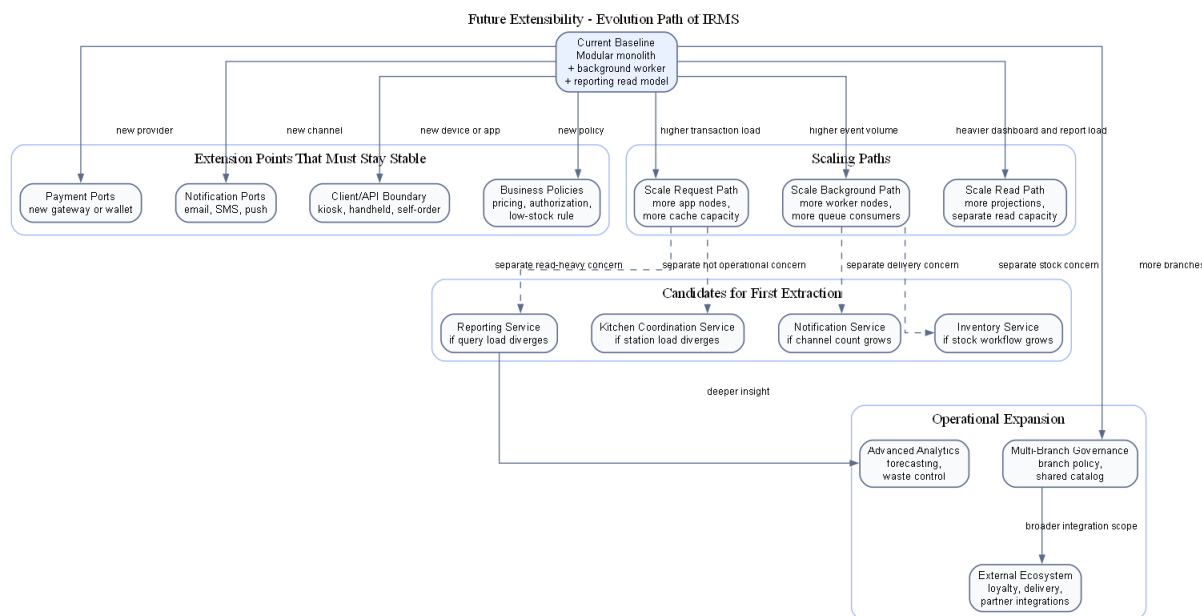
3.9.4 Nguyên lý phân tách giao diện

IRMS tránh các giao diện khổng lồ kiểu “dịch vụ tiện ích tổng hợp” hoặc “quản lý mọi hệ ngoài”. Thay vào đó, mỗi mô-đun chỉ phụ thuộc vào đúng giao diện mà nó cần. Billing chỉ cần cổng thanh toán và cổng biên lai. Inventory chỉ cần cổng cảnh báo. Reporting chỉ cần giao diện truy vấn hoặc giao diện làm mới mô hình đọc phù hợp. Cách chia này làm cho việc kiểm thử đơn vị gọn hơn, giảm phụ thuộc chéo và tránh việc mô-đun tuyển đầu phải nhìn thấy những giao diện quản trị không liên quan.

3.9.5 Nguyên lý đảo ngược phụ thuộc

Đây là nguyên lý bảo vệ lõi nghiệp vụ khỏi công nghệ cụ thể. Dịch vụ ứng dụng và mô hình miền nghiệp vụ phụ thuộc vào giao diện kho lưu trữ, giao diện chính sách và giao diện bộ kết nối; còn lớp hạ tầng mới hiện thực các giao diện đó bằng PostgreSQL, giao diện thanh toán hay nhà cung cấp thông báo cụ thể. Nhờ vậy, chiều phụ thuộc được giữ nhất quán: chi tiết công nghệ phụ thuộc vào nhu cầu nghiệp vụ, chứ không phải nghiệp vụ bị ép đi theo công nghệ. Chính điều này làm cho kiến trúc có thể tiến hóa dần mà không làm nứt phần lõi.

3.10 Khả năng mở rộng trong tương lai



Hình 27: Lộ trình mở rộng kiến trúc của IRMS trong tương lai

Hình 27 tập trung riêng vào câu hỏi mà hầu như mọi đề án kiến trúc đều phải trả lời tường minh: nếu phạm vi vận hành lớn hơn, số thiết bị tăng lên, số chi nhánh tăng lên hoặc yêu cầu tích hợp đa dạng hơn, kiến trúc hiện tại sẽ tiến hóa theo đường nào mà không phải viết lại phần lõi. Hình này không mô tả một trạng thái giả định mơ hồ

trong tương lai. Nó mô tả **đường tiến hóa hợp lý** từ kiến trúc hiện tại sang những khả năng mở rộng có xác suất xảy ra cao nhất đối với IRMS.

Ở trạng thái hiện tại, IRMS vận hành hiệu quả nhất như một **modular monolith theo năng lực nghiệp vụ** với bộ xử lý nền và mô hình đọc tách riêng. Đây là nền tảng tốt vì giao dịch lỗi vẫn gọn, ranh giới mô-đun vẫn rõ và chi phí vận hành chưa bị đẩy lên mức của hệ phân tán nặng. Từ nền này, hướng tiến hóa hợp lý nhất là tách dần Reporting, Notification và một phần Kitchen Coordination hoặc Inventory ra trước khi nghĩ đến việc tách tuyến giao dịch lỗi. Lý do là các vùng đó đã có nhịp xử lý, kiểu tải và mức nhất quán khác với giao dịch đồng bộ ở tuyến đầu.

Điểm quan trọng của hình là nó nêu rõ **điều kiện để mở rộng mà không phá kiến trúc**. Nếu muốn thêm phương thức thanh toán mới, hệ thống chỉ nên chạm vào lớp công thanh toán và chính sách liên quan. Nếu muốn thêm kênh thông báo, thay đổi phải chủ yếu nằm ở Notification Integration. Nếu muốn mở thêm chi nhánh, hệ thống cần tăng sức chứa ở lớp cấu hình chi nhánh, vùng dữ liệu và tầng triển khai, nhưng không được buộc viết lại mô hình đơn gọi món hay quyết toán. Nếu muốn có kiosk tự phục vụ hoặc thiết bị cầm tay mới, thay đổi chủ yếu nằm ở lớp client và giao diện lập trình ứng dụng, còn chuỗi miền nghiệp vụ chính vẫn giữ nguyên. Những điều kiện đó chính là thước đo thật sự của extensibility, chứ không chỉ là khả năng thêm lớp mã mới.

Sơ đồ cũng chỉ ra các **điểm mở rộng chiến lược** cần được giữ ổn định từ sớm: cổng thanh toán, cổng gửi thông báo, chính sách phân quyền, quy tắc định giá, quy tắc cảnh báo tồn kho, mô hình đọc báo cáo và các giao diện ứng dụng khách. Khi những điểm này được giữ ở đúng ranh giới, hệ thống có thể tiến hóa dần từng năng lực mà không cần đánh đổi sự ổn định của tuyến lõi. Đây là lý do mục “Future Extensibility” không nên chỉ được nói bằng một đoạn văn chung chung; nó cần một hình kiến trúc riêng để cho thấy đường phát triển của hệ thống là hữu hình và có kiểm soát.

Các hướng tiến hóa chính thể hiện trong hình.

- **Mở rộng tích hợp:** thêm phương thức thanh toán, kênh thông báo, thiết bị in hoặc thiết bị đầu cuối mới thông qua các cổng và bộ kết nối hiện có.
- **Mở rộng theo tải:** tăng số nút ứng dụng, nút bộ xử lý nền, năng lực đọc báo cáo hoặc tách riêng một số mô-đun có tải đặc thù.
- **Mở rộng theo phạm vi vận hành:** thêm nhiều chi nhánh, thêm chính sách theo chi nhánh và tăng năng lực quản trị tập trung mà không phải làm lại mô hình miền lõi.
- **Mở rộng theo dịch vụ:** chỉ tách thành dịch vụ mạng độc lập với những vùng có nhịp thay đổi hoặc mức tải thực sự khác biệt, thay vì tách đồng loạt vì lý do hình thức.

4 Thiết kế chi tiết

4.1 Góc nhìn lớp

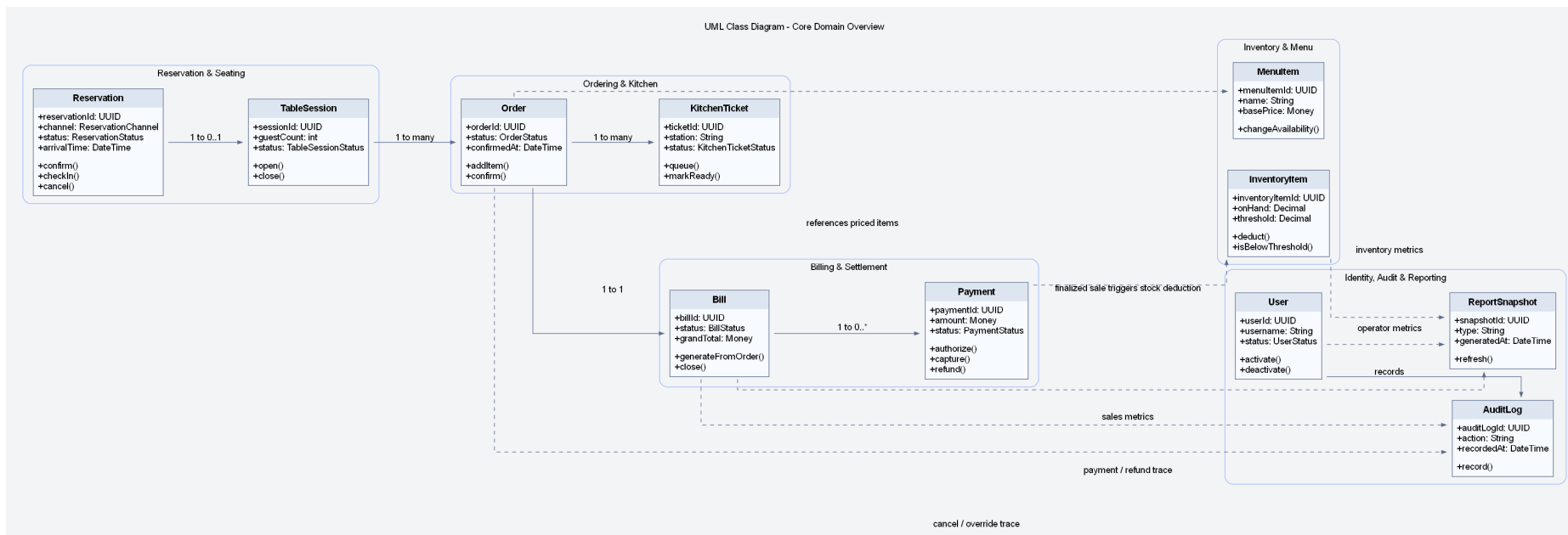
4.1.1 Phạm vi bao phủ của sơ đồ lớp

Sơ đồ lớp được tổ chức theo năm hướng chính nhằm bao phủ đầy đủ miền nghiệp vụ của IRMS:

1. **Ngữ cảnh khách hàng và chi nhánh:** sơ đồ thể hiện CustomerProfile và Branch để việc đặt bàn, lập hóa đơn, quản lý tồn kho và phân công nhân sự luôn được xác định rõ theo khách hàng và theo địa điểm vận hành.
2. **Bao phủ đầy đủ hơn phần tùy biến món:** sơ đồ thể hiện ModifierOption và OrderItemModifier để lưu được lựa chọn chi tiết ở mức tùy chọn, thay vì chỉ dừng ở mức nhóm modifier.
3. **Bao phủ khuyến mãi và giảm giá:** sơ đồ thể hiện PromotionCampaign và AppliedDiscount để hóa đơn phản ánh đúng các chương trình ưu đãi được áp dụng trong từng giao dịch.
4. **Bao phủ quản trị nhân sự theo ca:** sơ đồ thể hiện ShiftAssignment vì IRMS có nêu rõ nhu cầu quản trị và sắp lịch nhân viên.

5. Làm rõ các giao diện mở rộng: các giao diện cho thanh toán, thông báo và phân quyền được thể hiện riêng để chỉ ra rõ các điểm mở rộng và các chính sách nhạy cảm của hệ thống.

Sơ đồ lớp trong báo cáo này không được hiểu như một ERD thuần dữ liệu. Thay vào đó, nó được dùng để biểu diễn **hành vi nghiệp vụ, ranh giới aggregate và các điểm mở rộng**. Điều đó giải thích vì sao nhiều lớp có các operation như `confirm()`, `split()`, `markReady()`, `authorize()`, `issue()` thay vì chỉ liệt kê thuộc tính. Cách mô hình hóa này phù hợp hơn với mục tiêu của báo cáo, vì nó làm rõ hệ thống *vận hành như thế nào*, không chỉ *lưu trữ dữ liệu gì trong cơ sở dữ liệu*; phân biệt minh cho góc nhìn lớp, mức chi tiết của từng cụm lớp và mối liên hệ của chúng với kiến trúc tổng thể được dẫn ngay tới Bảng 20, Bảng 21 và Bảng 22.



Hình 28: Sơ đồ lớp UML tổng thể của miền lõi IRMS

Hình 28 là sơ đồ lớp ở mức nhìn toàn hệ thống, có nhiệm vụ thể hiện các cụm nghiệp vụ quan trọng và các liên kết chính giữa chúng. Thay vì chỉ tập trung vào một số lớp giao dịch quen thuộc như `Order` hay `Bill`, sơ đồ này giữ lại đầy đủ ngữ cảnh khách hàng, chi nhánh, đặt bàn, gọi món, bếp, thanh toán, tồn kho, quản trị, phân quyền và báo cáo. Nhờ đó, người đọc có thể nắm được **bức tranh miền nghiệp vụ tổng thể** trước khi đi sâu vào từng cụm lớp chi tiết hơn.

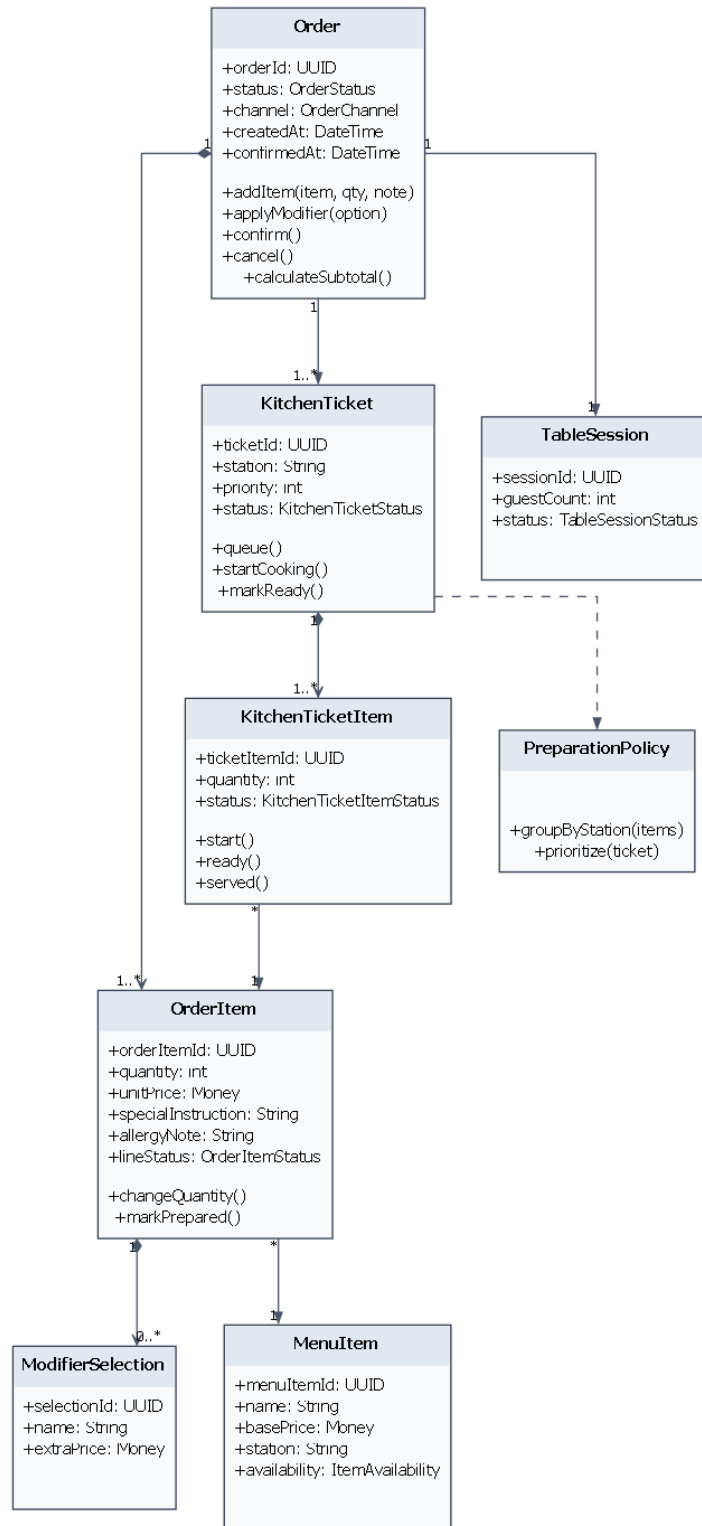
Các nhóm lớp chính thể hiện trong hình.

- **Ngữ cảnh khách hàng và chi nhánh:** `CustomerProfile` và `Branch` tạo nền cho việc gắn dữ liệu nghiệp vụ vào đúng khách hàng và đúng địa điểm vận hành.
- **Đặt bàn và phục vụ tại bàn:** `Reservation`, `WaitlistEntry`, `DiningTable` và `TableSession` biểu diễn vòng đời trước khi khách vào bàn và trong suốt thời gian bàn hoạt động.
- **Thực đơn và gọi món:** `MenuCategory`, `MenuItem`, `ModifierGroup`, `ModifierOption`, `Order`, `OrderItem` và `OrderItemModifier` mô tả cả cấu hình thực đơn lẫn dữ liệu giao dịch tại thời điểm khách gọi món.
- **Điều phối bếp:** `KitchenStation`, `KitchenTicket` và `KitchenTicketItem` thể hiện đường đi từ đơn gọi món sang đơn vị chế biến thực tế.
- **Quyết toán và thanh toán:** `Bill`, `BillSplit`, `Payment`, `Refund`, `Receipt`, `PromotionCampaign` và `AppliedDiscount` bao phủ luồng tính tiền của nhà hàng.
- **Tồn kho, cảnh báo và quản trị:** `InventoryItem`, `RecipeLine`, `StockTransaction`, `LowStockRule`, `User`, `Role`, `Permission`, `AuditLog`, `AuthorizationPolicy`, `ShiftAssignment` và `ReportSnapshot` đảm nhiệm phần kiểm soát, vận hành và phân tích.

Lý do thiết kế của sơ đồ này.

- Sơ đồ tổng thể được đặt ở mức đủ rộng để người đọc nhận ra ranh giới giữa các miền nghiệp vụ, nhưng chưa đi quá sâu vào chi tiết cài đặt của từng mô-đun.
- Các quan hệ được lựa chọn theo tiêu chí nghiệp vụ, nghĩa là chỉ thể hiện những liên kết có ý nghĩa vận hành rõ ràng như đặt bàn gắn với bàn ăn, phiên bản sinh đơn gọi món, đơn gọi món sinh phiếu bếp, và hóa đơn quyết toán phiên bản.
- Các điểm mở rộng như `PaymentGateway`, `NotificationSender` và `AuthorizationPolicy` được đưa vào ngay trong sơ đồ tổng thể để nhấn mạnh rằng mô hình lớp không chỉ mô tả dữ liệu, mà còn chỉ ra **ranh giới mở rộng và kiểm soát**.

Điểm mạnh của sơ đồ tổng thể là hỗ trợ kiểm tra nhanh phạm vi bao phủ của tài liệu đối với IRMS. Nếu thiếu một cụm lớp ở hình này, khả năng cao là cụm nghiệp vụ tương ứng cũng sẽ bị trình bày thiếu ở các phần sau. Theo nghĩa đó, đây là sơ đồ dùng để khóa phạm vi thiết kế trước khi đi vào chi tiết.



Hình 29: Sơ đồ lớp UML của cụm gọi món, thực đơn và bếp

Hình 29 đi vào cụm lớp có mật độ tương tác cao nhất trong vận hành hằng ngày của nhà hàng. Đây là nơi luồng nghiệp vụ bắt đầu từ dữ liệu thực đơn, đi qua quá trình chọn món và tùy biến, sau đó chuyển thành các đơn vị điều phối ở bếp. Vì vậy, sơ đồ này có vai trò quan trọng trong việc chứng minh rằng mô hình gọi món của IRMS không bị gián lược thành một bảng dữ liệu phẳng.

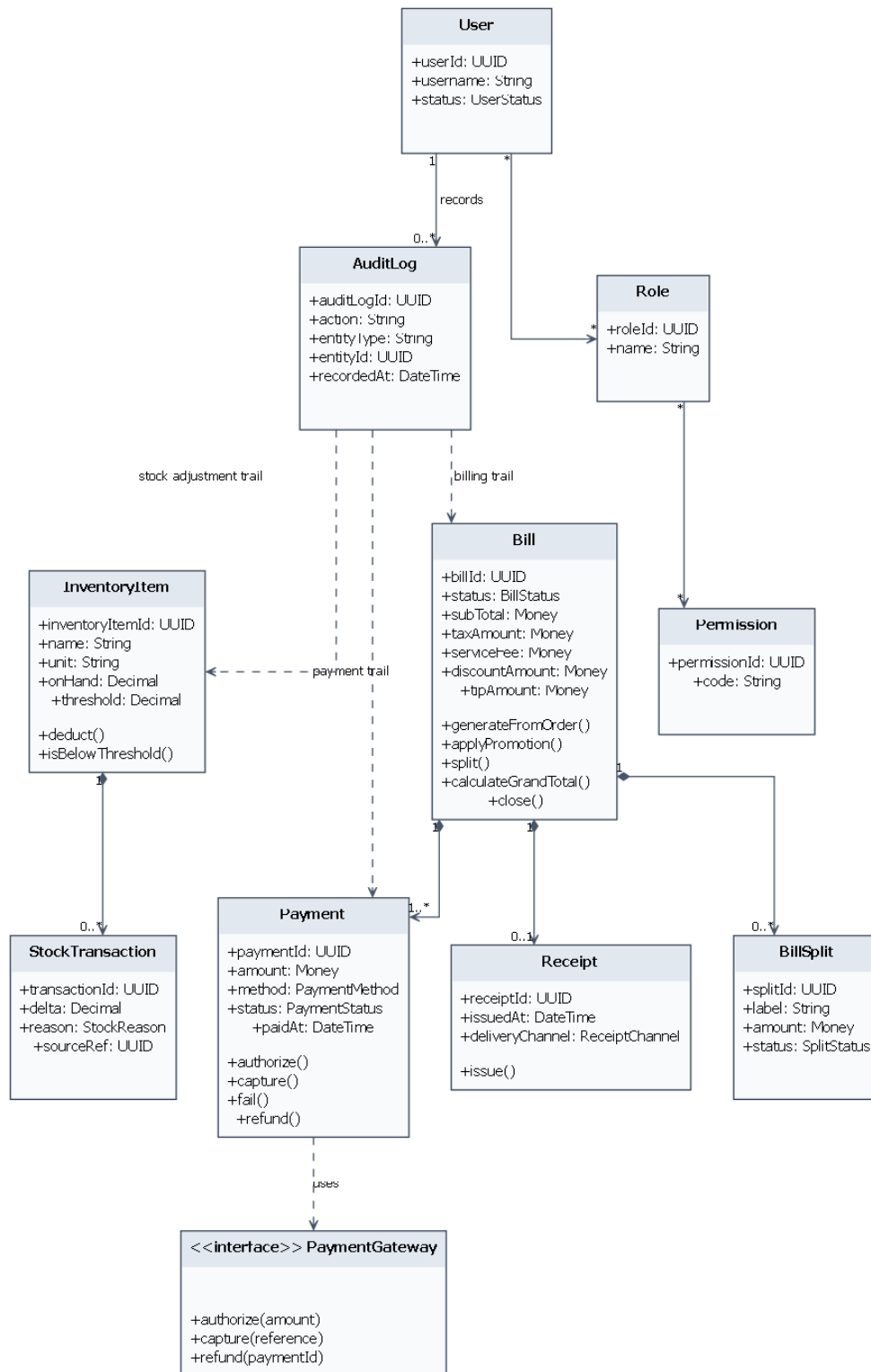
Các lớp trọng tâm thể hiện trong hình.

- **MenuCategory** và **MenuItem**: biểu diễn cấu trúc thực đơn và là nguồn gốc của giá cơ bản, trạng thái còn bán và thông tin điều phối.
- **ModifierGroup** và **ModifierOption**: chịu trách nhiệm ràng buộc việc tùy biến món, ví dụ giới hạn số lựa chọn tối thiểu và tối đa hoặc thay đổi giá do tùy chọn bổ sung.
- **Order** và **OrderItem**: là lõi của giao dịch gọi món; **Order** giữ trạng thái đơn tổng thể, còn **OrderItem** là đơn vị bán hàng cụ thể.
- **OrderItemModifier**: lưu ảnh chụp của tùy biến đã chọn tại thời điểm gọi món, giúp hệ thống không bị lệ thuộc ngược vào cấu hình thực đơn hiện tại.
- **KitchenStation**, **KitchenTicket** và **KitchenTicketItem**: chuyển dữ liệu từ miền bán hàng sang miền chế biến, đồng thời giữ được trạng thái của từng trạm bếp thay vì dồn toàn bộ tiến độ vào **Order**.

Lý do thiết kế của sơ đồ này.

- Tách **OrderItem** khỏi **KitchenTicketItem** để phân biệt rõ **đơn vị bán hàng** với **đơn vị điều phối chế biến**; đây là quyết định cần thiết nếu một món phải được định tuyến sang trạm bếp cụ thể.
- Dùng cơ chế ảnh chụp ở **OrderItem** và **OrderItemModifier** để bảo vệ tính đúng đắn của hóa đơn và phiếu bếp ngay cả khi thực đơn thay đổi sau thời điểm gọi món.
- Giữ **KitchenStation** như một lớp riêng để quy tắc định tuyến, mức ưu tiên và trạng thái bếp không làm phình to aggregate **Order**.

Ưu điểm của cụm lớp này là phản ánh được đồng thời **cấu hình thực đơn**, **dữ liệu giao dịch** và **trạng thái chế biến**. Nếu thiếu một trong ba lớp nghĩa đó, hệ thống rất dễ rơi vào tình trạng hoặc khó mở rộng nghiệp vụ, hoặc không theo dõi được đường đi thực sự của món từ màn hình gọi món xuống bếp.



Hình 30: Sơ đồ lớp UML của cum thanh toán, tồn kho và quản trị

Hình 30 tập trung vào vùng nhạy cảm nhất của kiến trúc, nơi các quyết định của hệ thống tác động trực tiếp đến tiền, tồn kho và quyền thao tác của người dùng. Vì đây là khu vực dễ phát sinh sai sót có hậu quả vận hành lớn, sơ đồ được trình bày chi tiết hơn để người đọc thấy rõ lớp nào chịu trách nhiệm giao dịch, lớp nào chịu trách nhiệm truy vết, và lớp nào đóng vai trò cổng mở rộng.

Các lớp trọng tâm thể hiện trong hình.

- **Cụm quyết toán:** Bill, BillSplit, AppliedDiscount và PromotionCampaign cùng nhau biểu diễn logic tổng hợp tiền, tách hóa đơn và áp dụng khuyến mãi.
- **Cụm thanh toán:** Payment, Refund, Receipt và PaymentGateway mô tả việc thu tiền, hoàn tiền, phát hành biên lai và giao tiếp với hệ thống ngoài qua giao diện trừu tượng.
- **Cụm tồn kho:** InventoryItem, RecipeLine, StockTransaction, LowStockRule và NotificationSender mô tả đường suy diễn từ món bán ra tới biến động tồn kho và cảnh báo mức tồn thấp.
- **Cụm quản trị và kiểm soát:** User, Role, Permission, AuditLog, AuthorizationPolicy, ShiftAssignment và ReportSnapshot tạo nền cho phân quyền, kiểm toán, lịch ca và báo cáo.

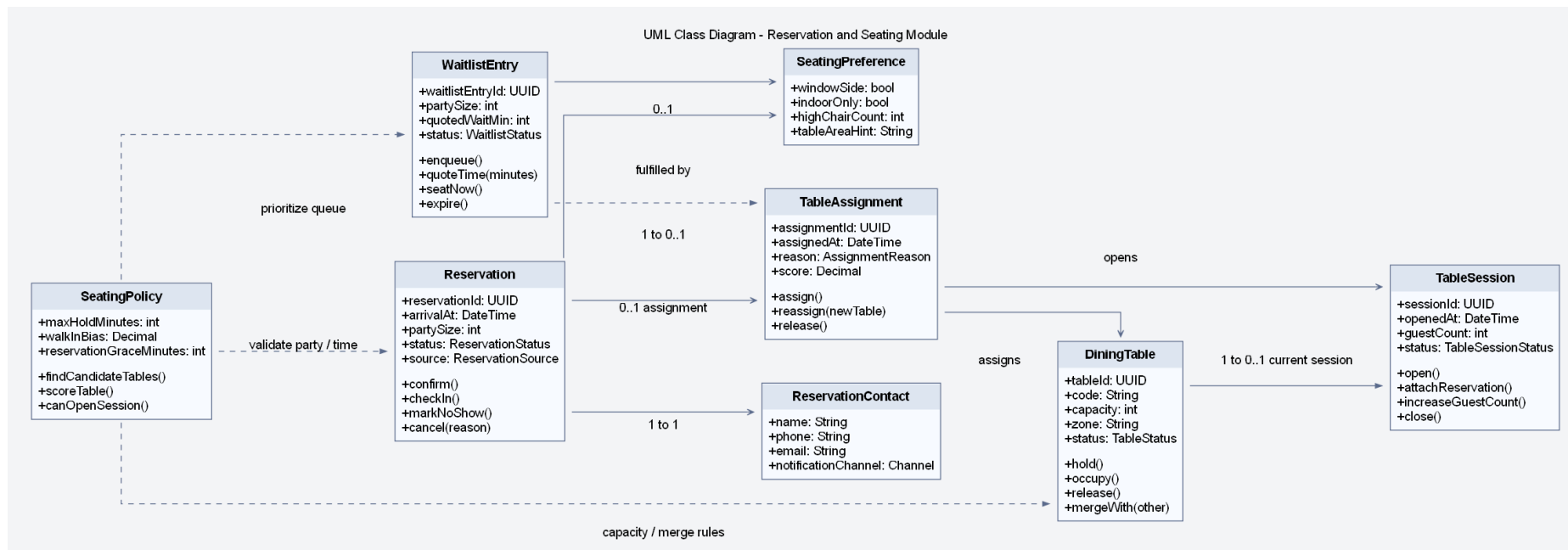
Lý do thiết kế của sơ đồ này.

- Đặt PaymentGateway và NotificationSender dưới dạng giao diện để miền nghiệp vụ không phụ thuộc trực tiếp vào một nhà cung cấp cụ thể.
- Tách Refund ra khỏi Payment để phản ánh đúng việc hoàn tiền là một hành động nghiệp vụ riêng, có lý do, ngưỡng phê duyệt và trạng thái theo dõi riêng.
- Đưa AuditLog và AuthorizationPolicy vào cùng sơ đồ với giao dịch để nhấn mạnh rằng kiểm soát quyền và truy vết không phải phần phụ, mà là một phần của thiết kế lõi.
- Giữ ReportSnapshot ở cụm này để thể hiện rõ dữ liệu báo cáo được suy diễn từ dữ liệu vận hành, thay vì can thiệp trực tiếp vào tuyến giao dịch nóng.

Ưu điểm lớn nhất của sơ đồ này là cho thấy kiến trúc lớp của IRMS không chỉ giải quyết việc “thực hiện đúng chức năng”, mà còn giải quyết việc “kiểm soát được hậu quả vận hành” khi xảy ra hoàn tiền, điều chỉnh kho, ghi đề giá hay tranh chấp thanh toán. Đây là điểm làm cho phần thiết kế chặt chẽ hơn dưới góc nhìn kiến trúc phần mềm trong môi trường vận hành thực tế.

4.2 Giải thích theo từng cụm lớp

Phần này được trình bày theo trình tự *quan sát sơ đồ* → *xác định cụm trách nhiệm* → *làm rõ lý do mô hình hóa*. Cách trình bày này giúp phần sơ đồ lớp không dừng lại ở mức liệt kê lớp, mà làm rõ vai trò trong miền, ranh giới trách nhiệm và quyết định thiết kế của từng mô-đun.



Hình 31: Sơ đồ lớp chi tiết cho module Reservation & Seating

Hình 31 đi vào module Reservation & Seating ở mức chi tiết và đúng tinh thần mô hình miền. Trọng tâm của module này không chỉ là “quản lý danh sách bàn”, mà là quản lý **năng lực phục vụ theo thời gian**: ai đến trước, ai có cam kết trước, bàn nào phù hợp, khi nào được mở phiên, và trên cơ sở nào quyết định gán bàn được đưa ra. Vì vậy, sơ đồ được tổ chức quanh ba trục: **nguồn nhu cầu** (Reservation, WaitlistEntry), **tài nguyên phục vụ** (DiningTable, TableSession) và **quy tắc điều phối** (TableAssignment, SeatingPolicy, SeatingPreference).

Các cụm lớp và trách nhiệm thể hiện trong hình.

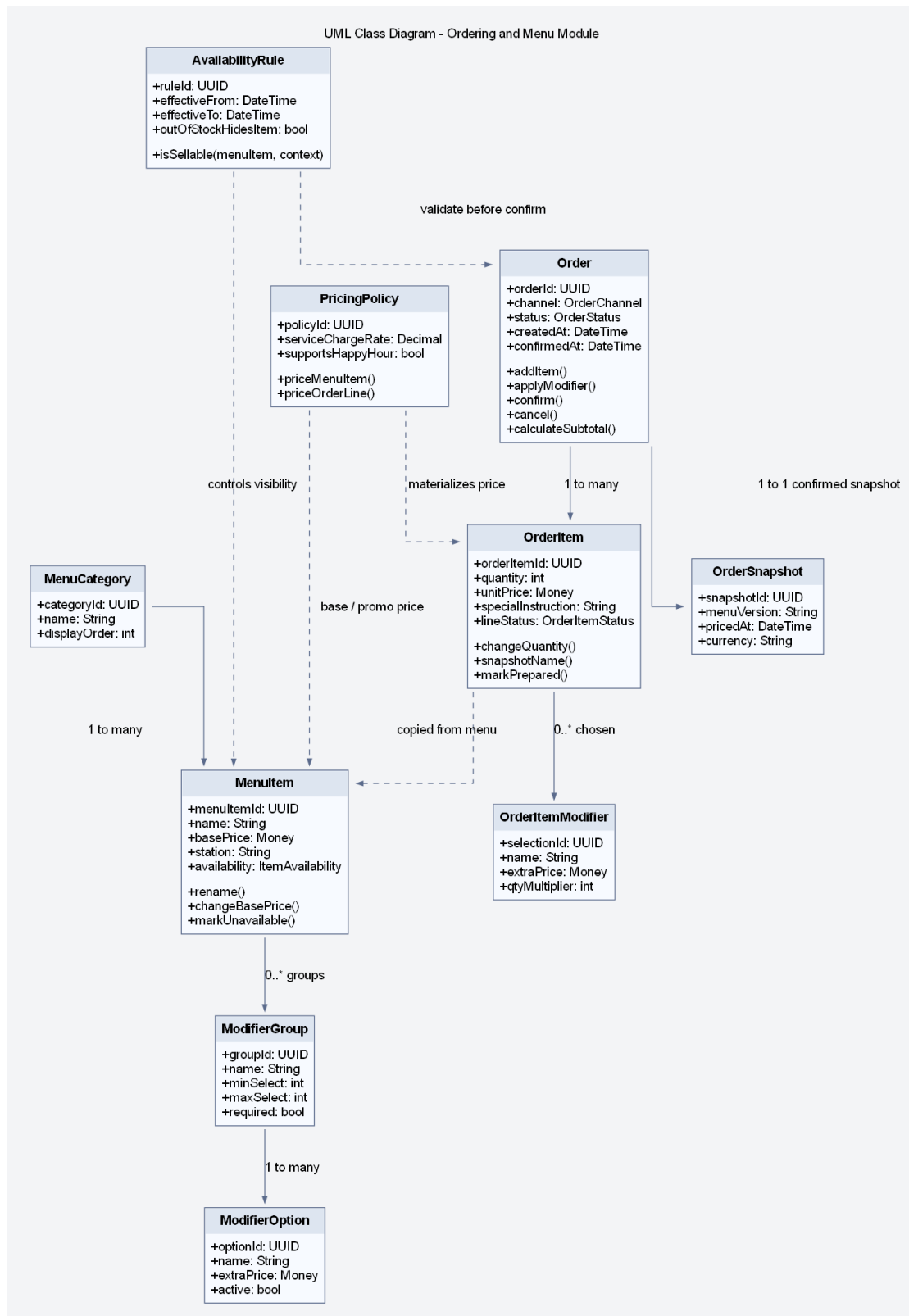
- **Nguồn nhu cầu trước khi phục vụ:** Reservation giữ thông tin đặt trước, trạng thái và các hành vi như xác nhận, check-in, no-show và hủy; ReservationContact tách riêng thông tin liên hệ để tránh dồn dữ liệu truyền thông vào thực thể đặt bàn.
- **Nguồn nhu cầu chưa được đáp ứng ngay:** WaitlistEntry mô hình hóa khách vắng lai hoặc trường hợp chưa có bàn phù hợp tại thời điểm hiện tại; điều này quan trọng vì waitlist không nên bị ép vào cùng vòng đời với reservation.
- **Tài nguyên vật lý và phiên vận hành:** DiningTable biểu diễn bàn vật lý với sức chứa và vùng; TableSession biểu diễn phiên phục vụ thực tế, độc lập với việc bàn đó có được đặt trước hay không.
- **Lớp điều phối:** TableAssignment lưu quyết định gán bàn cụ thể, còn SeatingPolicy và SeatingPreference làm rõ phân “cơ sở chọn bàn” thay vì để quy tắc nằm rải rác trong service.

Giải thích chi tiết theo từng cụm lớp.

- **Reservation không đồng nhất với TableSession.** Một reservation có thể bị hủy, no-show hoặc dời giờ mà không sinh phiên phục vụ. Ngược lại, một table session có thể được mở cho khách walk-in mà không có reservation. Việc tách hai lớp này là quyết định mô hình hóa quan trọng nhất của module.
- **WaitlistEntry là đối tượng hạng nhất của miền.** Nếu chỉ xem waitlist là một trạng thái phụ của reservation, hệ thống sẽ khó diễn tả đúng khách vắng lai, khách chưa chắc ngồi, khách đổi ý hoặc khách chấp nhận vùng bàn khác sau một khoảng chờ.
- **TableAssignment giữ lịch sử quyết định gán bàn.** Điều này hữu ích cho giải thích vận hành, tái gán bàn, và về sau có thể nối với audit hoặc analytics để đo hiệu quả xếp bàn theo giờ cao điểm.
- **SeatingPolicy là nơi đặt rule thay vì hard-code trong controller.** Các rule như thời gian giữ chỗ, ưu tiên walk-in hay reservation, giới hạn mở session và năng lực gộp bàn nên được gom vào chính sách có tên gọi rõ ràng.

Lý do thiết kế của sơ đồ này.

- Tách ReservationContact và SeatingPreference giúp thực thể Reservation gọn hơn, chỉ giữ phần nhận diện và trạng thái lõi.
- Dùng TableAssignment như đối tượng trung gian để lưu lý do và điểm chấm bàn, nhờ đó quyết định xếp bàn có thể giải thích được.
- Giữ TableSession như bản ghi riêng nhằm bảo vệ toàn bộ tuyến gọi món, hóa đơn và thanh toán khỏi việc phụ thuộc trực tiếp vào đặt bàn.



Hình 32: Sơ đồ lớp chi tiết cho module Ordering & Menu

Hình 32 mô tả mô-đun có cường độ vận hành cao nhất của toàn hệ thống. Về mặt miền, mô-đun này phải giải quyết mâu thuẫn giữa **thực đơn biến đổi liên tục** và **giao dịch cần được cố định**. Thực đơn có thể thay đổi thường xuyên: ẩn món, đổi giá, thêm tùy chọn món hoặc đổi giờ phục vụ. Trong khi đó, đơn gọi món đã xác nhận phải giữ nguyên đúng tên món, giá, tùy chọn và điều kiện áp dụng tại thời điểm xác nhận. Vì vậy, sơ đồ tách rõ hai nhóm đối tượng: **Menu*** là phần cấu hình có thể thay đổi, còn **Order*** là phần giao dịch đã được chốt.

Các cụm lớp và trách nhiệm thể hiện trong hình.

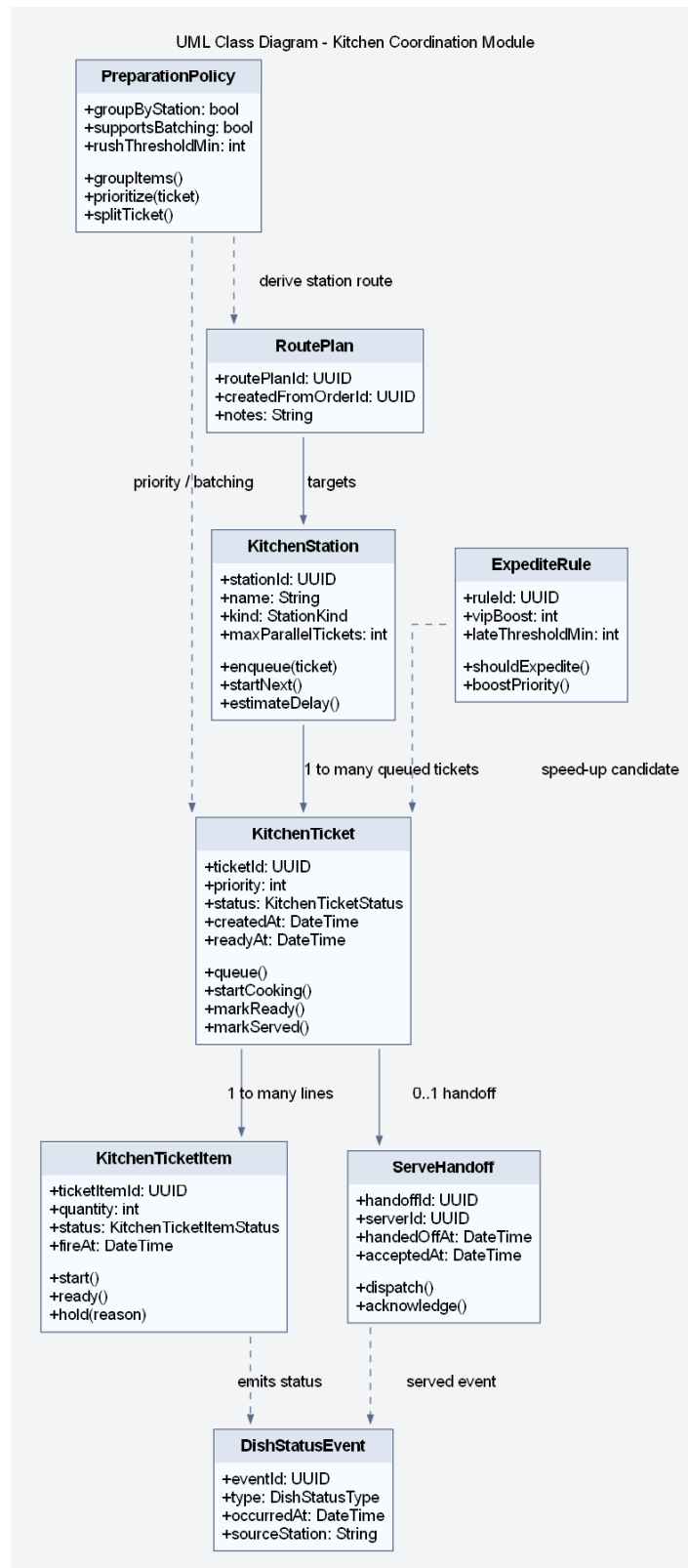
- **Cấu hình thực đơn:** `MenuCategory`, `MenuItem`, `ModifierGroup` và `ModifierOption` tạo nên cấu trúc menu mà quản lý có thể thay đổi.
- **Quy tắc khả dụng và định giá:** `AvailabilityRule` và `PricingPolicy` gom logic giờ bán, trạng thái hết hàng, giá cơ sở và các quy tắc điều chỉnh giá.
- **Giao dịch đơn gọi món:** `Order`, `OrderItem` và `OrderItemModifier` lưu lại món đã chọn, tùy chọn đã chọn, số lượng, ghi chú và trạng thái của từng dòng món.
- **Ảnh chụp giao dịch:** `OrderSnapshot` cho thấy tại thời điểm xác nhận, hệ thống có điểm neo để lần vết lại phiên bản menu và cách tính giá đã được áp dụng.

Giải thích chi tiết theo từng cụm lớp.

- **`MenuItem` không nên được dùng trực tiếp như `OrderItem`.** Nếu dùng trực tiếp, chỉ cần đổi tên hoặc giá món sau đó là lịch sử giao dịch và hóa đơn sẽ bị biến dạng.
- **`ModifierGroup` và `ModifierOption` là cấu trúc mang quy tắc.** Chúng không chỉ là dữ liệu hiển thị; chúng còn quyết định số lựa chọn bắt buộc, số lượng tối đa và phần cộng giá thêm.
- **`PricingPolicy` nên tách khỏi `MenuItem`.** Giá của một món không chỉ là `basePrice`; nó còn phụ thuộc khung giờ, phí phục vụ, chương trình áp dụng hoặc logic định giá theo bối cảnh. Tách chính sách này giúp mô-đun tuân thủ trách nhiệm đơn nhất và dễ mở rộng hơn.
- **`OrderSnapshot` khóa ngữ nghĩa tại thời điểm xác nhận.** Đây là điểm rất quan trọng khi cần kiểm toán, đối soát hóa đơn hoặc tái hiện đơn hàng trong trường hợp tranh chấp.

Lý do thiết kế của sơ đồ này.

- Việc tách rule và data giúp module vừa linh hoạt cho quản trị menu, vừa chắc chắn cho tuyến giao dịch.
- Đặt `AvailabilityRule` và `PricingPolicy` trong cùng sơ đồ để nhấn mạnh rằng `validate` và `price` phải là một phần của domain model, chứ không chỉ là service logic.
- Duy trì `OrderItemModifier` như thực thể riêng để ghi nhận modifier thực tế đã chọn thay vì chỉ giữ quan hệ tham chiếu tới modifier cấu hình.



Hình 33: Sơ đồ lớp chi tiết cho module Kitchen Coordination

Hình 33 làm rõ rằng điều phối bếp là một miền riêng, chứ không chỉ là một trạng thái phụ của đơn gọi món. Khi đơn gọi món đã được xác nhận, hệ thống bắt đầu giải quyết một bài toán khác: chia việc theo trạm, xếp hàng đợi, điều chỉnh mức ưu tiên, theo dõi tiến độ và bàn giao món đã hoàn tất cho tuyến phục vụ. Nếu dồn toàn bộ logic này vào Order hoặc OrderItem, mô hình sẽ nhanh chóng lẫn lộn giữa góc nhìn bán hàng và góc nhìn chế biến.

Các cụm lớp và trách nhiệm thể hiện trong hình.

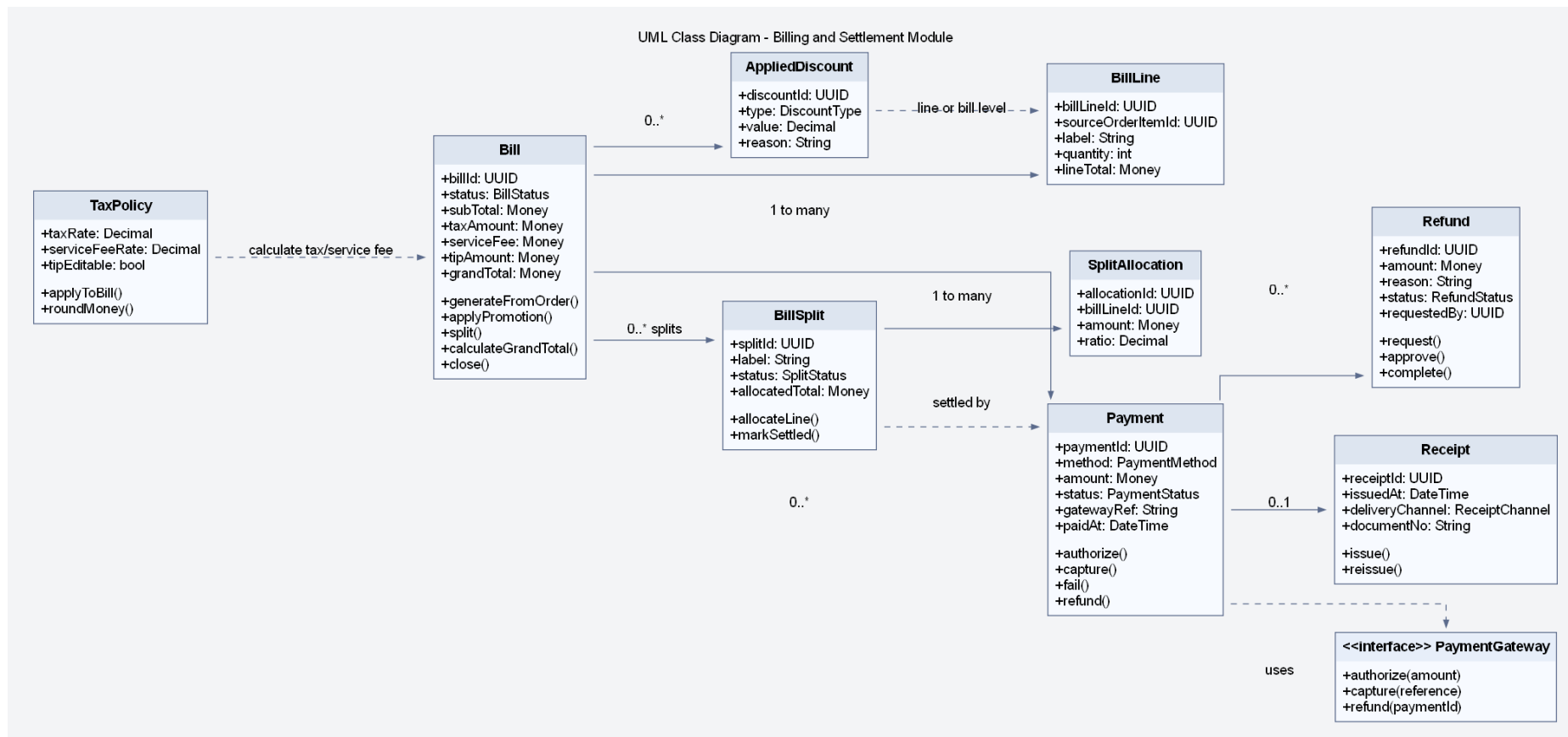
- **Tài nguyên bếp:** KitchenStation biểu diễn từng trạm với năng lực xử lý, loại trạm và hàng chờ tương ứng.
- **Đơn vị điều phối:** KitchenTicket là phiếu công việc ở mức phiếu bếp; KitchenTicketItem là đơn vị thực thi cụ thể theo món hoặc nhóm món.
- **Chính sách điều phối:** PreparationPolicy, RoutePlan và ExpediteRule gom logic tách phiếu, nhóm món, định tuyến và tăng ưu tiên.
- **Bàn giao trạng thái:** ServeHandoff và DishStatusEvent neo đường truyền trạng thái món ra khỏi bếp.

Giải thích chi tiết theo từng cụm lớp.

- **KitchenTicket là aggregate riêng.** Nó có vòng đời chờ, đang làm, sẵn sàng và đã bàn giao; vòng đời này không nên bị gộp vào trạng thái đơn gọi món.
- **KitchenTicketItem cho phép xử lý ở mức hạt mịn.** Trong thực tế, không phải toàn bộ phiếu bếp luôn đi cùng một tốc độ; từng dòng có thể bị giữ, sẵn sàng hay bị chậm riêng.
- **PreparationPolicy và ExpediteRule là nơi đặt thuật toán điều phối.** Nhờ đó, logic ưu tiên khách VIP, món bị trễ hoặc gom nhóm theo trạm bếp có vị trí rõ ràng trong mô hình.
- **ServeHandoff tạo điểm chốt giữa bếp và phục vụ.** Một món ở trạng thái ready chưa đồng nghĩa với trạng thái served; thêm lớp bàn giao giúp phân biệt rõ hai giai đoạn này.

Lý do thiết kế của sơ đồ này.

- Tách miền bếp thành một module có lớp riêng giúp hệ thống mở rộng thuận lợi hơn khi cần nhiều station hoặc nhiều màn hình bếp khác nhau.
- Dùng lớp sự kiện DishStatusEvent để cho phép mô hình đọc, thông báo hoặc phân tích tiến độ mà không chạm trực tiếp vào aggregate bếp.
- Mô hình này cũng làm rõ vì sao góc nhìn thành phần cần KitchenTicketFactory và KitchenApplications riêng.



Hình 34: Sơ đồ lớp chi tiết cho module Billing & Settlement

Hình 34 trình bày vùng quyết toán với độ sâu phù hợp cho báo cáo. Ở IRMS, bài toán quyết toán không chỉ dừng ở việc “tính tổng tiền”: hệ thống còn phải biểu diễn dòng hóa đơn, tách hóa đơn, phân bổ dòng vào từng phần thanh toán, áp khuyến mãi, tính thuế và phí, thu tiền theo một hay nhiều giao dịch thanh toán, phát hành biên lai và theo dõi hoàn tiền. Mỗi quyết định trong vùng này đều có hậu quả tài chính, vì vậy mô hình lớp phải được phân tách đủ rõ để tránh một lớp `Bill` quá lớn ôm toàn bộ trách nhiệm.

Các cụm lớp và trách nhiệm thể hiện trong hình.

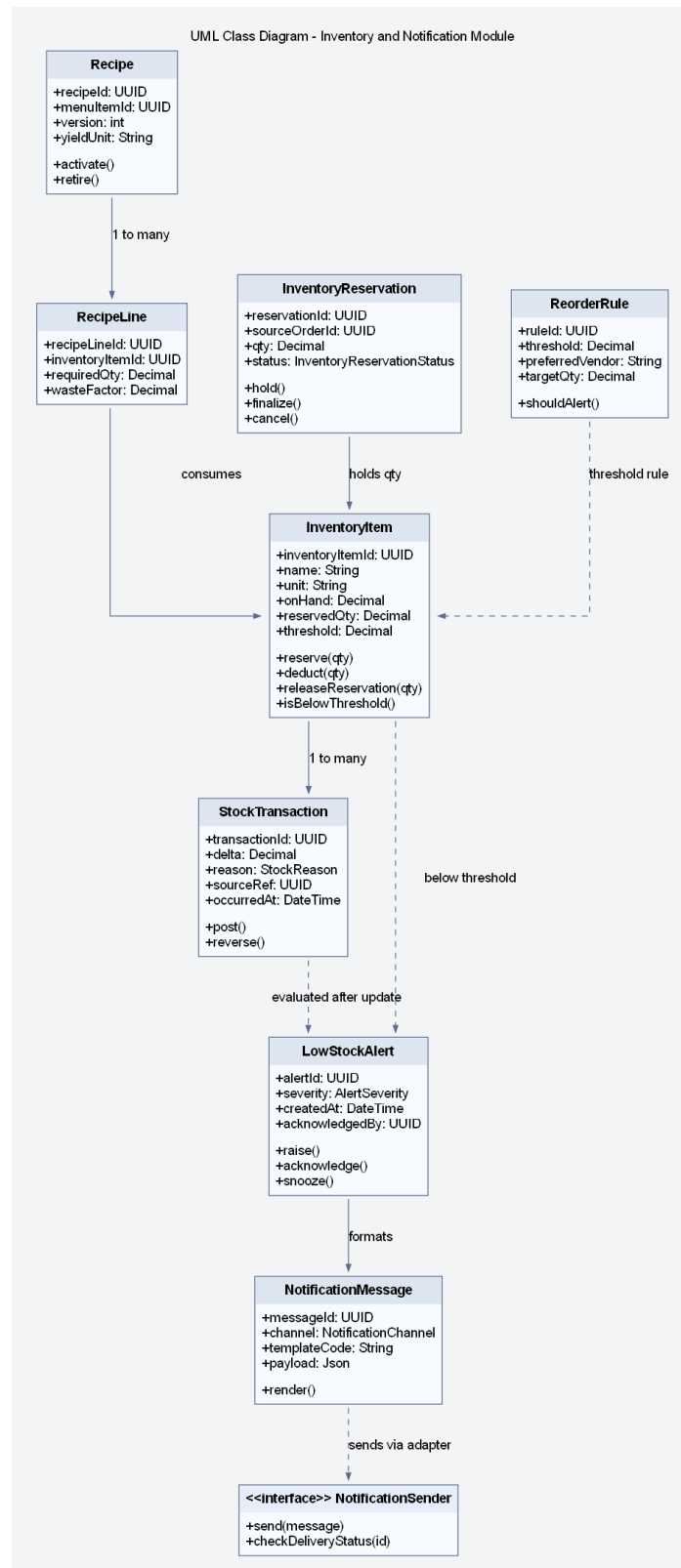
- **Hóa đơn và dòng hóa đơn:** `Bill` và `BillLine` là trung tâm quyết toán, nơi dữ liệu đơn gọi món đã xác nhận được gom lại thành đối tượng tài chính.
- **Tách hóa đơn:** `BillSplit` và `SplitAllocation` biểu diễn đúng nghiệp vụ chia hóa đơn, thay vì coi split chỉ là một vài cờ trạng thái hoặc danh sách tạm.
- **Thanh toán và hoàn tiền:** `Payment`, `Refund` và `Receipt` tạo thành vòng đời thu tiền, hoàn tiền và chứng từ.
- **Quy tắc quyết toán:** `AppliedDiscount`, `TaxPolicy` và giao diện `PaymentGateway` gom logic giảm giá, làm tròn, thuế và tích hợp ngoài.

Giải thích chi tiết theo từng cụm lớp.

- **`BillSplit` là đối tượng hạng nhất.** Trong nhà hàng, việc tách hóa đơn không phải thao tác hiếm; nó có nhãn, phạm vi, tổng phân bổ và trạng thái riêng. Mô hình hóa tường minh giúp hỗ trợ chia theo người, theo dòng món hoặc theo tỷ lệ.
- **`Payment` tách khỏi `Bill`.** Một hóa đơn có thể có nhiều giao dịch thanh toán; một giao dịch có thể thất bại, cần thử lại, hoặc bị hoàn tiền một phần. Dồn logic này vào hóa đơn sẽ làm `Bill` có quá nhiều lý do thay đổi.
- **`Refund` là thực thể riêng, không chỉ là số âm của `payment`.** Nó cần lý do, người yêu cầu, trạng thái và về sau có thể cần cơ chế phê duyệt.
- **`PaymentGateway` phải là lớp trừu tượng.** Lỗi quyết toán cần thực hiện các thao tác nghiệp vụ như ủy quyền, ghi nhận giao dịch và hoàn tiền, nhưng không nên phụ thuộc vào SDK hoặc giao thức đặc thù của từng nhà cung cấp.

Lý do thiết kế của sơ đồ này.

- Mô hình này bám sát tuyến giao dịch nhưng vẫn chừa đúng điểm mở cho bộ kết nối.
- Việc tách `TaxPolicy` và `AppliedDiscount` làm rõ đâu là dữ liệu nghiệp vụ, đâu là quy tắc tính toán.
- Sơ đồ cũng chuẩn bị tốt cho kiểm toán: hoàn tiền, tách hóa đơn và giao dịch thanh toán đều đã có chỗ neo riêng để nối với nhật ký kiểm toán hoặc luồng phê duyệt.



Hình 35: Sơ đồ lớp chi tiết cho module Inventory & Notification

Hình 35 mở rộng phần tồn kho theo hướng **truy vết được nguồn gốc biến động** và **phân biệt rõ dữ liệu kho với kênh cảnh báo**. Một hệ thống nhà hàng thực tế không thể chỉ có trường `onHand`; nó phải biết món nào dùng nguyên liệu nào, thời điểm nào thì giữ tạm nguyên liệu, khi nào thì trừ thật, và khi nào cần phát ra cảnh báo mức tồn thấp. Vì vậy, sơ đồ bổ sung các lớp `Recipe`, `RecipeLine`, `InventoryReservation`, `ReorderRule`, `LowStockAlert` và `NotificationMessage`.

Các cụm lớp và trách nhiệm thể hiện trong hình.

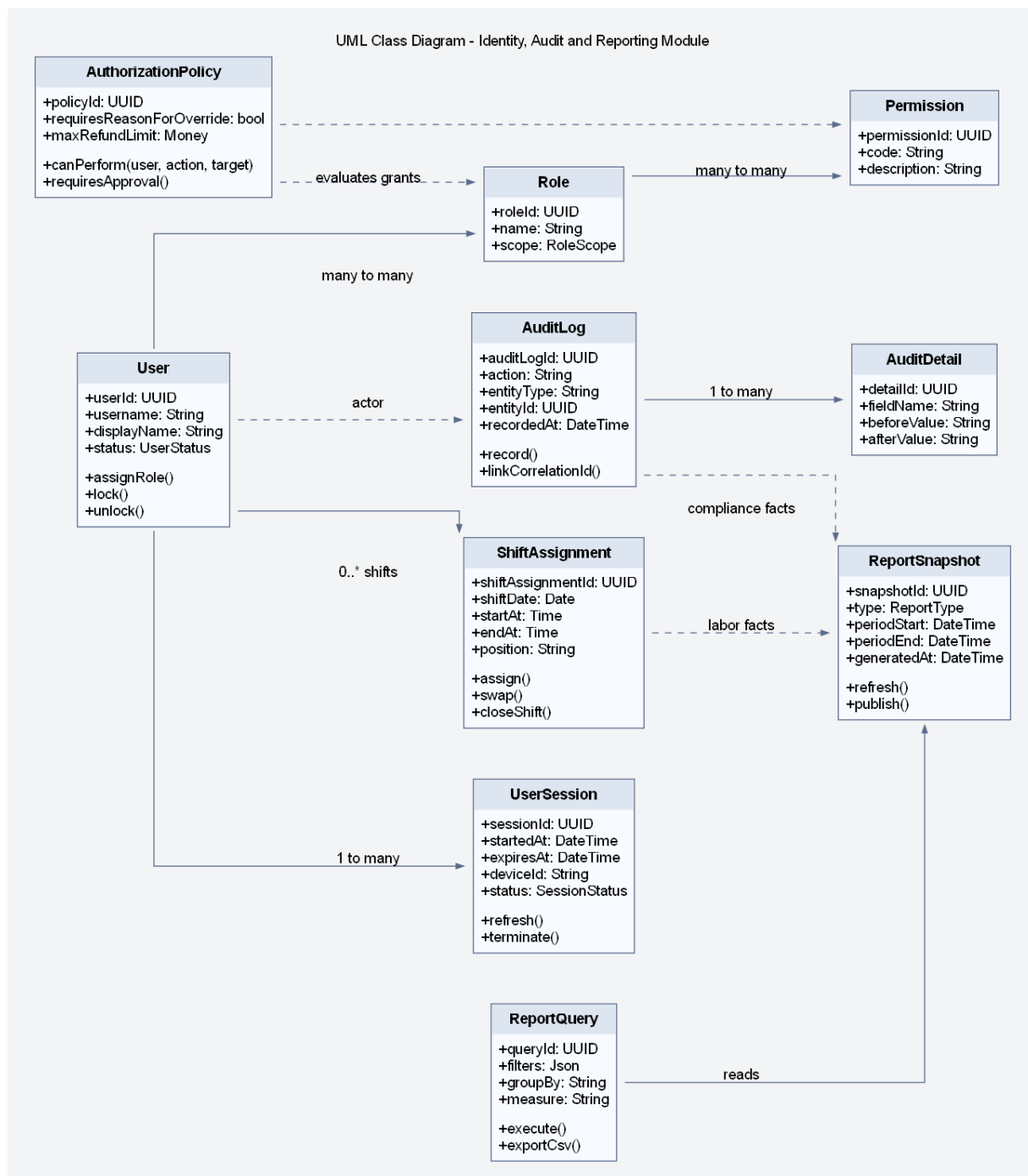
- **Số dư và giao dịch kho:** `InventoryItem` và `StockTransaction` giữ trạng thái kho hiện tại và lịch sử biến động.
- **Suy diễn mức tiêu hao:** `Recipe` và `RecipeLine` nối món bán ra với nguyên liệu cần dùng.
- **Giữ chỗ và chốt trừ:** `InventoryReservation` cho phép mô tả giai đoạn giữ tạm nguyên liệu trước khi thanh toán hoàn tất hoặc đơn gọi món được chốt.
- **Cảnh báo và kênh gửi:** `ReorderRule`, `LowStockAlert`, `NotificationMessage` và giao diện `NotificationSender` tách rõ điều kiện phát cảnh báo khỏi hạ tầng chuyển thông báo.

Giải thích chi tiết theo từng cụm lớp.

- **Recipe là cầu nối giữa thực đơn và kho.** Nếu thiếu lớp này, phần tiêu hao nguyên liệu sẽ bị mã hóa cứng trong logic gọi món hoặc quyết toán.
- **InventoryReservation giúp mô hình linh hoạt hơn.** Một số phương án triển khai muốn trừ kho ngay khi xác nhận đơn gọi món, số khác chỉ chốt hẳn khi thanh toán hoàn tất; có lớp giữ chỗ giúp mở được cả hai chiến lược.
- **StockTransaction là bằng chứng, không chỉ là lịch sử phụ.** Nó cho phép giải thích vì sao tồn kho thay đổi, hỗ trợ reconciliation và audit sau ca.
- **Notification cần đi qua lớp trừu tượng.** Cảnh báo tồn thấp là nghiệp vụ; gửi email, SMS hay thông báo đẩy lại là bài toán của bộ kết nối.

Lý do thiết kế của sơ đồ này.

- Mô hình tách được tồn kho khỏi kênh thông báo nhưng vẫn giữ liên hệ chặt thông qua `LowStockAlert`.
- Nó hỗ trợ tốt cho sơ đồ thành phần bất đồng bộ, nơi bộ xử lý nền có thể gửi cảnh báo sau khi mô-đun tồn kho phát hiện vượt ngưỡng.
- Nó cũng tạo điểm nối dữ liệu sạch hơn cho phần báo cáo doanh thu–tồn kho ở giai đoạn sau.



Hình 36: Sơ đồ lớp chi tiết cho module Identity, Audit & Reporting

Hình 36 trình bày phần governance ở mức phù hợp với kiến trúc của một hệ thống vận hành thực tế. Trong IRMS, user, role và permission mới chỉ là phần đầu. Hệ thống còn cần mô tả phiên đăng nhập (UserSession), chính sách kiểm tra quyền theo bối cảnh (AuthorizationPolicy), nhật ký kiểm toán đa thuộc tính (AuditLog, AuditDetail), phân công ca (ShiftAssignment) và ảnh chụp báo cáo cùng truy vấn báo cáo (ReportSnapshot, ReportQuery). Chỉ khi các lớp này được biểu diễn tường minh, phần “quản trị, kiểm toán, báo cáo” mới vượt khỏi mức mô tả khái quát.

Các cụm lớp và trách nhiệm thể hiện trong hình.

- **Danh tính và quyền:** User, Role, Permission và UserSession giữ nền tảng xác thực–phân quyền.
- **Chính sách truy cập:** AuthorizationPolicy gom logic bối cảnh như giới hạn hoàn tiền, yêu cầu

nêu lý do ghi đề hoặc yêu cầu phê duyệt.

- **Dấu vết thay đổi:** AuditLog và AuditDetail lưu hành động, thực thể bị tác động và giá trị trước-sau.
- **Nhân sự và phân tích:** ShiftAssignment, ReportSnapshot và ReportQuery nổi dữ liệu vận hành với góc nhìn quản trị.

Giải thích chi tiết theo từng cụm lớp.

- **AuthorizationPolicy cần là lớp riêng.** Nếu mọi quy tắc quyền đều nằm ở bộ điều khiển hoặc chú giải, hệ thống sẽ khó diễn tả các quyết định phụ thuộc ngữ cảnh nghiệp vụ như số tiền hoàn tiền hay lý do ghi đề.
- **AuditDetail giúp nhật ký kiểm toán có giá trị thực tế.** Chỉ lưu “ai làm gì” là chưa đủ; với thao tác nhạy cảm, cần biết trường nào đã đổi từ gì sang gì.
- **UserSession là đối tượng đáng được mô hình hóa.** Nó hỗ trợ khóa phiên, truy vết thiết bị và quản lý timeout, vốn là nhu cầu thực tế trong nhà hàng khi đổi ca hoặc bàn giao máy.
- **ReportSnapshot tách khỏi ReportQuery.** Snapshot là dữ liệu tổng hợp bền vững cho một kỳ; query là cách người dùng truy cập và lọc dữ liệu đó.

Lý do thiết kế của sơ đồ này.

- Module governance được biểu diễn chi tiết giúp phân bảo mật, kiểm toán và báo cáo giữ đúng vai trò kiến trúc, thay vì bị xem như lớp chức năng phụ.
- Nó tạo điểm nổi rõ ràng giữa sơ đồ lớp với góc nhìn thành phần hỗ trợ và quản trị đã trình bày ở phần trước.
- Nó cũng hỗ trợ tốt cho phân biện minh nguyên lý SOLID, đặc biệt là SRP, OCP và DIP ở vùng policy, audit và reporting.

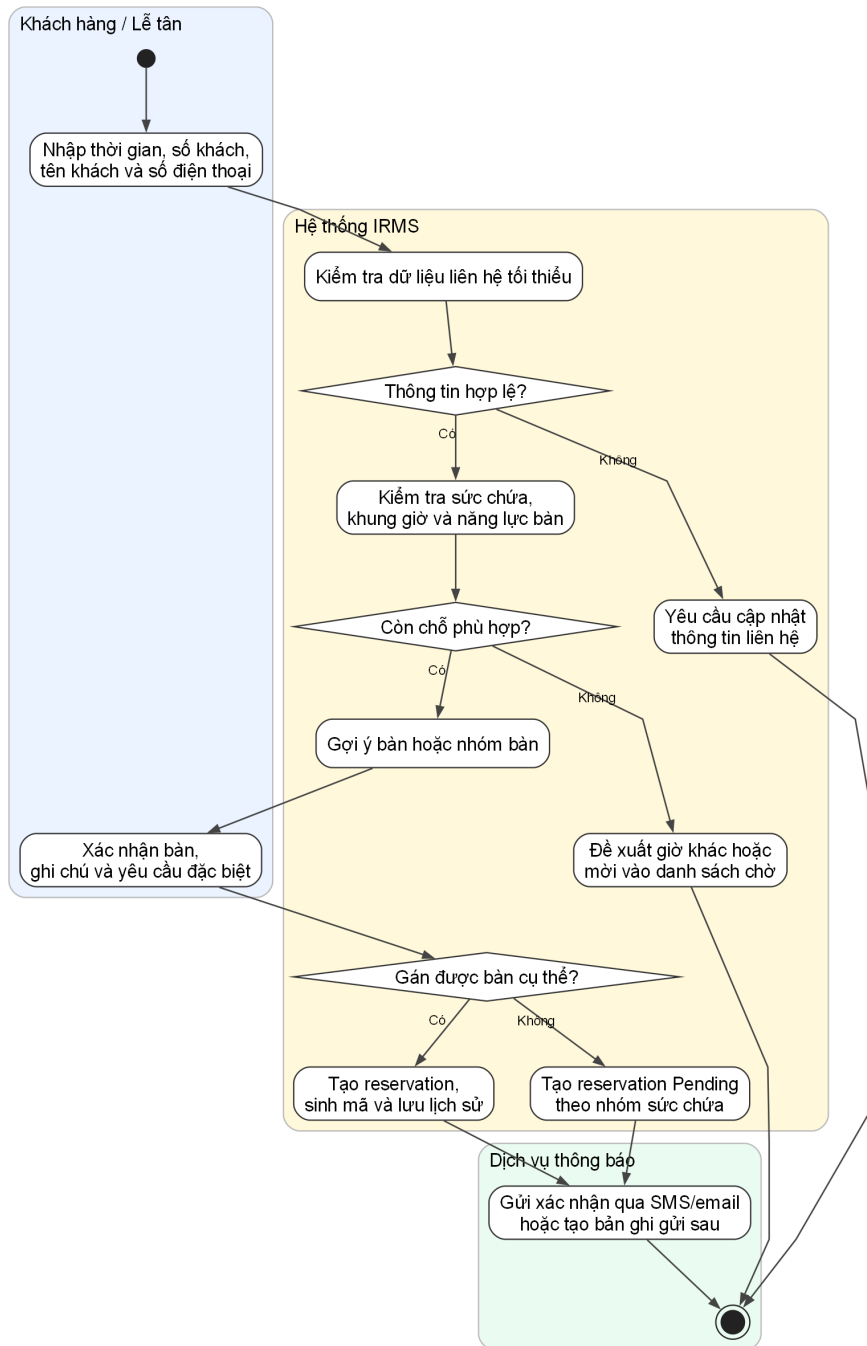
4.3 Thiết kế hành vi dựa trên ca sử dụng

Phần này mở rộng trực tiếp từ các ca sử dụng đã đặc tả ở trên. Phần đặc tả ca sử dụng xác định tác nhân, mục tiêu, điều kiện và kết quả nghiệp vụ; còn sơ đồ hoạt động diễn đạt thứ tự xử lý, điểm rẽ nhánh và nơi phát sinh ngoại lệ trong từng luồng vận hành. Vì vậy, các sơ đồ dưới đây được xây dựng theo đúng ranh giới vận hành hiện tại của IRMS: lễ tân, phục vụ, bếp, thu ngân, quản lý và hệ thống IRMS.

Một điểm quan trọng của nhóm sơ đồ này là mức độ chi tiết không hoàn toàn giống nhau. Những ca sử dụng nằm trên tuyến nghiệp vụ nóng như nhận khách, gọi món, gửi bếp, cập nhật chế biến, tạo hóa đơn, thanh toán, hoàn tiền và cập nhật tồn kho được mở rộng sâu hơn vì đây là các vị trí quyết định trực tiếp đến độ trễ phản hồi, độ tin cậy giao dịch và khả năng truy vết của toàn hệ thống.

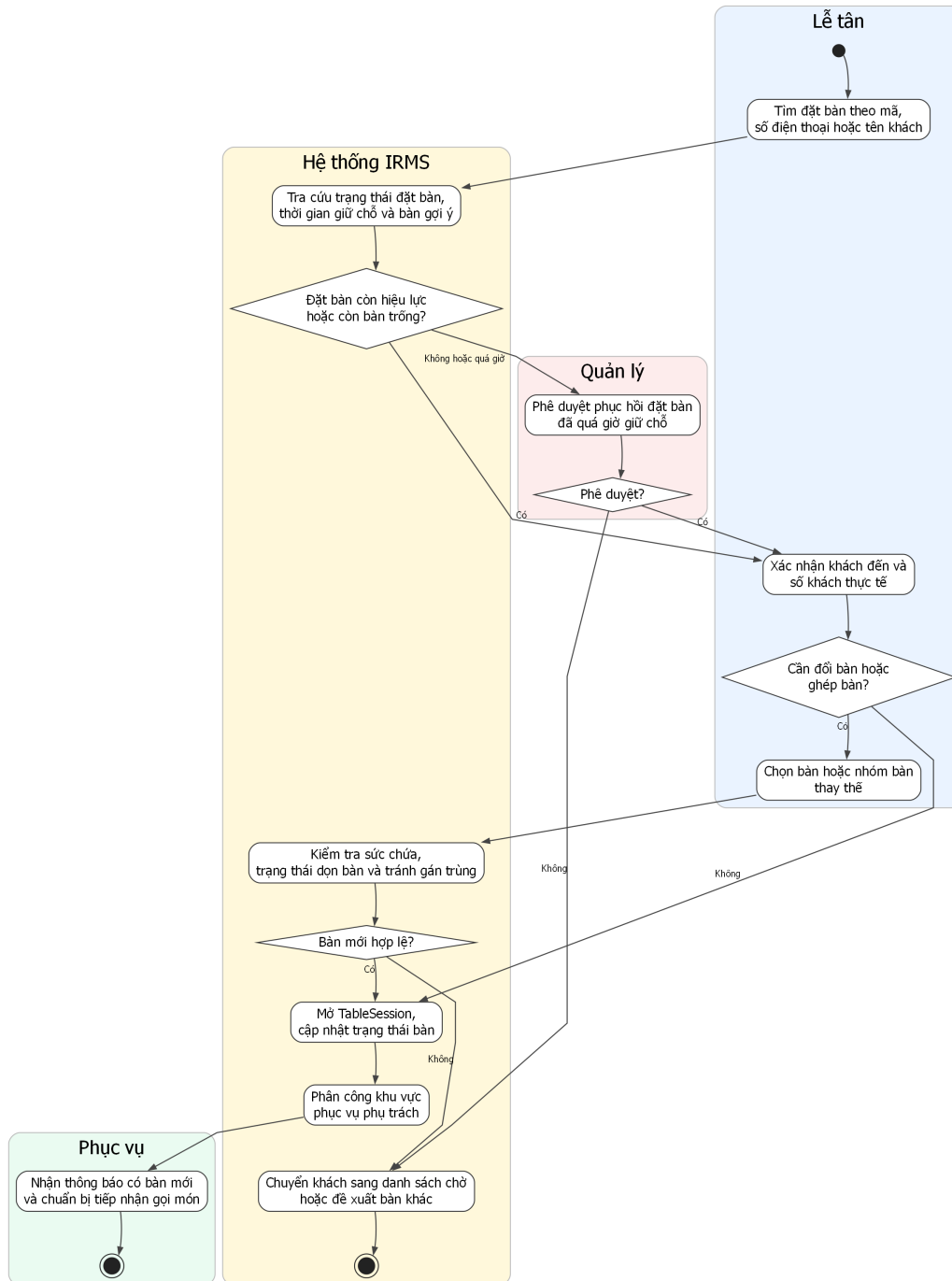
4.3.1 Nhóm đặt bàn và bàn ăn

UC-RES-01 -- Tạo đặt bàn



Hình 37: Sơ đồ hoạt động cho UC-RES-01 – Tạo đặt bàn

UC-RES-02 — Nhận khách và sắp bàn

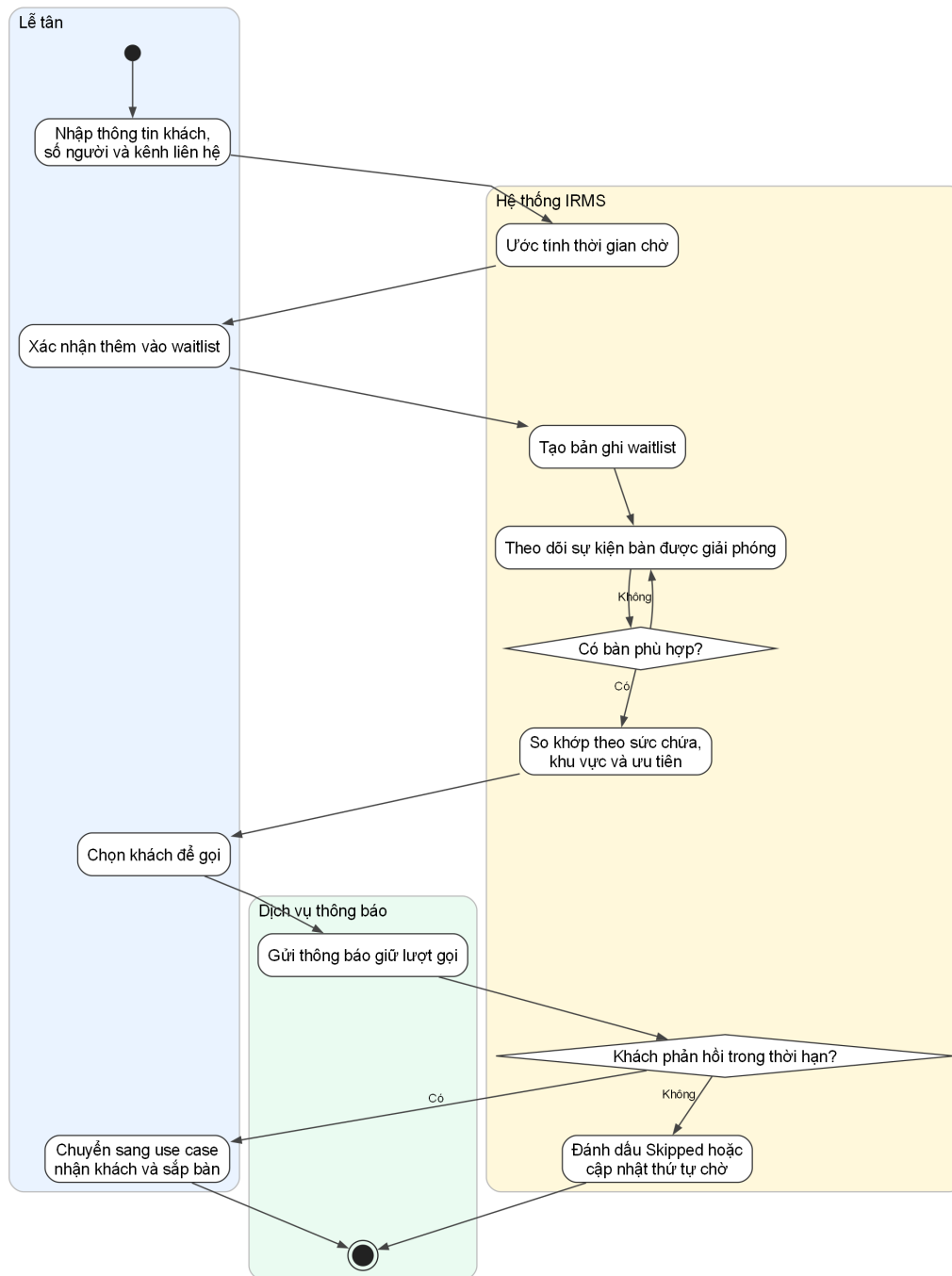


Hình 38: Sơ đồ hoạt động cho UC-RES-02 – Nhận khách và sắp bàn

Phân tích sơ đồ UC-RES-02. Luồng hoạt động trong trường hợp này bám sát thực tế tại quầy lễ tân: tìm đặt bàn, kiểm tra hiệu lực, đối chiếu số khách thực tế, xác định có cần đổi bàn hoặc ghép bàn hay không, sau đó mới mở TableSession. Điểm mấu chốt là việc **mở phiên phục vụ bàn** luôn diễn ra sau bước kiểm tra bàn khả dụng, nhờ đó báo cáo giữ được bất biến quan trọng rằng một bàn không có hai phiên phục vụ hoạt động cùng lúc.

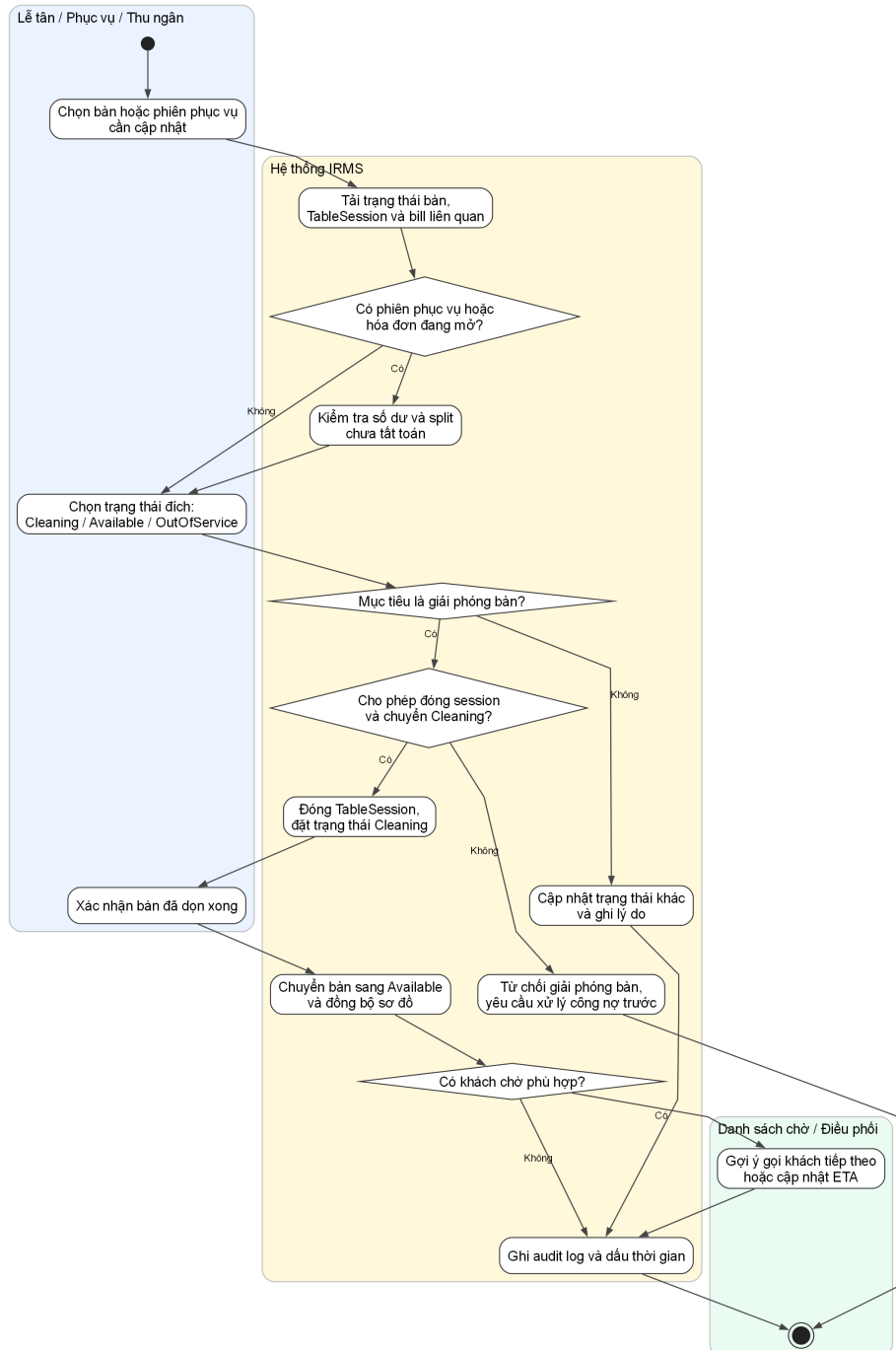
Ngoài ra, sơ đồ cũng làm rõ nhánh ngoại lệ khi đặt bàn đã quá giờ giữ chỗ. Trong tình huống này, hệ thống không tự động bỏ qua mà chuyển sang bước phê duyệt của quản lý. Cách mô tả này bám sát phân đặc tả ca sử dụng phía trên và phản ánh đúng yêu cầu kiểm soát nghiệp vụ của IRMS.

UC-RES-03 -- Quản lý danh sách chờ



Hình 39: Sơ đồ hoạt động cho UC-RES-03 – Quản lý danh sách chờ

UC-RES-04 -- Cập nhật trạng thái bàn và giải phóng bàn

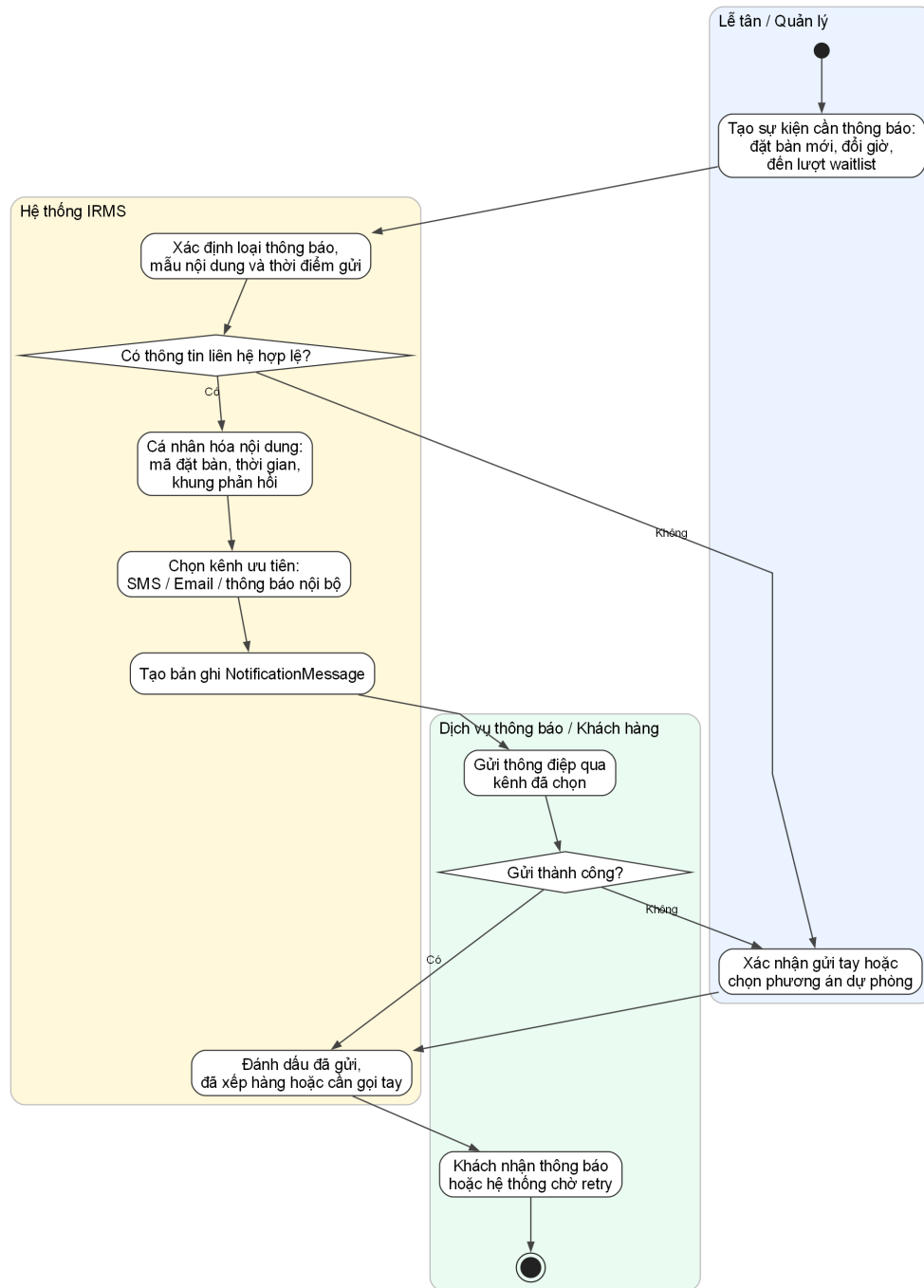


Hình 40: Sơ đồ hoạt động cho UC-RES-04 – Cập nhật trạng thái bàn và giải phóng bàn

Phân tích sơ đồ UC-RES-04. Luồng này làm rõ phần nằm giữa **kết thúc phục vụ** và **bắt đầu vòng quay bàn mới**. Trong vận hành thực tế, việc một bàn trở thành Available không thể chỉ là hệ quả ngầm sau thanh toán. Nó phải đi qua chuỗi kiểm tra rõ ràng: còn phiên phục vụ nào đang mở không, còn split hoặc công nợ nào chưa tất toán không, bàn đã chuyển sang Cleaning chưa, và nhân viên đã xác nhận dọn xong chưa. Nhờ đó, trạng thái bàn trên sơ đồ vận hành phản ánh đúng thực tế thay vì chỉ là ước đoán theo cảm tính của từng bộ phận.

Giá trị nổi bật của sơ đồ là khả năng nối được ba miền vốn thường bị mô tả rời nhau: TableSession, Billing và Waitlist. Khi bàn được giải phóng thành công, hệ thống không chỉ đổi một nhãn trạng thái; nó còn cập nhật nền cho việc gọi khách tiếp theo, tính vòng quay bàn và tránh lỗi gán trùng trong giờ cao điểm. Đây là lý do ca sử dụng này cần được tách riêng thay vì ẩn trong một mô tả chung về “đóng bàn”.

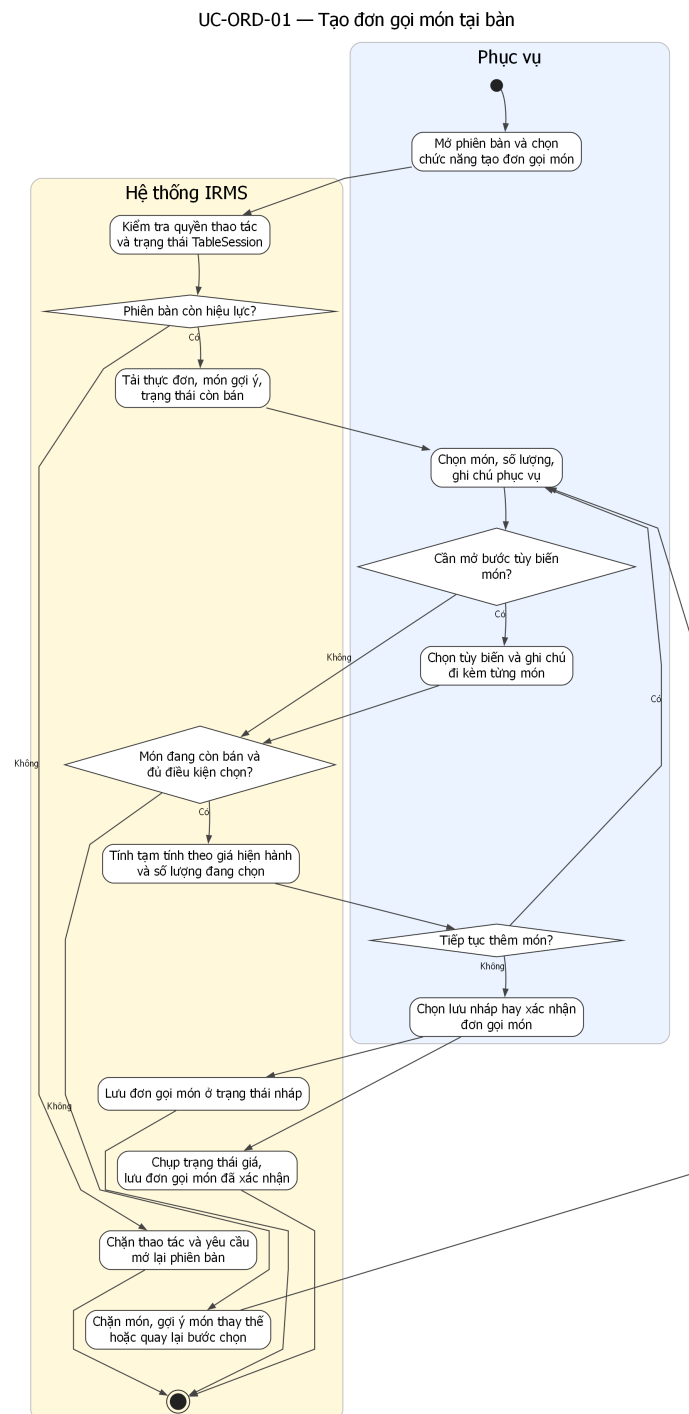
UC-RES-05 -- Gửi thông báo cho khách hàng



Hình 41: Sơ đồ hoạt động cho UC-RES-05 – Gửi thông báo cho khách hàng

Phân tích sơ đồ UC-RES-05. Sơ đồ thông báo khách hàng cho thấy đây không phải một thao tác phụ có thể bỏ qua, mà là một luồng nghiệp vụ có đầu vào, điều kiện và trạng thái riêng. Trong IRMS, việc thông báo đặt bàn thành công, nhắc giờ đến, gọi khách từ danh sách chờ hay xác nhận thay đổi không nên được mô tả như vài dòng tích hợp kỹ thuật ở rìa hệ thống. Trái lại, nó cần được hiểu là **hệ quả được kiểm soát của sự kiện nghiệp vụ**. Cách mô tả này giúp tách rõ phần nào thuộc trách nhiệm của lỗi nghiệp vụ, phần nào thuộc bộ kết nối gửi thông báo và phần nào cần cơ chế retry hoặc fallback.

4.3.2 Nhóm gọi món và điều phối bếp



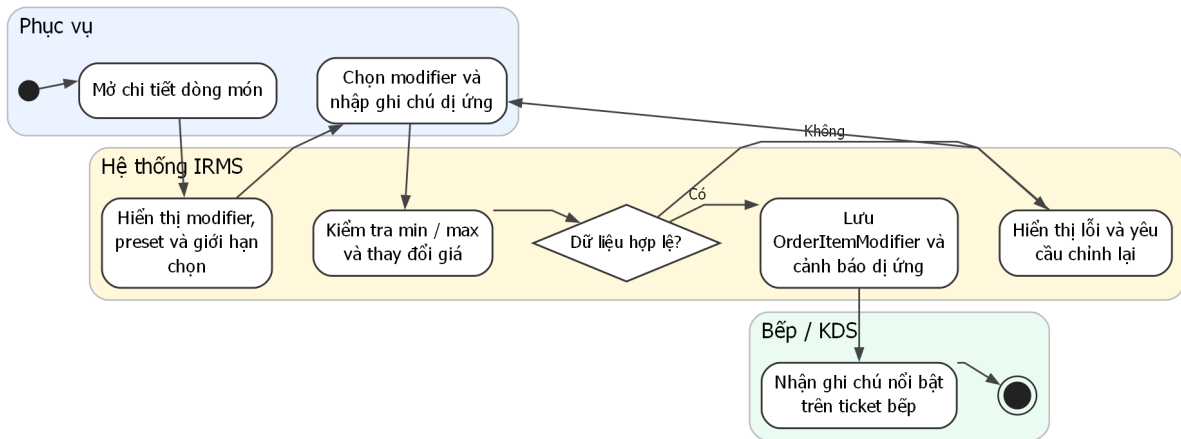
Hình 42: Sơ đồ hoạt động cho UC-ORD-01 – Tạo đơn gọi món tại bàn

Phân tích sơ đồ UC-ORD-01. Phần hoạt động này làm rõ các bước tải thực đơn, kiểm tra quyền thao tác trên phiên bản, chọn món, kiểm tra trạng thái còn bán, quay lại vòng chọn món nếu món vừa hết hàng, tính tạm tính và tách rõ hai nhánh “lưu nhập” và “xác nhận đơn”. Điều này quan trọng vì ca sử dụng gọi món không chỉ là thao tác nhập dữ liệu; nó còn là điểm chụp trạng thái giá và tính hợp lệ của món tại thời điểm nhân viên phục vụ thao tác.

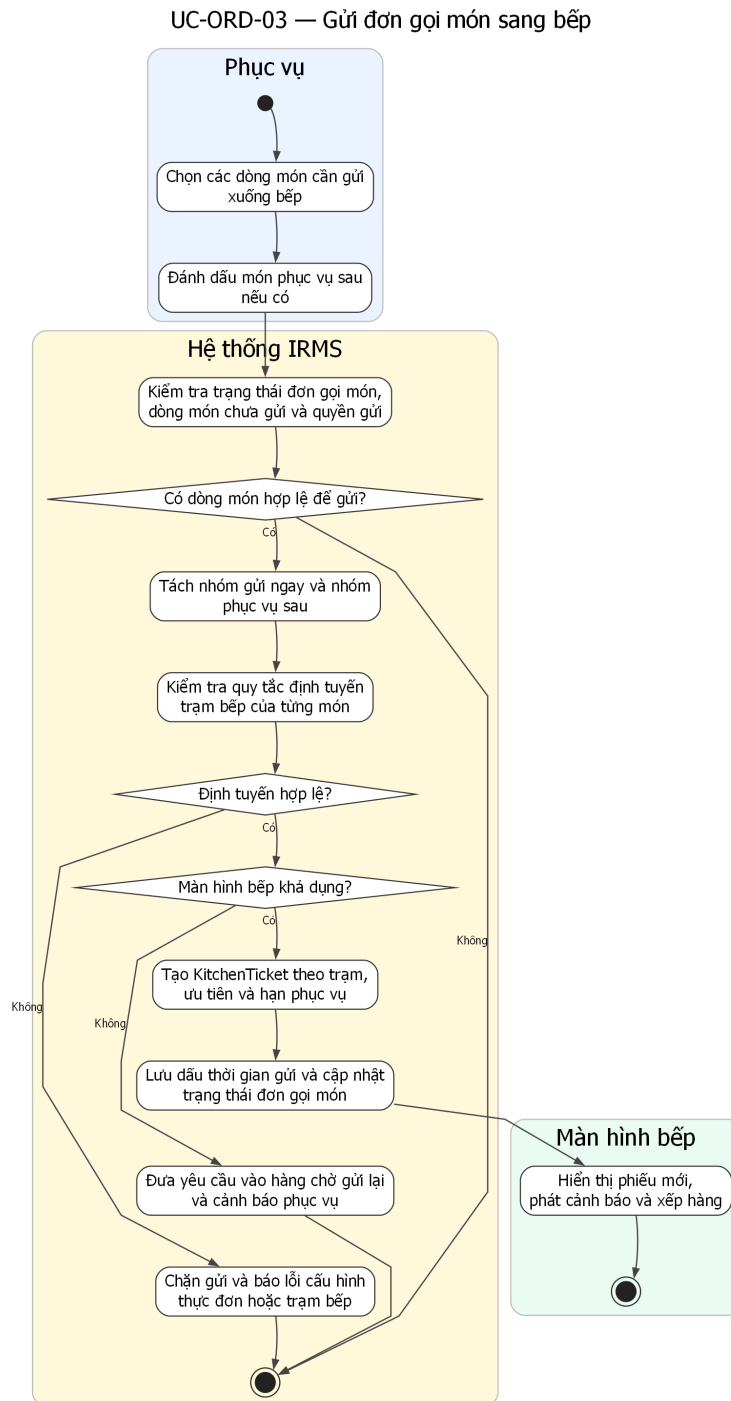
Sơ đồ cũng giữ đúng ranh giới với ca sử dụng gửi bếp. Ở đây, đơn gọi món chỉ được tạo và xác nhận ở mức nghiệp vụ gọi món; việc tạo phiếu bếp và điều phối sang trạm bếp được tách sang sơ đồ riêng, nhờ đó ranh giới

trách nhiệm giữa Ordering và Kitchen Coordination rõ ràng hơn.

UC-ORD-02 — Tùy biến món và ghi chú dị ứng



Hình 43: Sơ đồ hoạt động cho UC-ORD-02 – Tùy biến món và ghi chú dị ứng

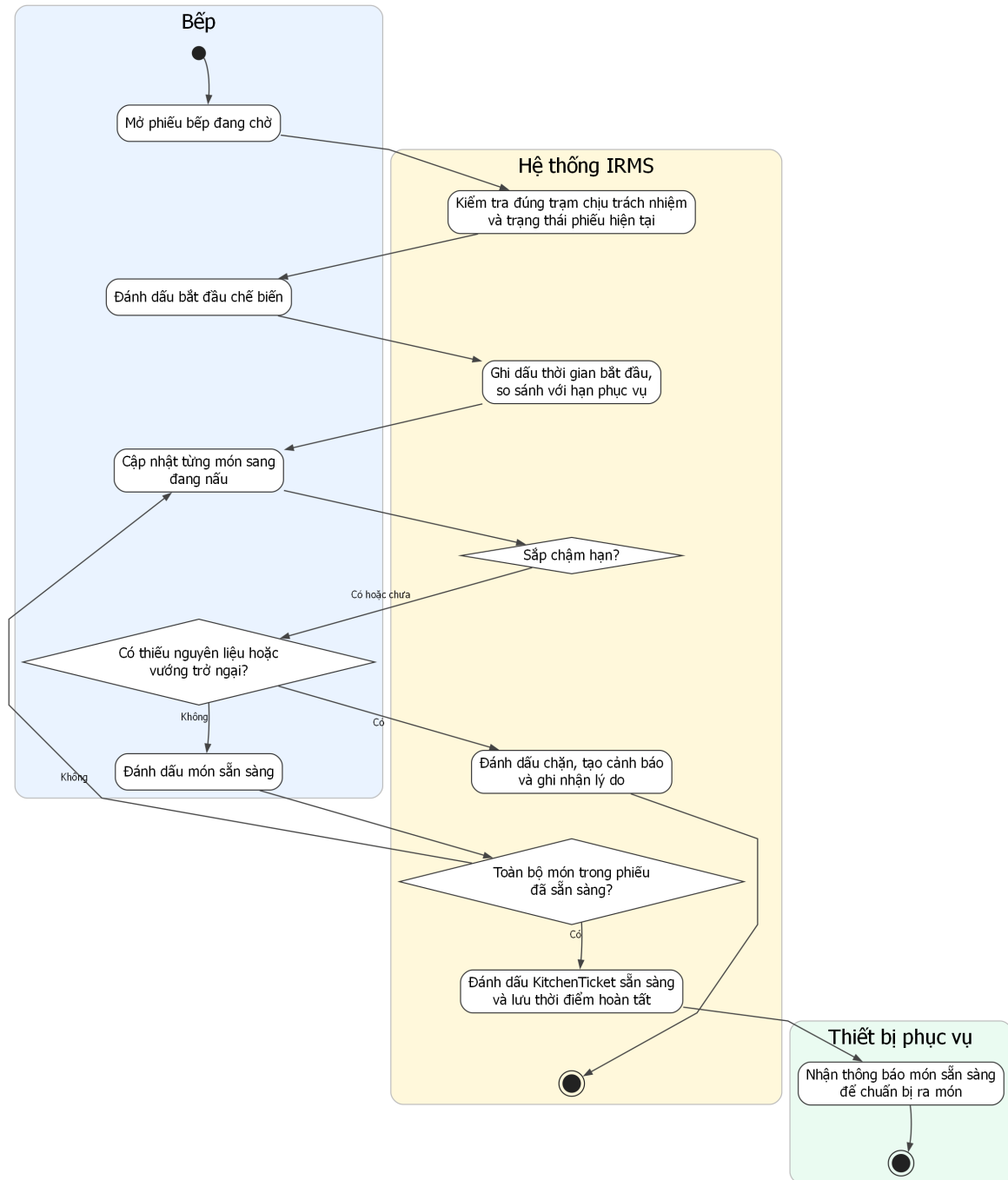


Hình 44: Sơ đồ hoạt động cho UC-ORD-03 – Gửi đơn gọi món sang bếp

Phân tích sơ đồ UC-ORD-03. Đây là một trong những sơ đồ hoạt động quan trọng nhất của báo cáo vì nó nằm đúng trên ranh giới giữa tuyển gọi món của phục vụ và tuyển điều phối của bếp. Luồng hoạt động này làm rõ các bước kiểm tra điều kiện gửi, tách riêng các dòng món gửi ngay và các dòng món phục vụ sau, kiểm tra quy tắc định tuyến theo trạm, tạo KitchenTicket, xếp hàng theo độ ưu tiên và cập nhật trạng thái đơn gọi món sau khi tạo phiếu.

Các nhánh ngoại lệ cũng được mô tả rõ ràng: nếu không xác định được trạm bếp hoặc màn hình bếp không khả dụng, hệ thống không được phép làm mất dữ liệu đã xác nhận mà phải chuyển sang hàng chờ gửi lại và phát cảnh báo cho phục vụ. Cách mô tả này phù hợp với yêu cầu phi chức năng về độ tin cậy và bảo toàn đơn đã xác nhận.

UC-KIT-01 — Cập nhật tiến độ chế biến

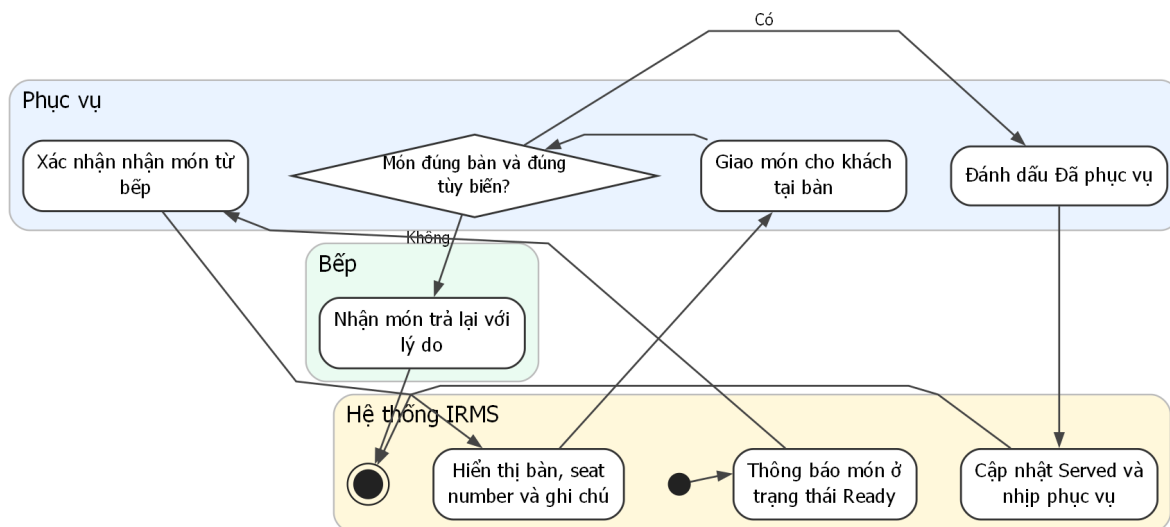


Hình 45: Sơ đồ hoạt động cho UC-KIT-01 – Cập nhật tiến độ chế biến

Phân tích sơ đồ UC-KIT-01. Luồng hoạt động ở đây bám đúng nhịp vận hành của bếp: nhận phiếu, kiểm tra trạm chịu trách nhiệm, đánh dấu bắt đầu, chuyển sang chế biến, ghi dấu thời gian cho từng lần đổi trạng thái, đánh giá nguy cơ chậm hạn và cuối cùng xác nhận món sẵn sàng. Mức chi tiết này giúp phân báo cáo không chỉ dừng ở mô tả “bếp cập nhật trạng thái”, mà còn cho thấy trạng thái nào cần được ghi nhận để phục vụ phân tích vận hành về sau.

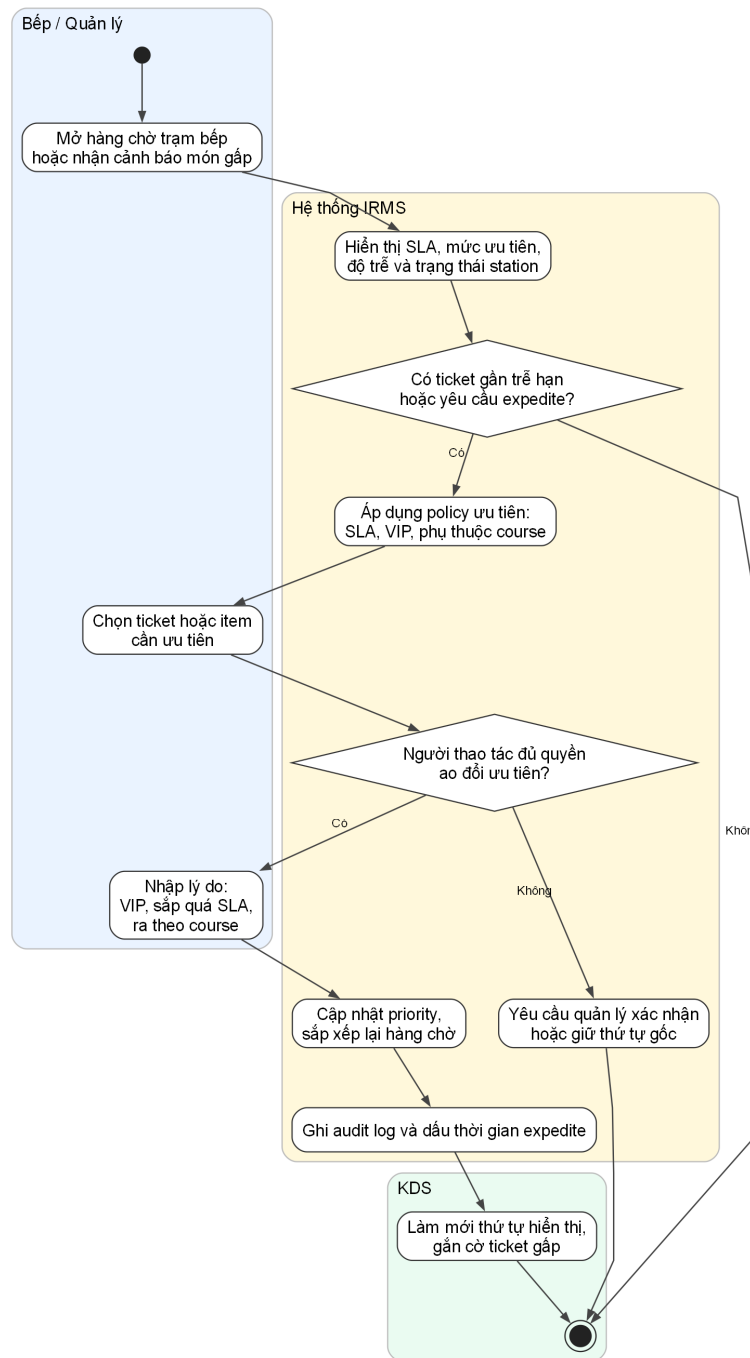
Phân nhánh “thiếu nguyên liệu hoặc bị chặn” cũng rất quan trọng. Trong trường hợp đó, hệ thống không được kết thúc im lặng mà phải đánh dấu chặn, phát cảnh báo và trả thông tin ngược về phía phục vụ. Điều này giữ cho luồng giao tiếp giữa bếp và khu vực phục vụ thống nhất với phần kiến trúc đã mô tả ở các góc nhìn trước.

UC-KIT-02 — Ra món cho bàn



Hình 46: Sơ đồ hoạt động cho UC-KIT-02 – Ra món cho bàn

UC-KIT-03 -- Ưu tiên hàng chờ bếp và xử lý món gấp

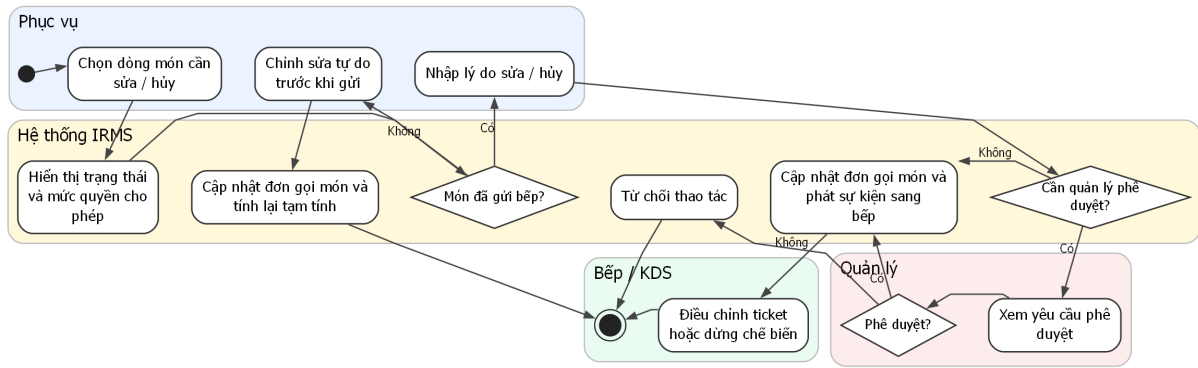


Hình 47: Sơ đồ hoạt động cho UC-KIT-03 – Ưu tiên hàng chờ bếp và xử lý món gấp

Phân tích sơ đồ UC-KIT-03. Luồng này làm rõ một điểm thường được nhắc đến trong tài liệu nhà hàng nhưng ít được mô hình hóa đầy đủ: **bếp không chỉ nhận phiếu bếp, mà còn phải điều phối thứ tự thực thi**. Trong giờ cao điểm, nếu mọi ticket đều bị xử lý đúng theo thứ tự đến mà không xét SLA, cấu trúc course, khách VIP hay món sắp trễ hạn, hệ thống KDS sẽ chỉ là một bảng hiển thị thụ động. Vì vậy, ca sử dụng ưu tiên hàng chờ bếp được trình bày như một luồng độc lập để phản ánh đúng yêu cầu “prioritize queues” của đề bài.

Về mặt kiến trúc, sơ đồ cũng cho thấy quyết định ưu tiên phải đi qua chính sách có kiểm soát, có ghi lý do và có giới hạn quyền, chứ không phải bất kỳ nhân viên nào cũng có thể tùy ý đổi thứ tự chế biến. Điều này giúp IRMS vừa hỗ trợ vận hành linh hoạt, vừa giữ được khả năng truy vết khi có khiếu nại về món ra chậm hoặc món bị đẩy lùi không hợp lý.

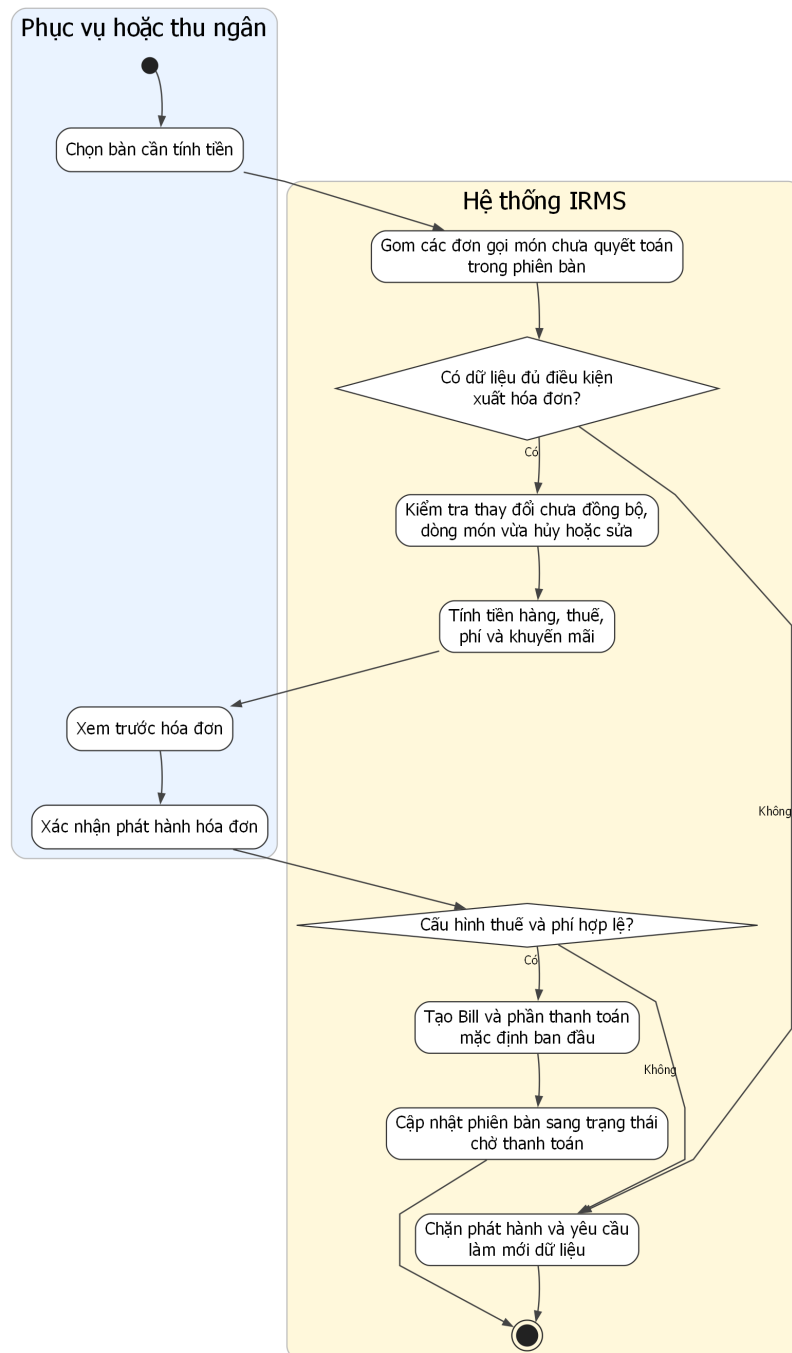
UC-ORD-04 — Sửa hoặc hủy dòng món



Hình 48: Sơ đồ hoạt động cho UC-ORD-04 – Sửa hoặc hủy dòng món

4.3.3 Nhóm hóa đơn, thanh toán và biên lai

UC-BIL-01 — Tạo hóa đơn cho bàn



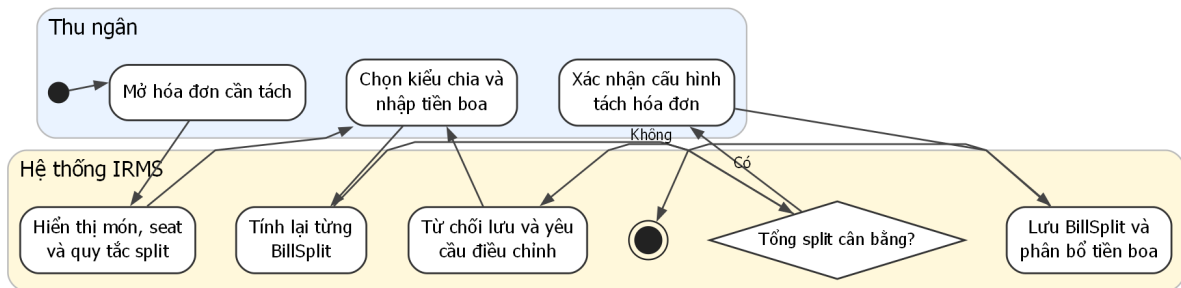
Hình 49: Sơ đồ hoạt động cho UC-BIL-01 – Tạo hóa đơn cho bàn

Phân tích sơ đồ UC-BIL-01. Sơ đồ tạo hóa đơn phản ánh đúng thứ tự xử lý của phần quyết toán: gom các đơn gọi món chưa quyết toán, kiểm tra điều kiện xuất hóa đơn, tính tiền hàng, thuế, phí và khuyến mãi, sau đó mới cho phép phục vụ hoặc thu ngân xem trước và xác nhận phát hành. Việc đặt bước “xem trước hóa đơn” trước “phát hành” là cần thiết vì đây là điểm cuối cùng để phát hiện sai lệch nếu một dòng món vừa bị hủy hoặc vừa có thay đổi cấu hình giá.

Sơ đồ cũng thể hiện rõ nhánh chặn phát hành khi dữ liệu chưa hợp lệ. Điều này phù hợp với đặc tả ca sử dụng ở trên và làm nổi bật rằng Bill không chỉ là phép cộng cuối cùng, mà là một thực thể nghiệp vụ có điều kiện

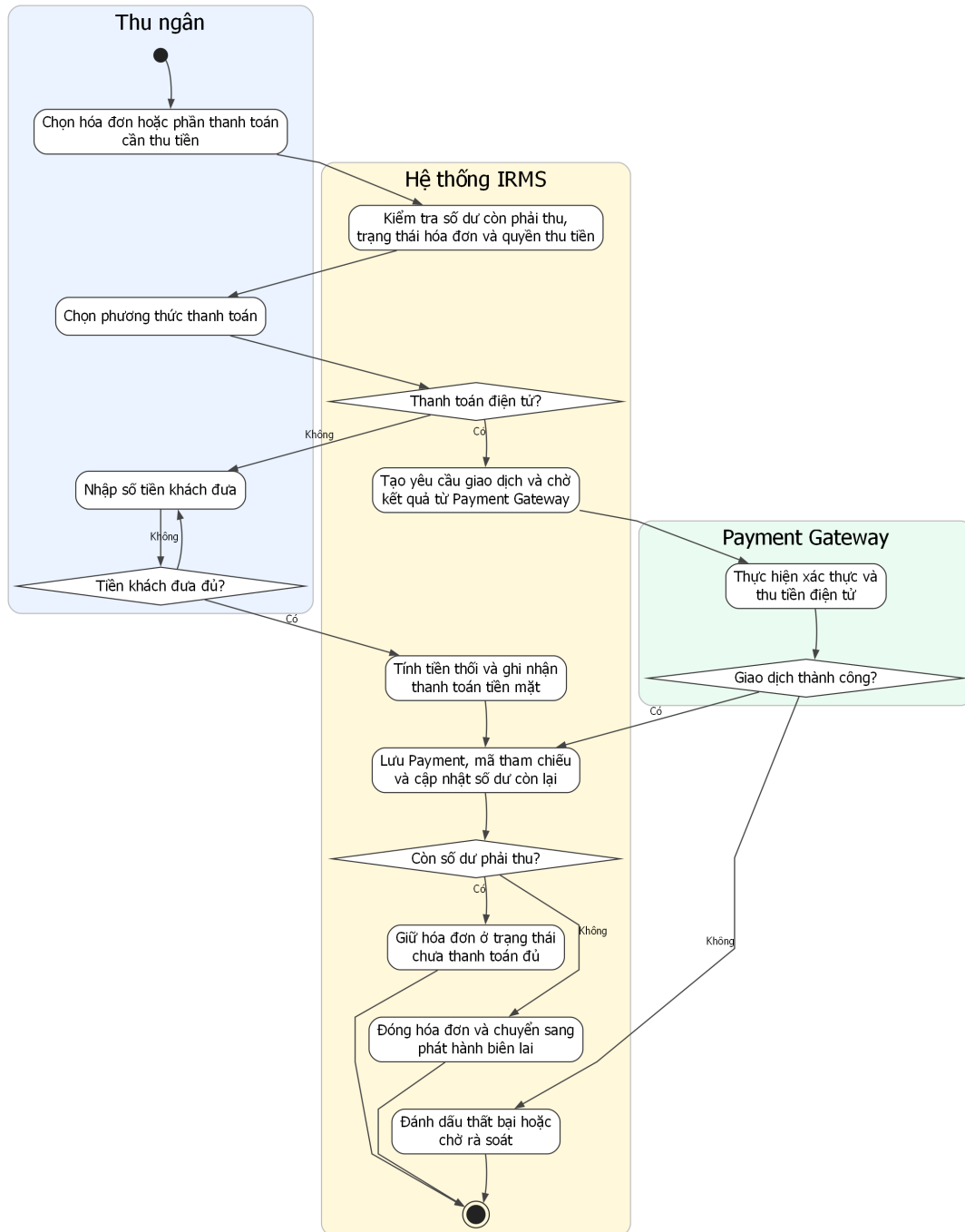
phát hành chặt chẽ.

UC-BIL-02 — Tách hóa đơn và thêm tiền boa



Hình 50: Sơ đồ hoạt động cho UC-BIL-02 – Tách hóa đơn và thêm tiền boa

UC-PAY-01 — Xử lý thanh toán

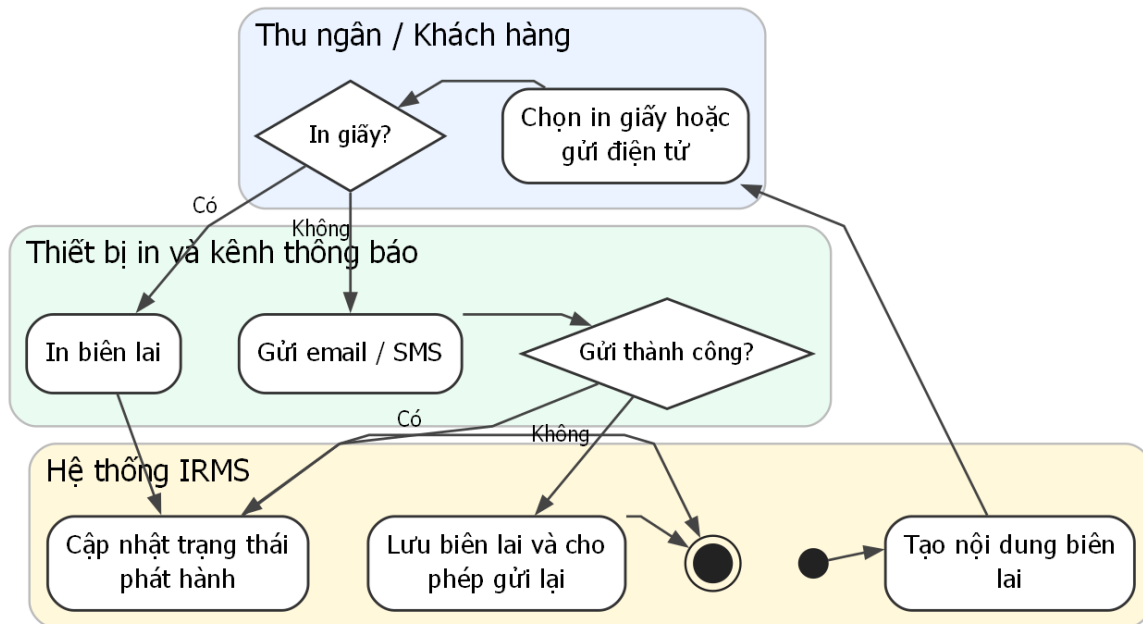


Hình 51: Sơ đồ hoạt động cho UC-PAY-01 – Xử lý thanh toán

Phân tích sơ đồ UC-PAY-01. Đây là sơ đồ hoạt động có ý nghĩa kiểm soát cao nhất trong toàn bộ phần hành vi. Luồng này làm rõ hai tuyến thanh toán khác nhau: thanh toán điện tử qua Payment Gateway và thanh toán tiền mặt tại quầy. Với thanh toán điện tử, hệ thống phải tạo yêu cầu giao dịch, gửi sang cổng thanh toán, nhận kết quả, lưu mã tham chiếu và xử lý riêng các trạng thái thành công, thất bại hoặc chờ rà soát. Với thanh toán tiền mặt, hệ thống tính số tiền thuế rồi mới cập nhật số dư còn lại.

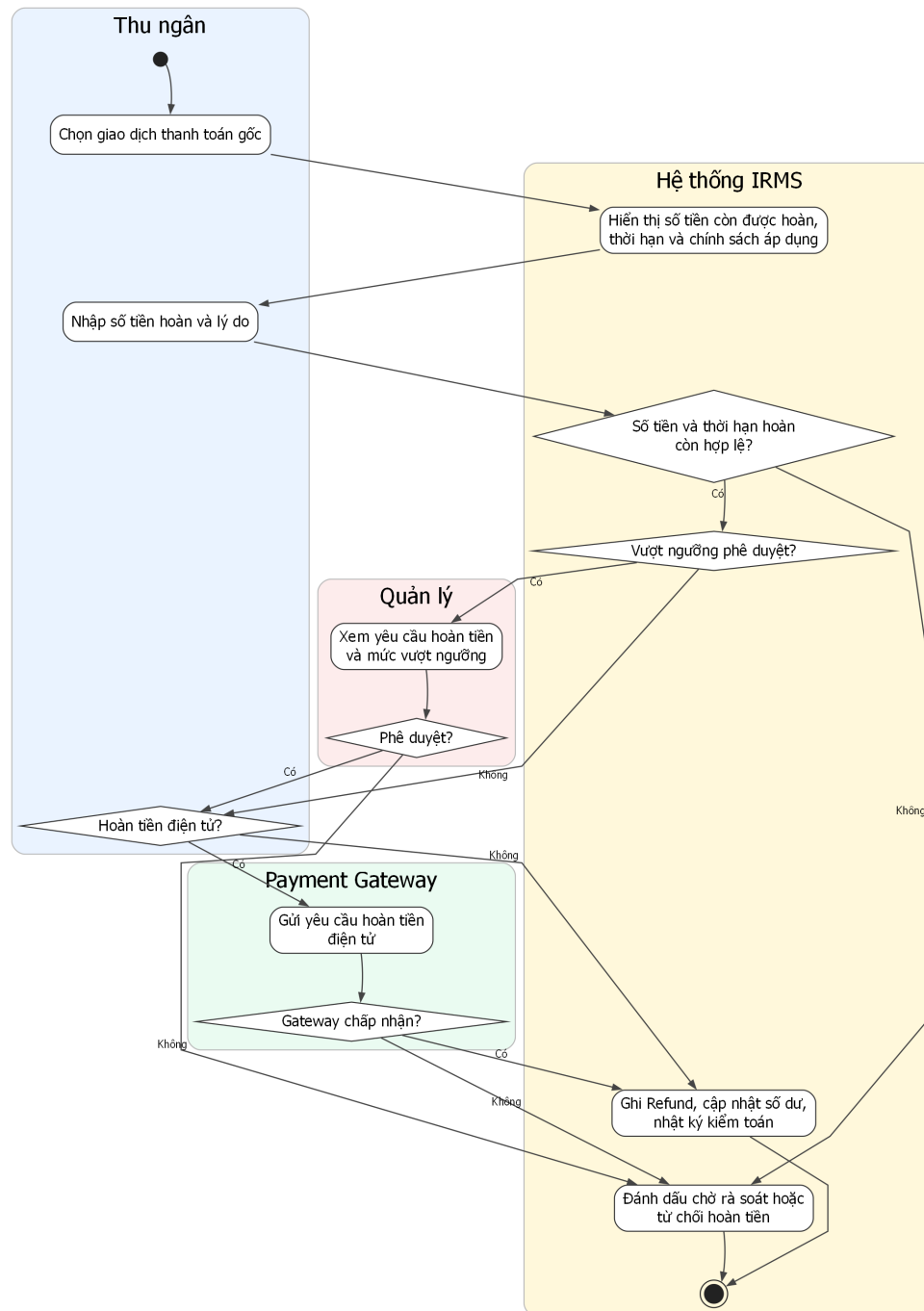
Sau bước ghi nhận thanh toán, hệ thống còn phải đánh giá hóa đơn đã được thanh toán đủ hay chưa. Nếu chưa đủ, hóa đơn vẫn mở; nếu đủ, hóa đơn được đóng và luồng phát hành biên lai mới được mở ra. Cách mô tả này bám đúng kiến trúc hiện tại của IRMS và tránh nhập nhằng giữa “đã có một giao dịch thanh toán” và “đã tắt toán hóa đơn”.

UC-PAY-02 — Phát hành biên lai



Hình 52: Sơ đồ hoạt động cho UC-PAY-02 – Phát hành biên lai

UC-PAY-03 — Hoàn tiền



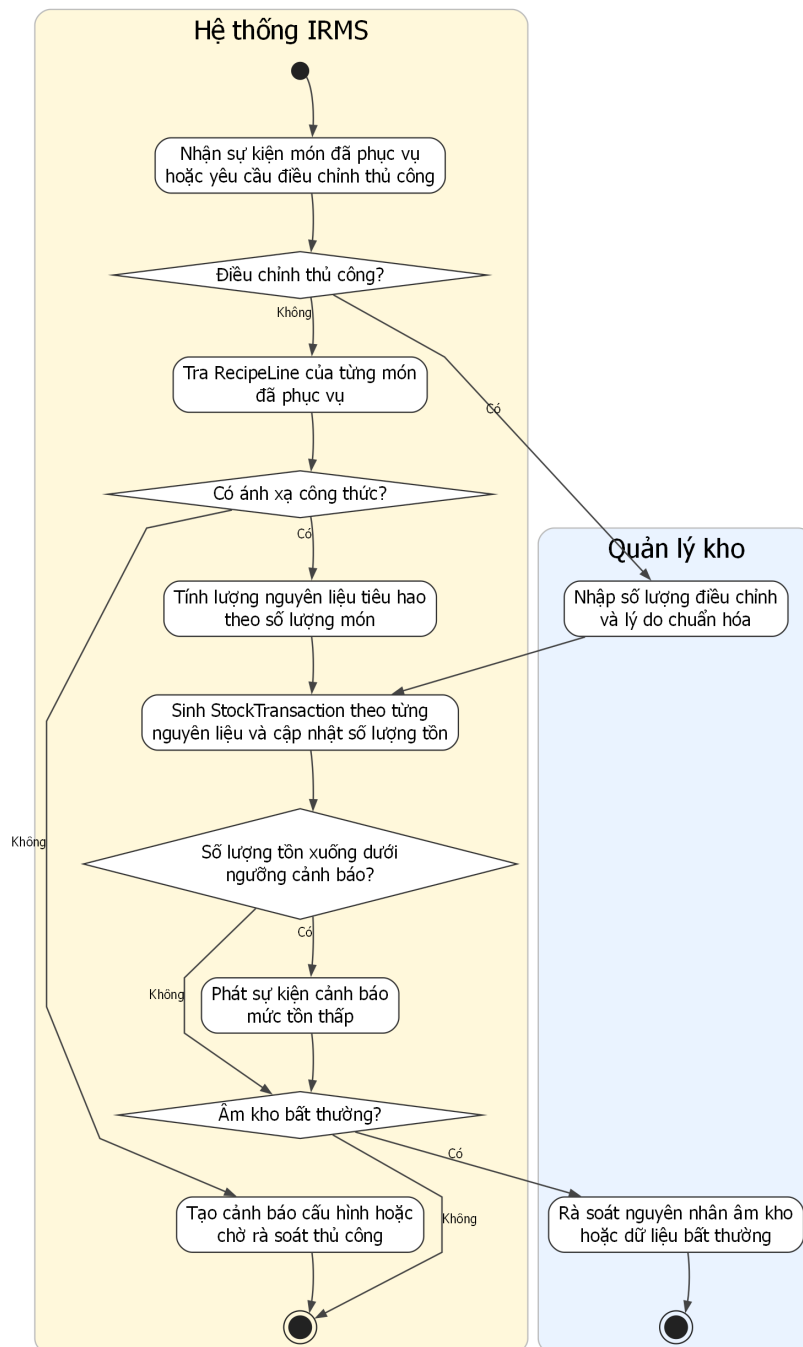
Hình 53: Sơ đồ hoạt động cho UC-PAY-03 – Hoàn tiền

Phân tích sơ đồ UC-PAY-03. Sơ đồ hoàn tiền hiện đã thể hiện đầy đủ hơn các điểm kiểm soát bắt buộc: chọn giao dịch gốc, kiểm tra số tiền còn được hoàn, nhập lý do, so ngưỡng phê duyệt, xin xác nhận của quản lý nếu cần, gửi yêu cầu hoàn tiền tới Payment Gateway hoặc ghi nhận hoàn tiền thủ công, rồi mới cập nhật sổ kiểm toán. Điều này rất quan trọng vì hoàn tiền là thao tác nhạy cảm nhất trong vùng thanh toán.

Một chi tiết đáng chú ý là sơ đồ đã tách rõ nhánh “chờ rà soát” khi cổng thanh toán không chấp nhận hoặc phản hồi không rõ ràng. Nhờ đó báo cáo có thể giải thích trạng thái nghiệp vụ chặt chẽ hơn, thay vì gom mọi trường hợp không thành công vào một nhãn lỗi duy nhất.

4.3.4 Nhóm tồn kho, cảnh báo và phân tích

UC-INV-01 — Cập nhật tiêu hao tồn kho sau phục vụ

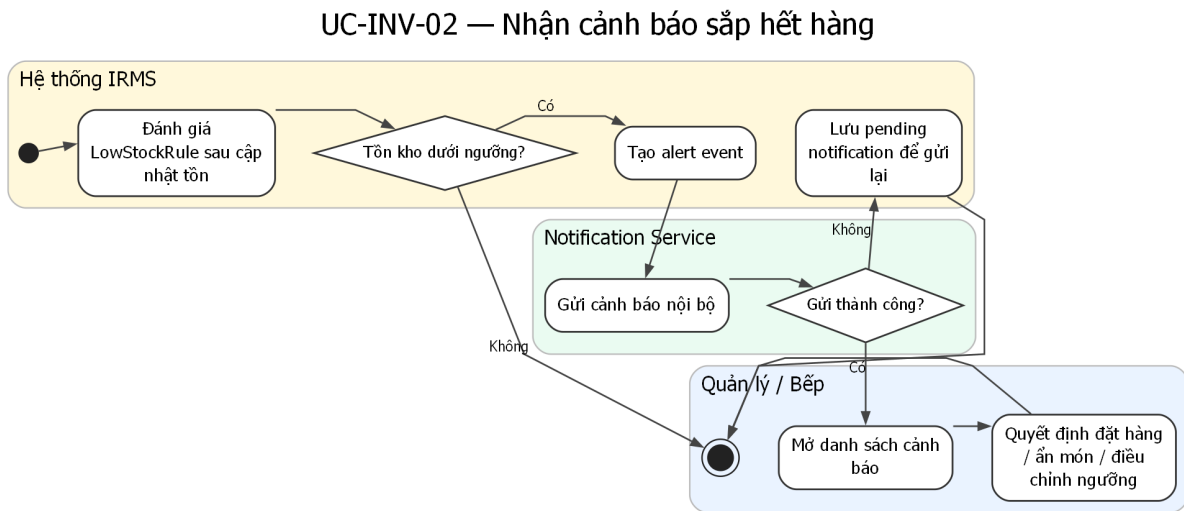


Hình 54: Sơ đồ hoạt động cho UC-INV-01 – Cập nhật tiêu hao tồn kho sau phục vụ

Phân tích sơ đồ UC-INV-01. Sơ đồ cập nhật tiêu hao tồn kho phản ánh đúng bản chất suy diễn của vùng tồn kho: nhận sự kiện từ món đã phục vụ hoặc điều chỉnh thủ công, tra công thức nguyên liệu, tính lượng tiêu hao theo số lượng món, sinh `StockTransaction` theo từng nguyên liệu, cập nhật số lượng tồn hiện có, rồi tiếp tục đánh giá các điều kiện bất thường. Đây là mô tả phù hợp với kiến trúc của IRMS, nơi vùng tồn kho không tự suy đoán món bán ra từ giao diện mà dựa vào tín hiệu nghiệp vụ từ tuyến phục vụ.

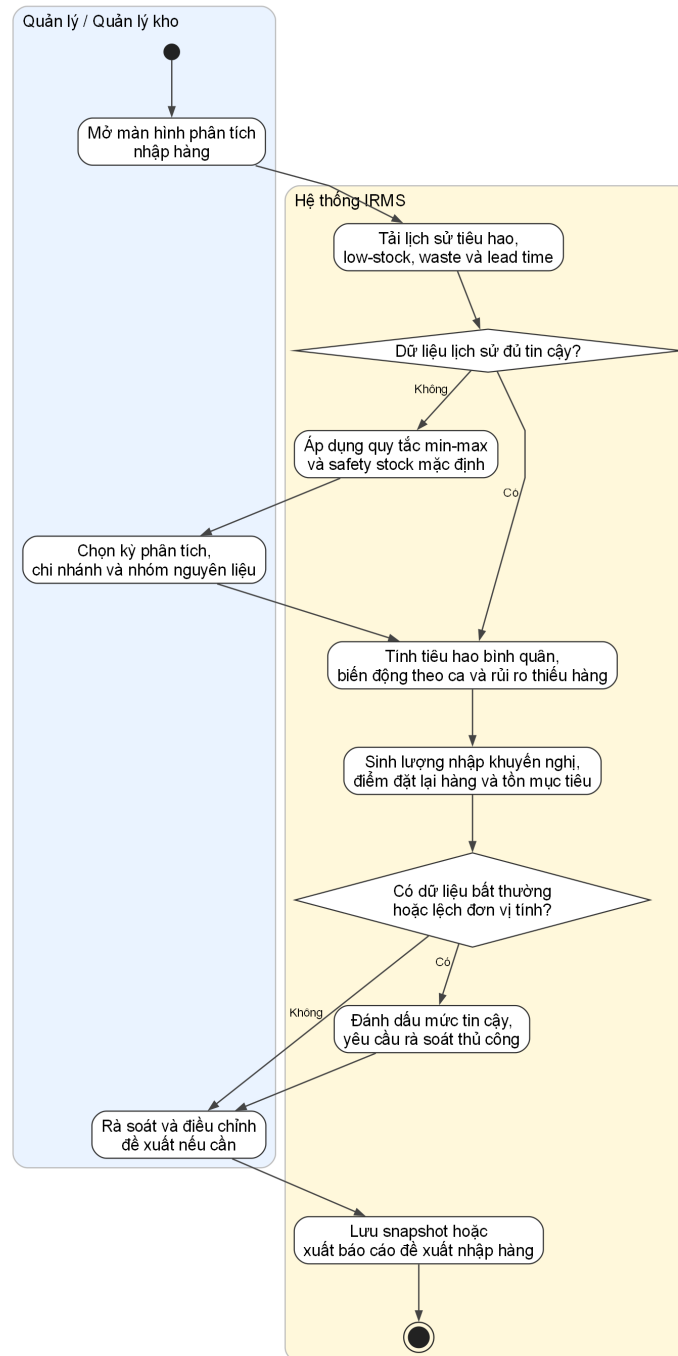
Sơ đồ cũng làm rõ hai nhánh ngoại lệ quan trọng: thiếu ánh xạ công thức và âm kho bất thường. Nhờ đó phần thiết kế hành vi bám sát hơn với ca sử dụng đã đặc tả, đồng thời nối được với sơ đồ cảnh báo sắp hết hàng ngay

bên dưới.



Hình 55: Sơ đồ hoạt động cho UC-INV-02 – Nhận cảnh báo sắp hết hàng

UC-INV-03 -- Phân tích tiêu hao và đề xuất nhập hàng

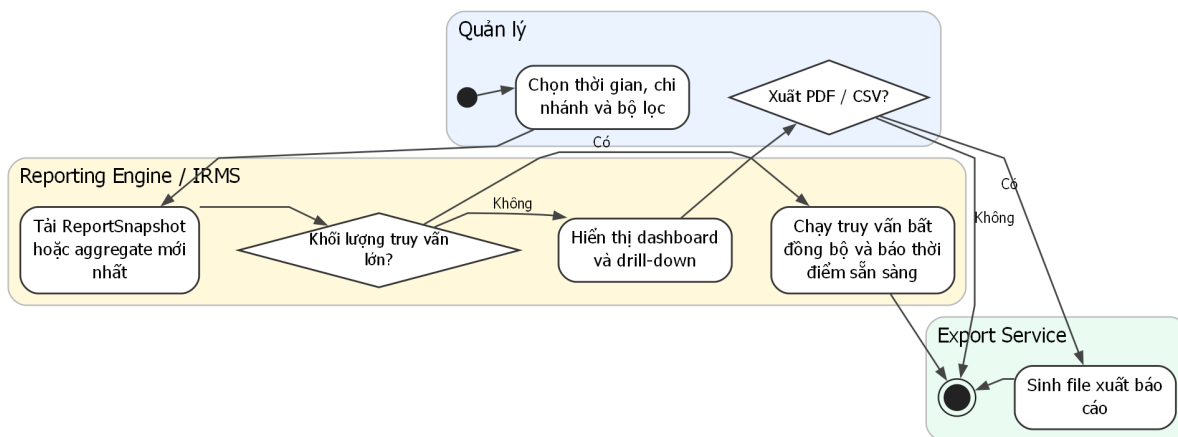


Hình 56: Sơ đồ hoạt động cho UC-INV-03 – Phân tích tiêu hao và đề xuất nhập hàng

Phân tích sơ đồ UC-INV-03. Ca sử dụng này làm cho phần tồn kho bám sát hơn với yêu cầu “historical usage helps predict reorder quantities and manage waste”. Nếu chỉ dừng ở cập nhật tiêu hao và cảnh báo sắp hết hàng, IRMS mới hỗ trợ tốt cho phản ứng vận hành ngắn hạn; nó chưa hỗ trợ đầy đủ cho quyết định chuẩn bị hàng hóa ở chu kỳ ngày, tuần hoặc mùa cao điểm. Vì vậy, sơ đồ chuyển trọng tâm của vùng tồn kho từ **theo dõi hiện trạng** sang **hỗ trợ ra quyết định**.

Điểm quan trọng của sơ đồ là không mô tả việc dự báo như một thuật toán mơ hồ, mà làm rõ chuỗi dữ liệu đầu vào: tiêu hao lịch sử, low-stock alert, hao hụt, lead time và mức tồn an toàn. Điều này có giá trị trong báo cáo kiến trúc vì nó cho thấy đề xuất nhập hàng là kết quả của mô hình dữ liệu và luồng xử lý đã có, không phải một chức năng được bổ sung rời rạc ở phần quản trị mà không có nền dữ liệu đủ tin cậy.

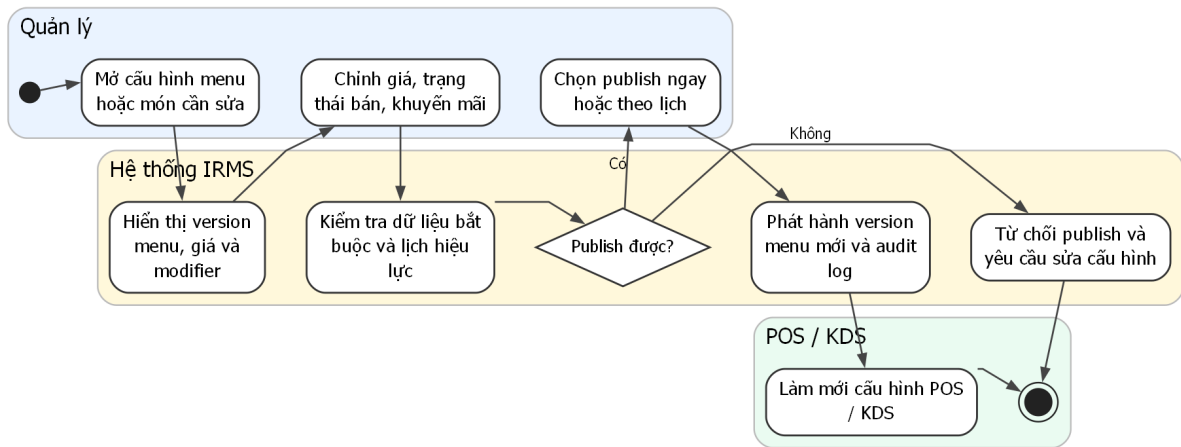
UC-REP-01 — Xem báo cáo doanh thu và vận hành



Hình 57: Sơ đồ hoạt động cho UC-REP-01 – Xem báo cáo doanh thu và vận hành

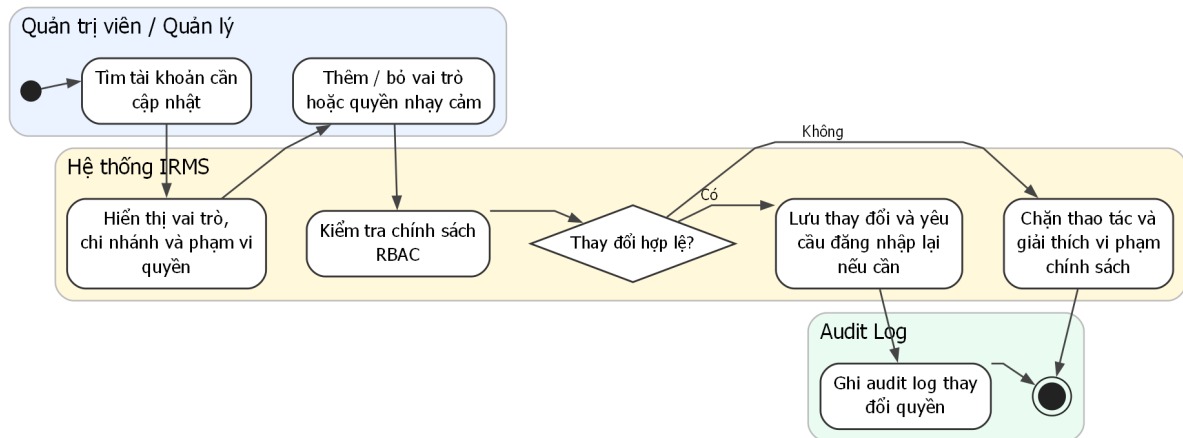
4.3.5 Nhóm quản trị cấu hình và vận hành

UC-ADM-01 — Quản lý menu và giá bán



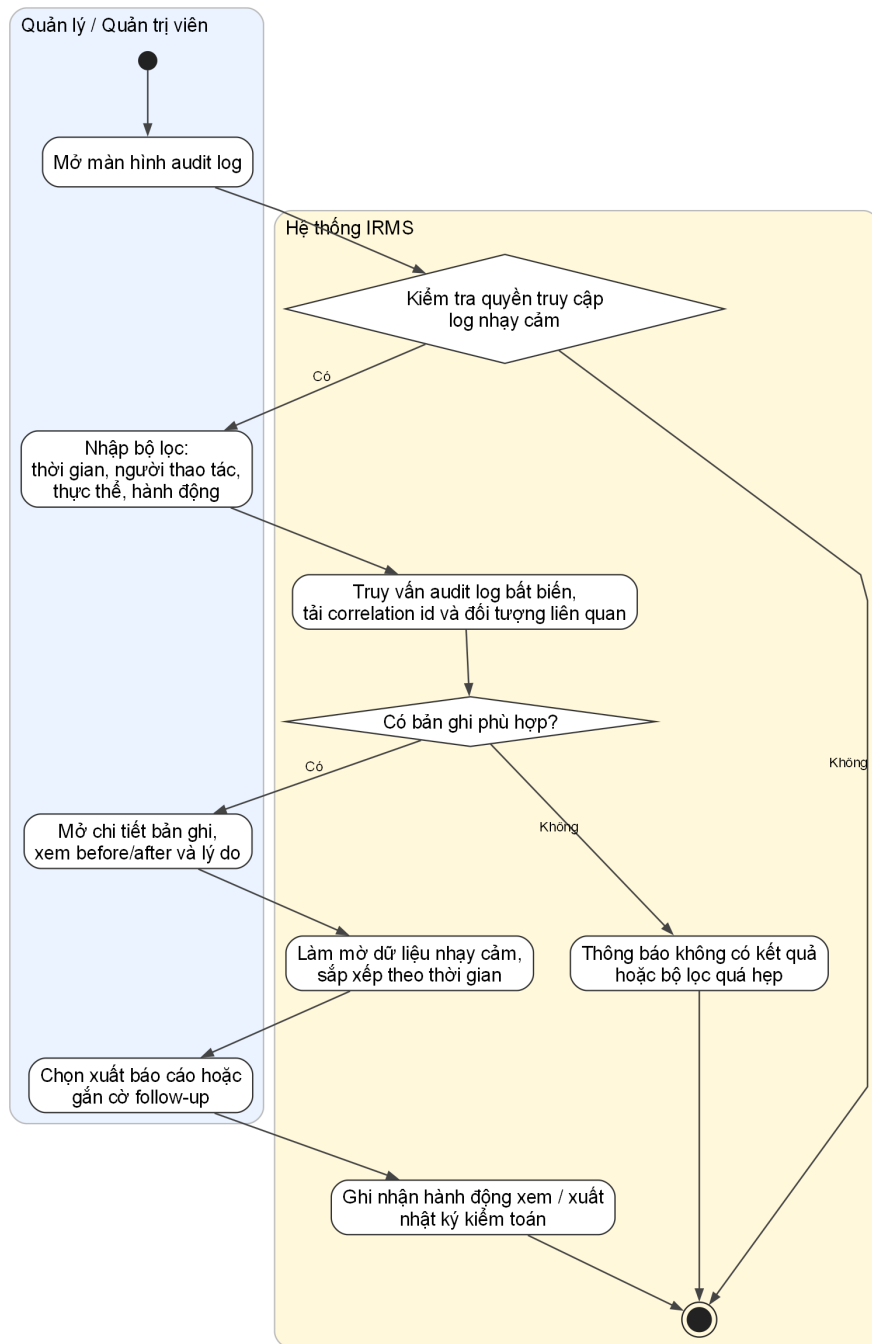
Hình 58: Sơ đồ hoạt động cho UC-ADM-01 – Quản lý thực đơn và giá bán

UC-ADM-02 — Quản lý vai trò và phân quyền



Hình 59: Sơ đồ hoạt động cho UC-ADM-02 – Quản lý vai trò và phân quyền

UC-ADM-03 -- Xem nhật ký kiểm toán và truy vết thao tác nhạy cảm

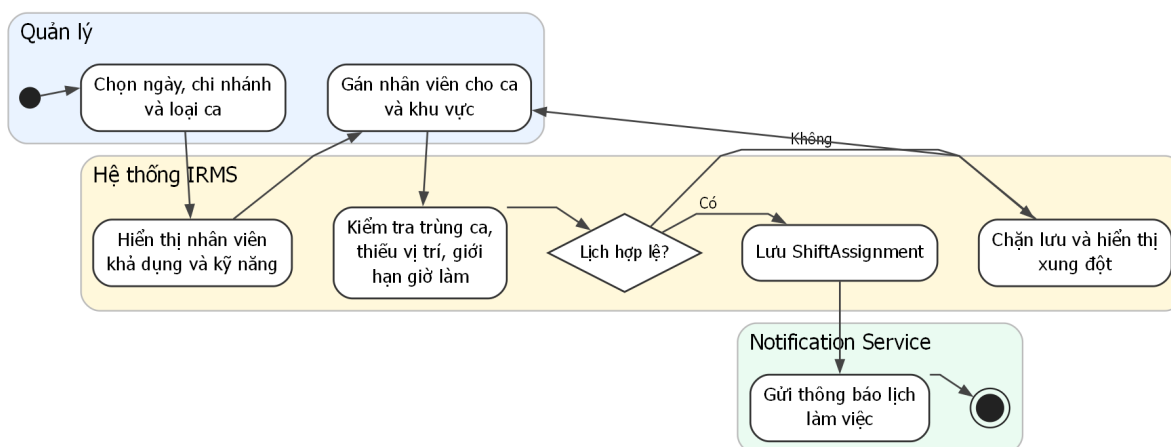


Hình 60: Sơ đồ hoạt động cho UC-ADM-03 – Xem nhật ký kiểm toán và truy vết thao tác nhạy cảm

Phân tích sơ đồ UC-ADM-03. Phần hoạt động này làm rõ rằng nhật ký kiểm toán không chỉ là nơi hệ thống “ghi lại cho có”, mà là một luồng sử dụng thực sự của khối quản trị. Khi có hoàn tiền bất thường, hủy món sau khi gửi bếp, ghi đề giá, mở lại hóa đơn hoặc thay đổi quyền, quản lý cần một ca sử dụng riêng để truy ngược chuỗi hành động, xem trước–sau, lý do và mã liên kết tới giao dịch gốc. Nếu thiếu ca sử dụng đó, phần audit trong báo cáo sẽ dễ bị hiểu như một thuộc tính kỹ thuật thay vì một khả năng kiểm soát vận hành.

Sơ đồ cũng nhấn mạnh rằng việc xem audit log nhạy cảm bản thân nó là một hành động cần kiểm soát. Đây là chi tiết quan trọng vì dữ liệu kiểm toán thường chạm vào thông tin nhân sự, quyết định tài chính và dấu vết thay đổi cấu hình. Do đó, mô hình hành vi không dừng ở bước “truy vấn log”, mà còn bổ sung bước kiểm tra quyền, làm mờ dữ liệu nhạy cảm và ghi nhận việc xem hoặc xuất log khi cần.

UC-OPS-01 — Quản lý ca làm nhân viên



Hình 61: Sơ đồ hoạt động cho UC-OPS-01 – Quản lý ca làm nhân viên

4.4 Phản tư về toàn bộ báo cáo và định hướng hiện thực

Điểm mạnh lớn nhất của báo cáo là đã duy trì được một đường chỉ đạo tương đối nhất quán từ bối cảnh nghiệp vụ, yêu cầu phi chức năng, ca sử dụng, kiến trúc, sơ đồ lớp cho tới phần hành vi. Tài liệu không đặt các sơ đồ như những hình minh họa rời rạc, mà dùng mỗi góc nhìn để làm rõ một lớp quyết định riêng của cùng một hệ thống. Nhờ vậy, người đọc có thể lần theo một logic liên tục: từ áp lực vận hành của nhà hàng, sang nhu cầu tách miền nghiệp vụ, sang cách tổ chức mô-đun, thành phần, dữ liệu và các cơ chế xử lý nền.

Điểm mạnh thứ hai nằm ở việc báo cáo đưa **ranh giới năng lực nghiệp vụ** vào vị trí trung tâm. Các phần đặt bàn, gọi món, bếp, quyết toán, tồn kho, báo cáo, phân quyền và kiểm toán đều được mô tả như những miền có trách nhiệm riêng thay vì bị hòa thành một khối xử lý chung. Nhờ đó, góc nhìn mô-đun, góc nhìn thành phần và góc nhìn lớp hỗ trợ lẫn nhau khá chặt chẽ. Khi xuất hiện mô-đun *Billing & Settlement* trong phần mô-đun, người đọc cũng thấy cụm lớp tương ứng trong phần lớp và thấy tuyến xử lý tương ứng trong phần thành phần. Sự nhất quán này làm cho lập luận kiến trúc có sức nặng hơn nhiều so với cách trình bày chỉ dừng ở danh sách chức năng.

Một kết quả tích cực khác là các nguyên lý **SOLID** không bị trình bày như một phần lý thuyết tách rời. **SRP** thể hiện ở việc mỗi mô-đun và mỗi lớp điều phối giữ một phạm vi trách nhiệm rõ ràng. **OCP** xuất hiện ở các điểm mở rộng như *PaymentGateway*, *NotificationSender*, *ReceiptPrinter* và các chính sách định giá, phân quyền. **LSP** được phản ánh qua yêu cầu mọi bộ kết nối cùng một vai trò phải giữ hợp đồng hành vi nhất quán. **ISP** thể hiện ở việc tránh tạo các giao diện lớn ôm mọi tích hợp ngoài. **DIP** là nguyên lý giúp giữ ổn định phần lõi khi các chi tiết công nghệ như cổng thanh toán, kênh thông báo hay cách lưu trữ thay đổi. Nhờ được gắn vào cấu trúc mô-đun, thành phần và lớp, **SOLID** ở đây mang ý nghĩa kiến trúc thực sự thay vì chỉ là phần diễn giải bổ sung.

Tuy nhiên, báo cáo cũng có những giới hạn cần được nhìn nhận thẳng thắn. Hệ thống được mô tả khá đầy đủ ở quy mô một nhà hàng hoặc một cụm nhỏ chi nhánh, nên các quyết định như modular monolith, mô hình đọc riêng cho báo cáo và bộ xử lý nền tách biệt là phù hợp. Dù vậy, khi quy mô tăng lên theo hướng nhiều chi nhánh, nhiều thiết bị, nhiều tích hợp thanh toán và nhiều chính sách giá khác nhau, kiến trúc sẽ cần tiến thêm một bước ở vùng triển khai, khả năng quan sát và chiến lược phân tách dữ liệu. Ngoài ra, một số phần như dự báo nhập hàng, điều phối bếp nâng cao hay xử lý sự cố tích hợp mới dừng ở mức kiến trúc và miền nghiệp vụ, chưa đi sâu tới mức thuật toán hoặc chính sách vận hành chi tiết.

Phần phản tư quan trọng nhất nằm ở kinh nghiệm tổ chức lập luận kiến trúc. Khi bám theo danh sách chức năng, tài liệu rất dễ dừng lại ở việc hệ thống có những gì. Khi chuyển trọng tâm sang giao dịch lõi, hệ quả phụ, biên mô-đun, điểm mở rộng và ranh giới dữ liệu, báo cáo mới bắt đầu thể hiện đúng bản chất của một tài liệu kiến trúc. Từ đó có thể rút ra một nguyên tắc rõ ràng: chất lượng báo cáo không nằm ở số lượng sơ đồ, mà nằm ở việc mỗi sơ đồ có bảo vệ được một quyết định thiết kế cụ thể hay không.

Giá trị lâu dài của tài liệu này là tạo ra một khung quyết định có thể tiếp tục dùng trong giai đoạn hiện thực. Những ranh giới như Reservation & Seating, Ordering, Kitchen Coordination, Billing & Settlement, Inventory, Reporting và IAM / Audit đã được neo tương đối rõ; vì vậy, phần mã nguồn về sau có thể dùng chính các ranh giới đó làm chuẩn để tự kiểm tra chất lượng tổ chức mã. Nếu giữ được vai trò này, báo cáo không chỉ là tài liệu trình bày, mà còn là bản thiết kế có khả năng dẫn hướng cho việc phát triển hệ thống.

A Phụ lục

A.1 Lý do lựa chọn kiến trúc và hồ sơ quyết định kiến trúc

Phụ lục này tổng hợp phần biện minh cho lựa chọn kiến trúc của IRMS dưới dạng **so sánh phương án** và **hồ sơ quyết định kiến trúc** (*Architecture Decision Records – ADR*). Mục tiêu của phần này không phải chỉ để nêu kiến trúc nào được chọn, mà còn để làm rõ **bối cảnh ra quyết định**, **các phương án đã cân nhắc**, **lập luận loại trừ** và **hệ quả thiết kế** của phương án cuối cùng. Cách trình bày này giúp phụ lục đóng vai trò như một hồ sơ giải thích quyết định, thay vì chỉ là phần bổ sung hình thức cho báo cáo chính.

Các mục trong phụ lục này không đứng riêng với thân báo cáo. Chúng được dùng để giải thích ngược cho các phần mô hình hóa ở Mục 2.1, phần tổng quan và các góc nhìn kiến trúc ở Mục 3.1, Mục 3.3, cùng phần thiết kế chi tiết ở Mục 4.1 và Mục 4.2.

A.1.1 So sánh các kiểu kiến trúc ứng viên

Bảng 19: So sánh các kiểu kiến trúc ứng viên cho IRMS

Kiểu kiến trúc	Ưu điểm chính	Hạn chế trong bối cảnh IRMS
Layered Monolith thuần	Dễ khởi tạo ban đầu, ít thành phần triển khai, đơn giản cho giao dịch nội bộ, thuận lợi nếu phạm vi hệ thống còn nhỏ.	Dễ trượt thành một khối lớn tổ chức theo tầng kỹ thuật thay vì theo năng lực nghiệp vụ. Trong bối cảnh IRMS, các vùng như gọi món, bếp, thanh toán, tồn kho và báo cáo rất dễ chồng lấn trách nhiệm nếu chỉ bám vào cấu trúc <i>controller-service-repository</i> . Điều này làm giảm khả năng kiểm soát ranh giới mô-đun và gây khó khăn cho bảo trì về sau.
Service-Based phân tán	Cho phép tách các năng lực thành các đơn vị triển khai độc lập hơn; thuận lợi khi cần mở rộng riêng một số chức năng hoặc tích hợp nhiều thành phần ngoài.	Làm tăng số lượng lời gọi qua mạng, nhu cầu truy vết liên dịch vụ, thử lại, quản lý hợp đồng dữ liệu và chi phí phối hợp triển khai. Với phạm vi của IRMS trong học phần, nhiều năng lực vẫn cần giao dịch gần nhau và cần phản hồi nhanh, nên phương án này tạo thêm độ phức tạp mà chưa mang lại lợi ích tương xứng.
Microservices	Cô lập mạnh về triển khai, thuận lợi khi mỗi năng lực có vòng đời phát hành riêng, đội phụ trách riêng và yêu cầu mở rộng độc lập rõ rệt.	Là phương án có chi phí vận hành cao nhất. Nó kéo theo các vấn đề khó như giao dịch phân tán, quan sát hệ thống, quản trị hợp đồng dịch vụ, điều phối lỗi và nhất quán dữ liệu. Mức độ phức tạp này không tương xứng với quy mô của một IRMS cỡ vừa trong bối cảnh đồ án học phần.
Event-Driven thuần	Phù hợp với các luồng phát tán sự kiện, xử lý nền, retry, mô hình chiếu và các hệ quả phụ như thông báo, tồn kho hoặc báo cáo.	Không phù hợp để áp dụng cho toàn bộ hệ thống. Nhiều tương tác lỗi của IRMS như tra cứu thực đơn, tạo đơn gọi món, lập hóa đơn và xử lý thanh toán đòi hỏi phản hồi tức thời, trạng thái rõ ràng và ngữ nghĩa lỗi minh bạch. Nếu dùng thuần sự kiện cho toàn hệ thống, tuyến đầu dễ trở nên khó giải thích và khó kiểm soát hơn.
Kiến trúc lai: Modular Monolith theo năng lực + phân lớp nội bộ + luồng hỗ trợ hướng sự kiện	Giữ được ranh giới năng lực nghiệp vụ rõ ràng; đơn giản hơn cho giao dịch lỗi; dễ triển khai, kiểm thử và tích hợp trong phạm vi học phần; đồng thời vẫn hỗ trợ tốt các hệ quả phụ qua sự kiện và bộ xử lý nền.	Đòi hỏi kỷ luật tổ chức mã nguồn để không làm mờ ranh giới mô-đun bên trong. Ngoài ra, khi quy mô tăng lên, một số năng lực có thể cần được tách dần thành đơn vị triển khai độc lập. Tuy nhiên, đây là hạn chế có thể chấp nhận được ở giai đoạn hiện tại.

Từ các góc nhìn mô-đun, thành phần và triển khai đã trình bày trong báo cáo chính, phương án phù hợp nhất cho IRMS không phải là một **kiến trúc dịch vụ thuần** theo nghĩa mỗi năng lực nghiệp vụ là một tiến trình mạng độc lập, mà là một **kiến trúc lai**. Ở mức logic, hệ thống được phân rã rõ theo các năng lực như đặt bàn và bàn ăn, thực đơn, gọi món, điều phối bếp, quyết toán, tồn kho, báo cáo, IAM và kiểm toán. Tuy nhiên, ở mức thực thi hiện tại, các năng lực này vẫn được đặt trong một **backend tập trung** để giữ cho giao dịch lỗi đơn giản hơn, giảm chi phí triển khai và phù hợp với phạm vi của đồ án.

Vì vậy, cách mô tả chính xác hơn là: IRMS được tổ chức như một **modular monolith theo năng lực nghiệp vụ**; bên trong từng mô-đun áp dụng **phân lớp nội bộ** để tách phần giao diện, điều phối ứng dụng, mô hình miền và truy cập dữ liệu; đồng thời hệ thống sử dụng **luồng hỗ trợ hướng sự kiện** cho những xử lý không cần phản hồi đồng bộ ngay lập tức, chẳng hạn như gửi thông báo, làm mới mô hình đọc, đồng bộ báo cáo, và một phần xử lý

tồn kho hoặc cảnh báo. Cách kết hợp này cho phép IRMS giữ được sự rõ ràng về kiến trúc mà không phải gánh toàn bộ độ phức tạp của một hệ phân tán nặng.

ADR-01: Lựa chọn kiến trúc lại cho IRMS

1. **Bối cảnh:** IRMS phải đồng thời hỗ trợ nhiều tuyến nghiệp vụ có tính chất rất khác nhau: đặt bàn và xếp bàn, gọi món tại bàn, điều phối bếp, lập hóa đơn, thanh toán, theo dõi tồn kho, báo cáo, quản lý quyền và kiểm toán. Kiến trúc được chọn cần giữ được ranh giới nghiệp vụ đủ rõ để dễ bảo trì, nhưng cũng phải khả thi trong phạm vi một đồ án học phần, nơi chi phí triển khai và vận hành cần được kiểm soát.
2. **Quyết định:** Chọn kiến trúc lại gồm:
 - **modular monolith theo năng lực** cho phần lõi triển khai;
 - **phân lớp nội bộ** bên trong từng mô-đun;
 - **luồng hỗ trợ hướng sự kiện** cho thông báo, cập nhật mô hình đọc, một phần tồn kho và báo cáo;
 - **luồng đồng bộ** cho các tương tác nghiệp vụ tuyến đầu như gọi món, lập hóa đơn và thanh toán.
3. **Trạng thái:** Đã phê duyệt.
4. **Lý do lựa chọn:** Phương án này cân bằng được hai yêu cầu quan trọng. Thứ nhất, nó giữ cho các giao dịch lõi đơn giản hơn so với kiến trúc phân tán, nhờ đó giảm rủi ro giao dịch phân tán, giảm độ phức tạp vận hành và tăng tính khả thi của phần hiện thực. Thứ hai, nó vẫn duy trì được ranh giới mô-đun rõ ràng theo năng lực nghiệp vụ, từ đó hỗ trợ bảo trì, kiểm thử và mở rộng có kiểm soát.
5. **Hệ quả kiến trúc:** Ở mức logic, các năng lực được mô tả như các mô-đun độc lập, có ranh giới trách nhiệm và dữ liệu rõ ràng. Ở mức triển khai hiện tại, chúng có thể cùng chạy trong một backend runtime. Nếu tải hệ thống hoặc độ phức tạp vận hành tăng trong tương lai, các năng lực như Reporting, Notification hoặc một phần Inventory là các ứng viên phù hợp để tách ra trước.
6. **Các phương án không được chọn:** Layered monolith thuần bị loại vì không giữ được ranh giới năng lực đủ rõ; microservices bị loại vì vượt quá nhu cầu và chi phí của bối cảnh hiện tại; event-driven thuần bị loại vì không phù hợp với các giao dịch cần phản hồi ngay.

ADR-02: Sử dụng API Gateway làm điểm vào thống nhất

1. **Bối cảnh:** IRMS phục vụ nhiều loại ứng dụng khách như máy tính bảng POS, màn hình KDS, thiết bị thu ngân và ứng dụng web cho quản lý. Nếu mỗi ứng dụng khách gọi trực tiếp vào nhiều thành phần phía sau, việc kiểm soát xác thực, định tuyến, giới hạn tốc độ, ghi nhật ký ở biên và quản lý phiên bản API sẽ bị phân tán.
2. **Quyết định:** Đặt API Gateway làm điểm vào thống nhất cho mọi ứng dụng khách. Gateway chịu trách nhiệm xác thực, chuyển tiếp yêu cầu, giới hạn tốc độ, ghi log ở biên, chuẩn hóa lỗi phản hồi và áp dụng các chính sách chung trước khi chuyển tiếp vào backend.
3. **Trạng thái:** Đã phê duyệt.
4. **Lý do lựa chọn:** Cách tổ chức này làm giảm sự phụ thuộc trực tiếp giữa ứng dụng khách và cấu trúc nội bộ của backend. Nó cũng tạo một điểm kiểm soát tập trung cho bảo mật, kiểm toán, quản lý phiên bản giao diện lập trình ứng dụng và quan sát hệ thống.
5. **Hệ quả kiến trúc:** Ứng dụng khách không cần biết chi tiết nội bộ của từng mô-đun hoặc từng luồng xử lý. Mọi chính sách chung được chuẩn hóa ở một điểm vào thống nhất. Tuy nhiên, gateway không thay thế logic nghiệp vụ của các mô-đun phía sau; nó chỉ đóng vai trò lớp biên kiểm soát và điều phối truy cập.
6. **Lưu ý triển khai:** Trong bối cảnh modular monolith, gateway không hàm ý rằng toàn bộ hệ thống đã là microservices. Nó được hiểu là **điểm vào thống nhất ở biên hệ thống**, giúp kiến trúc rõ ràng hơn ngay cả khi phần lõi vẫn đang chạy tập trung.

ADR-03: Kết hợp REST đồng bộ với sự kiện miền bất đồng bộ

1. **Bối cảnh:** IRMS có hai loại luồng xử lý khác nhau. Một số thao tác cần phản hồi ngay cho người dùng, ví dụ tra cứu thực đơn, tạo đơn gọi món, lập hóa đơn hoặc ghi nhận thanh toán. Trong khi đó, một số hệ quả phụ phù hợp hơn với xử lý nền hoặc xử lý phát tán, chẳng hạn gửi phiếu sang bếp, cập nhật tồn kho, làm mới báo cáo và gửi cảnh báo.
2. **Quyết định:** Dùng **REST đồng bộ** cho các lệnh và truy vấn ở ranh giới người dùng, đồng thời dùng **sự kiện miền bất đồng bộ** cho các tác vụ phát tán hoặc chỉ cần nhất quán sau cùng.
3. **Trạng thái:** Đã phê duyệt.
4. **Lý do lựa chọn:** Cách kết hợp này giữ được trải nghiệm tương tác nhanh, trạng thái lỗi rõ ràng và khả năng kiểm soát giao dịch tốt ở tuyến đầu. Đồng thời, nó giúp tách rời những luồng nền có thể retry, xử lý lại, mở rộng hoặc nối thêm bộ xử lý mới mà không làm nặng tuyến giao dịch chính.
5. **Hệ quả kiến trúc:** Mô hình xử lý của IRMS không thuần đồng bộ và cũng không thuần sự kiện. Thay vào đó, hệ thống dùng đồng bộ ở nơi người dùng cần phản hồi tức thời, và dùng bất đồng bộ ở nơi ưu tiên khả năng phát tán, giảm ghép nối và chịu lỗi tốt hơn. Điều này kéo theo nhu cầu quản lý idempotency, retry, theo dõi trạng thái xử lý nền và quan sát luồng sự kiện.
6. **Ví dụ áp dụng trong IRMS:**
 - CreateOrder, CreateBill, ProcessPayment đi theo tuyến đồng bộ.
 - OrderConfirmed, DishReady, DishServed, LowStockDetected, ReportRefreshRequested là các ví dụ phù hợp với tuyến bất đồng bộ.

ADR-04: Tách dữ liệu theo miền nghiệp vụ và dùng read model cho báo cáo

1. **Bối cảnh:** Nếu toàn bộ các vùng như đặt bàn, gọi món, bếp, quyết toán, tồn kho và báo cáo cùng đọc–ghi trực tiếp trên một lược đồ dùng chung, hệ thống sẽ khó kiểm soát khóa, khó tiến hóa cấu trúc dữ liệu và dễ làm cho truy vấn báo cáo tác động ngược lại tuyến giao dịch nóng.
2. **Quyết định:** Mỗi mô-đun sở hữu dữ liệu giao dịch theo miền tương ứng. Phần báo cáo không đọc trực tiếp đường ghi nghiệp vụ, mà sử dụng **read model** hoặc **snapshot** được cập nhật từ sự kiện miền hoặc từ một đường đồng bộ nhẹ có kiểm soát.
3. **Trạng thái:** Đã phê duyệt.
4. **Lý do lựa chọn:** Tách dữ liệu theo miền giúp giảm phụ thuộc chặt, tăng khả năng kiểm soát lược đồ và cho phép cô lập các điểm nóng truy cập. Việc dùng read model cho báo cáo giúp hệ thống vừa cung cấp góc nhìn quản trị phong phú, vừa tránh làm chậm POS hoặc KDS trong giờ cao điểm.
5. **Hệ quả kiến trúc:** Dữ liệu giao dịch và dữ liệu phân tích không còn bị trộn lẫn trong cùng một tuyến xử lý. Đổi lại, hệ thống phải chấp nhận một mức **trễ đồng bộ có kiểm soát** giữa đường ghi nghiệp vụ và mô hình đọc báo cáo. Đây là đánh đổi hợp lý vì báo cáo quản trị thường không cần đồng bộ tuyệt đối theo từng mili giây.
6. **Phạm vi áp dụng:** Nguyên tắc này đặc biệt quan trọng đối với các số liệu như doanh thu theo ca, vòng quay bàn, hiệu suất bếp, tiêu hao nguyên liệu và cảnh báo tồn kho.

ADR-05: Áp dụng RBAC và nhật ký kiểm toán cho tác vụ nhạy cảm

1. **Bối cảnh:** IRMS chứa nhiều thao tác nhạy cảm như ghi đè giá, hủy món sau khi gửi bếp, hoàn tiền, điều chỉnh tồn kho, công bố thực đơn và thay đổi vai trò hoặc quyền hạn. Nếu các thao tác này không được kiểm soát và ghi vết đầy đủ, hệ thống sẽ khó đáp ứng nhu cầu vận hành, đối soát và điều tra sau sự cố.
2. **Quyết định:** Áp dụng **RBAC** ở tầng IAM và lớp biên truy cập, đồng thời thực thi thêm

AuthorizationPolicy trong miền nghiệp vụ đối với các thao tác nhạy cảm theo ngữ cảnh. Song song với đó, hệ thống ghi **AuditLog** cho mọi hành động quan trọng, bao gồm người thực hiện, thời điểm, thao tác, thực thể bị tác động và thông tin giải thích cần thiết.

3. Trạng thái: Đã phê duyệt.

4. **Lý do lựa chọn:** RBAC giúp chặn truy cập sai vai trò ở mức tổng quát, nhưng chưa đủ để diễn tả các điều kiện nghiệp vụ tinh hơn như ngưỡng hoàn tiền, yêu cầu lý do ghi đề hoặc phê duyệt khi vượt quyền thông thường. Vì vậy, cần kết hợp kiểm soát theo vai trò với kiểm soát theo chính sách nghiệp vụ. Nhật ký kiểm toán là phần không thể thiếu để đảm bảo khả năng truy vết.

5. **Hệ quả kiến trúc:** Kiểm soát truy cập không còn chỉ là một lớp kỹ thuật ở biên, mà trở thành một phần của thiết kế miền. Điều này giúp IRMS phù hợp hơn với môi trường vận hành thực tế, nơi sai lệch về giá, thanh toán, kho và quyền thao tác đều có thể dẫn đến hậu quả nghiệp vụ rõ rệt.

6. Ví dụ áp dụng trong IRMS:

- hoàn tiền vượt ngưỡng cần kiểm tra chính sách và có thể cần phê duyệt;
- xem hoặc xuất audit log cần quyền riêng;
- thay đổi giá bán hoặc phân quyền phải được ghi kiểm toán đầy đủ.

A.2 Liên kết giữa phụ lục và các phần chính

Phần này cho thấy mỗi phụ lục kỹ thuật được dùng để đọc ngược và biện minh cho đúng phần tương ứng trong báo cáo chính. Nhờ vậy, khi một quyết định đã xuất hiện trong thân báo cáo, người đọc có thể lần ngay sang phụ lục phù hợp để xem lý do chọn kiến trúc, thành phần hoặc loại hình sơ đồ.

Bảng 20: Liên kết giữa các phần chính và các phụ lục biện minh

Phần chính	Điểm cần được giải thích	Phụ lục dùng để biện minh
Mục 2.1	Vì sao phần mô hình hóa mở đầu bằng sơ đồ ngữ cảnh, sơ đồ tổng quan ca sử dụng và sơ đồ hoạt động đầu-cuối thay vì đi thẳng vào đặc tả chi tiết.	Phụ lục A.3 giải thích giá trị của từng loại sơ đồ; Phụ lục A.1 cho thấy các sơ đồ này là nền để đọc kiến trúc, không phải hình minh họa rời rạc.
Mục 3.1	Vì sao IRMS được mô tả như modular monolith theo năng lực, có lớp biên web, tuyến nền và mô hình độc tách riêng.	Phụ lục A.1 chứa các ADR về kiểu kiến trúc, điểm vào, kiểu kết nối và dữ liệu đọc; Phụ lục A.4 giải thích vì sao dùng các thành phần nền tương ứng.
Mục 3.3 và Mục 3.8	Vì sao báo cáo đồng thời dùng góc nhìn logic, mô-đun, thành phần, triển khai và nguyên tắc thiết kế.	Phụ lục A.3 giải thích vai trò của từng góc nhìn; Phụ lục A.2 giúp nối mỗi góc nhìn với đúng phần giải thích tương ứng.
Mục 4.1 và Mục 4.2	Vì sao phần thiết kế chi tiết dùng sơ đồ lớp ở mức aggregate, điểm mở rộng và hành vi thay vì chỉ mô tả dữ liệu; vì sao các sơ đồ hoạt động chi tiết đi theo từng cụm nghiệp vụ.	Phụ lục A.3 giải thích lý do chọn góc nhìn lớp và hoạt động; Phụ lục A.4 giải thích vì sao các điểm mở rộng như thanh toán, thông báo, phân quyền được đưa vào mô hình chi tiết.

A.3 Lý do chọn các góc nhìn và loại sơ đồ

Mỗi loại sơ đồ trong báo cáo trả lời một câu hỏi kiến trúc riêng: ranh giới hệ thống ở đâu, phạm vi ca sử dụng được gom như thế nào, luồng nóng đi qua những mắt xích nào, ranh giới mô-đun nào cần giữ ổn định và mô hình

lớp nào phản ánh đúng hành vi miền nghiệp vụ.

Bảng 21: Lý do chọn từng góc nhìn và loại sơ đồ trong báo cáo

Loại sơ đồ / góc nhìn	Dùng ở phần nào	Lý do chọn
Sơ đồ ngữ cảnh	Mục 2.1	Dùng để chốt ranh giới hệ thống, các vai trò vận hành và các tích hợp ngoài trước khi đi sâu vào kiến trúc. Nếu không có hình này, các mô-đun ở chương kiến trúc dễ bị đọc như cấu trúc kỹ thuật nội bộ thay vì phản ánh bối cảnh thật của IRMS.
Sơ đồ tổng quan ca sử dụng	Mục 2.1	Dùng để gom 25 ca sử dụng thành các gói nghiệp vụ lớn, nhờ đó người đọc nhìn ra phạm vi bắt buộc và các tuyến chức năng nóng trước khi đi vào đặc tả từng mã. Đây là lớp nối giữa yêu cầu chức năng và phân rã kiến trúc.
Sơ đồ hoạt động đầu-cuối	Mục 2.1	Dùng để khóa tuyến vận hành xuyên suốt từ tiếp nhận khách đến giải phóng bàn, qua đó làm rõ những trạng thái nào phải đồng bộ mạnh và những hệ quả nào có thể đẩy sang bất đồng bộ.
Góc nhìn logic	Mục 3.3	Dùng để ánh xạ hệ thống thành các năng lực nghiệp vụ, qua đó kiểm tra xem kiến trúc có phản ánh đúng vận hành nhà hàng hay chưa. Đây là góc nhìn phù hợp để nói về kết tụ, phụ thuộc và ranh giới quyết định ở mức khái niệm.
Góc nhìn mô-đun và thành phần	Mục 3.3	Dùng để chuyển từ bản đồ năng lực sang cấu trúc thực thi bên trong runtime: mô-đun nào giữ trách nhiệm nào, hợp đồng nào đi qua bộ kết nối, luồng nào nên đồng bộ và luồng nào nên phát tán qua sự kiện.
Góc nhìn triển khai và phân bổ	Mục 3.3	Dùng để trả lời hệ thống chạy ở đâu, tuyến dữ liệu và tuyến nền đi qua đâu, và cách cô lập lỗi hoặc mở rộng tải được thực hiện như thế nào. Đây là phần giúp nối quyết định mô-đun với hạ tầng thực thi.
Sơ đồ lớp	Mục 4.1	Dùng để thể hiện aggregate, hành vi miền nghiệp vụ và các điểm mở rộng như PaymentGateway, NotificationSender, AuthorizationPolicy; vì vậy nó phù hợp hơn ERD thuần dữ liệu trong bối cảnh báo cáo kiến trúc.
Sơ đồ hoạt động chi tiết	Mục 4.2	Dùng để kiểm chứng từng ca sử dụng trọng yếu bằng chuỗi thực thi cụ thể, nhất là ở những vị trí nhạy cảm như gửi bếp, thanh toán, hoàn tiền, cập nhật tồn kho và kiểm toán.

A.4 Lý do chọn các thành phần và loại hình kiến trúc nền

Phần này tổng hợp cơ sở giải thích cho các thành phần và kiểu tổ chức xuất hiện trong sơ đồ tổng quan, mô-đun, thành phần và triển khai; đồng thời nối từng lựa chọn với yêu cầu vận hành của IRMS.

Bảng 22: Lý do chọn các thành phần và loại hình kiến trúc nền của IRMS

Thành phần / loại hình	Lý do dùng trong IRMS	Liên hệ trực tiếp trong báo cáo
ReactJS cho lớp giao diện	Phù hợp với nhiều màn hình thao tác nhanh theo vai trò như lễ tân, phục vụ, bếp, thu ngân và quản lý; thuận lợi cho đồng bộ trạng thái giao diện và điều hướng thống nhất.	Xuất hiện trong Mục 3.1 như lớp biên người dùng, đồng thời là điểm vào chung cho các kịch bản ở Mục 2.1.
Nginx / API edge boundary	Tạo biên truy cập rõ ràng, gom xác thực sơ bộ, định tuyến, chuẩn hóa phản hồi và che giấu cấu trúc nội bộ khỏi client.	Biện minh cho ADR-02 trong Phụ lục A.1 và xuất hiện trực tiếp trong sơ đồ tổng quan kiến trúc.
Modular monolith theo năng lực	Phù hợp với IRMS vì các giao dịch lõi như gọi món, bếp và thanh toán cần gần nhau, nhưng vẫn phải tách ranh giới mô-đun để dễ bảo trì và mở rộng có kiểm soát.	Là trục chính của ADR-01; được thể hiện xuyên suốt ở Mục 3.1 và Mục 3.3.
Outbox / background worker	Tách các hệ quả phụ như làm mới read model, gửi thông báo, in biên lai và phát cảnh báo khỏi tuyến giao dịch nóng; hỗ trợ retry và giảm ghép nối.	Biện minh cho ADR-03 và xuất hiện trong sơ đồ tổng quan, mô-đun, thành phần và triển khai.
PostgreSQL giao dịch	Phù hợp để neo giữ tính nhất quán của các quyết định có hậu quả nghiệp vụ cao như xác nhận đơn, đóng hóa đơn, hoàn tiền và truy vết kiểm toán.	Gắn với ADR-04 và các sơ đồ dữ liệu / triển khai trong Mục 3.3.
Read model cho báo cáo	Giữ báo cáo và phân tích không làm nặng tuyến giao dịch nóng, đồng thời chấp nhận độ trễ nhỏ có kiểm soát cho dữ liệu quản trị.	Biện minh bởi ADR-04; liên kết trực tiếp với các sơ đồ báo cáo và triển khai.
Payment gateway adapter	Cô lập giao thức và nhà cung cấp thanh toán khỏi lỗi nghiệp vụ; cho phép thay đổi cách ủy quyền, capture hoặc refund mà không kéo logic tài chính vào mã hạ tầng.	Xuất hiện ở Mục 3.1 và Mục 4.1 như một điểm mở rộng bắt buộc của vùng thanh toán.
Notification sender / receipt printer adapter	Tách những tích hợp có độ biến động cao khỏi miền nghiệp vụ; thuận tiện cho retry, fallback và thay đổi nhà cung cấp.	Nối trực tiếp giữa phần kiến trúc tổng quan với phần lớp / hoạt động chi tiết ở Mục 4.2.
RBAC + AuthorizationPolicy + audit log	Bảo vệ các thao tác nhạy cảm như hoàn tiền, ghi đề giá, đổi quyền, xem audit log; đồng thời giữ khả năng truy vết khi điều tra sự cố.	Được biện minh bởi ADR-05 và được dùng trong cả góc nhìn kiến trúc lẫn góc nhìn lớp chi tiết.

A.5 Phân công công việc

Phần phân công dưới đây được trình bày theo hướng mỗi thành viên phụ trách một mảng chính, đồng thời đều tham gia phối hợp rà soát và hiệu chỉnh để bảo đảm tính liên kết giữa các phần của báo cáo. Cách phân công này phản ánh vai trò chính của từng thành viên nhưng vẫn nhấn mạnh tính cộng tác chung của toàn nhóm trong quá trình hoàn thiện tài liệu.

Bảng 23: Phân công công việc của Nhóm 8

MSSV	Thành viên	Phạm vi đóng góp chính
2311896	Đỗ Quang Long	Phụ trách tổng hợp bối cảnh nghiệp vụ, hệ thống hóa yêu cầu chức năng và phi chức năng, đồng thời phối hợp rà soát để bảo đảm các yêu cầu được phản ánh nhất quán trong các phần phân tích và thiết kế.
2311982	Lê Hoàng Luân	Phụ trách tổ chức cấu trúc tài liệu LaTeX, chuẩn hóa hình thức trình bày và tích hợp các thành phần nội dung, đồng thời phối hợp với các thành viên khác để bảo đảm bố cục báo cáo và hệ thống sơ đồ thống nhất.
2310990	Lê Thúy Hiền	Phụ trách xây dựng phần đặc tả use case và mô tả các luồng hoạt động nghiệp vụ chính, đồng thời phối hợp đối chiếu với phần yêu cầu và phần kiến trúc để giữ sự liên thông giữa mô tả nghiệp vụ và giải pháp thiết kế.
2213698	Nguyễn Hải Trung	Phụ trách các góc nhìn mô-đun, thành phần và kết nối của hệ thống, đồng thời phối hợp với các phần khác để bảo đảm các quyết định kiến trúc bám sát yêu cầu nghiệp vụ và thống nhất với cấu trúc tổng thể của báo cáo.
2211384	Tô Thế Hưng	Phụ trách xây dựng các sơ đồ lớp và rà soát mô hình miền nghiệp vụ, đồng thời phối hợp đối chiếu với phần kiến trúc và phần hành vi để bảo đảm cấu trúc lớp phản ánh đúng các trách nhiệm và điểm mở rộng của hệ thống.
2310737	Trần Nhật Đăng	Phụ trách phần triển khai, phân bổ và phân tư kiến trúc, đồng thời phối hợp rà soát cách diễn đạt và mối liên kết giữa các chương để bảo đảm báo cáo có tính nhất quán và hoàn chỉnh ở mức toàn cục.